你缺失的那門手機課 V1.8

v1.8

ClassTechStar

目录

一、基礎篇（第 0-7 章） **2**

第零章：嗨，開始之前 **2**

一、這本書能幫你做什麼？（和不能做的） 2

二、我們的“暗號”說明 2

三、安卓和 iOS，咱都說說 2

四、手機容量：為啥買了 128G，可用卻不到 110G？ 2

五、你的手機是一座“數字資產庫” 3

第一章：你好，我的私人電腦 **5**

一、手機硬體“五臟六腑”：深度解析 5

二、作業系統：安卓 VS iOS VS 鴻蒙 11

三、手機介面全解析 12

四、手機產品體系與命名規則 13

五、基帶與天線：手機通訊的幕後功臣 14

六、你的手機，到底在扮演多少角色？ 15

附：安卓手機記憶體拓展解析 16

第一章附：鴻蒙作業系統專題 18

第二章：整理你的數字房間——檔案管理 **22**

一、三層儲存體系：你的數字倉庫 22

二、認識“檔案管理器” 22

三、檔案系統結構：深入瞭解 23

四、資料備份與恢復：給你的資料上保險 23

五、檔案安全：保護你的隱私 24

六、空間不夠？來場大掃除 24

七、常見檔案格式全解析 24

八、圖片與檔案系統管理：讓資產井井有條 26

第三章：讓手機接上“生命線”——網路 **28**

〇、聯網不迷茫——分清 Wi-Fi 與流量的“路”與“車” 28

一、行動網路（4G/5G）與流量 29

二、Wi-Fi：連線、分享與安全 30

三、藍芽：短距離無線通訊的萬能選手 31

四、紅外遙控：手機變身萬能遙控器 32

五、NFC：碰一碰的魔法 32

六、個人熱點：把手機變 Wi-Fi 路由器 33

第四章：安裝和管理 App——手機的“技能包” **34**

一、官方商店：最安全的“應用超市” 34

二、純淨/防護模式：開啟手機的“防毒牆” 34

三、第三方來源與“電子寄生蟲”：風險極高！ 35

四、裝、更新、解除安裝 36

五、許可權管理：你是 App 的“老闆” 36

六、軟體安裝包的構成要素 37

第五章：讓手機更懂你——基礎設定 **39**

一、聲音與震動 39

二、顯示與亮度 39

三、桌布、主題與桌面 39

四、效率設定：讓手機成為你的“外掛大腦” 40

五、輔助功能與無障礙：科技向善的力量 41

第六章：守護你的數字生活——安全與隱私 **46**

一、鎖屏：第一道物理防線 46

二、“查詢我的手機”：終極防盜工具 46

三、系統更新：打補丁，堵漏洞 47

四、識別與防範電信詐騙 47

五、抵禦“開盒”與社工攻擊：斬斷隱私洩露鏈條 48

六、隱私保護：你的資料是你的財產 49

七、丟失 / 被盜後的“急救”清單 50

八、SIM 卡與 eSIM 安全 50

第七章：前沿功能全解析 **52**

一、衛星通訊：沒訊號也能求救 52

二、無網通：近距離通訊新方式 53

三、車禍檢測：關鍵時刻救命 53
四、星閃技術：下一代無線連線 54

二、軟體篇（第 8-14 章） 57

第八章：通訊：簡訊、電話與社交 57
一、電話 App 的進階功能 57
二、簡訊與 iMessage/RCS 59
三、微信/QQ：國民級社交應用程式的管理 60
四、視訊通話對比 61
五、電子郵件：在手機上配置與使用 62

第九章：移動辦公與學習 64
一、文件處理：在掌中編輯與創作 64
二、筆記應用：從隨手記到知識體系 64
三、掃描與 OCR：把紙質世界搬進手機 65
四、網盤怎麼選：沒有最好的，只有最適合你的 65
五、一個真實的移動辦公場景 66

第十章：移動支付與生活服務 67
一、到底怎麼繫結銀行卡？ 67
二、付款碼是“誰看誰收錢” 67
三、這些安全設定，花 5 分鐘設好能省很多心 68
四、公交地鐵：手機就是交通卡 68
五、生活繳費、外賣、打車 68
六、用手機搞定醫保、社保、政務 69

第十一章：攝影與圖片管理 70
一、你是不是也犯過這些錯？ 70
二、幾個立刻提升出片率的小技巧 70
三、人像模式與專業模式 71
四、後期修圖：不要過度，恰到好處就行 71
五、照片太多怎麼辦？整理策略 72
六、截圖與錄屏 72
第十一章附：專業影像技術知識 73

第十二章：影音娛樂與閱讀 81

一、聽音樂：選哪個 App，怎麼聽才舒服 81
二、播客（Podcast）：通勤路上的“耳朵伴侶” 81
三、看影片：從播放器到流媒體的正確開啟方式 82
四、電子書閱讀：手機上也能好好看書 82
五、手機遊戲：玩得爽，也不傷手機 83

第十三章：工具類 App 推薦 **85**
一、效率工具：管理你的時間與注意力 85
二、健康運動 87
三、旅行出行 88
四、自動化工具啟蒙 90
五、瀏覽器與搜尋引擎：你的資訊入口 90
六、實用系統工具：清理、防騷擾與安全防護 93
七、天氣預警、自然災害預警與地震預警 95
八、跨裝置協同：遠控電腦、快傳與帳號安全 97

第十四章：密碼管理器專題 **101**
一、為什麼你需要密碼管理器？ 101
二、密碼管理器的工作原理 101
三、加密原理簡介 101
四、主流密碼管理器推薦 103
五、密碼強度真實評估 103
六、Passkey：無密碼登入的未來 105
七、硬體安全金鑰 107
八、家庭/團隊密碼共享 108
九、密碼管理器使用最佳實踐 110

三、進階篇（第 15-20 章） **113**

第十五章：電池科學與續航最佳化 **113**
一、電池是怎麼回事？迴圈與健康度 113
二、充電玄學？快充、無線充與黃金法則 113
三、快充協議全解析：為什麼你的充電器充不快？ 114
四、電池新技術：矽碳負極電池 117
五、這才是真正的省電技巧 117

第十六章：賬號體系與雲服務 **119**

一、你的手機賬號，到底管著多少東西？ 119
二、三個必須做的事，現在就做 119
三、iCloud：蘋果使用者的“空氣” 120
四、同步和備份，別搞混了 121
五、安卓使用者的賬號矩陣 121
六、家庭共享：讓家人也受益 121

第十七章：手機與其他裝置的聯動 123

一、與電腦協同：手機和電腦不再是兩個世界 123
二、投屏到電視：全家一起看 123
三、與智慧手錶/手環：健康管家長在手腕上 124
四、與智慧家居：手機是家裡的“總開關” 124
五、藍芽耳機與音箱：無線的自由 125
六、與車載系統：開車時的安全搭檔 126
七、聯動中的安全和隱私 126

第十八章：資料備份與遷移 128

一、先搞清楚：你手機裡哪些東西丟了就沒了 128
二、整機備份：一次搞定所有東西 128
三、分類備份策略：重要的單獨另存一份 129
四、換機指南：從舊手機到新手機 130
五、備份的黃金法則：三二一原則 130

第十九章：故障排查與自救 132

一、手機變卡、發熱——最常被問的問題 132
二、應用閃退、無響應——App 不聽話了 132
三、充不進電或充電慢——可能不是電池的鍋 133
四、網路連不上、Wi-Fi 老是斷 133
五、螢幕問題：觸控不靈、閃屏、留印子 134
六、無法開機、卡在 Logo 介面——先別慌 134
七、聲音問題：喇叭不響、通話聲音小 134
八、找官方維修，避開那些坑 135
第二十章：開發者模式與高階許可權 136
第二十章附：Termux——Android 裝置上的行動式 Linux 終端環境 143

四、超越篇（第 21-26 章） 147

第二十一章：行動網路與資訊保安深探 147

一、VPN：網路裡的“隱形斗篷” 147
二、公共 Wi-Fi 的“甜蜜陷阱”——比你想象的危險 147
三、HTTPS 是怎麼保護你的？——SSL/TLS 握手 148
四、DNS 安全：你的“網路電話本”安全嗎？ 149
五、“無痕瀏覽”真的能隱身嗎？ 149
六、瀏覽器隱私：別讓瀏覽器把你賣了 149
七、實用安全清單：保護自己從現在開始 150

第二十二章：AI 如何在手機中工作 152

一、AI 相機：你的“後期暗房”搬進了手機 152
二、語音助手：你的“聲音秘書”如何聽懂你？ 152
三、端側 AI vs. 雲端 AI 152
四、AI 大模型與手機：你的隨身智慧助手 152
五、幾個典型應用場景與實操 154
六、提示詞最佳化：讓 AI 聽你指揮的“溝通公式” 155
七、AI 的侷限與挑戰：保持清醒的判斷力 158

第二十三章：程式設計思維與自動化啟蒙 160

一、自動化到底是個啥？ 160
二、iOS“快捷指令”——蘋果使用者的自動化神器 160
三、安卓自動化——Tasker 和它的朋友們 161
四、自動化思維的日常應用 162
五、踩坑指南：自動化的常見問題 163

第二十三章附：GitHub 入門指南——從程式碼託管到協作平臺 165

一、GitHub 到底是什麼？ 165
二、Git 和 GitHub 的區別（關鍵概念!） 165
三、GitHub 的核心概念與功能 166
四、註冊與上手：五分鐘建立你的第一個倉庫 167
五、基本工作流：從本地到雲端 168
六、團隊協作流程：Pull Request 是怎麼工作的 168
七、GitHub 的典型應用場景 169
八、給新手的幾點建議 169
九、總結：為什麼每個人都該瞭解 GitHub 170

第二十四章：Root 權限與 BL 鎖深度解析 **172**
一、深入理解 Bootloader 172
二、解鎖 BL 鎖的詳細流程（以小米為例） 173
三、Root 的多種方案 174
四、刷機入門 174
五、三星 Knox：BL 解鎖的不可逆代價 174
六、常見問題解答（Q& A） 175

第二十五章：二手手機處理與數字遺產 **176**
一、舊手機的“安樂死”：安全出售或轉贈指南 176
二、數字遺產：你的社交靈魂如何安放？ 177
三、環保的最後一程 181

第二十六章：當下手機技術前沿 **183**
一、AI 手機：是營銷噱頭，還是真變革？ 183
二、各廠商 AI 戰略對比：誰在認真做，誰在貼牌？ 183
三、衛星通訊：從旗艦“噱頭”到中端標配 184
四、摺疊屏生態成熟度：能當主力機了嗎？ 184
五、6G 與 Wi-Fi 7：下一代連線的速度與邊界 185
六、矽碳負極電池：續航終於迎來材料級突破 185
七、本章速覽：一句話記住這些前沿 186

五、實戰篇（第 27-32 章） **188**

第二十七章：爸媽也能輕鬆用——適老化手機完全指南 **188**
零、先把心態擺正：這不是“老人用不來”，是“手機沒做好” 188
一、第一步：開啟“簡易模式”——手機變清爽了 188
二、如果不開簡易模式，手動調也可以——“大字 + 大圖示”五步走 189
三、防詐騙——這章請一定帶著長輩一起看 190
四、遠端協助——你在外地也能幫爸媽修手機 191
五、微信的“適老改造”——跟長輩說這幾句就夠了 191
六、手機丟了怎麼辦？幫長輩做個“應急預案” 192
七、無線耳機和助聽器相容——被忽視的好功能 192
八、健康相關功能——手機不只是通訊工具 192

第二十八章：花對錢買到心儀機——手機選購終極指南 **194**
零、三步選機法：需求預算機型 194

一、核心引數怎麼看——不用全懂，看懂這五項就夠了 194
二、按預算分檔推薦（以 2026 年中市場為基準） 195
三、按場景選機——你的需求是什麼？ 197
四、五個容易踩的坑——買之前對照看一眼 197
五、什麼時候買最劃算？ 198
六、渠道怎麼選——在哪買靠譜？ 198

第二十九章：數字健康——和手機建立健康關係 200
一、先看看你的“螢幕時間”——數字讓人清醒 200
二、給 App 設定“使用限額”——給自己裝個限速器 200
三、“勿擾模式”的正確用法——不是關掉通知，是關掉焦慮 201
四、護眼不靠 App 靠習慣——20-20-20 法則 201
五、“低頭族”的頸椎自救指南 201
六、睡眠與手機——臥室裡最該改的習慣 202
七、資訊焦慮症——學會“選擇性失明” 202

第三十章：投屏、演示與車載互聯 204
一、無線投屏三大陣營——別搞混了 204
二、有線投屏——更穩定、零延遲 204
三、手機變電腦——桌面模式 205
四、車載互聯——開車用手機的正確方式 205
五、會議室投屏實戰——不再手忙腳亂 206

第三十一章：短影片拍攝與內容創作入門 207
一、拍之前先想清楚：你要拍給誰看？ 207
二、手機拍攝基礎設定——開拍前必做的 5 件事 207
三、運鏡五招——讓畫面“動”起來 208
四、剪輯入門——剪映就夠了 208
五、封面的重要性——70% 的人只看封面決定看不看 208
六、內容創作的三條心法 209

第三十二章：手機安全急救手冊 210
場景零：地震 / 極端天氣——手機怎麼幫你保命 210
場景一：手機丟了/被偷了——黃金 30 分鐘 211
場景二：被騙了——72 小時黃金處理期 211
場景三：手機中了病毒/木馬——怎麼辦？ 212
場景四：帳號被盜——快速奪回控制權 212

場景五：收到勒索資訊——不要怕，不要付 213
場景六：AI 換臉/AI 擬聲詐騙——眼見不再為實 213
場景七：SIM swap 攻擊（SIM 劫持）——你的號碼被“偷”走了 214

六、展望篇與車聯篇（第 33-34 章） 218

第三十三章：未來展望——手機的下一程 218
一、AR/VR 與空間計算：手機之外的“第二桌面” 218
二、腦機介面：思想成為互動媒介 218
三、手機的“隱身”與新身份基石 219
四、倫理思考：我們，還是手機的主人嗎？ 219
五、保持連線，也保持獨立——你的數字生活哲學 219

第三十四章：車機互聯深度指南——手機與汽車的融合 221
一、車機互聯的三大路線 221
二、CarPlay / Android Auto / HiCar 對比 221
三、鴻蒙座艙：手機與車機的無縫流轉 222
四、造車新勢力的車機系統 222
五、手機當車機——能用嗎？ 223
六、實用技巧 223

兒童與手機專題（第 35 章） 226

第三十五章：兒童與手機——數字時代的教育指南 226
一、幾歲給孩子手機？ 226
二、家長控制系統全攻略 227
三、青少年模式 228
四、網路素養教育 228
五、健康使用習慣 229
六、數字遺產與兒童 229

第三十六章：手機與智慧家居——讓家變聰明 231
一、智慧家居協議：選誰很重要 231
二、三大生態對比 232
三、手機當家居中樞 232
四、入門套件推薦 233

五、安全注意事項 233

七、AI 程式設計實戰篇（第 37-47 章） 236

第三十七章：正規化革命——從“寫程式碼”到“指揮 AI 寫程式碼” 236

一、Vibe Coding 的誕生：一句話引發的運動 236
二、2026 年現狀：Agentic Engineering 時代 236
三、核心認知轉變：從 Coder 到 Conductor 237
四、為什麼方法論比工具更重要 238

第三十八章：AI 程式設計五大思維原則 240

一、原則一：Think Before Coding（先思考，再編碼） 240
二、原則二：Simplicity First（簡單至上） 241
三、原則三：Surgical Changes（精準修改） 242
四、原則四：Goal-Driven Execution（目標驅動執行） 242
五、原則五：Iterate and Verify（迭代驗證） 243
六、Harness Engineering：Agent = Model + Harness 244

第三十九章：上下文工程——讓 AI 理解你的意圖 246

一、專案記憶檔案：AI 的“長期記憶” 246
二、規範驅動開發（SDD） 247
三、上下文視窗管理 248
四、上下文工程的實踐要點 248

第四十章：需求工程——把想法變成 AI 可執行的指令 250

一、從模糊想法到結構化需求 250
二、需求拆解的分層藝術 251
三、PRD 寫作模板與實踐 251
四、實戰：從想法到方案的完整推演 252

第四十一章：2026 年 AI 程式設計工具全景 254

一、三大流派與選擇框架 254
二、IDE 內建 AI 流派 255
三、終端 Agent 流派 255
四、雲端 Builder 流派 256
五、工具會變，方法論不變 256

第四十二章：大型語言模型全景與 API 實戰 258

一、國際主流大型語言模型：三強格局 258
二、國產主流大型語言模型：百花齊放 260
三、模型選擇決策框架 263
四、API 呼叫入門：從零開始接入大型語言模型 264
五、Token Plan 購買指南 267
六、實戰：在專案中接入大型語言模型 API 270

第四十三章：對話工程——與 AI 高效協作的對話技術 273
一、Prompt 分層策略 273
二、除錯對話：如何描述錯誤讓 AI 快速定位 274
三、迭代策略：小步快跑 vs 大步重構 274
四、多輪對話的上下文管理 275

第四十四章：從 0 到 1 實戰——完整專案全流程 277
一、專案一：極簡個人工具站（入門級） 277
二、專案二：全棧記帳系統（進階級） 278
三、專案三：AI 增強應用（進階） 279
四、通用實戰流程清單 280

第四十五章：品質保障——讓 AI 寫的程式碼可靠 282
一、測試驅動：讓 AI 寫測試 282
二、程式碼審查技巧 283
三、版本控制：Git 在 AI 程式設計中的角色 283
四、常見陷阱與避坑指南 284

第四十六章：進階之路——從使用者到架構師 286
一、MCP 協議與工具擴充套件 286
二、自定義工作流與 Skills 286
三、多 Agent 協作模式 287
四、持續學習路徑 287

第四十七章：速查手冊與資源 289
一、思維模型速查卡 289
二、工具選擇決策樹 289
三、常見問題 FAQ 290
四、推薦學習資源 290
五、結語：工具在手，方法論在心 291

參考文獻 292

八、安卓開發實戰篇（第 48-58 章） 294

第四十八章：認識安卓——從作業系統到開發生態 294

一、安卓的身世：從“小機器人”到“全球最大系統” 294
二、用什麼語言寫：Java 還是 Kotlin？ 295
三、一個 App 是怎麼誕生的？ 296
四、安卓技術棧全景：你的“武器清單” 296
五、安卓 vs iOS：開發者的第一堂“選邊課” 297

第四十九章：搭建開發環境——Android Studio 與第一個專案 299

一、安裝 Android Studio 299
二、建立你的第一個專案 299
三、跑起來：模擬器與真機 300
四、工程結構拆解：每個檔案是幹嘛的 301
五、小試牛刀：把“Hello”換成你的名字 301
六、常見報錯急救：新手前兩週的“急診室” 302

第五十章：Kotlin 快速上手——安卓開發的第一語言 303

一、變數與型別：給資料起名字 303
二、函式：把邏輯打包成“按鈕” 304
三、條件與迴圈：讓程式會“思考” 304
四、空安全：Kotlin 的“保命符” 305
五、類與物件：造“零件”的模具 305
六、lambda 與進階函式：Kotlin 的“語法糖” 306
七、寫給零基礎的語法速查卡 306

第五十一章：介面開發——Jetpack Compose 入門 308

一、Compose 的思想：介面就是函式 308
二、基礎元件：文字、按鈕、輸入框、圖片 309
三、佈局三件套：Column、Row、Box 309
四、狀態：讓介面“活”起來 310
五、主題與深色模式：一次配置，全域性生效 311
六、預覽、動畫與實時編輯：Compose 的“外掛” 311

第五十二章：四大元件與 Activity 生命週期 313

一、四大元件：安卓的“四梁八柱” 313
二、Activity 生命週期：安卓開發的第一張“藏寶圖” 313
三、Intent：App 裡外通行的“車票” 314
四、多頁面與導航 315
五、順帶認識：Fragment 是什麼（以及為什麼本書沒教） 315

第五十三章：資料儲存與網路請求 317
一、儲存方案怎麼選：先看這張表 317
二、DataStore：存點小配置 317
三、Room：給資料安個“真正的家” 318
四、Retrofit：和伺服器說話 319
五、JSON 解析：給網路資料“翻譯” 319
六、協程：安卓併發的“普通話” 320
八、協程：安卓併發的“普通話” 320

第五十四章：專案一——我的記帳本（本地版） 322
一、需求思考：先問五個問題 322
二、資料層：讓“賬”有個家 322
三、MVVM：介面與邏輯“分家” 323
四、介面：列表頁 + 錄入頁 324
五、編輯與刪除：讓資料“能進能出” 325
六、驗收：三層檢查 + 兩個進階挑戰 326
八、驗收：三層檢查 + 兩個進階挑戰 326

第五十五章：專案二——天氣查詢 App（網路版） 327
一、申請免費天氣 API 327
二、網路層：Retrofit + JSON 解析 327
三、三種介面狀態：載入中 / 成功 / 失敗 328
四、實戰介面程式碼：把天氣畫出來 329
五、介面與互動細節 329
七、介面與互動細節 329

第五十六章：打磨與發佈——測試、除錯與上架 Google Play 331
一、除錯三板斧：Logcat、斷點、UI 檢查器 331
二、測試：給程式碼上保險 331
三、簽名打包：給 App 蓋上“印章” 332
四、上架 Google Play 全流程 333

五、商店素材怎麼做：第一印象就是下載量 333
五、效能最佳化與崩潰上報小清單 333

第五十七章：進階之路——權限、通知與現代架構 335
一、執行時權限：先敲門，再進門 335
二、通知：在正確的時間說有用的話 336
三、架構進階：依賴注入與模組化 336
四、2026 年的新玩法：AI 輔助安卓開發 337
五、Compose 效能最佳化與除錯工具 337

第五十八章：速查手冊與資源 339
一、常用程式碼速查卡 339
二、常見問題 FAQ 339
三、推薦學習資源 340
四、Gradle 常用命令速查 340
五、結語：從第一個 App 到第 N 個 App 341
七、結語：從第一個 App 到第 N 個 App 341
參考文獻 341

九、iOS 開發實戰篇（第 59-69 章） 343

第五十九章：認識 iOS——蘋果生態與開發生態 343
一、iOS 的身世：一部手機改變一個行業 343
二、圍牆花園：蘋果生態的“規矩”與“紅利” 343
三、用什麼語言寫：Objective-C 還是 Swift？ 344
四、一個 App 是怎麼誕生的？ 344
五、iOS 技術棧全景：你的“武器清單” 345

第六十章：搭建開發環境——Xcode 與第一個專案 347
一、準備一臺 Mac 347
二、安裝 Xcode 347
三、建立你的第一個專案 348
四、工程結構拆解：每個檔案是幹嘛的 348
五、真機除錯：把 App 裝進你的 iPhone 349
五、Xcode 效率技巧：新手的五個“快速鍵外掛” 349
六、真機除錯：把 App 裝進你的 iPhone 349
七、小試牛刀 + 常見報錯急救 350

九、小試牛刀 + 常見報錯急救 350

第六十一章：Swift 快速上手——蘋果開發的第一語言 351

一、變數與型別：給資料起名字 351
二、可選值（Optional）：Swift 的“防崩護身符” 352
三、函式與閉包：把邏輯打包成“按鈕” 352
四、結構體與協議：Swift 的“樂高”體系 353
五、寫給零基礎的語法速查卡 354

第六十二章：介面開發——SwiftUI 入門 355

一、SwiftUI 的思想：宣告式，不是命令式 355
二、基礎檢視：文字、按鈕、輸入框、圖片 356
三、佈局三件套：VStack、HStack、ZStack 356
四、狀態與繫結：讓介面“活”起來 357
五、預覽、主題與動畫：SwiftUI 的“外掛” 357
六、常用修飾符速查：一頁在手，介面不愁 358

第六十三章：檢視、狀態與導航 360

一、狀態管理的“資料流全景” 360
二、NavigationStack：頁面跳轉的“棧” 361
三、跳出視窗與底部標籤：sheet 與 TabView 361
四、List：iOS 的“國民列表” 362
五、程式生命週期：前臺與後臺 363

第六十四章：資料儲存與網路請求 365

一、儲存方案怎麼選：先看這張表 365
二、UserDefaults：存點小配置 365
三、SwiftData：給資料安個“真正的家” 366
四、Codable：給 JSON“翻譯” 367
五、URLSession + async/await：和伺服器說話 367
六、網路錯誤也要“分級”：別把一切報錯都丟給使用者 368

第六十五章：專案一——我的記帳本（本地版） 370

一、需求思考：先問五個問題 370
二、資料層：讓“賬”有個家 370
三、MVVM：介面與邏輯“分家” 371
四、介面：列表頁 + 錄入跳出視窗 372

五、編輯、刪除與彙總 373
六、驗收：三層檢查 + 兩個進階挑戰 373

第六十六章：專案二——天氣查詢 App（網路版） 375
一、申請免費天氣 API 375
二、網路層：URLSession + Codable 375
三、三種介面狀態：載入中 / 成功 / 失敗 376
四、實戰介面程式碼：把天氣畫出來 377
五、互動細節與收藏城市 377
六、網路請求怎麼除錯：列印、抓包與模擬慢網 378

第六十七章：打磨與發佈——測試、除錯與上架 App Store 379
一、除錯三板斧：print、斷點、預覽 379
二、測試：給程式碼上保險 379
三、簽名與打包：給 App 蓋上“印章” 380
四、上架 App Store 全流程 381
五、商店素材怎麼做：第一印象就是下載量 381
六、崩潰上報與效能最佳化小清單 381

第六十八章：進階之路——權限、通知與現代架構 383
一、權限：先敲門，再進門 383
二、通知：在正確的時間說有用的話 384
三、架構進階：MVVM 深入與可測試性 384
四、2026 年的新玩法：AI 輔助 iOS 開發 385
五、Swift 併發要點：@MainActor 與 Task 385

第六十九章：速查手冊與資源 387
一、常用程式碼速查卡 387
二、常見問題 FAQ 387
三、Xcode 高頻快速鍵速查 388
四、推薦學習資源 389
六、推薦學習資源 389
五、結語：從第一個 App 到第 N 個 App 389
參考文獻 389

附錄 A：新手機開箱“保姆級”流程 392
開箱驗機“三部曲”： 392

啟用與初始設定“重點提醒”： 392
裝 App“安全第一”： 392

附錄 B：動手練習小參考 **393**

附錄 C：極簡術語“黑話”對照表 **394**

附錄 D：靠譜資源去哪裡找？ **395**
官方支援是首選： 395
實用網站與博主： 395
安全自查一句話清單： 395

附錄 E：感謝與說明 **396**
6 版更新要點 396
7 版更新要點 396
8 版更新要點 398
內容增補：實用軟體與騷擾攔截 398
內容增補：天氣預警 / 自然災害預警 / 地震預警 398
內容增補：跨裝置協同與效率工具 399

附錄 F：摺疊屏選購與使用專題 **400**
一、摺疊屏現在發展到什麼程度了？ 400
二、什麼人適合買摺疊屏？ 400
三、選購要點：看這 6 個指標 401
四、摺疊屏獨有的使用技巧 402
五、保養要點：讓摺疊屏活得久一點 402
六、摺疊屏的“坑”和心理預期 403
七、主流品牌怎麼選（2026 年） 403

結語：保持好奇，保持清醒 **405**

附錄 G：全書速查索引 **406**

附錄 H：版本路線圖 **410**

一、基礎篇（第 0-7 章）

第零章：嗨，開始之前

在正式探索前，咱們先聊點基本規則，確保後面說的你都懂。別急，磨刀不誤砍柴工嘛。

一、這本書能幫你做什麼？（和不能做的）

它能幫你系統性地搞懂手機怎麼用，明白功能背後的道理，而且無論你是安卓黨還是蘋果派，大多數思路是通用的。它會帶著你動手練習，讓你從“會用”升級到“懂用”。

它不是某一款特定手機的說明書，也不是追新品的科技新聞，更不是教你沉迷手機的指南。說實話，如果你期待的是“如何用手機刷影片更爽”，那可能要失望了——我們更關心的是如何讓手機真正為你服務，而不是反過來。

二、我們的“暗號”說明

為了說得更清楚，書裡會用一些小標記，這些標記就像我們之間的默契暗號：

- **[按鈕名]**：代表螢幕上具體的按鈕或 App 名字，比如 [設定]、[確定]。
- **>**：代表操作順序，比如” 點 [設定] > [聲音]”，就是先點設定，再點聲音。
- **加粗**：非常重要！可能是關鍵概念或安全警告，請務必留意。
- *斜體*：一些補充的小想法或比喻，幫你更好地理解。

三、安卓和 iOS，咱都說說

你猜怎麼著，有些東西叫法不同，但乾的事兒一樣。比如：應用（安卓）和 App（iOS），設定（都叫設定），檔案管理（安卓）和檔案（iOS）。知道是一回事就行，別被名詞繞暈了。

另外，現在安卓陣營裡還有華為的鴻蒙系統，它有自己的特色，後面我們會專門講到。總之，不管你用什麼手機，這本書都能幫到你。

四、手機容量：為啥買了 128G，可用卻不到 110G？

容量單位大家知道：1TB = 1024GB，1GB = 1024MB，1MB = 1024KB……但這裡有個坑：硬碟廠商通常按 1000 進位制算，而系統按 1024 進位制算，所以你買到的“128G”硬碟，

實際容量會“縮水”。

但你手機容量縮水的原因主要有三個：

1. **系統自己佔了地方**：作業系統就像房子的承重牆，必須存在。現在的系統動輒佔用 10-20GB，這很正常。
2. **廠商預裝的應用**：新手機裡自帶的那些 App 也佔空間，有的能解除安裝，有的不能。
3. **分割區和格式化損耗**：技術原因，就像書包的襯布也佔體積。

買手機的小建議

現在 128GB 的手機基本不夠用了，256GB 起步比較穩妥。如果你喜歡拍照、玩遊戲，建議直接上 512GB 甚至更大。記住，儲存空間這東西，寧可富餘，不可捉襟見肘。

來動手：看看你的手機”身份證”

開啟 [設定]，找到關於手機（或類似選項），記下：手機型號、系統版本、總儲存空間和剩餘空間。這是瞭解它的第一步。順便看看電池健康度，心裡有個數。

五、你的手機是一座“數字資產庫”

在開始具體知識之前，我們先建立一個核心認知：**手機早已不只是通訊工具，它是我們最重要的數字資產庫**。聯絡人、聊天記錄、家庭照片、工作文件、銀行 App、付款碼、健康資料、電子證件……這些資料加在一起，價值往往超過手機本身。

為什麼這很重要？

如果手機丟了或壞了，你損失的不僅是幾千塊錢的硬體，**更可能是多年積累的不可再生資料**——孩子的成長照片、過世親人的語音、沒備份的重要合同。科學地整理與防護習慣，能從三個維度保護你：

- **提升執行效率**：規範的檔案分類讓系統查詢更快，手機長期不卡頓。
- **延長使用壽命**：合理的儲存管理減少晶片讀寫負擔，電池和儲存都更耐用。
- **保障資料安全**：分層管理 + 定期備份 + 許可權收斂，讓”資料災難”遠離你。

我們的核心理念：“預防為主，防治結合”

就像養生不能等到得病才看醫生，**手機管理也要從“日常習慣”抓起**。本書反覆強調的不是某個高階技巧，而是這些樸素的原則：

- **預防為主**：裝 App 只走官方商店、照片發之前先脫敏、密碼不重複——這些”小動作”成本極低，但能擋住絕大多數風險。
- **定期體檢**：每月一次整理下載資料夾、每季度一次許可權審計、每半年一次全量備份——固定節奏比臨時救火有效一百倍。

- **留有退路：**開啟雲同步、記好賬號密碼、寫一份”數字遺產清單”——確保意外發生時你和家人都能從容應對。

第一章：你好，我的私人電腦

你的手機，其實是你口袋裡最私人的超級計算機。它比當年把人類送上月球的計算機強大無數倍，卻安靜地躺在你的掌心。今天，咱們就來好好認識這位“沉默的巨人”。

一、手機硬體“五臟六腑”：深度解析

手機雖小，五臟俱全。讓我們像解剖課一樣，逐個認識這些精密的元件。

SoC（系統級晶片）：手機的“大腦 + 心臟”

SoC 全稱 System on Chip，翻譯過來就是“把整個系統塞進一塊晶片”。它不只是 CPU（中央處理器），而是把 CPU、GPU（圖形處理器）、NPU（神經網路處理器）、ISP（影象訊號處理器）、基帶（負責通訊）等等全部整合在一起。

這就像把一個城市的政府大樓、發電廠、通訊中心、警察局全部建在一棟樓裡，效率極高。

CPU：通用計算的核心

CPU 是 SoC 中最核心的元件，負責處理絕大多數計算任務。手機 CPU 採用 ARM 架構，與電腦的 x86 架構不同，ARM 架構以精簡指令集（RISC）為特色，功耗更低，更適合移動裝置。

CPU 大小核設計：現代手機 CPU 普遍採用“大小核”架構（ARM 的 big.LITTLE 技術），將核心分為不同效能等級：

- **超大核/大核**：效能最強，負責重度任務。如遊戲載入、應用啟動、複雜計算。頻率最高（3GHz+），但功耗也最大。
- **中核**：效能與功耗平衡，處理日常中等負載任務。刷微博、看影片、多工切換主要靠它。
- **小核/能效核**：功耗最低，處理輕量級後臺任務。待機、聽音樂、接收訊息等，幾乎不費電。

三個等級的核心協同工作：系統排程器根據任務負載自動分配——輕量任務交給小核省電，重活交給大核猛幹，幹完再切回小核。這就是為什麼手機既能流暢打遊戲，又能待機一整天。

各品牌架構特點：

- **高通驍龍 8 Elite：**採用 Oryon 自研 CPU 架構（源自 Nuvia 收購），2 個超級核（4.32GHz）+ 6 個效能核（3.53GHz），全部基於臺積電第二代 3nm 製程。徹底告別 ARM 公版架構，單核和多核效能均實現跨越式提升。GPU 為 Adreno 830，採用切片架構，圖形效能提升 40%，Hexagon NPU 算力約 45 TOPS，AI 推理效能提升 45%。（注：三星 Galaxy 定製版頻率更高，超級核可達 4.47GHz。）
- **聯發科天璣 9400：**1 個超大核（Cortex-X925，3.62GHz）+ 3 個效能核（Cortex-X4，3.3GHz）+ 4 個能效核（Cortex-A720，2.4GHz），全大核架構延續，臺積電第二代 3nm 製程。GPU 為 Immortalis-G925（12 核），峰值效能提升 41%，光線追蹤效能提升 40%。AI 處理器為第八代 NPU 890，支援端側大模型執行、端側 LoRA 訓練和端側影片生成。
- **蘋果 A18 Pro：**2 個效能核（約 4.04GHz）+ 4 個能效核（約 2.4GHz），臺積電第二代 3nm 製程。6 核 GPU 支援硬體級光線追蹤，16 核 NPU（約 35 TOPS）為端側 AI 和 Apple Intelligence 提供強大算力。蘋果自研架構在單核效能和能效比上依然領先，統一記憶體架構為 AI 推理提供高頻寬優勢。
- **華為麒麟 9020：**搭載於 Mate 70 系列，採用自研泰山 CPU 架構，NPU 和 ISP 能力持續增強，支援 5.5G 和衛星通訊，代表國產晶片的最新突破。

GPU：圖形渲染引擎

GPU（Graphics Processing Unit）專門負責圖形和平行計算任務。它的核心數量遠多於 CPU（動輒數百上千個核心），但每個核心的功能相對簡單，適合同時處理大量相似的計算。

圖形渲染流程：

1. **頂點處理：**計算 3D 模型中每個頂點的位置座標。
2. **光柵化：**將 3D 圖形轉換為 2D 螢幕上的畫素點。
3. **畫素著色：**為每個畫素計算顏色、紋理、光照效果。
4. **輸出合成：**將所有圖層合成最終畫面，送到螢幕顯示。

GPU 對日常使用的影響：不只是遊戲！介面滑動、動畫效果、影片播放、照片濾鏡、AR 應用都依賴 GPU。GPU 效能弱的手機，滑動介面可能掉幀，遊戲更是直接卡頓。

各品牌 GPU：高通使用自研 Adreno GPU；聯發科使用 ARM Mali GPU；蘋果使用自研 GPU，效能和能效都是行業標杆。

NPU：AI 加速引擎

NPU（Neural Processing Unit）是專門為神經網路運算設計的處理器。相比 CPU 和 GPU，NPU 在處理 AI 任務時效率更高、功耗更低。

神經網路運算方式：NPU 的核心運算是矩陣乘法和卷積運算——這正是神經網路中最基礎、最頻繁的計算。NPU 透過硬體級最佳化，可以並行處理大量矩陣運算，速度比 CPU 快幾十倍，功耗卻低得多。

AI 處理流程：

1. **資料輸入：**攝像頭影象、麥克風音訊、感測器資料等。
2. **預處理：**調整資料格式和尺寸，適配神經網路輸入要求。
3. **推理計算：**NPU 載入訓練好的模型，對輸入資料進行前向計算，輸出結果。
4. **後處理：**將 AI 輸出轉換為可用的結果，如人臉座標、文字內容、場景標籤。

實際應用場景：

- **計算攝影：**夜景合成、人像虛化、HDR 處理、AI 美顏。
- **語音助手：**語音識別、自然語言理解、語音合成。
- **實時翻譯：**AI 字幕、對話翻譯。
- **智慧識別：**文字識別（OCR）、物體識別、場景識別。
- **端側大模型：**最新旗艦晶片已支援在手機端執行小型 AI 大模型（7B 引數級別），實現離線 AI 對話、文件摘要、智慧寫作等功能。驍龍 8 Elite 的 Hexagon NPU 算力約 45 TOPS，天璣 9400 的 NPU 890 同樣支援端側大模型（LLM 推理速度提升 80%），蘋果 A18 Pro 的 16 核 NPU 提供約 35 TOPS 算力。
- **AI 影象處理：**AI 消除（一鍵去除照片中的路人）、AI 擴圖（擴充圖片邊界）、AI 風格轉換、文字生成圖片等。

ISP：影像訊號處理器

ISP 處理攝像頭傳來的原始資料，把光線訊號變成好看的照片。

效能怎麼看？看型號！蘋果 A18 Pro、高通驍龍 8 Elite、聯發科天璣 9400、華為麒麟 9020 都是旗艦級別。中端看驍龍 8s Gen3/7+ Gen3、天璣 8400。別隻看核心數，架構和製程（奈米數）更重要。3nm 製程是目前最先進的，能效比遠優於 5nm 和 4nm。

運存（RAM）：臨時工作臺

運存就像你的辦公桌，越大，能同時攤開的檔案就越多。12GB 是現在的主流，16GB 適合重度使用者，24GB 以上則是遊戲玩家和極客的選擇。

運存型別也很重要：LPDDR5 比 LPDDR4X 更快更省電。蘋果雖然運存不大，但系統最佳化好，所以看起來“夠用”。

運存不夠會怎樣？App 會被系統殺掉後臺，切換時需要重新載入。這就是為什麼有時候切個微信回來，遊戲就沒了。

儲存（ROM/快閃記憶體）：長期倉庫

儲存就像你的書櫃，裝所有照片、影片、App 的地方。現在主流是 UFS 3.1 或 UFS 4.0，速度比老式的 eMMC 快好幾倍。

為什麼儲存速度重要？它決定了 App 開啟速度、遊戲載入速度、照片儲存速度。用久了之後，慢速儲存會讓手機越來越卡。

螢幕：你的眼睛

螢幕是你和手機互動的主要視窗，好壞直接影響體驗。

- **材質**：OLED 色彩豔麗、對比度高、更省電（顯示黑色時畫素不發光），但低亮度下可能有頻閃傷眼；LCD 更護眼、成本低，但對比度和色彩不如 OLED。
- **解析度**：1080P 是主流，2K 更清晰但更耗電。蘋果的”視網膜屏”概念就是：在一定距離下，人眼分辨不出畫素點。
- **重新整理率**：60Hz 是標準，90Hz/120Hz 更流暢，滑動時絲般順滑。高重新整理率會耗電，所以很多手機支援自適應重新整理率（LTPO），根據內容動態調整。
- **觸控取樣率**：影響觸控響應速度，遊戲玩家要關注，越高越跟手。
- **調光方式**：DC 調光比 PWM 調光更護眼，低亮度下不容易眼睛累。現在很多手機支援高頻 PWM 調光，也是為了護眼。

螢幕 HDR 技術

HDR（High Dynamic Range）讓螢幕同時展現更亮的亮部和更暗的暗部，畫面層次感大幅提升。

- **實現原理**：普通螢幕的亮度範圍有限（SDR），最亮和最暗之間的差距不夠大。HDR 螢幕的峰值亮度更高（通常 1000 尼特以上），對比度更大，色域更廣，能顯示更多細節。
- **主流標準**：HDR10（最基礎，免費開源）、HDR10+（三星推動，動態後設資料）、Dolby Vision（杜比視界，畫質最佳但需授權）。手機支援哪個標準取決於螢幕硬體和系統授權。
- **實際效果**：看 HDR 影片時，陽光更刺眼、陰影更深邃、色彩更豐富。前提是影片本身是 HDR 製作的，螢幕也支援 HDR 顯示。

影像 HDR 技術

拍照時的 HDR 和螢幕 HDR 是兩回事。影像 HDR 是透過演算法解決“亮處過曝、暗處死黑”的問題。

- **演算法原理**：手機在按下快門時，實際拍攝了多張不同曝光的照片（欠曝保留亮部細節、過曝保留暗部細節、正常曝光保留中間調），然後將它們合成為一張高動態範圍的照片。

- **處理流程**：多幀捕獲對齊補償（防手抖）曝光融合/色調對映色彩校正輸出最終照片。
- **效果最佳化**：現在手機已經能做到” 實時 HDR”——取景器裡看到的畫面就是 HDR 處理後的效果，所見即所得。蘋果的 Smart HDR 和安卓的自動 HDR 都非常成熟。

電池與充電：能量之源

電池是手機最“短命”的部件，用個兩三年就會明顯老化。現在主流容量是 5500-7500mAh。

- **充電功率**：安卓陣營已經卷到了 100W 甚至 200W，十幾分鍾充滿；蘋果還在 20-30W 徘徊。快充傷電池嗎？用原裝配件問題不大，手機有溫控保護。
- **無線充電**：方便但效率低、發熱大。適合辦公室隨手充，不適合急著出門時用。
- **反向充電**：手機變成充電寶，給耳機、手錶充電，應急用。

攝像頭：光與影的魔法

現在手機攝像頭動輒三四個，各有分工：

- **主攝**：最常用的，感測器最大，畫質最好。
- **超廣角**：拍風景、建築，視野更寬。
- **長焦**：拍遠處，有光學變焦和數碼變焦之分。
- **微距**：拍極近距離的小東西。

畫素數不是全部！感測器尺寸（底大一級壓死人）、光圈大小（f/1.8 比 f/2.4 大）、防抖（OIS 光學防抖很重要）都影響畫質。計算攝影（AI 演算法）現在越來越重要，能把多張照片合成一張好照片。

潛望式長焦鏡頭

傳統長焦鏡頭需要較長的光學路徑，會讓手機變得很厚。潛望式長焦透過一個 90 度的稜鏡將光線折射，讓光路在手機內部橫向展開，從而在保持手機輕薄的同時實現 5 倍甚至 10 倍光學變焦。

- **結構原理**：光線從鏡頭進入稜鏡折射 90 度沿手機寬度方向穿過鏡片組到達感測器。相當於把一個豎著的望遠鏡橫著放。
- **光學特性**：光學變焦倍數高（5x-10x），畫質遠超數碼變焦。但感測器尺寸通常比主攝小，暗光表現一般。
- **成像優勢**：拍月亮、拍遠處建築、拍舞臺特寫，是普通手機無法企及的。

OIS 光學防抖 vs EIS 電子防抖

防抖是影響拍照和錄影畫質的關鍵技術，兩種方式原理完全不同：

- **OIS（光學防抖）**：透過物理移動鏡頭或感測器來抵消手抖。鏡頭內部有微型陀螺儀檢測抖動方向和幅度，然後驅動馬達反向移動鏡片組。優點是不損失畫質，缺點是成本高、佔用空間。
- **EIS（電子防抖）**：透過軟體演算法裁切和偏移畫面來抵消抖動。手機錄製稍大的畫面範圍，然後根據陀螺儀資料裁切出穩定的區域。優點是不需要額外硬體，缺點是會裁切畫面（視野變小），暗光下可能模糊。
- **實際選擇：**拍照優先看 OIS，錄影兩者結合效果最好。現在很多旗艦手機支援"Sensor-Shift"感測器位移防抖，比傳統鏡頭防抖效果更好。

光學變焦 vs 數碼變焦

- **光學變焦**：透過物理移動鏡片組改變焦距，畫質不損失。手機上的光學變焦通常只有固定倍數（如 2x、3x、5x、10x），不能像相機那樣連續變焦。
- **數碼變焦**：放大畫面中心區域，本質是裁切 + 插值放大。放大越多，畫質越差。100 倍數碼變焦看著很酷，但實際畫質已經很糊了。
- **混合變焦**：結合光學和數碼，在光學變焦倍數之間用多幀合成演算法彌補畫質損失。效果比純數碼好，但仍不如光學。

音訊：揚聲器與獨顯晶片

單揚聲器 vs 雙揚聲器：

- **單揚聲器**：只有底部一個發聲單元，聲音偏向一側，沒有立體感。中低端手機常見。
- **雙揚聲器（立體聲）**：頂部和底部各一個揚聲器，形成立體聲場。看影片、聽音樂時左右聲道分明，沉浸感強很多。聽筒兼做頂部揚聲器是常見方案。
- **音質差異**：雙揚聲器不只是" 聲音更大"，關鍵是空間感和層次感。用單揚聲器看大片，總覺得少了點什麼。

遊戲手機獨顯晶片：

部分遊戲手機和效能手機配備了獨立的顯示處理晶片，它的作用是：

- **插幀**：將 30fps 的遊戲畫面透過演算法插值到 60fps 或 120fps，畫面更流暢。這個過程在獨顯晶片上完成，不佔用 SoC 的 GPU 資源。
- **超分**：將低解析度畫面透過 AI 演算法提升到高解析度，畫質更清晰。
- **降功耗**：因為插幀和超分由獨顯晶片負責，SoC 的負載降低，發熱和功耗都減少，遊戲更穩定。

GPS 定位：單頻與雙頻

手機定位主要依賴 GPS 等衛星導航系統。

- **單頻 GPS**：只接收 L1 頻段訊號，定位精度約 3-5 米。在開闊地帶夠用，但在高樓密集的城市峽谷中，訊號反射會導致定位偏移較大。
- **雙頻 GPS**：同時接收 L1 和 L5 兩個頻段訊號。L5 頻段波長更長、抗干擾能力更強，雙頻聯合解算可以消除大部分電離層誤差，定位精度可達 1-3 米。在城市中導航、打車定位時優勢明顯。
- **多星座支援**：現代手機不僅支援 GPS，還支援北斗（中國）、GLONASS（俄羅斯）、Galileo（歐洲）等多個衛星系統。同時連線的衛星越多，定位越快越準。

感測器：手機的小觸角

手機裡藏著十幾個感測器，默默工作：

- **加速度計 + 陀螺儀**：感知手機姿態，玩遊戲轉向、自動旋轉螢幕。
- **光線感測器**：自動調節螢幕亮度。
- **距離感測器**：打電話時手機貼近耳朵自動熄屏。
- **指紋感測器**：現在主流是屏下光學或超聲波指紋。
- **面部識別**：蘋果的 Face ID 是 3D 結構光，安全級別高；很多安卓是 2D 照片識別，安全性一般。
- **NFC**：近場通訊，刷公交卡、門禁卡、銀行卡。
- **氣壓計**：測海拔高度，爬樓梯都能檢測到。
- **霍爾感測器**：檢測磁吸殼的開合。

二、作業系統：安卓 VS iOS VS 鴻蒙

硬體是身體，作業系統是靈魂。三大系統各有特色：

- **安卓**：像開源社群，手機廠商可以深度定製，選擇多。你可以換主題、裝第三方應用商店、折騰各種玩法。但也因為開放，碎片化嚴重，不同品牌體驗差異大。
- **iOS**：像精心打理的花園，蘋果軟硬體一手抓，體驗統一流暢。應用質量高，隱私保護好，但自由度低，很多事得按蘋果的規矩來。
- **鴻蒙**：華為的新物種，主打分散式能力，可以讓不同裝置無縫協同。後面我們會專門講。

三者生態都極其豐富，選你喜歡的生活方式就好。沒有絕對的好壞，只有適不適合。

三、手機介面全解析

手機介面是連線外部世界的橋樑，不同介面差異很大：

Micro USB

- **物理特性**：扁平梯形介面，分正反面（插反了插不進去）。已經在手機上基本淘汰，但部分低端裝置和老裝置仍在使用。
- **傳輸限制**：僅支援 USB 2.0 標準，最高傳輸速率 480Mbps，充電功率通常不超過 10W。
- **缺點**：正反面不通用、插拔壽命短、傳輸速度慢、充電功率低。

Lightning（蘋果閃電介面，已淘汰）

- **物理特性**：蘋果自研的 8 針介面，正反都可以插。iPhone 14 及更早機型使用，現已全面淘汰。
- **傳輸限制**：Lightning 介面物理上僅支援 USB 2.0 標準，最高傳輸速率 480Mbps。即使使用 USB-C 轉 Lightning 線，傳輸速度也受限於 Lightning 介面本身。
- **淘汰原因**：歐盟透過法規要求 2024 年起所有手機統一使用 USB-C 介面，蘋果從 iPhone 15 系列開始全面改用 Type-C。Lightning 介面正式退出歷史舞臺。
- **注意**：蘋果 MFi 認證的資料線內含認證晶片，非認證線可能無法正常充電或傳輸資料。如果你仍在使用舊款 iPhone，建議備好 Lightning 資料線。

USB Type-C

- **物理特性**：24 針腳的橢圓形介面，正反都可以插。已成為手機和多數電子裝置的標準介面。歐盟已立法要求 2024 年起所有手機、平板、耳機等便攜裝置統一使用 USB-C 介面，全球其他地區也在跟進。
- **傳輸協議**：Type-C 只是介面形態，實際傳輸能力取決於內部協議。支援 USB 2.0（480Mbps）、USB 3.0/3.1/3.2（5-20Gbps）甚至 USB4（40Gbps）。
- **重要區分**：並非所有 Type-C 介面都支援 USB 3.0！很多手機雖然用 Type-C 介面，但內部只連線了 USB 2.0 的引腳，傳輸速度和老介面一樣慢。iPhone 16/17 標準版僅支援 USB 2.0 速度，Pro 版才支援 USB 3.0（需使用 USB-C to USB-C 線）。
- **充電能力**：支援 USB PD 等快充協議，最高可支援 240W 充電。
- **影片輸出**：支援 DisplayPort Alt Mode，可以透過轉接線輸出影片到顯示器。

5mm 音訊介面

- **物理特性**：圓形 3 段或 4 段插孔，3 段傳輸左右聲道音訊，4 段多一個麥克風通道。
- **特點**：模擬訊號傳輸，即插即用，不需要驅動或轉換。音質穩定，延遲為零。
- **現狀**：大部分旗艦手機已取消 3.5mm 介面，需要 Type-C 轉接線或使用藍芽耳機。部分中端機和遊戲手機仍保留。

四、手機產品體系與命名規則

面對各品牌眼花繚亂的產品線，搞清楚命名規則能幫你快速定位適合自己的機型。

主流品牌產品系列：

- **蘋果**：iPhone（標準版）/ iPhone Plus（大屏版）/ iPhone Pro（專業版）/ iPhone Pro Max（大屏專業版）/ iPhone SE（平價版）/ iPhone 16e（平價 AI 版，搭載 A18 晶片和蘋果自研 C1 基帶）。2026 年 iPhone 17 系列全面搭載 A19 Pro 晶片，AI 能力進一步提升。
- **華為**：Mate 系列（商務旗艦）/ P/Pura 系列（影像旗艦）/ Mate X 系列（摺疊旗艦）/ nova 系列（年輕中端）/ 暢享系列（入門）
- **小米**：小米數字系列（旗艦）/ 小米 MIX（探索旗艦/摺疊）/ Redmi K 系列（效能旗艦）/ Redmi Note 系列（中端）/ Redmi 數字系列（入門）
- **OPPO**：Find X 系列（旗艦）/ Reno 系列（中端）/ A 系列（入門）/ 一加數字系列（效能旗艦）
- **vivo**：X 系列（影像旗艦）/ iQOO 數字系列（效能旗艦）/ S 系列（自拍中端）/ Y 系列（入門）
- **三星**：Galaxy S 系列（旗艦）/ Galaxy Z Fold/Flip（摺疊旗艦）/ Galaxy A 系列（中端）

常見字尾含義：

- **Pro**：Professional 的縮寫，專業版。通常在標準版基礎上升級攝像頭、螢幕、處理器等核心配置。
- **Max**：最大版。螢幕最大、電池最大，但配置不一定比 Pro 高。
- **Ultra**：極致版。通常代表該系列的最高配置，堆料最猛，價格也最高。如三星 S24 Ultra、小米 14 Ultra。
- **SE**：Special Edition 特別版，實際多為平價版。配置縮水，價格降低。如 iPhone SE。
- **Plus**：加大版。螢幕比標準版大，電池也大，但配置基本一致。

- **Lite/青春版**：輕量版。配置精簡，價格更低。
- **Note**：原本指大屏手機，現在各品牌用法不一。Redmi Note 是中端系列。
- **Neo**：新銳版。通常指效能取向的中端機型。

五、基帶與天線：手機通訊的幕後功臣

手機能打電話、上網，靠的是基帶晶片和天線這對搭檔。

基帶晶片：

基帶是手機中負責無線通訊的處理器，它將數字訊號調製成無線電波發出去，也將接收到的無線電波解調成數字訊號。基帶效能直接影響通話質量、上網速度和訊號穩定性。高通是最大的手機基帶供應商，聯發科、三星也有自研基帶。蘋果從 iPhone 16e 開始搭載自研的 C1 基帶晶片，標誌著蘋果正式進入自研基帶時代，C1 基帶採用 4nm 製程，功耗表現優異；iPhone 16/16 Plus/16 Pro/16 Pro Max 仍使用高通基帶（驍龍 X75 5G），預計後續機型將逐步過渡到自研基帶。更早的 iPhone XS/11 系列使用 Intel 基帶時訊號表現不佳，是不少使用者的痛點。

天線設計：

手機天線藏在機身內部，通常圍繞邊框佈置。天線設計直接影響訊號接收效能：

- **MIMO 技術**：多入多出，使用多個天線同時收發資料，提升速度和穩定性。4×4 MIMO 比 2×2 MIMO 速度更快。
- **天線數量**：旗艦機通常有十幾個天線（蜂窩、Wi-Fi、藍芽、NFC、GPS 各需要獨立天線）。
- **金屬機身的影響**：金屬會遮蔽訊號，所以金屬機身手機需要設計天線帶（邊框上的塑膠條）。玻璃和塑膠機身對訊號影響較小。

網路技術演進：

- **4G LTE**：理論峰值速率下行 1Gbps/上行 500Mbps（Cat.18），實際體驗通常 50-150Mbps。覆蓋最廣，夠用。
- **5G NR**：理論峰值速率下行 10Gbps+。Sub-6GHz 頻段覆蓋較好，mmWave 毫米波速度極快但覆蓋極小。
- **5G-A（5G Advanced）**：5G 的增強版，也叫 5.5G。下行速率可達 10Gbps，上行 1Gbps，連線數密度和定位精度大幅提升。2024 年起中國移動、電信、聯通開始在重點城市部署 5G-A 網路，華為 Mate 70/80 系列、小米 15/16 系列等旗艦手機已支援 5G-A。5G-A 還帶來了”通感一體”能力，基站可以同時通訊和感知環境，為低空經濟（無人機配送）和車聯網提供基礎設施。

網路頻段分類：

手機支援的網路頻段決定了它能在哪些國家、哪些運營商的網路上使用：

- **FDD-LTE Band（B 系列）**：如 B1、B3、B5 等，4G 頻段。不同國家使用不同頻段，中國主要用 B1/B3/B5/B8/B34/B38/B39/B40/B41。
- **TDD-LTE Band**：4G 的另一制式，中國大量使用 B38/B39/B40/B41。
- **5G NR Band（N 系列）**：如 n1、n3、n28、n41、n78、n79 等。n78（3.5GHz）是中國 5G 主力頻段，n41（2.5GHz）移動使用較多。
- **購買建議**：如果需要出國使用，購買前確認手機是否支援目標國家的頻段。國行手機通常對中國頻段支援最全，但可能缺少部分海外頻段。

六、你的手機，到底在扮演多少角色？

它早已不只是電話：

1. **通訊工具**：電話、簡訊、微信、郵件……
2. **智慧相機**：拍照、錄影、掃描文件……
3. **錢包**：支付、公交卡、銀行卡、會員卡……
4. **資訊門戶**：新聞、天氣、股票、知識……
5. **遊戲機**：從消消樂到 3A 大作……
6. **移動辦公室**：文件、表格、會議、協作……
7. **健康助手**：計步、心率、睡眠、運動……
8. **智慧家居遙控器**：控制燈、空調、電視、掃地機器人……
9. **數字身份證**：電子身份證、電子駕照、健康碼……
10. **娛樂中心**：音樂、影片、電子書、播客……

想想看，一臺小小的裝置，承載了多少生活！所以，學會用好它，就是學會更好地生活。

實操一下：瞭解硬體配置

在 [設定] 裡，看看能不能找到處理器型號、執行記憶體（RAM）和機身儲存（ROM）的具體大小。對你手機的能力有個數。安卓使用者可以下載“安兔兔”或“DevCheck”檢視詳細硬體資訊。

附：安卓手機記憶體拓展解析

買手機時，你常能看到“*12GB+8GB* 運存”這樣的宣傳——好像憑空多了 *8GB* 執行記憶體。但它真的能讓手機變快嗎？會不會反而傷手機？這一節我們就把“記憶體拓展”這項技術徹底講清楚。

一、本質定義

安卓手機的“記憶體拓展”（又稱記憶體擴充、記憶體融合、RAM Plus 等）**並非真正物理增加執行記憶體晶片**，而是一項系統級軟體技術：透過將手機部分內部儲存空間（ROM/UFS 快閃記憶體）動態劃轉，模擬為虛擬執行記憶體（RAM）使用。

打個比方：物理 RAM 是“高速工作臺”，拓展記憶體則是“臨時儲物間”。當工作臺不夠用時，系統會把暫時不用的資料搬到儲物間，需要時再搬回來。儲物間容量大，但來回搬運肯定比直接放在工作臺上慢。

二、技術原理

主流廠商的實現方式主要有兩種：

zRAM 壓縮

在 RAM 內部劃分一塊壓縮資料空間，將不活躍的應用程式碼進行壓縮儲存，壓縮率約 2:1。資料仍在 RAM 中，速度快、不損耗快閃記憶體，但需 CPU 參與壓縮解壓。

swap 分割區

直接在 UFS 快閃記憶體中劃分固定空間作為虛擬記憶體，將後台程式資料寫入儲存。依賴儲存頻寬，讀寫速度遠低於實體記憶體。

廠商宣傳的“12GB+8GB 運存”通常是 12GB 物理 RAM+8GB 快閃記憶體虛擬記憶體的組合。值得注意的是，zRAM 預設並不佔用機身儲存，只有在 zRAM 被資料完全填滿時，系統才會在儲存上建立交換檔案，且這種情況極少發生。

三、速度差距是關鍵瓶頸

這項技術的根本侷限在於速度差異。目前主流的 UFS 4.0 快閃記憶體順序讀速約 4200MB/s，僅為 LPDDR5X 實體記憶體的 1/10-1/15。因此在輕負荷的 zRAM 壓縮階段速度接近真實 RAM，但一旦進入高負載場景，速度會驟降到幾十分之一。

四、適用場景與收益邊界

記憶體拓展並非萬能，效果因機型和使用場景而異：

- **有用的情況**：6GB 實體記憶體 +UFS 3.1 以上、剩餘空間 ≥30GB 的機型，開啟 3GB 拓展後後臺保活數可多 3-4 個，過載率降低 68%，適合外賣小哥、網約車司機等重度多工使用者。
- **日常場景**：對微信多開、網頁標籤保活、即時通訊訊息響應等改善顯著，後臺應用冷啟動耗時明顯縮短。
- **有限場景**：對大型遊戲載入、4K 影片剪輯等高頻寬場景提升有限。
- **有害的情況**：8GB 記憶體 +UFS 2.2 且剩餘空間不足的機型，開啟後遊戲幀率波動反而擴大，刷短影片多耗電，切換卡頓。
- **無效的情況**：16GB 以上大記憶體機型收益幾乎為零，最多多保活 1 個 APP，卻會讓 CPU 負載增加 12%、發熱上升 2-3℃。對 RAM 大於 12GB 的機型，建議直接關閉以避免無意義的壓縮計算。

五、各廠商命名與開啟方式

不同品牌叫法各異：小米叫“記憶體擴充”、華為稱“記憶體融合”、三星叫“RAM Plus”、vivo 命名為“Extended RAM”、一加稱作“Memory Extension”。該功能屬系統原生能力，與安卓底層版本無直接繫結關係。

一般開啟路徑為：進入【設定】【更多設定】找到【記憶體】或【效能】子項開啟“記憶體拓展”並選擇容量檔位（通常 2GB-7GB），重啟後生效。開啟後系統會顯示如“8GB+4GB”的記憶體組合。

注意事項

- 記憶體拓展會佔用部分機身儲存空間，請確保剩餘儲存不少於 20-30GB，否則可能影響系統流暢度。
- 快閃記憶體作為虛擬記憶體會有讀寫損耗，雖然現代 UFS 快閃記憶體壽命較長，但長期高頻 swap 仍可能加速儲存老化。
- 如果手機本身實體記憶體較小（如 4GB 及以下），開啟記憶體拓展只能緩解燃眉之急，無法從根本上解決卡頓問題。

小結

記憶體拓展是一項成熟的系統級最佳化技術，能在硬體不變的前提下提升中低端機型的多工能力與後臺保活率，但它受限於快閃記憶體與實體記憶體的巨大速度鴻溝，並非“萬能神藥”。是否開啟、開多大，需結合自身機型的實體記憶體大小、快閃記憶體規格、剩餘儲存空間及使用習慣來判斷。

來動手：看看你的手機有沒有記憶體拓展

1. 開啟【設定】，搜尋” 記憶體擴充”、” 記憶體融合”、”RAM Plus” 或”Extended RAM”。
2. 找到後檢視當前是否開啟，以及拓展容量設定是多少。

3. 對照本節內容判斷：你的機型實體記憶體多大？快閃記憶體規格是什麼？剩餘儲存空間是否充足？

4. 嘗試關閉或調低拓展容量，觀察一週後臺保活和發熱變化，找到最適合自己的設定。

第一章附：鴻蒙作業系統專題

鴻蒙（*HarmonyOS*）是華為自主研發的作業系統，從 *2019* 年釋出至今，已經走過了幾個大版本。它不只是“又一個安卓替代品”，而是有著獨特設計理念的新物種。讓我們一起深入瞭解這個正在崛起的作業系統。

一、鴻蒙是什麼？為什麼它與眾不同？

鴻蒙的核心理念是“萬物互聯”。不同於傳統作業系統主要服務於單一裝置，鴻蒙從設計之初就考慮了多裝置協同。

分散式架構是鴻蒙的靈魂。簡單說，它可以把多個裝置虛擬成一個“超級終端”。你的手機、平板、電腦、手錶、電視，不再是孤立的個體，而是一個整體的“器官”，可以互相呼叫對方的能力。

舉個例子：你在用手機看影片，想在大屏上看，傳統方式是投屏；而在鴻蒙上，你可以直接把影片“流轉”到電視上繼續看，手機變成遙控器。這不是簡單的投屏，而是應用本身在跨裝置執行。

二、鴻蒙 4.3：基於 AOSP 的成熟版本

鴻蒙 4.3 是當前華為手機上廣泛部署的版本。需要說明的是，鴻蒙 4.3 相容安卓開源專案（AOSP）的程式碼框架，因此它能夠完整相容安卓應用——你在華為應用市場下載的大部分 App，本質上仍然是安卓應用，透過鴻蒙的相容層執行。但鴻蒙 4.3 並非簡單套殼安卓，它在 AOSP 基礎上加入了大量華為自研的分散式能力、方舟引擎等技術。

核心特性：

- **安卓應用相容**：由於基於 AOSP 開發，鴻蒙 4.3 可以正常執行幾乎所有安卓應用，使用者幾乎感受不到差異。
- **方舟引擎**：華為自研的編譯器和執行時，讓應用執行更流暢、更省電。相比早期版本，流暢度提升明顯。
- **超級終端**：在控制中心，可以直觀地看到周圍的其他鴻蒙裝置，拖拽即可連線。手機呼叫電腦的鍵盤滑鼠、平板呼叫手機的攝像頭，都很方便。
- **服務卡片**：類似 iOS 的小元件，但更靈活。可以上滑 App 圖示展開卡片，顯示關鍵資訊，不用開啟 App。

- **純淨模式**：只允許從官方應用市場安裝應用，減少惡意軟體風險。
- **隱私中心**：集中管理應用許可權，檢視應用行為記錄。

使用者介面特點：

- **雜誌鎖屏**：鎖屏介面顯示精美圖片和文章，劃走即進入桌面。
- **萬能卡片**：桌面圖示可以展開成卡片，顯示更多資訊。
- **超級中轉站**：一個臨時儲存區，可以把文字、圖片、檔案暫存，然後拖到任意 App 中使用。跨應用分享變得無比絲滑。
- **智慧搜尋**：全域性搜尋，可以搜應用、檔案、設定、網頁。

三、鴻蒙 NEXT（鴻蒙 6）：華為自主作業系統的真正起點

鴻蒙 NEXT（HarmonyOS NEXT 5.0）與鴻蒙 4.3 有著根本性的區別——它採用華為自研的鴻蒙微核心，不再依賴 Linux 核心和 AOSP 程式碼，也**不再直接支援安卓應用**。這是鴻蒙從“相容安卓的過渡形態”正式走向“完全獨立的作業系統”的關鍵一步。

重大變革：

- **完全自主核心**：鴻蒙 NEXT 使用華為自研的核心，不再包含任何 AOSP 程式碼。系統更精簡、更安全、更高效。
- **不再支援安卓應用**：這是最大的變化。鴻蒙 NEXT 只能執行原生鴻蒙應用，無法安裝和執行 APK 檔案。升級前請確認你常用的 App 已經有鴻蒙原生版本。
- **原生應用生態**：華為正在全力推動開發者將應用遷移到鴻蒙原生平臺。目前主流 App 如微信、支付寶、淘寶等已推出鴻蒙原生版本，但部分小眾應用可能尚未適配。
- **星盾安全架構**：全新的安全體系，從硬體到軟體的全鏈路保護。支援更細粒度的許可權控制，比如” 模糊定位”——應用只能知道你大概在哪，不知道精確位置。
- **AI 深度融合**：系統級 AI 能力，不只是某個 App 有 AI。比如 AI 字幕可以實時翻譯影片，AI 摳圖可以智慧識別主體。
- **跨裝置無縫流轉**：更進一步的分散式能力。應用狀態可以在裝置間無縫切換，你在手機上編輯的文件，走到電腦前自動在電腦上開啟。
- **元服務**：無需安裝即可使用的輕量級服務。比如掃二維碼直接開啟某個功能，不用先下載 App。

升級鴻蒙 NEXT 前必讀

升級到鴻蒙 NEXT 後，你的手機將無法執行安卓應用。請務必在升級前確認：(1) 你常用的 App 已有鴻蒙原生版本；(2) 你的工作或生活中沒有必須依賴安卓應用的場景。升級後

無法回退到鴻蒙 4.3。

操作邏輯變化：

- **手勢導航**：左右邊緣滑動返回，底部上滑回桌面，上滑停頓進多工。
- **智慧多窗**：支援分屏、懸浮窗、平行視界等多種多工形態。
- **暢連通話**：華為的視訊通話服務，支援螢幕共享、美顏、背景虛化。

四、鴻蒙生態：裝置與應用

鴻蒙不只是手機系統，而是一個覆蓋多種裝置的生態系統。

支援的裝置型別：

- **手機與平板**：核心裝置，功能最完整。
- **智慧手錶與手環**：獨立執行或與手機協同。
- **智慧屏（電視）**：家庭娛樂中心。
- **智慧音箱**：語音互動入口。
- **汽車**：鴻蒙座艙，智慧駕駛艙體驗。
- **智慧家居**：冰箱、空調、洗衣機等。

應用生態現狀：

鴻蒙應用市場已有數百萬應用，主流 App 基本都有。華為也在積極推動開發者開發原生鴻蒙應用。截至 2026 年中，微信、支付寶、淘寶、抖音、美團、京東等頭部應用均已推出鴻蒙原生版本，日常使用基本無障礙。部分小眾應用和行業應用可能仍在適配中，升級前建議確認你常用的 App 是否已支援。

五、鴻蒙實用技巧

- **超級終端**：從螢幕右上角下滑開啟控制中心，點選” 超級終端”，可以看到周圍的其他裝置，拖拽連線。
- **超級中轉站**：長按文字/圖片/檔案，選擇” 複製到中轉站”，然後在其他 App 中長按貼上，或拖拽出來。
- **服務卡片**：上滑 App 圖示展開卡片，長按卡片可以調整大小或新增到桌面。
- **智慧多窗**：從螢幕左右邊緣向內滑動並停頓，開啟側邊欄，點選應用以懸浮窗開啟。
- **隱私保護**：設定 > 隱私，可以檢視應用許可權使用記錄，設定模糊定位等。

注意事項

鴻蒙系統在不斷更新迭代，不同版本可能有些差異。本文描述基於鴻蒙 4.3 和 6 的主要特性，具體功能請以你的手機實際版本為準。建議保持系統更新，以獲得最新功能和安全補丁。

現在就試：體驗鴻蒙特色功能

1. 開啟控制中心，找到” 超級終端”，看看周圍有沒有其他鴻蒙裝置。
2. 試試” 超級中轉站”：複製一段文字到中轉站，然後在備忘錄中拖出來。
3. 上滑幾個常用 App 圖示，看看它們的服務卡片顯示什麼內容。
4. 去設定 > 隱私，檢視最近哪些應用使用了敏感許可權。

第二章：整理你的數字房間——檔案管理

手機用久了會亂，好習慣從管理檔案開始。就像你的房間需要定期整理一樣，你的數字空間也需要打理。一個井井有條的手機，不僅找東西方便，執行也更流暢。

一、三層儲存體系：你的數字倉庫

1. **機身儲存**：核心區，速度快，系統和 App 都在這。這是你手機最寶貴的空間，要精打細算。
2. **SD 卡（如果支援）**：擴充衣櫃，速度慢點，適合放電影、照片等大檔案。注意：現在很多旗艦機不支援 SD 卡了。
3. **雲端儲存**：保險櫃 + 同步網盤。防丟失，也能在多個裝置間同步。iCloud、百度網盤、OneDrive 都是常用選擇。

二、認識“檔案管理器”

安卓一般有個 [檔案] 或 [檔案管理] App，iOS 是自帶的 [檔案] App。它們是你檢視手機所有檔案的“資源管理器”。

常見資料夾含義：

- **DCIM**：相機拍攝的照片和影片。
- **Pictures**：其他圖片，如截圖、儲存的圖片。
- **Download**：瀏覽器下載的檔案。
- **Music**：音樂檔案。
- **Movies**：影片檔案。
- **Documents**：文件檔案。
- **Android**：安卓系統資料夾，裡面是各 App 的資料，一般不要手動刪除。

三、檔案系統結構：深入瞭解

安卓基於 Linux，檔案系統有一定的層級結構。雖然普通使用者不需要深入瞭解，但知道一些基本概念有幫助。

儲存分割區：

- **/system**：系統分割區，存放作業系統檔案。普通使用者無權訪問，需要 root 許可權。
- **/data**：資料分割區，存放使用者資料和應用資料。這是你日常使用的空間。
- **/sdcard 或 /storage/emulated/0**：使用者可訪問的儲存空間，你的照片、下載檔案都在這裡。

App 資料儲存：

- **內部儲存**：App 的私有空間，其他 App 無法訪問。解除安裝 App 時會被刪除。
- **外部儲存**：App 在使用者可訪問區域儲存的檔案，如照片、下載內容。解除安裝 App 時不一定會被刪除。

四、資料備份與恢復：給你的資料上保險

資料無價，備份是必須養成的習慣。

備份策略：

1. **雲備份**：自動、方便，但需要網路和雲空間。iCloud、華為雲、小米雲等都支援。
2. **本地備份**：用資料線連線電腦，用官方工具備份。速度快、空間大，但需要手動操作。
3. **雙重備份**：雲備份 + 本地備份，最安全。

備份什麼？

- **照片和影片**：最佔空間，也最珍貴。
- **通訊錄**：存在雲賬號裡最方便。
- **微信聊天記錄**：用微信自帶的備份功能。
- **重要文件**：存在雲盤或電腦上。

五、檔案安全：保護你的隱私

- **敏感檔案加密**：很多手機有” 安全資料夾” 或” 私密空間” 功能，可以把敏感檔案放進去，需要密碼或指紋才能訪問。

- **刪除不等於銷燬**：普通刪除只是標記空間可覆蓋，資料還在。要徹底刪除，需要用專門的安全擦除工具，或者多次覆蓋寫入。

- **雲端安全**：選擇信譽好的雲服務商，開啟雙重認證，定期檢查登入裝置。

六、空間不夠？來場大掃除

- **清理快取**：App 的臨時記憶，清了能騰地方。在設定 > 儲存裡可以清理。

- **查詢大檔案**：用檔案管理器按大小排序，揪出佔空間的” 元兇”。

- **刪除殘留**：解除安裝 App 後，有時會留下垃圾檔案，手動清掉。

- **清理相簿**：刪除重複照片、模糊照片、不需要的截圖。

- **清理微信**：微信是空間大戶，定期清理聊天記錄和快取很有必要。

七、常見檔案格式全解析

手機裡各種檔案字尾名讓人頭大，搞懂它們能幫你更好地管理檔案：

格式	型別	特性	推薦開啟方式
JPG/JPEG	圖片	有失真壓縮，體積小，相容性最好。手機拍照預設格式。	相簿/任意圖片檢視器
PNG	圖片	無失真壓縮，支援透明背景。截圖、圖示常用。體積比 JPG 大。	相簿/任意圖片檢視器
MP3	音訊	有失真壓縮，體積小，相容性極好。最通用的音訊格式。	任意音樂播放器
FLAC	音訊	無失真壓縮，音質與 CD 一致，體積比 MP3 大 3-5 倍。發燒友首選。	音樂播放器/專業播放 App
WAV	音訊	無壓縮原始音訊，音質最好但體積巨大。專業錄音常用。	音樂播放器
M4A	音訊	AAC 編碼，比 MP3 同位元速率音質更好。蘋果生態常用。	Apple Music/音樂播放器
MP4	影片	最通用的影片封裝格式，相容性極好。手機錄製的影片預設格式。	相簿/任意影片播放器
MKV	影片	開源封裝格式，支援多音軌、多字幕。電影資源常用。	VLC/nPlayer
DOC/DOCX	文件	Word 文件。DOCX 是壓縮格式，體積更小。DOC 是舊版格式。	WPS/Office
PPT/PPTX	演示	PowerPoint 簡報。PPTX 是壓縮格式。	WPS/Office
XLS/XLSX	表格	Excel 電子表格。XLSX 是壓縮格式。	WPS/Office
PDF	文件	跨平臺格式，排版固定不可編輯。合同、論文、電子書常用。	Adobe Reader/WPS
ZIP	壓縮	最通用的壓縮格式，跨平臺相容好。	系統自帶/解壓 App
RAR	壓縮	壓縮率比 ZIP 高，但需要專用軟體解壓。	解壓 App
ISO	映象	光碟映象檔案，完整複製光碟內容。系統安裝包常用。	解壓 App/虛擬光碟機

練一練：整理”下載”資料夾

開啟檔案管理器 > 進入“下載”資料夾。看看裡面都有啥？把沒用的刪掉，有用的分門別類（比如新建“工作資料”、“表情包”資料夾放進去）。順便看看儲存空間，瞭解哪些型別檔案佔用最多。

八、圖片與檔案系統管理：讓資產井井有條

上一節我們學會了“認識資料夾”，但光認識還不夠。本節要講的是**如何主動建立一套可長期執行的管理規範**，讓你的檔案隨時找得到、不丟失、不被偷窺。

檔案分類與命名規範

混亂的根源是沒有規則。建立規則其實很簡單，關鍵在於**堅持**。

- **”時間 + 事件”命名法**：資料夾名字裡帶時間戳和事件描述，方便日後回憶與檢索。例如 202606_ 旅行票據、2025 年終總結、2024 孩子生日。這樣既能看到時間線，也能一眼知道內容。
- **三級分類體系**：
  - **一級**：按檔案型別（文件 / 圖片 / 影片 / 音訊）
  - **二級**：按使用場景（工作 / 生活 / 學習）
  - **三級**：按具體事件或專案（如”2026Q2_ 專案 A”）
- **月度清理計劃**：每月固定一個日期（建議發工資那天或月初 1 號），對”下載”目錄進行集中整理：**歸檔有用的、刪除無用的、轉存珍貴的**。一年 12 次小整理，比攢一年大掃除輕鬆得多。

本月任務清單

挑一個空閒的週末，按以下順序操作：

1. 在檔案管理器中建立 2026Q2_ 專案 A 形式的本月工作資料夾；
2. 把所有”下載”目錄裡本週的檔案過一遍，留下有用的、歸檔、刪除沒用的；
3. 給今年的家庭照片建立 2025 家庭、2026 家庭這種年度資料夾。

冗餘檔案處理策略

手機用久了會“長胖”，多半是冗餘檔案在作怪。

- **72 小時自動清理**：微信、QQ 等社交 App 接收的臨時檔案（表情包、聊天圖片、小影片），大多看一次就沒用了。在 [微信] > [設定] > [通用] > [儲存空間] 裡開啟”自動清理”，把過期快取設成 72 小時。手機瞬間清爽不少，還不丟重要記錄。

- **儲存空間定向清理**：安卓 [設定] > [儲存] > [清理加速]，iOS [設定] > [通用] > [iPhone 儲存空間]，系統會按” 快取 / 臨時檔案 / 可解除安裝 App / 大檔案” 分類列出。重點清理：(1) 快取檔案；(2) 已過期安裝包（升級 App 後留下的舊版安裝包）；(3) 重複檔案（同名同尺寸的照片）。
- **自動分類規則**：多數品牌相簿 App 支援智慧分類：人物、風景、文件、截圖。開啟後，新拍的合影會自動歸入” 人物 > 張三”，掃描的合同會自動歸入” 文件”。**配合每月整理，可極大降低” 找不到東西” 的機率**。

圖片安全處理規範

圖片是手機裡最“敏感”的資料型別——一張隨手拍就能暴露你的位置、裝置、時間。我們建議**建立“雙重保護”工作流**：

傳送證件照/敏感截圖的標準流程

1. **可見資訊打碼**：用系統自帶或第三方工具（如美圖秀秀、Snapseed）**塗抹身份證號中間幾位、車牌號、門牌、姓名、金額**等關鍵欄位；
2. **後設資料剝離**：傳送前在相簿中點” 分享” 或” 匯出”，系統會詢問是否保留位置資訊，選擇**” 不保留”**。這樣拍攝裝置型號、GPS 座標、拍攝時間就不會跟著圖片一起發出去；
3. **臨時水印**（可選）：在敏感截圖上加” 僅供 XX 辦理使用” 的文字水印，避免被濫用。

後設資料（EXIF）清理的常用方法（基礎操作，深度防護見第六章“照片脫敏處理”）：

- **系統自帶**：iOS [設定] > [隱私與安全性] > [相機]，可關閉” 精確位置”；分享時選” 僅當前照片” 會**自動剝離大部分 EXIF**。
- **安卓**：多數品牌相簿 App 在” 分享” 時會提示” 是否移除位置資訊”，勾選即可。
- **專業工具**：ExifTool（電腦端）、Metapho（iOS）、Photo Metadata Remover（安卓）可以徹底清除所有後設資料。

對於需要**長期儲存**的敏感圖片（身份證、銀行卡、戶口本掃描件、護照），建議存入**加密相簿**：

- iPhone：[照片] > 選中照片 > 隱藏（僅 Face ID 驗證後可檢視）；
- 華為/小米：[相簿] > 長按照片 > ” 移入保險櫃”；
- 第三方加密 App：**360 安全相簿**、**騰訊相簿管家（隱私相簿）**、**KeepSafe** 等，單獨密碼保護，隱蔽性更好。

第三章：讓手機接上“生命線”——網路

沒網，智慧手機就“智障”了一半。網路是現代手機的生命線，理解它、用好它、保護它，是每個手機使用者的必修課。

〇、聯網不迷茫——分清 Wi-Fi 與流量的“路”與“車”

很多人的手機螢幕上方同時亮著 Wi-Fi 和訊號圖示，但對二者的本質區別卻一頭霧水。最經典的誤區就是：**連著 Wi-Fi 卻刷不出頁面**——因為路由器斷網了，而手機並沒有自動切迴流量。要徹底弄懂這個問題，我們需要建立一個“路與車”的認知模型。

核心比喻：路與車

把網際網路想象成一座城市。**“路”**是你手機到網際網路之間的通道，**“車”**是跑在路上的資料。Wi-Fi 和流量是兩條不同的路，但資料（車）本身是一樣的。

本質區別：本地通道 vs 遠端高速

Wi-Fi 本質上是一種**區域網技術**，它就像你家樓下無線鋪設的路，負責把資料從路由器傳到手機上。但如果寬頻欠費或光纜斷了，路雖然還在，外面的“高速公路”卻斷了，自然哪兒也去不了。而網路流量（移動資料）則是透過運營商的 4G/5G 基站傳輸，相當於你直接開上了運營商的高速專線，只要有訊號就能直連網際網路。

費用與場景：固定月租 vs 按量計費

Wi-Fi 通常是一次性投入（寬頻月費），只要電費不斷，全家人無限使用，非常適合下載電影、打遊戲等大流量場景。而流量則按使用量計費，超出套餐可能會降速或額外扣費，更適合在地鐵、出差等移動場景下處理輕量級任務。

來試試：檢查你的流量套餐

開啟手機的 [設定] > [行動網路] > [流量管理]，看看你本月還剩多少流量、哪些 App 最耗流量。心裡有數，出門才不慌。

安全考量：公共 Wi-Fi 是隱秘的“釣魚場”

在安全性上，移動資料通常比公共 Wi-Fi 更安全。移動資料由運營商加密，而公共 Wi-Fi 確實容易被駭客搭建為“偽熱點”截獲未加密的流量。但這裡要區分清楚：在 HTTPS 全面普及的今天，攻擊者能截獲的是**你訪問了哪些網站**（域名），以及**未加密的 HTTP 內容**；而微信訊息、銀行 App 的登入密碼等經過 HTTPS 加密的資料，攻擊者**無法直接讀取**——除非實施更復雜的 SSL 剝離攻擊且使用者忽略了證書警告。不過即便如此，在外進行支付操作時，直接用手機流量仍然是最穩妥的選擇。

安全提醒

在機場、商場、咖啡廳等公共場所，不要連線名字可疑的免費 Wi-Fi（如“Free_WiFi_2024”）。即使連了，也絕不要在上面登入銀行 App 或輸入密碼。

手機端判斷：看懂狀態列的“暗語”

如何知道手機此刻在用什麼聯網？以華為手機為例，狀態列中運營商訊號旁或 WLAN 訊號旁出現**雙向箭頭圖示**，即代表正在使用對應方式上網；若兩者同時亮起，則說明網路加速生效，手機正同時利用兩種通道。

iPhone 使用者則更直觀：Wi-Fi 圖示亮著就是在用 Wi-Fi，不亮時看右上角的訊號格就是在用流量。部分安卓手機還會在狀態列顯示“5G”或“4G”字樣，直接告訴你當前行動網路型別。

一、行動網路（4G/5G）與流量

行動網路是你隨時隨地聯網的基礎。從 2G 到 5G，速度提升了上千倍，但原理其實差不多。

網路制式：

- **4G LTE**：目前覆蓋最廣，速度夠用（理論上最高 100Mbps+），耗電相對低。
- **5G**：速度快（理論上最高 10Gbps），但覆蓋還在完善，耗電高。適合下載大檔案、看高畫質影片。
- **5G SA vs NSA**：SA 是獨立組網，NSA 是依託 4G 基站。SA 體驗更好，但需要運營商和手機都支援。

流量管理：

- **流量是什麼**：就是手機上網產生的資料量。看影片、刷圖最費流量，文字和語音相對省。
- **省流小貼士**：在設定裡開啟” 省流量模式”，Wi-Fi 下再更新 App 或看高畫質影片。
- **訊號格 ≠ 網速**：訊號強只代表連線好，網速還看基站忙不忙、你離基站多遠。

重要警告：出國前關閉【資料漫遊】!

在國外使用移動資料可能產生天價賬單！出國前務必在設定裡關閉資料漫遊，或者提前購買當地流量卡/國際漫遊套餐。

二、Wi-Fi：連線、分享與安全

Wi-Fi 是家庭和辦公室的聯網主力，速度快、不費流量。但 Wi-Fi 其實也在不斷進化，不同標準之間差異很大。

連線 Wi-Fi：

去 [設定] > [WLAN] 選擇網路輸密碼。現在很多手機支援掃碼連線，更方便。

分享密碼：

安卓和 iOS 都能讓已連線的手機快速分享密碼給朋友。安卓生成二維碼，iOS 可以靠近自動分享。

Wi-Fi 標準演進：

Wi-Fi 標準由 IEEE 制定，每一代都有顯著提升：

標準	協議	頻段	最高速率	特點
Wi-Fi 4	802.11n	2.4/5GHz	600Mbps	老舊標準，覆蓋還行
Wi-Fi 5	802.11ac	5GHz	3.5Gbps	目前最普及，速度夠用
Wi-Fi 6	802.11ax	2.4/5GHz	9.6Gbps	多裝置效能強，延遲低
Wi-Fi 6E	802.11ax	2.4/5/6GHz	9.6Gbps	增加 6GHz 頻段，不擁堵
Wi-Fi 7	802.11be	2.4/5/6GHz	46Gbps	極速傳輸，多鏈路聚合

- **Wi-Fi 6 的關鍵技術：**OFDMA（正交分頻多重進接）讓多個裝置同時傳輸，MU-MIMO（多使用者多入多出）提升多裝置併發效率，TWT（目標喚醒時間）讓裝置更省電。
- **Wi-Fi 6E：**在 Wi-Fi 6 基礎上增加了 6GHz 頻段，提供更多通道，減少干擾。需要手機和路由器都支援。
- **Wi-Fi 7：**引入 MLO（多鏈路操作），可同時使用多個頻段傳輸資料，大幅降低延遲。320MHz 超寬通道讓速率飛躍。驍龍 8 Elite Gen5、天璣 9400 等旗艦晶片平臺均已支援 Wi-Fi 7，小米 17、華為 Mate 80、三星 S26 等機型可使用 Wi-Fi 7 網路（需配合 Wi-Fi 7 路由器）。

- **2.4GHz vs 5GHz**：2.4GHz 穿牆好但速度慢、干擾多；5GHz 速度快但穿牆弱。Wi-Fi 6E/7 的 6GHz 頻段速度最快，但穿牆最弱，適合同一房間內使用。

公共 Wi-Fi 風險：

公共 Wi-Fi 可能有駭客竊聽資料或設釣魚熱點。守則：

- 避免登入銀行賬戶、輸入密碼。
- 最好用 VPN 加密連線。
- 不要自動連線陌生 Wi-Fi。
- 用 HTTPS 網站（位址列有鎖圖示）。

三、藍芽：短距離無線通訊的萬能選手

藍芽可能是你手機上使用頻率最高的無線技術之一——無線耳機、智慧手錶、車載系統、鍵盤滑鼠，都靠它連線。

技術原理：

藍芽工作在 2.4GHz ISM 頻段，使用跳頻擴頻技術（FHSS）將資料分散在 79 個頻道上快速切換傳輸，有效避免干擾。藍芽的通訊距離分為三個等級：Class 1 約 100 米、Class 2 約 10 米（手機常用）、Class 3 約 1 米。

版本演進：

- **藍芽 4.0**：引入低功耗藍芽（BLE），讓手環、心率帶這類小電池裝置也能無線連線，續航從天變成月。
- **藍芽 4.2**：提升傳輸速度和隱私保護，支援 IPv6 直連。
- **藍芽 5.0**：傳輸速度翻倍（2Mbps），傳輸距離 4 倍（理論 300 米），廣播能力 8 倍。支援雙音訊（同時連兩副耳機）。
- **藍芽 5.1**：引入方向定址功能（AoA/AoD），定位精度可達釐米級。
- **藍芽 5.2**：LE Audio 新標準，支援 LC3 編碼（音質更好、功耗更低）、多流音訊、廣播音訊。
- **藍芽 5.3/5.4**：進一步最佳化功耗和連線穩定性，增強週期性廣播。

LE Audio 普及情況：2025-2026 年，LE Audio 已進入快速普及階段。主流品牌的新款真無線耳機（如華為 FreeBuds Pro 5、小米 Buds 6 Pro、三星 Galaxy Buds3 Pro 等）均已支援 LE Audio 和 LC3 編碼，提供更低延遲和更好音質。購買新耳機時，建議優先選擇支援 LE Audio 的產品。

應用場景：

- **音訊傳輸**：無線耳機、車載音響。藍芽 5.2 的 LE Audio 讓音質和延遲都有質的飛躍。
- **資料傳輸**：近距離傳檔案、照片。雖然速度不如 Wi-Fi Direct，但勝在簡單。
- **裝置配對**：智慧手錶、手環、鍵盤滑鼠、遊戲手柄。
- **位置服務**：藍芽信標（Beacon）用於室內導航、物品追蹤（如蘋果 AirTag）。
- **智慧家居**：藍芽 Mesh 網路控制燈泡、門鎖等。

四、紅外遙控：手機變身萬能遙控器

紅外遙控是最古老的無線控制技術之一，但在手機上依然實用。部分安卓手機（如小米、華為部分機型）內建了紅外發射器，可以遙控電視、空調、機頂盒等家電。

工作原理：

紅外遙控透過紅外發光二極體發出特定頻率（通常 38kHz）的紅外光脈衝訊號，家電的紅外接收器解碼這些脈衝執行對應操作。不同品牌使用不同的編碼協議，常見的有 NEC、RC5、RC6、SONY 等。手機裡的紅外 App 內建了各品牌的編碼庫，選擇品牌和型號就能匹配。

侷限性：

- **方向性**：紅外是直線傳播，需要對著裝置發射，不能隔牆。
- **距離短**：有效距離一般 5-8 米。
- **蘋果不支援**：iPhone 沒有紅外發射器，無法使用此功能。

五、NFC：碰一碰的魔法

NFC（Near Field Communication，近場通訊）是一種極短距離（10 釐米以內）的高頻無線通訊技術，工作頻率 13.56MHz，傳輸速率最高 424Kbps。

通訊原理與模式：

NFC 基於 RFID 技術演變而來，支援三種工作模式：

- **讀寫模式**：手機讀取 NFC 標籤中的資訊，如讀取 NFC 貼紙觸發動作。
- **點對點模式**：兩臺 NFC 裝置互相傳輸資料，如 Android Beam（已停用）。
- **卡模擬模式**：手機模擬一張智慧卡，這是最常用的模式——公交卡、門禁卡、銀行卡都是這個模式。

安全機制：

- **超短距離**：10 釐米內的通訊距離本身就是物理安全保障，遠端竊聽幾乎不可能。
- **安全晶片**：手機內建獨立的安全晶片（SE）或使用 HCE（主機卡模擬），支付金鑰儲存在安全區域，即使手機被 root 也無法提取。
- **指紋/面容驗證**：支付時需要生物識別確認，即使手機丟失也無法盜刷。
- **令牌化**：Apple Pay、華為 Pay 等不傳輸真實卡號，而是使用一次性令牌，即使被截獲也無法重複使用。

典型應用：

- **公交卡/門禁卡**：把實體卡資訊寫入手機，刷手機就行。
- **移動支付**：Apple Pay、華為 Pay 等，把銀行卡繫結到手機，碰一碰支付。
- **裝置配對**：NFC 碰一碰快速配對藍芽耳機、音箱。
- **身份驗證**：NFC 身份證讀取、電子駕照驗證。

六、個人熱點：把手機變 Wi-Fi 路由器

自己流量多時，可以開熱點給平板、電腦用。

- **開啟方法**：設定 > 個人熱點/網路共享，開啟它。
- **注意事項**：很耗電和流量，記得設個複雜密碼，並限制連線裝置數。
- **USB 共享**：用資料線連電腦，比無線熱點更穩定、更快。

動手試試：設定並管理熱點

1. 開啟 [設定]，找到” 個人熱點” 或” 網路共享”，開啟它，設好名稱和密碼。
2. 用另一臺裝置（比如電腦）搜尋並連線這個 Wi-Fi。
3. 在手機端看看，能不能管理已連線的裝置，或踢掉不認識的裝置。

第四章：安裝和管理 App——手機的“技能包”

App 讓手機無所不能，但也要管好它們。就像你會給新員工安排工位、分配許可權一樣，安裝 *App* 也需要一套規範流程。

一、官方商店：最安全的“應用超市”

- **安卓**：Google Play（國外）或各品牌自帶商店（如華為應用市場、小米應用商店）。
- **iOS**：只有 App Store。
- **鴻蒙**：華為應用市場。

核心安全意義：經過稽核、更新有保障、支付安全。行動指南：**優先只用官方商店下載！**

二、純淨/防護模式：開啟手機的“防毒牆”

主流國產品牌（華為、小米、OPPO、vivo、榮耀）都內建了**純淨模式**或**防護模式**，這是系統級別的安全開關。**強烈建議所有使用者都開啟**。

它到底是什麼？

純淨模式（也叫“防護模式”）的**核心功能是阻斷第三方 App 的安裝**：當它開啟時，系統會**只允許從官方應用商店安裝 App**，從瀏覽器、二維碼、聊天軟體等渠道下載的安裝包會被直接攔截，從源頭杜絕惡意軟體。

適用人群

- **老年使用者**：分不清” 官方” 和” 假冒” 廣告，純淨模式是最穩妥的護身符。
- **對應用許可權不敏感的使用者**：不需要安裝小眾 App、測試版、外服遊戲。
- **希望提升裝置安全性的所有使用者**：哪怕是發燒友，開著也比不開強——真有臨時需要，再單獨關一下即可。

開啟路徑（以華為/小米為例）

- **華為/鴻蒙**：[設定] > [系統和更新] >**[純淨模式]**，選擇” 開啟”。
- **小米**：[設定] > [隱私保護] >**[保護隱私]** 或 [設定] > [應用設定] > [應用安裝]，關閉” 安裝未知應用”。
- **OPPO/vivo**：[設定] > [安全] >**[安裝外部來源應用]**，逐個關閉授權。

應用商店的安全檢測機制

應用商店不是“擺設”，它背後有完整的安全流水線：

- **病毒掃描**：每個 App 上架前會經過多家防毒引擎聯合掃描，識別已知木馬和病毒特徵。
- **許可權稽核**：商店會審查 App 申請的許可權是否” 合理”——一個手電筒 App 要通訊錄許可權？直接駁回或下架。
- **開發者認證**：上架需要企業資質、個人身份證或信用卡認證。出問題能找到人，不是” 黑App”。
- **持續監控**：已上架的 App 也會被持續復掃，新發現的惡意行為會被強制下架。

三、第三方來源與“電子寄生蟲”：風險極高！

從網頁或不明連結下載 App，可能夾帶病毒、竊取隱私、盜刷支付。這裡我們特別提一個新概念：**“電子寄生蟲”**。

什麼是“電子寄生蟲”？

指的是那些**被植入木馬的“破解版”軟體**——你以為是“省了錢”，實際是“中了招”。常見形式：

- ” 破解版”、” 免費版”、” 去廣告版” 付費軟體；
- ” 刷贊”、” 搶紅包”、” 遊戲外掛” 等灰色工具；
- 群聊裡” 小廣告” 推薦的所謂” 內部版”App。

這些程式表面能用，背後可能在**偷通訊錄、讀簡訊、監聽麥克風、後臺挖礦**。純淨模式是防範它們**第一道防線**——直接不讓你裝。

第二道防線建議：

- **定期全盤病毒掃描**：使用手機自帶安全中心（如華為手機管家、小米安全中心）每月掃一次。
- **謹慎授予敏感許可權**：通訊錄、簡訊、位置、相機、麥克風——這五項” 高危許可權” 必須嚴格審批。

- **關注耗電與流量異常**：手機突然發燙、電量掉得快、流量半夜跑——八成是後臺藏了”寄生蟲”。

如果萬不得已要裝第三方 App：

1. 確保來源絕對可信。
2. 安裝前仔細看許可權要求。
3. 事後用安全 App 掃描。

終極建議：在系統設定裡，關掉“安裝未知應用”的許可權。

四、裝、更新、解除安裝

- **安裝**：商店裡點” 獲取” 或” 安裝”，裝完首次開啟會問你要許可權。
- **更新：必須更！**主要是修復安全漏洞。建議開啟自動更新。
- **解除安裝**：長按圖示拖到解除安裝區。想清得更徹底，去 [設定] > [應用管理] 裡找到並解除安裝。

五、許可權管理：你是 App 的“老闆”

App 要許可權（比如通訊錄、位置）有時合理（地圖要位置），有時不合理（手電筒要通訊錄）。

- **許可權選項**：” 始終允許”、” 僅在使用時允許”、” 禁止”、” 每次詢問”。
- **管理原則**：
  1. 最小化：非必要不給。
  2. 事後審計：定期去許可權設定裡看看誰在” 越權”。
  3. 情境化：像打車軟體，只在用時給位置許可權。

重點盯防的許可權

- 通訊錄/簡訊：可能被用來發垃圾簡訊。
- 實時位置：可能被追蹤行蹤。
- 相機/麥克風：可能被偷拍偷錄。
- 檔案訪問：可能讀取你的隱私檔案。

六、軟體安裝包的構成要素

你有沒有想過，從應用商店下載的那個安裝包，裡面到底裝了什麼？瞭解安裝包的構成，能幫你更好地理解 App 是怎麼工作的，也能在遇到問題時知道該從哪裡排查。

安卓 APK 檔案結構

APK（Android Package Kit）本質是一個 ZIP 壓縮包，解壓後可以看到以下核心目錄和檔案：

- **AndroidManifest.xml**：應用的" 身份證" 和" 說明書"。記錄了應用包名、版本號、所需許可權、支援的系統版本、入口 Activity 等核心資訊。系統安裝應用時首先讀取這個檔案。
- **classes.dex**：應用的核心程式碼。Java/Kotlin 原始碼編譯後變成 Dalvik 位元組碼，存在這個檔案裡。一個應用可能有多個 dex 檔案（如 classes2.dex、classes3.dex）。
- **resources.arsc**：編譯後的資源索引表。記錄了所有資源的 ID 和對應關係。
- **res/目錄**：資原始檔夾。包含圖片、佈局檔案、字串、顏色定義等。不同解析度的圖片放在不同的 drawable 資料夾裡（如 drawable-hdpi、drawable-xhdpi）。
- **assets/目錄**：原始資原始檔夾。存放不經過編譯的資源，如網頁檔案、字型、音訊等。App 執行時可以透過 AssetManager 讀取。
- **lib/目錄**：本地庫資料夾。存放編譯好的.so 檔案（C/C++ 程式碼編譯產物），按 CPU 架構分類（如 arm64-v8a、armeabi-v7a）。遊戲和需要高效能運算的 App 通常有這個目錄。
- **META-INF/目錄**：簽名資訊資料夾。包含 MANIFEST.MF、CERT.SF、CERT.RSA 三個檔案，用於驗證 APK 的完整性和簽名。

iOS IPA 檔案結構

IPA（iOS App Store Package）同樣是壓縮包，結構略有不同：

- **Payload/目錄**：核心內容所在。裡面有一個.app 資料夾，包含應用的所有內容。
- **Info.plist**：應用的配置檔案，類似安卓的 AndroidManifest.xml。記錄了應用 ID、版本、許可權需求、支援的裝置等。
- **可執行檔案**：與.app 資料夾同名的主程式，是編譯後的 Mach-O 格式二進位制檔案。
- **Frameworks/目錄**：應用依賴的動態庫和框架。
- **資原始檔**：圖片、音訊、影片等資源，通常存放在.app 資料夾內。
- **_CodeSignature/目錄**：程式碼簽名資訊，用於驗證應用的完整性和來源。

簽名驗證機制

為什麼從官方商店下載的 App 更安全？簽名驗證是關鍵。

- **安卓簽名**：開發者用私鑰對 APK 進行簽名，系統安裝時用公鑰驗證。如果 APK 被篡改（比如植入病毒），簽名就會失效，系統會拒絕安裝。Google Play 還要求 App Signing，由 Google 保管簽名金鑰，進一步保護應用安全。
- **iOS 簽名**：蘋果的稽核更嚴格。每個 IPA 必須有蘋果頒發的證書簽名，否則無法安裝。這就是為什麼 iOS 只能從 App Store 或企業證書安裝應用。企業證書簽名的應用有效期有限，且蘋果可以隨時吊銷。
- **簽名的作用**：
  1. 驗證來源：確認應用確實來自聲稱的開發者。
  2. 驗證完整性：確保應用在傳輸過程中未被篡改。
  3. 防止冒充：惡意應用無法偽裝成其他應用。

安裝包大小與最佳化

你可能會發現，應用商店顯示的 App 大小和安裝後佔用的空間不一樣。這是因為：

- **下載大小**：壓縮後的 APK/IPA 大小。
- **安裝大小**：解壓後的大小，通常比下載大小大很多。
- **執行時佔用**：App 執行時還會產生快取、資料檔案，佔用更多空間。

現代 App 的最佳化手段：

- **App Bundle（安卓）**：Google Play 的動態分發技術。使用者下載時只獲取適合自己裝置的資源（如只下載對應螢幕解析度的圖片），大大減少下載大小。
- **App Thinning（iOS）**：類似的技術，蘋果會根據裝置自動裁剪不需要的資源。
- **按需下載**：遊戲等大型應用可以先下載核心包，後續再下載資源包。

小知識：為什麼有的 App 安裝包特別大？

遊戲類 App 通常最大，因為包含大量高畫質貼圖、3D 模型、音訊檔案。社交類 App 也比較大，因為要支援各種功能模組。工具類 App 通常最小。如果你發現某個工具類 App 特別大，要警惕是否捆綁了不必要的功能。

實操環節：審查 3 個 App 的許可權

選 3 個常用 App（如社交、購物、工具類），進入 [設定] 的應用許可權管理頁，逐項審查，把不必要的敏感許可權關掉或改成“僅在使用時允許”。

第五章：讓手機更懂你——基礎設定

花幾分鐘設定，體驗提升巨大。這些設定就像給手機“調音”，讓它更符合你的使用習慣。

一、聲音與震動

設個喜歡的鈴聲和通知聲。靈活使用靜音模式（開會）、勿擾模式（睡覺），世界瞬間清淨。

- **鈴聲**：選一個你能聽到的、不尷尬的。
- **震動模式**：靜音時靠震動提醒。
- **勿擾模式**：可以設定例外，比如允許通訊錄來電。
- **音量分離**：媒體音量、鈴聲音量、鬧鐘音量可以分開調。

二、顯示與亮度

- **自動亮度**：讓手機根據環境光自動調節，省心。
- **護眼/夜覽模式**：過濾藍光，晚上看手機舒服點。
- **深色模式**：省電（對 OLED 屏特別有效），且夜間不刺眼。很多 App 也支援深色模式。
- **螢幕重新整理率**：部分安卓機可調，越高（如 120Hz）滑動越絲滑。
- **自動鎖屏**：設定 15 秒或 30 秒，安全又省電。
- **字型大小**：覺得字小？調大一點。

三、桌布、主題與桌面

換張好看的桌布（靜態或動態），心情都變了。安卓品牌手機常有主題商店，能整套換圖示和風格。

桌面小元件：天氣、日程、音樂控制放桌面，一步直達。iOS 和安卓都支援，長按桌面空白處新增。

四、效率設定：讓手機成為你的“外掛大腦”

這部分是“一旦設定就回不去”的功能，每個都能讓你的日常效率顯著提升。

專注 / 免打擾模式：奪回被通知奪走的注意力

- **核心思想**：不是” 少看手機”，而是” 在該專注時專注，該聯絡時再聯絡”。
- **使用建議**：
  • 工作模式：僅允許” 星標” 聯絡人和必要 App 透過；
  • 睡眠模式：自動開啟，僅允許鬧鐘；
  • 健身 / 閱讀 / 會議：根據當前活動定製。
- **進階玩法**：用自動化 + 位置 / 時間觸發，讓模式自動切換。例如：iOS 快捷指令可設定” 到達公司即自動進入工作模式”。

通知管理：讓紅點不再“奪命連環 call”

- **關閉營銷通知**：90% 的推送其實你並不需要。
- **按重要性排序**：把人際通訊類（電話、簡訊、即時通訊）置頂；新聞、購物、社交平臺類可設為” 靜默推送”。
- **分組通知**：把同類通知摺疊在一起，避免一次解鎖被幾十條訊息淹沒。

快捷手勢：讓操作快人一步

- **截圖**：電源 + 音量下（iOS 全面屏）/電源 + 音量下（安卓通用）。
- **截長圖 / 滾動截圖**：普通截圖後點選” 長截圖” 選項。
- **一鍵掃碼**：控制中心 / 快捷設定裡新增” 掃一掃” 圖示。
- **輕點背面（iPhone）**：在 [設定] > [輔助功能] > [觸控] > [輕點背面] 中繫結” 截圖” 或” 鎖屏” 等動作。
- **語音喚醒**：Siri / 小愛同學 / 小藝 / 小布，喊一聲就能查天氣、設鬧鐘。

輸入法：最被低估的效率神器

- **選對輸入法**：
  • 搜狗 / 百度：詞庫大，流行語更新快；
  • 訊飛：語音輸入識別率最高，開會記筆記神器；
  • Gboard（谷歌）：簡潔、跨平臺同步強、支援滑行輸入；
  • iOS 自帶：隱私最好，與系統深度整合。
- **必備開關**：智慧糾錯、聯想詞、雲詞庫同步、上滑輸入數字符號。

- **隱私提示**：慎用” 整句聯想” 或” 個性化推薦” 功能，可能上傳輸入內容。

桌面與多工：讓工作流順暢起來

- **App 分屏 / 畫中畫**：看影片時回訊息，上網時記筆記。
- **小窗 / 懸浮窗**：微信 / 抖音支援” 小窗模式”，回覆完即收起。
- **常用 App 置頂**：Dock 欄（螢幕底部）只放最高頻的 4 個，減少查詢時間。

” 上工第一天” 檢查清單

拿到新手機或大版本更新後，建議花 10 分鐘過一遍這些設定：
(1) 鎖屏密碼 + 生物識別 (2) 通知分組 + 關閉營銷推送 (3) 字型大小 / 重新整理率 / 深色模式
(4) 預設輸入法 / 瀏覽器 / 郵件 (5) 緊急聯絡人 / 醫療卡 (6) 備份設定

五、輔助功能與無障礙：科技向善的力量

別被“輔助功能”這個名字嚇到，這些功能人人能用。它們最初是為殘障人士設計的，但對普通人也很有用——臨時手受傷抱娃、地鐵上耳機壞了聽不清、戴老花鏡看小字……都能救場。說到底，無障礙設計體現了**科技向善**的理念：讓每個人都能平等地享受數字生活。

視障使用者的“眼睛”——螢幕閱讀器全攻略

螢幕閱讀器是視障使用者使用手機的核心工具。它會把螢幕上**所有可視內容朗讀出來**，使用者透過手勢和語音操作手機。下面分別介紹 iOS “旁白”和安卓 TalkBack 的使用方法。

iOS 旁白（VoiceOver）

開啟路徑：[設定] > [輔助功能] > [旁白]，開啟開關。也可三擊側邊鍵（或主螢幕鍵）快速開關。
基本手勢：

1. 單指滑動：逐元素瀏覽螢幕內容；
2. 雙指輕掃：上下/左右整頁滾動；
3. 單指雙擊：啟用當前焦點元素（相當於” 點選”）；
4. 三指輕掃：切換頁面；
5. 單指三擊：放大或選擇（與普通三擊不同）。

個性化設定：[旁白] > [語音]，可調節**朗讀語速**（預設約 50%，可調到 70-80% 聽感更流暢）、**音調**、切換不同語音包。iOS 自帶“增強語音”和“語音控制器”，前者能減少呼吸聲和停頓，聽感更自然。

安卓 TalkBack

開啟路徑：[設定] > [輔助功能] > [TalkBack]，開啟開關。或長按音量上下鍵 3 秒快速啟用（部分品牌支援）。
基本手勢：

1. 單指左右滑動：切換焦點元素；
2. 單指雙擊：啟用元素；
3. 雙指上下/左右滑：滾動；
4. 三指單擊：進入閱讀模式（連續朗讀）；
5. 四指單擊：暫停/繼續朗讀。

個性化設定：[TalkBack] > [文字轉語音設定]，可調整**語速**、**音調**、切換**語音引擎**（建議安裝 Google 文字轉語音引擎，朗讀更自然）。

讓小字變大、讓模糊變清晰——視覺調整

放大手勢

- **iOS**：[設定] > [輔助功能] > [縮放]，開啟後**三指輕點螢幕**即可放大。預設全屏縮放，可選視窗縮放（區域性放大窗）。**雙指開合可調節倍數（1.5x 至 15x）**，三指雙擊鎖定當前縮放比例。

- **安卓**：[設定] > [輔助功能] > [放大]，三指點選放大，再三指點選退出；或直接**三指點選螢幕三下**啟用臨時放大（最多 8 倍）。

高對比度文字 / 粗體文字

- **iOS**：[設定] > [輔助功能] > [顯示與文字大小]，開啟” 粗體文字”、” 增強對比度”、” 降低透明度” 等開關。” 智慧反轉” 可在淺色背景下讓文字保持黑色，避免白底白字。

- **安卓**：[設定] > [顯示] > [字型大小]，可調節” 小、普通、大、超大” 四檔。華為/小米額外提供” 粗體” 和” 高對比度文字” 開關。

色彩濾鏡（色盲友好）

- **iOS**：[設定] > [輔助功能] > [顯示與文字大小] > [色彩濾鏡]，提供**紅/綠/藍/黃四種色盲模式**，以及” 灰度” 和” 反轉顏色” 選項。建議先做一次色覺測試（系統自帶）再選對應濾鏡。

- **安卓**：[設定] > [輔助功能] > [顏色校正]，含**三種色盲模式** + ” 反轉顏色” 開關。

AI 影象描述：讓 AI 當你的“眼睛”

當螢幕閱讀器遇到圖片時，傳統做法是跳過或只讀出“圖片”兩個字。但現在 AI 可以**用一句話描述圖片內容**，對識別驗證碼、讀圖、識別選單非常有幫助。

各系統 AI 影象描述支援情況

- **iOS 15+**：在 [設定] > [輔助功能] > [旁白] > [語音描述] 中開啟” 影象描述”。系統會呼叫**本地 AI 模型**自動生成圖片文字描述（需 iPhone 12 及以上）。
- **新版 TalkBack 應用**：Google TalkBack 在較新版本中整合**圖片問答**功能，可以問” 這張圖裡有什麼”，AI 實時回答（不嚴格依賴安卓系統版本，更新 TalkBack 應用即可體驗）。
- **第三方 App**：”Seeing AI”（iOS）、”Lookout”（安卓）、” 小藝幫幫”（華為）都是優秀的 AI 視覺助手，可識別物品、讀檔案、識別人臉表情。

使用場景

試著對著一張街拍照片問“這張圖裡有幾個人？他們在做什麼？”，或者開啟外賣 App，讓 AI 讀出選單圖裡的菜品名——**這是真正的“科技讓生活更平等”**。

聽障使用者的“耳朵”——實時字幕全攻略

實時字幕（Live Captions）是**系統級的語音轉文字**功能，聽障使用者在嘈雜環境、圖書館、外語場景下都特別有用。

實時字幕開啟與最佳化

iOS：[設定] > [輔助功能] > [實時字幕]，開啟開關。系統會檢測裝置所有音訊（通話、影片、播客、FaceTime、語音備忘錄），自動顯示字幕在螢幕上方。
安卓 12+：按**音量鍵**點選彈出框中的“實時字幕”圖示；或在 [設定] > [輔助功能] > [實時字幕] 中開啟。
準確率最佳化技巧：

1. 儘量在**安靜環境**下使用，遠離空調、機器轟鳴；
2. 說話者**咬字清晰、語速適中**時識別率最高；
3. 多人對話會降低準確率，可使用” 只顯示一人” 模式；
4. 支援中文普通話、粵語、英語等主流語種，**小語種識別率較低**；
5. 說話中途有背景音樂或人聲干擾，識別會斷斷續續。

通話場景雙通道：語音轉文字 + 文字轉語音

當使用者**既聽不到也說不了話**時（如突發失聲、喉部手術恢復期、嚴重口吃），手機還能當“傳話員”。

通話實時轉譯（TTY / RTT）

iOS：[設定] > [輔助功能] > [RTT/TTY]，開啟後通話時螢幕上會顯示鍵盤，對方的語音會被實時打成文字，**你打的字也會被念出來**給對方聽。
安卓：[設定] > [輔助功能] > [實時轉錄] + [聲音通知]。“實時轉錄”將對方說話實時顯示為文字；“聲音通知”在不便語音時將你的輸入轉為語音播放。
相容性：

- **RTT（Real-Time Text）**：雙方都需支援，文字邊打邊顯示，無延遲，主流運營商已支援；
- **TTY（電傳打字）**：需專用 TTY 裝置，已逐步被 RTT 取代；
- 微信/QQ 的**語音轉文字功能**也是替代方案：長按語音訊息”轉文字”。

面向所有人的“小但實用”功能

- **輕點背面**（iOS 14+）：[設定] > [輔助功能] > [觸控] > [輕點背面]，可繫結**雙擊/三擊**觸發截圖、鎖屏、控制中心等動作。雙手拿快遞時特別好用。
- **輔助觸控 / 懸浮球**：iOS [輔助功能] > [觸控] > [輔助觸控]，安卓 [輔助功能] > [懸浮導航]。虛擬按鈕替代手勢或物理按鍵，按鍵不靈或單手操作時救場。
- **切換控制**：透過外部開關、單次點選、頭部動作控制手機，**為肢體障礙使用者設計**。iOS [輔助功能] > [切換控制]，安卓 [輔助功能] > [開關控制]。
- **聲音識別**（iOS）：[輔助功能] > [聲音識別]，開啟後手機會”聽”門鈴、嬰兒哭聲、火警警報、警笛等聲音，**識別到就震動 + 螢幕通知**，對聽障使用者和睡眠期間非常有價值。
- **LED 閃爍提醒**：iOS [輔助功能] > [音訊/視覺] > [LED 閃爍以示提醒]，安卓 [設定] > [通知] > [閃光燈提醒]。來電和通知時閃光燈閃爍，**嘈雜環境無聲也能感知**。
- **單聲道音訊**：[設定] > [輔助功能] > [音訊] > [單聲道音訊]，將左右聲道合併。單耳聽力受損的使用者不會丟失另一半聲音資訊。

蘋果的無障礙設計哲學

蘋果在無障礙領域投入巨大，每年都會更新和新增輔助功能。蘋果官網有專門的無障礙頁面，詳細介紹各項功能的使用方法。iOS 的無障礙功能通常比安卓更完善、體驗更流暢，這也是很多殘障使用者選擇蘋果的重要原因。

花 30 秒體驗一下

找一段空閒時間，做以下體驗：

1. 開啟**旁白 / TalkBack**，試著不靠眼睛用三指滑動瀏覽微信；
2. 開啟**實時字幕**，對著手機說話看識別準確率；
3. 用**三指點選**放大一段文字到 200%，體驗視障使用者的日常；

4. 關掉之前所有功能，思考” 如果我永遠需要這些功能，我會希望它怎麼設計”。

第六章：守護你的數字生活——安全與隱私

手機安全 = 隱私安全 + 財產安全。在這個資料即資產的時代，保護手機就是保護你自己。

一、鎖屏：第一道物理防線

鎖屏是你手機的第一道防線，也是最容易被忽視的。

安全等級排序：

1. **密碼/PIN 碼**：安全度最高，是最終保障（忘記指紋/臉時靠它）。
2. **指紋識別**：安全與便利的完美平衡。
3. **人臉識別**：最方便，但安全性取決於技術（3D 結構光安全，2D 照片易騙過）。
4. **圖案解鎖**：方便但容易留下痕跡被猜到。

設定強鎖屏：

- 務必設定一個複雜密碼（不要用生日、123456）。
- 開啟生物識別。
- 鎖屏時間設成” 立即” 或最短。

二、“查詢我的手機”：終極防盜工具

必須開啟！它能：

- **定位**：手機丟哪了，在地圖上找。
- **播放聲音**：讓手機響鈴，適合在家找。
- **丟失模式**：遠端鎖死手機並顯示聯絡號碼。

- **遠端擦除**：最後一招，清除所有資料（但擦除後無法再定位）。

去 [設定] 裡，蘋果找“查詢我的 iPhone”，安卓找“查詢我的裝置”或品牌類似服務，確保它是綠色開啟狀態。

三、系統更新：打補丁，堵漏洞

系統更新首要目的是修復安全漏洞，就像給房子補牆洞。所以，看到系統更新提示，儘量第一時間升級。

當然，大版本更新（比如 iOS 17 到 iOS 18）可能會有相容性問題，可以觀望幾天再升。

四、識別與防範電信詐騙

在 AI 技術迅猛發展的今天，傳統的“眼見為實”認知模式已完全失效。詐騙分子正利用深度偽造（Deepfake）等 AI 技術，實現了從“廣撒網”式詐騙到“精準圍獵”式犯罪的全面升級。

AI 時代最常見的幾種詐騙套路

(1) AI 換臉擬聲詐騙

不法分子透過非法蒐集目標物件在社交媒體釋出的高畫質照片或僅需幾秒鐘的語音樣本，利用 AI 深度偽造技術克隆出足以亂真的“親友影片”或“語音”。常見劇本包括：**突發車禍、緊急借款、遭遇綁架**等。此類詐騙利用親情、友情等情感紐帶，**迷惑性極高**。

(2) 冒充權威 / 客服詐騙

AI 技術能夠精準模擬官方客服、公檢法機關人員的聲音與語調，甚至可以合成高度逼真的虛假轉賬截圖、電子公文、通緝令等檔案，透過**製造恐慌心理或權威壓力**，誘導受害者轉賬或開啟螢幕共享功能，從而竊取賬戶資訊。

(3) AI 投資理財詐騙

透過 AI 演算法偽造“穩賺不賠”的虛假收益截圖、虛擬量化選股軟體介面或“大師級”投資分析報告，以高額回報為誘餌，售賣高價投資服務或誘導受害者將資金轉入其控制的交易賬戶，**最終捲款跑路**。

破局之法——“照妖鏡”防護思路

AI 可以克隆聲音和麵容，但也不是無懈可擊，有幾個破綻可以利用：

- **動作驗證法**：進行視訊通話時，主動要求對方完成預設的連貫動作指令，例如” 用手在臉前快速揮過”、” 側臉展示耳朵輪廓” 或” 連續快速眨眼 5 次”。目前的 AI 換臉技術在處理複雜動態遮擋、快速運動場景和光影變化時，**極易出現畫面卡頓、面部扭曲或邊緣穿模**等破綻。

- **暗號核驗法**：建立並使用預設的私密核驗問題，例如” 我們上次一起聚餐時你點的什麼菜”、” 我送你的第一個禮物是什麼” 等只有雙方知曉的細節。若對方出現明顯遲疑或答非所問，應**立即終止通話並啟動安全預案**。
- **二次核實法**：嚴格遵守公安部網安局釋出的” **四不一核實**” 原則——不輕信陌生資訊、不隨意轉賬匯款、不洩露個人敏感資訊、不傳播可疑內容。涉及任何錢財往來時，務必透過**已知的原號碼回撥或當面確認**。進行轉賬操作時，建議選擇”24 小時到賬” 模式，為潛在詐騙識別和資金挽回預留充足時間視窗。

傳統常見套路（仍要警惕）：

- **中獎/刷單返利**：利誘，先給你點甜頭再騙大錢。
- **網戀詐騙**：殺豬盤，培養感情後騙錢。
- **冒充熟人**：盜號後借錢（AI 時代更難分辨）。

手機自帶防護：

- 騷擾電話攔截。
- 驗證碼防竊取。
- 簡訊連結安全檢測。

防騙口訣：

不輕信、不透露、不點選、不轉賬、有疑問先核實！遇到“急事”先冷靜 5 分鐘，幾乎所有詐騙都會在這一刻露出馬腳。

五、抵禦“開盒”與社工攻擊：斬斷隱私洩露鏈條

“開盒”（社會工程學攻擊）的核心原理在於**系統性蒐集、分析並拼湊個人數字碎片，構建完整的個人畫像**。防範的關鍵在於為個人隱私劃定明確紅線，建立多層次防護體系。

照片脫敏處理

基礎 EXIF 清理方法見第二章“圖片與檔案系統管理”。在社交媒體分享照片前，還需進一步**徹底刪除照片的後設資料（EXIF 資訊）**，包括拍攝時間、裝置型號、GPS 位置座標等敏感資料。對照片中包含的**門牌號碼、車牌資訊、車票條形碼、身份證號**等關鍵資訊進行嚴格打碼或模糊處理，避免無意中洩露個人行蹤軌跡。

- iOS：[設定] > [隱私與安全性] > [相機] 中可關閉” 精確位置”；分享時選” 僅當前照片” 可剝離後設資料。
- 安卓：部分相簿 App（如 Google 相簿）分享時會提示是否保留位置資訊。

- 通用工具：ExifTool、Metapho 等可檢視/編輯 EXIF。

應用許可權收斂

基礎許可權管理方法詳見第四章“許可權管理”。在“防開盒”場景下，還需進一步定期審查並關閉電商類、工具類 App 的**麥克風、通訊錄、位置資訊**等非必要許可權，防止 App 在後臺“偷聽”日常對話或非法收集個人關係網路；對長期閒置但仍佔用高許可權的應用程式應及時解除安裝，從源頭減少隱私洩露風險點。

- 看到” 通訊錄許可權” 申請時，先問一句：這 App 真有必要知道我所有聯絡人嗎？
- 三個月沒用過的 App，果斷解除安裝。許可權是越用越緊的，不是越用越松。

智慧裝置安全設防

智慧攝像頭、智慧門鎖、智慧音箱等 IoT 裝置在初次使用時必須**立即修改預設登入密碼**，設定包含大小寫字母、數字和特殊符號的高強度獨立密碼；不常用的遠端訪問功能應保持關閉狀態，定期檢查裝置韌體更新並及時修補安全漏洞，**防止駭客入侵併窺探家庭隱私空間**。

真實案例警示

2022 年某地警方通報，駭客透過破解某品牌家用攝像頭的預設密碼（很多使用者從未修改），竊取了上千戶家庭的臥室畫面並在網上兜售。**不改預設密碼 = 把家大門敞開**。

六、隱私保護：你的資料是你的財產

手機裡的照片、聊天記錄、位置軌跡，本質上都是你的“數字資產”。一旦洩露，輕則廣告騷擾，重則被人肉、詐騙甚至敲詐。

App 許可權的“最小化原則”：

- **位置**：打車/地圖用” 始終允許”；其他 App 用” 僅使用時允許”。
- **通訊錄**：除非必要（如微信加好友）才給，否則一律禁止。
- **相簿 / 全部照片**：iOS 14+ 起可授權” 選定的照片”，安卓 13+ 同樣支援精細授權。
- **麥克風 / 攝像頭**：平時關掉，需要時再開啟。懷疑被偷錄時，滑出控制中心看是否有” 小綠點/小橙點”。
- **後臺跟蹤**：iOS 14.5+ 起強制要求 App 使用 IDFA 前必須詢問；安卓可關閉” 廣告 ID”。

隱私管理實戰：

- **隱私報告（iOS）**：[設定] > [隱私與安全性] > [App 隱私報告]，檢視哪些 App 在偷偷訪問你的位置、相機、麥克風。

- **許可權使用記錄（安卓）**：[設定] > [隱私] > [許可權使用記錄]，看誰在什麼時候呼叫了敏感許可權。
- **限制廣告跟蹤**：在系統設定裡關閉” 廣告 ID” 或” 限制廣告追蹤”，能減少” 被精準廣告騷擾”。

七、丟失 / 被盜後的“急救”清單

手機丟了不要慌，按以下順序處理：

1. **立即凍結 SIM 卡**：聯絡運營商掛失，避免被用於接收驗證碼。
2. **遠端鎖定**：透過雲服務（iCloud / 品牌雲）開啟” 丟失模式”。
3. **修改重要密碼**：登入過手機的所有重要賬號（支付、銀行、郵箱）立即改密碼。
4. **報案**：到派出所報案，記錄 IMEI 號碼。手機找回時也便於證明身份。
5. **通知親友**：防止騙子以你名義借錢。
6. **（最後）遠端擦除**：確認找回無望時再操作，擦除後定位功能失效。

八、SIM 卡與 eSIM 安全

- **設定 SIM 卡 PIN**：開啟後，每次重啟手機或換卡都需要輸入 PIN 碼，防止” 拔卡換手機” 繞過鎖屏。
- **eSIM 優勢**：沒有實體卡可被拔出，安全性更高；同時支援多號切換，旅行時非常方便。
- **二次換卡風險**：警惕” 運營商二次換卡” 騙局——騙子冒用你的身份補辦 SIM 卡，從而接管你的手機號。設定” 補卡需機主本人持身份證到營業廳辦理” 是最有效的防線。

千萬別做的事

- 用生日 / 123456 / qwerty 等弱密碼；
- 把驗證碼發給任何人（哪怕是” 客服”）；
- 點選簡訊裡的不明連結，特別是 00 開頭的境外號碼；
- 在公共 Wi-Fi 下登入網銀或支付；
- 給陌生人開啟” 螢幕共享” 或” 遠端控制”。

試著啟用並模擬” 遠端鎖定”

1. 確保你的” 查詢我的手機” 功能已開啟。
2. 在電腦瀏覽器或另一部手機上，登入對應的雲服務賬號（iCloud.com 或品牌雲服務網站）。

3. 找到” 查詢裝置” 功能，嘗試對你自己的手機發起一次” 播放聲音” 或模擬” 鎖定” 指令，熟悉整個操作流程。

第七章：前沿功能全解析

科技日新月異，手機功能也在不斷進化。衛星通訊、無網通、車禍檢測、星閃技術……這些聽起來很科幻的功能，其實已經走進我們的生活。讓我們一起來了解這些前沿技術。

一、衛星通訊：沒訊號也能求救

衛星通訊是近年來手機領域的一大突破。當你在荒郊野外、深山老林，手機完全沒有訊號時，衛星通訊可能就是救命稻草。

工作原理：

手機直接與軌道上的通訊衛星建立連線，傳送求救資訊。目前主要支援傳送簡訊和位置資訊，不能打電話。

支援機型：

- **蘋果**：iPhone 14 系列及以後，支援透過衛星傳送緊急 SOS。iPhone 17 系列進一步支援衛星傳送 iMessage 和簡訊，且擴充了更多地區的覆蓋。
- **華為**：Mate 50 系列開始支援北斗衛星訊息（僅文字），Mate 60 Pro 系列進一步支援天通衛星通話（可打電話），Mate 70 系列衛星通訊能力進一步增強。
- **小米/OPPO/vivo**：2024 年起旗艦機型陸續支援北斗衛星訊息功能。
- **發展趨勢**：衛星通訊正從高階旗艦專屬功能向下普及，未來中端機型也有望支援。中國北斗衛星系統在國產手機上的支援最為廣泛。

使用場景：

- 戶外探險、登山遇險。
- 海上遇險。
- 自然災害導致地面通訊中斷。

使用方法：

1. 確保在開闊地帶，能看到天空。
2. 開啟手機的衛星通訊功能（通常在緊急呼叫介面）。
3. 按照螢幕指引，對準衛星方向。
4. 傳送預設的求救資訊和位置。

小知識

衛星通訊需要手機有特殊的硬體天線，不是所有手機都支援。另外，衛星通訊通常需要付費訂閱服務，但緊急求救功能一般是免費的。

二、無網通：近距離通訊新方式

“無網通”是指在沒有行動網路和 Wi-Fi 的情況下，兩臺手機之間直接通訊的技術。

技術原理：

利用藍芽或專用的無線電頻率，在短距離內（通常幾百米到幾公里）建立點對點連線。

應用場景：

- **戶外活動：**徒步、露營時隊友間通訊。
- **災難救援：**地震、洪水後通訊中斷時。
- **大型活動：**演唱會、體育賽事人多訊號差時。

支援平臺：

- **蘋果：**iOS 17 的”NameDrop” 和” 離線找朋友” 功能。
- **華為：**暢連 App 支援無網通話。
- **小米：**小米妙享中心的部分功能。

三、車禍檢測：關鍵時刻救命

車禍檢測是利用手機感測器檢測嚴重車禍，並自動撥打緊急電話的功能。

工作原理：

手機透過加速度計、陀螺儀、氣壓計、麥克風等多個感測器，綜合判斷是否發生了嚴重碰撞。檢測到車禍後，手機會：

1. 彈出倒計時介面，詢問你是否沒事。
2. 如果倒計時結束你沒有響應，自動撥打緊急電話。
3. 向緊急聯絡人傳送你的位置。

支援機型：

- **蘋果**：iPhone 14 系列及以後。
- **安卓**：部分高階機型陸續支援。

注意事項：

- 功能需要開啟才能使用。
- 可能會誤觸發（比如過山車、劇烈運動），及時取消即可。
- 依賴感測器和演算法，不是 100% 可靠，但多一層保障。

四、星閃技術：下一代無線連線

星閃（SparkLink）是華為牽頭研發的新一代無線通訊技術，目標是在某些場景下替代藍芽和 Wi-Fi。它在關鍵指標上對標甚至超越現有方案：空口速率可達 900Mbps（遠超藍芽的 50Mbps），延遲低至 20 μ s（藍芽約 15ms），單網路支援 4096 個裝置同時連線（藍芽通常 8 個以內）。

技術特點：

- **超低延遲**：端到端延遲僅 20 微秒（0.02ms），比藍芽低兩個數量級，遊戲和音樂同步幾乎無感知。
- **高速率**：單鏈路峰值速率 900Mbps，是藍芽音訊的數十倍，可傳輸無損音訊甚至高畫質影片。
- **低功耗**：待機功耗與藍芽相當，但傳輸效率更高，單位資料的能耗更低。
- **廣覆蓋 + 高併發**：連線距離比藍芽遠 50% 以上，單網路支援上千裝置同時線上。

應用場景：

- **無線耳機：**更低延遲，玩遊戲音畫同步。
- **智慧手錶：**更穩定的連線。
- **智慧家居：**更多裝置同時連線。
- **車載互聯：**更流暢的手機-車機連線。

支援裝置：

目前主要是華為生態裝置，如華為 Mate 60 系列、部分華為耳機和手錶。未來會有更多品牌支援。

技術展望

星閃技術代表了無線通訊的未來方向。隨著更多廠商加入，我們可能會看到更多支援星閃的裝置，享受更好的無線體驗。

去檢查一下：你的手機是否支援這些功能

1. 去設定裡搜尋” 衛星”，看看是否有衛星通訊相關選項。
2. 搜尋” 車禍檢測” 或” 緊急 SOS”，檢查是否開啟。
3. 如果你的手機是華為，看看是否支援星閃裝置連線。
4. 瞭解這些功能的使用方法，關鍵時刻可能派上用場。

二、軟體篇（第 8-14 章）

第八章：通訊：簡訊、電話與社交

通訊是手機最原始、最核心的使命。現在，它已從簡單的通話和簡訊，演變成一個包含即時通訊、社交網路和電子郵件在內的複雜生態系統。

一、電話 App 的進階功能

現代手機的電話應用程式，早已超越了一個簡單的撥號盤，它整合了錄音、騷擾攔截、智慧識別等強大工具。

通話錄音（注意法律合規性）：

- 作用：記錄重要通話內容，用於工作紀要、採訪、客戶承諾或爭議留存。
- 法律提示：在中國境內，通話錄音作為證據使用的前提是取得對方同意，或用於維護自身合法權益。
- 單方錄音 vs 雙方同意：法律層面，”未經對方同意”取得的錄音並非當然無效。最高人民法院相關司法解釋中，只要錄音取得方式未嚴重侵害他人合法權益、未違反法律禁止性規定、未違背公序良俗，仍可作為證據提交。但部分省份法院在民事審判中對”偷偷錄音”的採信標準更嚴格；刑事案件則通常要求更嚴格的合法性審查。
- 各地差異：北京、上海、廣東等地法院對單方錄音作為民事證據較為寬容；而涉及商業秘密、個人隱私的錄音，無論在哪個省份都需要特別謹慎，可能因侵害隱私權而被排除。
- 如何操作：安卓通常在通話介面有明確的”錄音”按鈕；iOS 系統不提供原生通話錄音功能，需藉助第三方 App 或使用帶錄音功能的藍芽耳機。

錄音使用紅線

- 不得將錄音用於敲詐、傳播或公開侮辱他人。
- 涉及第三方隱私（如偶然錄到他人對話）的內容不得隨意擴散。
- 重要場景建議在通話開始時口頭告知”本次通話將被錄音”，既合規也更具證據效力。

騷擾電話治理深度內容：

- 原理：電話 App 透過網路查詢龐大的號碼庫，在來電時顯示” 騷擾電話”、” 詐騙電話” 或公司名稱。
- 如何開啟：安卓進入電話 App > 設定 > 騷擾攔截；iOS 進入設定 > 電話 > 來電阻止與身份識別。各品牌詳細開關路徑與推薦開啟項，見第十三章「實用系統工具」中的品牌對照表。
- 主流標記平臺：國內號碼標記主要由三家資料服務商提供——騰訊手機管家、360 手機衛士、電話邦（Tapir）。電信業者（移動/聯通/電信）的” 機主姓名” 顯示、” 騷擾攔截” 功能也常呼叫這些資料。手機廠商（華為、小米、OPPO、vivo）的標記庫多與上述服務商合作。
- 被誤標記怎麼辦：如果你正常工作號碼被錯誤標記為” 外賣”” 快遞”” 騷擾電話”，可透過以下管道申訴：電話邦：訪問號碼申訴平臺（haoma.tk.cn）提交身份證明。
- 360：透過 360 手機衛士 App 內” 號碼申訴” 功能提交。
- 騰訊：透過” 騰訊手機管家” 小程式” 號碼申訴” 頁面提交。
- 三大電信業者：中國移動使用者可傳送簡訊”KTFSR” 到 10086 開通防騷擾功能；電信使用者關注” 天翼防騷擾” 公眾號；聯通使用者在” 聯通手機管家” 中設定。

防騷擾進階：開啟電信業者的” 高頻呼叫攔截”（如移動的高頻呼叫限制），可自動攔截 AI 語音外呼。

手機自帶攔截常見能力：來電識別 / 智慧攔截：按號碼庫自動標記並攔截騷擾、詐騙、廣告推銷。

黑名單 / 白名單：手動拉黑已知騷擾號；把通訊錄、快遞外賣號加入白名單，避免誤殺。

簡訊關鍵詞過濾：包含” 中獎”” 貸款”” 點選連結” 等關鍵詞的簡訊自動歸入攔截箱。

iOS 靜音未知來電：不在通訊錄中的來電只靜音進語音信箱，不打斷你；適合騷擾電話多的使用者，但要留意漏接陌生重要電話。

勿擾模式聯動：夜間或開會時只允許通訊錄來電，配合定時開關，睡眠品質立刻改善。

實用軟體怎麼配合：優先用系統自帶攔截，再疊加國家反詐中心與電信業者防騷擾；清理類可選清濁等輕量工具，但同一類安全/清理軟體裝一套即可，詳見第十三章。

VoLTE/VoNR 是什麼及如何開啟：

- VoLTE（Voice over LTE）：基於 4G 網路的語音通話技術，通話時不再回落到 2G/3G，能邊打電話邊上網，音質比傳統通話更清晰，接通更快。
- VoNR（Voice over NR）：基於 5G 的語音通話，是 VoLTE 的下一代演進。延遲更低、音質更好，並能支援高畫質視訊通話。需 5G 手機 +5G 資費方案 + 當地 5G 覆蓋三者具備才能使用。

- 開啟方法：iOS 在” 設定 > 蜂窩網路 > 蜂窩資料選項 > 語音與資料” 中開啟”VoLTE”；安卓在” 設定 > 行動網路 > 移動資料 > 進階” 中找到”VoLTE 高畫質通話” 開關。電信業者通常預設開啟，若發現通話時網路斷開、無法上網，多半是 VoLTE 未啟用。

小知識

如果你的手機在通話時上網速度突然變慢或斷網，多半是 VoLTE/VoNR 未開啟，回落到了 2G/3G 網路。開啟後即可實現“邊打電話邊刷影片”。

二、簡訊與 iMessage/RCS

傳統簡訊並未被淘汰，它仍是接收驗證碼、銀行通知和官方資訊的可靠管道。

- iMessage：蘋果裝置間透過網際網路傳送的訊息，顯示為藍色氣泡。支援高品質圖片影片、已讀回執等，完全免費且端到端加密。
- RCS：由全球電信業者和谷歌推動的下一代通訊協議。功能類似 iMessage 或微信，但基於電話號碼。
- iMessage 與 RCS 的互操作性：iOS 18 起，蘋果開始支援 RCS 標準，這意味著 iPhone 與安卓手機之間可透過 RCS 傳送高畫質圖片、已讀回執和打字狀態提示，告別” 綠色氣泡” 的體驗斷層。但前提是雙方電信業者均開通 RCS，且手機系統版本足夠新。

RCS 在國內的部署情況：

- 中國移動：較早部署 RCS，部分省份已支援”5G 訊息” 業務，可傳送多媒體訊息、互動卡片。需手機終端支援且在營業廳開通。
- 中國電信：透過” 電信 5G 訊息” 提供服務，部分資費方案預設支援，老使用者需傳送指定簡訊指令開通。
- 中國聯通：在主要城市部署 5G 訊息，但終端覆蓋參差不齊，許多安卓機型仍需手動在簡訊 App 中開啟” 增強資訊” 或”5G 訊息” 開關。
- 實際體驗：國內 RCS 推廣速度慢於預期，主要受微信生態強勢、終端適配不統一等因素影響。日常通訊仍以微信為主，RCS 更多承擔” 官方推送” 角色（如銀行通知、政務訊息）。

驗證碼安全紅線

- 不要轉發驗證碼：任何讓你” 讀出驗證碼” 或” 轉發簡訊” 的人都可能是詐騙分子，包括自稱” 客服”” 快遞員”” 公檢法” 的人。驗證碼 = 你的錢袋子。
- 防釣魚簡訊：警惕”ETC 過期”” 醫保卡凍結”” 快遞丟失理賠” 等簡訊中的連結，正規機構不會透過簡訊連結索要銀行卡號、密碼。
- 號碼識別：銀行官方簡訊通常來自 95 開頭短號或 1069+ 企業簽名，凡是從普通 11 位手機號發來的” 銀行通知” 基本都是詐騙。

- 雙因子備份：繫結重要帳號的手機號不要隨意更換；如必須換號，務必先解綁所有關鍵帳號，否則新機主可能透過簡訊驗證碼重置你的密碼。

三、微信/QQ：國民級社交應用程式的管理

微信和 QQ 已超越通訊工具，成為集社交、支付、資訊獲取於一體的“超級 App”。

聊天紀錄備份與遷移：

1. 在電腦上登入 PC 版或 Mac 版微信。
2. 點選左下角選單 > [遷移與備份] > [備份與還原]。
3. 選擇” 備份聊天紀錄至電腦”，在手機上選擇需要備份的聊天。
4. 遷移到新手機：在原手機微信 [我] > [設定] > [通用] > [聊天紀錄遷移與備份] > [遷移] > [遷移到手機/平板]，掃碼即可完成。

儲存空間清理深度方法：

- 常規入口：微信 [我] > [設定] > [通用] > [儲存空間]，可清理快取和按時間、檔案類型精細化清理聊天紀錄。
- 聊天內清理：進入某個聊天 > 右上角”⋯” > [聊天紀錄] > [管理儲存空間]，可單獨清理某聊天中的圖片、影片、檔案，保留文字。
- 原圖/原影片清理：群組聊天中接收的高畫質原圖影片最佔空間，可在 [管理儲存空間] 中勾選” 原圖” 批次刪除。
- 關閉自動下載：[設定] > [通用] > [照片、影片、檔案和通話]，關閉” 自動下載”，避免在 WiFi 下默默快取大量群檔案。
- 小程式快取：在 [發現] > [小程式] 中長按不常用的小程式刪除，可釋放其本地快取。
- 聊天紀錄壓縮備份：電腦版微信備份時勾選” 壓縮”，可將幾 GB 的聊天紀錄壓縮到原體積的 30%-50%。

群管理技巧：

- 群公告：群主/管理員在群組聊天右上角”⋯” > [群公告] 中發佈，全員強提醒，適合重要通知。新成員入群時自動跳出。
- 群待辦：群主可將置頂訊息設為” 群待辦”，所有成員頭頂會出現紅色提示，完成點選後自動消失。適合活動報名、任務收集。
- 群接龍：在輸入框輸入”# 接龍” 可發起接龍，適合點餐、統計報名。接龍結果自動彙總到群組聊天頂部，避免刷屏。

- 群直播：群組聊天右上角”+” > [群直播]，可發起僅供本群觀看的直播，適合企業內訓、家族聚會、產品內測。

- 摺疊該群組聊天：在群列表向左滑動群名 > [摺疊]，可將不重要的群收納到” 摺疊的群組聊天” 中，僅在新 @ 你時提醒。

- 僅群主/管理員 @ 所有人：群成員 500 人以上時，普通成員無法 @ 所有人，避免被打擾。

朋友圈權限精細化管理：

- 分組可見性：發佈時點選” 誰可以看”，選擇” 部分可見” 或” 不給誰看”，基於預先設定的標籤分組（如” 同事”” 家人”” 客戶”）精準推送。

- 允許朋友檢視範圍：[我] > [設定] > [朋友權限] > [朋友圈] > [允許朋友檢視朋友圈的範圍]，可選” 最近三天/一個月/半年/全部”，建議設為” 半年” 以平衡隱私與社交。

- 遮蔽某條朋友圈：長按朋友圈某條動態 > [不看他/她的朋友圈] 或 [不再關注]，對方不會知道。

- 朋友圈廣告管理：[我] > [設定] > [關於微信] > [個人資料與權限] > [個人化廣告管理]，可關閉基於行為的個性化推薦。

- 陌生人十條：[朋友權限] > [朋友圈] > [允許陌生人檢視十條朋友圈]，關閉可避免陌生人翻看你的過往動態。

四、視訊通話對比

不同視訊通話工具各有側重，根據場景選擇合適的工具能讓溝通更順暢。

- 微信影片：國民級覆蓋，老少咸宜。適合親友日常聯絡、快速溝通。最多支援 20 人影片（需小程式” 群組影片”），缺點是網路差時畫質下降明顯。

- FaceTime：蘋果生態專屬，畫質優秀、端到端加密。iOS 15 起支援” 連結” 邀請安卓/Web 使用者加入，但主持人必須是蘋果裝置。適合家庭跨裝置溝通。

- Zoom：國際會議首選，支援最多 100 人免費 40 分鐘會議。螢幕共享、虛擬背景、分組討論室等功能完善，適合跨國/遠端協作。

- 騰訊會議：國內辦公首選，與微信深度整合，掃碼即入。支援 300 人會議（付費版），雲端錄製、實時字幕、會議紀要功能強大，適合企業內訓和大型會議。

- 使用場景建議：1 對 1 日常聯絡選微信；蘋果使用者之間選 FaceTime；正式商務會議選 Zoom 或騰訊會議（國內選騰訊，跨境選 Zoom）。

網路頻寬要求與用量消耗參考：

- 微信視訊通話：約需 500Kbps-1Mbps 頻寬，標準畫質每分鐘約消耗 3-5MB 用量，高畫質約 8-12MB。
- FaceTime：標清約 1Mbps，720p 約 2Mbps，1080p 需 4Mbps 以上。每分鐘用量約 5-15MB。
- Zoom/騰訊會議：720p 約 1.5-3Mbps，1080p 需 3-5Mbps。每小時用量約 300-800MB，多人會議主持人用量更大。
- 建議：長時間視訊通話儘量使用 WiFi；行動網路下可主動降低畫質（在 App 設定中關閉”高畫質影片”）以節省用量。

視訊通話畫質最佳化技巧：

- 光線：背對窗戶、面朝光源（自然光最佳），避免逆光導致人臉變黑。
- 角度：手機相機略高於眼睛，避免雙下巴；橫屏通話時雙手持握更穩定。
- 音訊：使用耳機可避免回聲和嘯叫；安靜環境下不要開”喇叭”模式。
- 網路：優先 WiFi；4G/5G 影片時儘量靠近窗戶避免訊號衰減；多人會議可關閉其他影片畫面僅保留發言人。
- 虛化背景：騰訊會議、Zoom 均支援 AI 虛化，可遮蔽雜亂背景。注意穿戴純色衣物，避免與背景色相近導致”綠幕穿幫”。

小貼士

網路卡頓時，先關閉影片保留語音，比直接結束通話更禮貌。重要會議前可用“騰訊會議”自帶的“網路檢測”功能提前測試頻寬。

五、電子郵件：在手機上配置與使用

- 新增信箱帳戶：iOS 在設定 > 郵件 > 帳戶 > 新增帳戶；安卓通常有”電子郵件”App。
- 使用技巧：為不同信箱帳戶設定不同的通知方式；善用滑動手勢快速歸檔、刪除或標記郵件。

IMAP vs POP3 vs Exchange 協議區別：

- POP3：從伺服器下載郵件後預設刪除原件，僅本地保留。適合只用一臺裝置、需要離線閱讀的場景。缺點是換裝置後看不到歷史郵件，已不推薦。
- IMAP：郵件始終儲存在伺服器，本地僅作快取。多裝置（手機/電腦/平板）同步狀態一致——一端已讀，全部已讀。是當前主流推薦協議。

- Exchange（含 ActiveSync）：微軟協議，除郵件外還同步日曆、聯絡人、任務。企業信箱首選。Outlook、網易企業信箱、騰訊企業信箱均支援。
- 選擇建議：個人信箱用 IMAP；公司信箱優先 Exchange，其次 IMAP；儘量不要用 POP3，除非有特殊歸檔需求。

企業信箱配置方法：

1. iOS：[設定] > [郵件] > [帳戶] > [新增帳戶] > 選擇”Microsoft Exchange” 或” 其他”。
2. 安卓：在自帶的” 電子郵件”App 或 Outlook App 中選擇” 新增帳戶”，輸入完整信箱地址和密碼。
3. 伺服器填寫（以 Exchange 為例）：伺服器地址通常為”mail. 公司域名.com” 或”outlook.office365.com”，域名填公司簡稱。
4. 若提示” 無法驗證伺服器”，可手動輸入：傳入伺服器（IMAP）埠 993，加密 SSL；傳出伺服器（SMTP）埠 465 或 587，加密 TLS。
5. 登入後建議在 [帳戶設定] 中開啟” 推送”（Push）以實時接收郵件，並關閉” 郵件預覽” 以保護商業機密。

小提示

企業信箱通常需要管理員開通“用戶端授權密碼”或“IMAP/SMTP 授權碼”，不能直接用網頁登入密碼。登入失敗時先到網頁版信箱設定中生成授權碼再填入手機。

實操：備份一次微信聊天紀錄到電腦

1. 在電腦上啟動備份：開啟電腦版微信 > 左下角選單 > [遷移與備份] > [備份與還原]。
2. 選擇備份到電腦：點選” 備份聊天紀錄至電腦”。
3. 在手機上選擇備份內容，等待備份完成。

第九章：移動辦公與學習

我以前也覺得手機上辦公就是“湊合一下”，直到有一次出差忘帶電腦，硬是用手機完成了標書修改和合同簽署。說實話，手機辦公的瓶頸往往不在手機本身，而在你不熟悉它的能力邊界。

一、文件處理：在掌中編輯與創作

在手機螢幕上編輯文件，跟電腦上感覺確實不一樣。最大的問題是螢幕小、手指粗、排版容易跑偏。但有幾個技巧能讓體驗好不少：

- **WPS Office**：本土化做得最好，完全相容微軟格式（docx/xlsx/pptx），內建的 PDF 工具集很豐富。我特別喜歡它的” 語音備註” 功能——看合同的時候不方便打字，直接對著手機說幾句意見，它就自動轉成文字批註貼在文件裡。另外 WPS 內建了海量免費模板，做簡歷、做彙報 PPT 的時候能省很多時間。
- **Microsoft 365**：如果你公司用正版 Office，這個就是最佳選擇。跟桌面版的格式相容性幾乎完美，不會出現 WPS 開啟某些複雜排版會輕微移位的情況。深度整合 OneDrive，你在電腦上改到一半的文件，手機上開啟就是最新版本。
- **石墨文件/騰訊文件**：這兩個是” 線上協同” 的代表。適合多人同時編輯一份文件，每個人改的地方實時顯示。我用石墨文件跟朋友一起做旅行攻略，兩個人同時往表格裡填資訊，效率比一個人弄高多了。

實用工作流：我現在的習慣是——重要檔案存 OneDrive，日常編輯用 WPS，團隊協作開石墨文件。檔案自從上了雲之後，再也沒出現過“忘帶隨身碟”的尷尬。

二、筆記應用：從隨手記到知識體系

記筆記這件事，很多人一上來就想搞一套很複雜的系統，結果堅持不了三天。我的建議是先輕後重——先養成“記”的習慣，再考慮“管理”的問題。

- **隨手記——系統自帶就夠**：iOS 備忘錄啟動極快，而且支援掃描文件、手寫、錄音。安卓各品牌的便籤也不差，鎖屏狀態下就能快速啟動。這些工具適合記臨時想法、購物清單、會議要點。關鍵是啟動快，想到了就能記，不用等 App 載入。

- **稍後讀——收集資訊**：微信的” 浮窗” 功能其實是個隱藏神器。看到好文章來不及細看，點右上角” 浮窗”，它就會縮成一個小圓點貼在螢幕邊緣，隨時可以點開繼續讀。比轉發到” 檔案傳輸助手” 方便多了。另外，Cubox 或 Pocket 這類稍後讀工具，可以跨平臺收集你在任何地方看到的好內容。

- **搭建體系——Notion 和飛書**：當你覺得零散的筆記開始不好找了，就可以上 Notion 了。這玩意真的很靈活——你可以建一個” 讀書筆記資料庫”，自動按書名、作者、打分來組織；也可以搭一個” 專案看板”，把工作任務的進度視覺化。飛書的” 多維表格” 也類似，而且國內訪問速度更快。我用 Notion 管理我所有的寫作素材、學習筆記和專案進度，已經用了三年多了。

一個實用的筆記策略：我自己的習慣是“三件套”——臨時想法用系統便籤（秒開），待讀文章用微信浮窗（順手），需要長期積累的知識放 Notion（體系化）。別一上來就想把所有東西塞進一個工具裡，這樣反而容易放棄。

三、掃描與 OCR：把紙質世界搬進手機

手機掃描這件事，用好了真的能省掉很多跑列印店的麻煩。我上次辦簽證需要一堆材料的電子版，全都是用手機搞定的。

- **iPhone 使用者**：iOS 備忘錄自帶的掃描功能已經很好了。開啟備忘錄點相機圖示選” 掃描文稿”，對準檔案它會自動識別邊框並矯正透視。拍完之後可以調顏色——選” 黑白” 模式出來的效果最接近掃描器，” 灰度” 適合有圖片的資料。

- **安卓使用者**：很多品牌的相機自帶” 文件掃描” 模式。如果找不到，去應用商店下載 Microsoft Lens（免費，而且跟 OneDrive 打通）。

- **OCR 文字識別**：拍完的掃描件，如果需要把上面的文字提取出來編輯，就需要 OCR 了。微信自帶的” 提取文字” 功能最簡單——長按圖片就能用，識別準確率還不錯。如果大量處理，可以用白描 App 或者全能掃描王。對於中文識別，百度的 OCR 引擎效果比較好；WPS 的圖片轉文字功能也很準。

- **掃描小技巧**：放在光線充足的地方拍，最好有檯燈從側面打光，避免在文件上留下你的手影。如果檔案折角了，先用手壓平再拍。多頁檔案可以連續掃描後合併成一個 PDF，不用一頁頁單獨存。

四、網盤怎麼選：沒有最好的，只有最適合你的

選網盤這件事我琢磨了很久，最後發現其實是看你的“主力生態”是什麼。

- **蘋果全家桶 iCloud**：無縫同步到讓你感覺不到它的存在。你在手機上拍的照片，幾分鐘後電腦上就自動有了。而且蘋果的隱私保護做得最嚴格。但缺點也很明顯：跨平臺分享給安卓或 Windows 使用者比較麻煩，免費只有 5GB，擴容要花錢。

- **Windows 主力 OneDrive**：如果你用的是 Windows 電腦 +Office 365，OneDrive 就是天生一對。Word 裡儲存檔案，自動就存到雲上了。1TB 空間配合 Office 訂閱很劃算。手機端 App 的自動備份照片功能也很好用。
- **國內分享百度網盤**：說實話，百度網盤的下載限速一直被批評，但它的分享生態確實無人能敵。中文網際網路上大量的學習資料、影視資源都是用百度網盤分享的。如果只是偶爾下載別人分享的檔案，不充會員也能湊合用。如果你是重度的資源收集者，會員該開就開吧，省下的時間值錢多了。
- **阿里雲盤/夸克網盤**：這兩年是後起之秀。阿里雲盤上傳下載不限速（目前是），夸克網盤跟夸克瀏覽器深度繫結，搜尋資料的時候可以直接儲存到網盤，很方便。

我的組合方案：iCloud 管個人照片和日常檔案（隱私優先），百度網盤存資源和接收分享（生態優先），OneDrive 管工作文件（Office 整合優先）。三個網盤各司其職，互不干擾。

五、一個真實的移動辦公場景

說個我自己的真實經歷吧。有一次出差到外地，客戶臨時要改合同條款。我的工作流是這樣的：

1. 在酒店收到客戶微信發來的修改意見（文字訊息 + 一張手寫批註的照片）。
2. 開啟 OneDrive 找到合同原件，用 WPS 開啟編輯，參照客戶的微信訊息逐條修改條款。
3. 改完後用 WPS 的” 輸出為 PDF” 功能匯出，發到客戶微信群裡確認。
4. 客戶提了第二輪意見，我用 WPS 的畫筆工具直接在 PDF 上圈出修改位置，配合語音批註說清楚改動思路。
5. 確認無誤後，用 WPS 的” 電子簽名” 功能線上簽署，發給對方同樣簽署。
6. 最後把簽好的合同 PDF 存檔到百度網盤和 iCloud 各一份，同時在 Notion 的工作日誌裡記了一筆。

整個過程從開始到結束大概四十分鐘，全程沒開電腦。說實話，三年前我根本不信手機能幹這些事，但現在確實可以。關鍵是提前熟悉這些工具，別等到需要的時候才手忙腳亂地下載註冊。

來試試：搭建你的移動辦公工作流

1. 選定你的主力文件 App（WPS 或 Microsoft 365），登入並關聯雲盤。
2. 開啟備忘錄或便籤，用掃描功能拍一份文件，匯出為 PDF 發給自己。
3. 選一個網盤 App，設定為手機照片自動備份（選” 僅 WiFi 下備份” 省流量）。
4. 在 Notion 或飛書裡新建一頁，試著搭建一個簡單的” 工作日誌” 模板。

第十章：移動支付與生活服務

說實話，我爸媽那一輩人對“手機付錢”這件事，一開始是抗拒的。“錢在手機裡，被人偷了怎麼辦?”這個擔心很正常，但現實是：你褲兜裡揣著的現金，丟的機率比手機支付被盜刷的機率高得多。手機支付不是把錢存進手機，它只是一把鑰匙，一把開啟你銀行賬戶的鑰匙。

一、到底怎麼繫結銀行卡？

不管你是用支付寶還是微信，第一次綁卡都要走這幾步：

1. 開啟 App，找到” 我的” 或” 錢包”。點” 新增銀行卡”。
2. 輸入卡號。現在大部分 App 都支援直接拍照識別卡號，不用一個個數字敲。
3. 銀行會發一條驗證碼簡訊到你預留的手機號上。填進去，設定一個 6 位數字的支付密碼。
4. 這個支付密碼和你的銀行卡密碼、鎖屏密碼，最好是三個不同的密碼，別圖省事設成一樣的。

有個小竅門：如果擔心安全問題，可以去銀行 App 裡開一張 II 類或 III 類電子賬戶，專門用來繫結手機支付。這張卡裡只放平時零花的錢，就算出了事，損失也有限。

二、付款碼是“誰看誰收錢”

很多人搞不清楚付款碼和收款碼的區別，這個區別很重要。

- **付款碼**：你展示給收銀員掃的那個條形碼 + 二維碼。這個碼相當於你的” 電子錢包”，誰掃了誰就能從你賬上劃走錢。所以**千萬別截圖發給任何人**。這個碼每分鐘自動重新整理一次，就是為了防止被盜用。
- **收款碼**：別人掃了這個碼給你轉賬。這個碼可以截圖儲存，可以發給別人，安全。

認準這個區別，能避開很多詐騙陷阱。凡是有人讓你“把付款碼截圖發過來”的，百分之百是騙子。

三、這些安全設定，花 5 分鐘設好能省很多心

- **指紋/面容支付**：在支付寶和微信的支付設定裡，把生物支付開啟。這樣每次付款要按指紋或掃臉，就算手機丟了，撿到的人也沒法花你的錢。

- **安全鎖**：支付寶和微信都有” 安全鎖” 功能。開啟之後，進入 App 的” 錢包” 頁面需要額外驗證一次身份。路徑：支付寶我的設定賬號與安全安全中心安全鎖。

- **免密支付檢查**：很多人簽約了各種免密支付，自己都忘了。去支付寶和微信的” 支付設定” 裡，找到” 免密支付/自動扣款”，看看簽約了哪些服務，把不需要的關掉。

- **登入裝置管理**：定期看看哪些裝置登入過你的賬號。如果發現不認識的裝置，立刻踢掉。路徑：支付寶設定賬號與安全登入裝置管理。

四、公交地鐵：手機就是交通卡

現在出門坐公交地鐵，已經很少人用實體的交通卡了。手機刷閘機，有兩條路可以走：

- **手機錢包裡的虛擬交通卡**：在蘋果錢包、華為錢包、小米錢包裡直接開通一張虛擬交通卡。這個方案的好處是：刷的時候不用解鎖手機，直接靠上去就行，響應速度最快，而且沒網也能刷。部分城市的虛擬交通卡還有和實體卡一樣的換乘優惠。

- **支付寶/微信的乘車碼**：在 App 裡搜尋” 乘車碼”，開通你所在城市的公交或地鐵乘車碼。這個方案的好處是：不用充值，直接從餘額或繫結的銀行卡裡扣。但需要聯網，而且刷之前要開啟 App 找到碼。

我個人更推薦用手機錢包的虛擬交通卡。雖然第一次要充個 10 塊 20 塊進去，但用起來真的方便太多，尤其是趕地鐵的時候，不用停下來解鎖手機找 App。

五、生活繳費、外賣、打車

- **水電氣繳費**：在支付寶或微信的” 生活繳費” 裡，把你家的水、電、燃氣戶號繫結好。以後每個月收到賬單，直接點進去就能交。戶號一般在你家的紙質賬單上能找到，或者打客服電話問。

- **外賣比價**：同一家店在美團和餓了麼上的價格和優惠經常不一樣。下單前兩邊都看一眼，有時候能省好幾塊。另外，美團那個” 拼好飯” 功能，和朋友一起點，價格能便宜不少。

- **打車安全**：在滴滴或者高德打車裡，記得把” 緊急聯絡人” 和” 行程分享” 設定好。打車的時候，App 會自動把你的行程分享給你設定的聯絡人。另外，深夜打車優先選” 計程車” 而不是” 快車”，計程車有行業監管，安全係數更高一些。

六、用手機搞定醫保、社保、政務

這幾年的變化很大，以前要跑大廳排隊的很多事，現在手機上幾分鐘就辦完了。

- **醫保電子憑證**：在支付寶或微信裡搜” 醫保電子憑證”，啟用之後，去醫院看病、藥店買藥直接刷手機就行，不用帶實體醫保卡。第一次需要人臉識別驗證，之後每次用的時候會生成一個動態碼。
- **社保查詢**：下載” 掌上 12333”App 或者用支付寶裡的” 市民中心”，可以查社保繳納記錄、養老金賬戶餘額、失業金申領等。
- **個稅**：每年 3 月到 6 月的個稅彙算清繳，在” 個人所得稅”App 上就能搞定。記得看看有沒有可以申報的專項附加扣除（子女教育、房貸利息、贍養老人等），這些都能幫你少交稅或者退稅。

記住這幾條，防騙保平安

- 公檢法、銀行、客服，不會讓你轉賬到” 安全賬戶”。
- 任何人讓你把付款碼截圖發過去，都是騙子。
- 陌生連結別點，尤其是”ETC 過期”” 社保卡停用” 這類簡訊。
- 網購退款會原路返回，不需要你提供銀行卡號。

現在就做：給你的支付 App 做一次安全體檢

1. 開啟支付寶和微信，確認已開啟指紋/面容支付。
2. 找到” 安全鎖” 功能並開啟。
3. 檢查” 免密支付/自動扣款” 列表，關掉不再需要的服務。
4. 檢視” 登入裝置” 列表，移除不認識的裝置。
5. 啟用你的醫保電子憑證，下次去醫院就不用帶實體卡了。

第十一章：攝影與圖片管理

說句實話，大多數人用手機拍照，拍出來的效果遠遠不如手機本身的潛力。不是你技術不行，而是很多小設定和小習慣你沒注意到。這一章不講虛的，就說那些你能馬上用上的東西。

一、你是不是也犯過這些錯？

先說說最常見的幾個問題，你看看中了幾條：

- **鏡頭是髒的**：這個真的是最普遍的問題。掏手機的時候指紋按在鏡頭上，拍出來霧濛濛的，還以為手機壞了。養成習慣，拍照前用衣角擦一下鏡頭。
- **從來不調曝光**：預設模式下，點一下螢幕是對焦，但如果你在點完之後上下滑動，是可以調節亮度的。很多人拍的照片要麼太暗要麼太亮，就是不知道這個操作。
- **閃光燈永遠開著**：手機 LED 閃光燈拍出來的效果真的很醜，光線又硬又冷，把人拍得像鬼。除非真的伸手不見五指，否則關掉它。暗光環境下，用夜景模式遠比閃光燈效果好。
- **放大靠兩根手指拉伸**：在螢幕上雙指縮放是數碼變焦，本質就是裁切畫面然後放大，畫質會明顯下降。如果想拍遠處的，直接用取景框上的變焦按鈕（0.5x/1x/2x/3x）。其中 0.5x（超廣角）和 1x（主攝）是原生光學鏡頭，不損失畫質；2x/3x 則多為" 光學品質變焦"——透過高畫素感測器裁切實現，畫質接近光學變焦但並非獨立鏡頭。超過 3x 的變焦通常就是混合變焦或數碼變焦了，畫質會有明顯下降。

二、幾個立刻提升出片率的小技巧

這些不需要什麼理論知識，用一次就能看到效果：

- **三分法構圖 + 網格線**：在相機設定裡開啟" 網格線"（也叫參考線）。拍照時把你想突出的東西放在四條線的交叉點上，畫面立刻就會" 對" 很多。這是攝影裡最簡單也最有效的一個規則，沒什麼玄學，就是符合人眼的視覺習慣。
- **找一找" 引導線"**：拍風景或者街景的時候，留意一下畫面裡有沒有自然的線條——馬路、欄杆、牆壁、一行樹——讓這些線條從畫面邊緣延伸到主體。這個技巧用好了，路人看你拍的照片會以為你學過攝影。

- **別老站著拍**：大部分人拍照永遠是自己眼睛的高度，所以拍出來的視角千篇一律。試試蹲下來從低角度拍，或者找個高處往下拍，同一個地方出來的感覺完全不一樣。拍小孩和寵物尤其需要蹲下來，平視才能拍出自然的感覺。

- **光線是第一要素**：中午大太陽下拍人像，臉上的陰影又硬又醜。最好的光線是清晨和傍晚的" 黃金時段"，陽光是斜的、顏色偏暖，拍什麼都好看。陰天的光線雖然不出彩，但勝在均勻柔和，拍人像反而比大太陽下容易出片。

- **夜景模式使用要領**：開啟夜景模式之後，手機會自動連拍多張然後合成。這時候最關鍵的是——手絕對不能抖。能靠牆就靠牆，能放桌上就放桌上，實在不行就屏住呼吸。夜景模式需要你穩住 2 到 4 秒，抖了畫面就糊了。

三、人像模式與專業模式

- **人像模式**：它能拍出背景模糊、主體清晰的效果（淺景深）。手機因為鏡頭物理尺寸小，真實虛化能力有限，所以靠的是 AI 演算法來實現——簡單說就是 AI 把你的人摳出來，然後模糊背景。用的時候保持 1-2 米距離效果最好，離太遠或太近演算法都容易出錯。還有，拍人像的時候，注意背景裡不要有太複雜的線條，否則 AI 扣圖會在頭髮邊緣出現" 翻車"。

- **專業模式**：如果你想要完全掌控畫面效果，可以試試專業模式。幾個關鍵引數要知道：ISO 越低畫質越乾淨，但畫面會暗；快門越慢進光越多，但手抖會糊；白平衡管色彩的冷暖，調不好照片會偏藍或偏黃。一般人不需要常年用專業模式，但瞭解這幾個引數至少能讓你知道相機在幹什麼。

四、後期修圖：不要過度，恰到好處就行

很多人對修圖有誤解，覺得修了就不真實了。其實相機拍出來的照片本來就和肉眼看到的不一樣，適當調整是為了還原你當時看到的畫面，而不是造假。

- **Snapseed**：谷歌出的免費修圖工具，功能專業但沒有廣告。我日常修圖的流程是：先用" 調整圖片" 拉一拉亮度和對比度用" 突出細節" 銳化一下用" 白平衡" 糾正偏色最後" 裁剪" 成想要的構圖。Snapseed 的" 區域性" 功能很好用，可以只調畫面中某一個區域，不用全圖一起動。

- **醒圖/美圖秀秀**：這兩個適合修人像。但我覺得有個原則：美顏濾鏡拉到 30% 以內就好，超過這個數就假了，皮膚像塑膠一樣。自然比完美好看。

- **Lightroom 手機版**：如果你對色彩有追求，這個是最專業的。它有曲線調整、HSL 分色調節這些高階功能。不過對大多數人來說有點殺雞用牛刀。

- **修圖工作流建議**：我自己的習慣是先拍好原片，回家後在電腦螢幕或 iPad 上慢慢修，手機螢幕太小，色彩也不太準。如果必須在手機上修，記得把螢幕亮度調到中等偏上，別在省電模式下修圖。

五、照片太多怎麼辦？整理策略

手機相簿裡堆了幾千張照片，想找一張特定的翻半天——這個體驗大家都有。幾個實用的整理方法：

- **善用搜尋**：現在 iPhone 和大部分安卓手機的相簿都支援智慧搜尋。你可以搜”貓””海邊””身份證””花”這些關鍵詞，系統會用 AI 識別畫面內容幫你找出來。還可以搜日期和地點，比如”去年五月””北京”。這個功能知道的人不多，但真的很好用。

- **定期清理垃圾**：截圖、重複的照片、拍糊了的廢片，這些才是佔空間的元兇。iOS 相簿有”重複專案”自動合併功能；安卓可以去檔案管理裡搜”screenshots”資料夾，批次刪不需要的截圖。我每個月花五分鐘清理，就能騰出好幾個 G。

- **建立相簿，別全靠預設排序**：系統預設按時間排序沒錯，但對於”孩子的成長照””旅行照片””重要證件”這種跨時間段的內容，你應該手動建相簿歸類。蘋果的”共享相簿”還能跟家人一起往裡面加照片，一個人建，全家都能看。

- **備份是底線**：照片是最不可替代的資料——檔案丟了可以重建，聊天記錄沒了也就沒了，但孩子的照片、老人的影片丟了就真的沒了。一定要開雲備份。iCloud 照片最省心，安卓這邊各品牌都有自己的雲服務，或者用百度網盤/阿里雲盤自動備份。原則是：至少有一份雲備份，別把雞蛋放在同一個籃子裡。

六、截圖與錄屏

- **截圖**：同時按電源鍵 + 音量加鍵，所有手機通用的方法。很多安卓手機還支援三指下滑截圖、指關節雙擊截圖。長截圖（滾動截圖）在安卓上很成熟——截完一張後點”滾動”按鈕，手機會自動往下翻頁並拼接成一張長圖，適合截聊天記錄和網頁。

- **錄屏**：從控制中心新增”螢幕錄製”，點一下開始錄，再點停止。錄之前長按錄製按鈕可以選擇是否錄製麥克風聲音——你要做教程解說就開麥，只是錄操作畫面就關掉。錄完的影片會自動存到相簿裡。

練一練：用手機完成一次完整的拍照 + 修圖 + 整理

1. 開啟相機設定裡的”網格線”，找一個有自然光的地方拍三張照片（注意擦鏡頭、調曝光）。
2. 選最好的一張，用 Snapseed 做基礎調整（亮度、對比度、裁剪）。
3. 給這張照片建一個專屬相簿，命名為”我的作品”。
4. 檢查一下雲備份有沒有開啟，確保照片已經在雲端存了一份。

第十一章附：專業影像技術知識

想要真正理解手機攝影和影片的奧秘，你需要了解一些專業的影像技術知識。這些知識不僅能幫你更好地使用手機相機，還能讓你在選擇裝置時更有判斷力。

一、解析度、幀率與位元速率：影片質量的三要素

解析度（Resolution）：

解析度決定了畫面的清晰度，用畫素數表示。

- **720P (HD)**：1280×720 畫素，入門級清晰度。
- **1080P (Full HD)**：1920×1080 畫素，目前最主流的解析度。
- **2K (QHD)**：2560×1440 畫素，比 1080P 更清晰。
- **4K (UHD)**：3840×2160 畫素，超高畫質，細節豐富。
- **8K**：7680×4320 畫素，未來標準，目前裝置支援有限。

解析度越高，畫面越清晰，但檔案也越大，對儲存和處理能力要求越高。

幀率（Frame Rate）：

幀率決定了畫面的流暢度，用每秒幀數（fps）表示。

- **24fps**：電影標準幀率，有獨特的”電影感”。
- **30fps**：電視和網路影片常用，流暢自然。
- **60fps**：高幀率，適合運動場景和遊戲錄製，非常流暢。
- **120fps/240fps**：超高幀率，可用於慢動作影片。

幀率越高，畫面越流暢，慢動作效果越好，但檔案也越大。

位元速率（Bitrate）：

位元速率決定了影片的資料密度，用每秒資料量（Mbps）表示。

- 位元速率越高，畫質越好，細節保留越多，但檔案也越大。
- 位元速率過低會導致畫面出現馬賽克、色塊等壓縮痕跡。
- 專業影片拍攝通常需要高位元速率以保留更多後期處理空間。

二、影片編碼格式：H.264、H.265 與 ProRes

H.264 (AVC)：

- **特點：**最廣泛使用的編碼格式，相容性極好。
- **優點：**幾乎所有裝置都能播放，壓縮效率適中。
- **缺點：**相比新格式，壓縮效率較低，檔案較大。
- **適用場景：**網路影片分享、普通使用者日常拍攝。

H.265 (HEVC)：

- **特點：**H.264 的升級版，壓縮效率提升約 50%。
- **優點：**相同畫質下檔案更小，支援更高解析度和幀率。
- **缺點：**編碼和解碼需要更強的處理器，部分老裝置不相容。
- **適用場景：**4K 影片拍攝、儲存空間有限的場景。

Apple ProRes：

- **特點：**蘋果開發的專業影片編碼，幾乎無失真壓縮。
- **優點：**畫質極高，後期處理空間大，色彩準確。
- **缺點：**檔案極大，需要大量儲存空間，處理需要專業裝置。
- **適用場景：**專業影片製作、電影級拍攝。

ProRes RAW：

- **特點：**結合了 ProRes 和 RAW 格式的優勢。
- **RAW 是什麼：**未經處理的原始感測器資料，保留最大資訊量。
- **優點：**後期調整空間極大，可以改變曝光、白平衡等引數而不損失畫質。
- **缺點：**檔案極其巨大，需要專業後期軟體處理。
- **適用場景：**高階專業製作。iPhone 15 Pro 系列及以後支援拍攝 Apple ProRes 影片（最高 4K 60fps），iPhone 17 Pro 進一步支援 ProRes RAW 直接機內錄製。

三、HDR：高動態範圍技術

HDR（High Dynamic Range）技術可以同時保留畫面中最亮和最暗部分的細節。

- **HDR10**：最基礎的 HDR 標準，相容性好。
- **HDR10+**：三星推動的標準，支援動態後設資料。
- **Dolby Vision**：杜比開發的頂級 HDR 標準，畫質最佳，但需要授權。
- **HLG**：廣播級 HDR 標準，相容普通顯示器。

拍攝 HDR 影片時，需要支援 HDR 的顯示器才能看到完整效果。

四、色彩空間與色深

色彩空間：

- **sRGB**：標準色彩空間，相容性最好。
- **Display P3**：蘋果推廣的廣色域標準，色彩更豐富。
- **Rec.2020**：HDR 影片的標準色彩空間，色域最廣。

色深：

- **8-bit**：1670 萬色，普通影片標準。
- **10-bit**：10.7 億色，專業影片標準，色彩過渡更平滑。
- **12-bit**：680 億色，高階專業標準。

實用建議

- 日常拍攝：1080P/4K + 30fps + H.265 編碼，平衡畫質和檔案大小。
- 運動拍攝：1080P/4K + 60fps，保證流暢度。
- 專業創作：4K + ProRes（如果支援），保留最大後期空間。
- 儲存建議：高位元速率影片很佔空間，建議定期備份到電腦或雲盤。

去探索一下：你的手機影片設定

1. 開啟相機 App，切換到影片模式。
2. 檢視解析度和幀率選項，嘗試不同的組合。
3. 如果支援，檢視是否有 HDR、ProRes 等高階選項。
4. 拍攝幾段不同設定的影片，對比畫質和檔案大小。

五、RAW 格式與 HEIF/JPEG 對比

什麼是 RAW 格式：

RAW 格式是相機感測器記錄的“原始資料”——未經任何壓縮和處理的純原始資訊。它就像生鮮食材，需要後期“烹飪”才能成品。手機因為儲存和算力有限，預設拍出來的都是已經“烹飪好”的 JPEG 或 HEIF，RAW 需要專門開啟。

RAW vs HEIF vs JPEG 對比：

- **JPEG**：最老的格式，相容性最好（任何裝置都能開啟）。8-bit 色深，有失真壓縮，檔案最小（一張照片 2-5MB）。後期空間有限，強行提亮暗部容易出現色塊。
- **HEIF/HEIC**：蘋果主推的新格式，同樣畫質下檔案只有 JPEG 的一半（一張照片 1-3MB）。支援 10-bit 色深和透明通道。缺點是部分老裝置和 Windows 需要安裝外掛才能開啟。
- **RAW**：未經壓縮的原始資料，檔案最大（一張照片 20-100MB）。色深通常 12-16bit，後期空間最大。缺點是必須用專業軟體（Lightroom、Snapseed 等）才能開啟和處理，無法直接分享。

為什麼專業攝影師要拍 RAW：

- **白平衡可後期無損調整**：JPEG 拍錯了白平衡（比如白熾燈下偏黃），後期糾正會損失畫質；RAW 可以重新設定色溫，完全無損。
- **曝光容差大**：JPEG 欠曝 2 檔的暗部基本是死黑一片，提亮後全是噪點；RAW 能從欠曝 3-4 檔的暗部” 救” 出可用細節。過曝同理。
- **暗部和高光細節更豐富**：RAW 的動態範圍遠大於 JPEG，亮部不過曝、暗部不死黑的餘地大得多。
- **銳化、降噪、鏡頭矯正都在後期可控**：相機的機內處理是” 一錘子買賣”，RAW 可以反覆調整，每次都從原始資料出發。

手機如何開啟 RAW：

- **iPhone ProRAW**：僅 iPhone 12 Pro 及以上 Pro 機型支援。開啟” 設定相機格式” 中開啟”Apple ProRAW”。然後在相機介面頂部會出現一個”RAW” 按鈕，點亮即可。檔案約 25MB/張，建議有需求時再開。
- **安卓 Motion Raw / Ultra RAW**：各品牌叫法不同。小米/OPPO/vivo 旗艦機型在相機” 專業模式” 或” 更多” 裡能找到 RAW 開關；華為在” 專業模式” 中開啟 RAW；三星 Galaxy S 系列在” 專業” 模式裡有 RAW 選項。檔案大小通常在 20-50MB。
- **通用建議**：日常隨手拍沒必要開 RAW，檔案太大佔空間；只有在重要場景（風光、人像、需要後期）時才開啟。

六、計算攝影原理

現代手機攝影已經離不開“計算攝影”——也就是靠演算法而不僅是鏡頭和感測器來生成照片。理解這幾個核心原理，能讓你更明白手機在按下快門的瞬間做了什麼。

多幀合成：

手機按下一次快門，其實拍了多張照片。演算法把多幀對齊、加權融合，得到一張高質量成片。

- **夜景模式**：1-3 秒內連拍多張欠曝照片，對齊後疊加——噪點互相抵消、亮度累加。結果就是暗光也能拍出乾淨的畫面。
- **HDR（高動態範圍）**：分別拍攝欠曝、正常、過曝三張（或多張），合成一張亮部和暗部都有細節的照片。逆光場景尤其有效。
- **Deep Fusion（蘋果）/Photonic Engine（光子引擎）**：針對中低光場景，逐畫素分析多幀並融合，重點提升紋理和細節，毛衣、毛髮這類細節會明顯更清晰。

語義分割：

AI 不只是把整張圖一刀切處理，而是先“看懂”畫面裡有什麼——天空、人臉、草地、建築、衣服——然後對每個區域分別應用不同的處理策略。

- **天空**：單獨增強藍色、壓低亮度，避免過曝。
- **人臉**：識別膚色區域，單獨提亮、平滑膚色，不動五官和頭髮細節。
- **草地/樹葉**：增強綠色飽和度，提升細節銳化。
- **背景虛化**：人像模式下，AI 識別主體輪廓，背景模糊、主體清晰。

深度學習的應用：

- **AI 場景識別**：相機自動識別” 美食”” 寵物”” 夜景”” 文件” 等場景，並對應最佳化引數。比如美食模式會增強暖色調和飽和度，讓食物看起來更誘人。
- **AI 人像虛化**：手機鏡頭物理上無法做到單反那種淺景深，靠深度學習模型預測主體邊緣，再對背景做模糊。最新演算法已經能正確處理頭髮絲、玻璃邊緣這些細節。
- **AI 消除/AI 擴圖**：選中畫面中的路人或雜物，AI 基於上下文” 補全” 被遮擋的部分。這是生成式 AI 在影像領域的應用。

計算攝影 vs 光學攝影：

- **計算攝影的優勢**：小體積也能得到好畫質，暗光、逆光等極端場景也能出片，普通使用者不需要懂攝影知識就能拍出” 好看” 的照片。

- **計算攝影的劣勢**：畫面有時” 過度處理”——銳化過度、色彩失真、天空顏色不自然；多幀合成在主體移動時會留下重影；細節是” 算出來的” 而非真實記錄的。
- **光學攝影的優勢**：大底感測器 + 大光圈鏡頭能真實捕捉光線，畫質自然，後期空間大；不會因為演算法” 腦補” 產生偽影。
- **趨勢**：現代旗艦手機是” 光學 + 計算” 結合——用大底感測器獲取真實資訊，用演算法最佳化呈現。兩者並不對立。

七、鏡頭型別與焦段

現代旗艦手機通常配三到四顆鏡頭，每顆對應不同焦段。理解焦段，能讓你在不同場景下選對鏡頭。

常見鏡頭與等效焦距：

- **超廣角（13-24mm 等效）**：視野最廣，適合風光、建築、室內空間。能拍到眼前所見的大場面，但邊緣會有畸變（直線變彎），拍人像時人臉會被拉伸。適合” 大場景”，不適合” 特寫”。
- **主攝（24-28mm 等效）**：日常最常用的鏡頭，畫質最好、光圈最大。兼顧視野和畫質，拍人拍景都能用。大多數手機的”1x” 就是主攝。
- **長焦（70-200mm 等效）**：用於拍遠景、人像特寫。長焦能壓縮空間感（遠處的山和近處的人看起來更近），拍人像能獲得更自然的五官比例和背景虛化。
- **潛望式長焦（200mm+ 等效）**：透過反射式光路設計，在手機厚度內實現更長焦距。能在很遠距離拍清楚細節，適合演唱會、體育賽事、野生動物。旗艦機型才有。

感測器尺寸與畫素大小的權衡：

感測器是“底”，決定能接收多少光線。行業裡說“底大一級壓死人”——同樣畫素數下，感測器越大，單個畫素也越大，進光量越多，畫質越好。

- **大底感測器**：進光量大，夜景畫質好，動態範圍寬。缺點是手機要做厚（鏡頭模組凸起），成本高。
- **高畫素小底**：白天光線充足時，高畫素能拍出更清晰的照片，方便後期裁切。但暗光下每個畫素進光不足，噪點多。
- **主流旗艦選擇**：主攝多為 1/1.3 英寸到 1 英寸（部分機型已用上 1 英寸大底），畫素數 5000 萬左右——這是畫質和體積的平衡點。

畫素融合技術（Pixel Binning）：

為了兼顧白天高畫素和夜晚大畫素，手機廠商用“畫素融合”——把多個小畫素合併成一個大畫素輸出。

- **四合一畫素（Quad Bayer）**：5000 萬畫素感測器，輸出 1250 萬畫素照片。每個輸出畫素由 4 個相鄰畫素合成，進光量提升 4 倍，畫質更乾淨。
- **九合一畫素（Nonacell）**：用於 1 億畫素以上的感測器，輸出約 1100-1200 萬畫素。同樣原理，進光量更大。
- **實際效果**：預設拍照模式下，你拿到的就是融合後的低畫素照片——畫質好、檔案小。需要極致細節時（比如拍文件、遠景），可以手動切到" 高畫素模式" 輸出全解析度。

八、特殊拍攝模式

除了日常拍照，手機裡還有幾個特殊模式，用對了能拍到平時拍不出的畫面。

全景模式：

- 拍攝時按提示箭頭緩慢平穩地橫向移動手機，不要快——快了會出現拼接錯位。
- 保持手機垂直方向穩定，只橫向平移。如果上下晃動，地平線會變波浪。
- 有移動的人或車時慎用——拼接時會出現" 半個身體" 或" 鬼影"。
- 全景模式也可以豎著拍——豎拍橫拼接，適合拍高聳的建築或瀑布。

慢動作與延時攝影：

- **慢動作**：用 120fps 或 240fps 高幀率拍攝，回放時變成慢鏡頭。適合拍水滴、運動瞬間、寵物跳躍。光線要充足，慢動作模式下進光時間短，暗光畫質差。
- **延時攝影**：間隔拍攝多張照片合成快進影片。適合拍車流、雲彩移動、日落、花朵開放。拍攝時長幾分鐘到幾小時不等，務必用三腳架固定手機，手持必糊。
- **小技巧**：延時攝影拍雲彩時，把手機架好，拍 15-30 分鐘，最後得到 10 秒左右的延時影片，效果震撼。

星空/天文模式：

- 需要遠離城市光汙染，找完全沒有路燈干擾的郊外或山區。城市裡拍不出銀河。
- 必須用三腳架，手持必糊。手機要穩穩架住幾秒到幾十秒。
- 天氣要晴朗無雲，月亮不要太大（滿月會蓋住星光）。農曆月初或月末的暗夜最適合。
- iPhone 的" 夜間模式" 會自動識別星空並延長曝光時間（最長 30 秒）；安卓旗艦的" 星空模式" 或" 天文模式" 效果類似。要耐心等待，按下快門後不要動手機。

微距模式：

- 用於拍攝花朵、昆蟲、珠寶、文字特寫等小物體。能貼近到 1-2 釐米距離對焦。
- 部分手機有專門的微距鏡頭（多為 200 萬畫素或 500 萬畫素的小底鏡頭），畫質一般；旗艦機用超廣角鏡頭兼顧微距，畫質更好。
- 貼近拍攝時鏡頭離主體很近，會遮擋光線，必要時用檯燈或手電補光。
- 對焦距離極近，景深很淺，主體前後移動幾毫米就可能脫焦。拍攝時手要穩，最好讓主體保持不動。

靜態攝影實用建議

- 日常拍攝：HEIF/JPEG 預設即可，檔案小、畫質夠用。
- 重要場景（風光、人像、需要後期）：開啟 RAW，保留最大後期空間。
- 選對鏡頭：風光用超廣角，人像用長焦，日常用主攝。
- 夜景和星空：務必固定手機，開夜景模式或天文模式。
- 大底手機（如 1 英寸感測器機型）畫質上限更高，但操作習慣仍是關鍵。

動手試試：摸清你的相機能力

1. 檢視你的手機是否支援 RAW 拍攝（設定相機格式），試著開啟並拍攝一張，對比同一場景的 JPEG 和 RAW 檔案大小。
2. 切換到超廣角、主攝、長焦三個焦段，分別拍同一處遠景，體會視角差異和畫質差異。
3. 找一個有夜景模式或星空模式的場景，固定手機後拍攝一張夜景照片。
4. 嘗試用全景模式橫拍一張大場景，注意保持手機平穩移動。

第十二章：影音娛樂與閱讀

手機現在基本上已經取代了 *MP3*、電視遙控器、*Kindle* 和掌上遊戲機。但說實話，很多人只是用了這些 *App* 最基本的功能，還有很多好用的地方沒發現。這一章我們就來聊聊怎麼讓手機娛樂更順手、更舒服。

一、聽音樂：選哪個 App，怎麼聽才舒服

音樂 App 現在功能都差不多，但各有各的側重點，選對了能省不少心：

- **網易雲音樂**：它的強項是歌單社群和評論區。你想聽" 適合下雨天聽的歌" 或者" 學累了放鬆用的純音樂"，搜一下就有別人整理好的歌單。評論區有時候比歌還精彩。缺點是部分歌曲版權不全，有些歌聽著聽著就灰了。
- **QQ 音樂**：版權最全，周杰倫、五月天這些熱門歌手的基本上只有 QQ 音樂有。音質選項也很豐富，會員可以開無損音質。如果你對" 想聽的歌都能搜到" 這件事很在意，QQ 音樂是最穩妥的選擇。
- **Apple Music**：蘋果使用者的首選。介面乾淨、沒有廣告、音質好，而且支援無損和杜比全景聲。但它的推薦演算法不如國內 App 懂中文使用者的喜好，歌單文化也沒有網易雲那麼發達。適合喜歡簡潔體驗、不太在乎社交功能的人。
- **Spotify**：全球使用者最多，推薦演算法公認最強。" 每週發現" 和" 日推" 準確率高得嚇人，用越久越懂你。缺點是免費版廣告多，而且某些中文歌版權覆蓋不全。需要特殊網路才能用。

幾個聽歌小技巧：如果你用的是藍芽耳機，去音樂 App 設定裡看看有沒有“音效調節”或“均衡器”，稍微調一下低音和高音，效果提升明顯。另外，流媒體聽歌很費流量，記得在設定裡把“僅 WiFi 下播放無損”開啟，或者提前下載好喜歡的歌單離線聽。

二、播客（Podcast）：通勤路上的“耳朵伴侶”

播客這個東西，沒聽過的人覺得跟自己無關，一旦開始聽就回不去了。本質就是“電臺節目的 Netflix 版”——有人聊科技，有人講歷史，有人讀故事，你想聽什麼型別的都有。

- **小宇宙 App**：國內做得最好的播客 App，介面清爽，沒有亂七八糟的功能。它的” 每日推薦” 和” 編輯精選” 幫你發現新節目，評論區也很活躍。我個人聽的最多的是《忽左忽右》（文化歷史類）和《不合時宜》（社會議題類），每期都像在旁聽幾個聰明人聊天。

- **蘋果播客**：iPhone 自帶，不需要額外下載 App。和 CarPlay 的整合最好，開車的時候很方便。

- **收聽技巧**：大部分播客 App 都支援 1.2 倍速或 1.5 倍速播放，習慣了之後能省不少時間。我一般 1.2 倍聽中文，1.5 倍聽英文。另外一定要開” 自動下載新節目” 和” 播完自動刪除”，這樣你的手機不會堆滿聽過的音訊檔案。

三、看影片：從播放器到流媒體的正確開啟方式

很多人不知道，手機上可以播的格式遠比流媒體 App 裡提供的多。一個好的本地播放器能解決很多問題。

- **VLC 播放器**：開源免費，幾乎什麼格式都能播——MKV、AVI、FLAC、甚至是 ISO 光碟映象。如果你下載了一個影片在電腦上能播但手機上播不了，八成是格式問題，裝個 VLC 就解決了。它還支援外掛字幕，把.srt 字幕檔案和影片放在同一個資料夾，VLC 會自動載入。

- **nPlayer**：如果你想要更精緻的體驗，這個付費 App 值得買。介面好看，手勢控制很順手——左邊上下滑調亮度，右邊上下滑調音量，左右滑快轉倒轉。支援 DTS 和杜比音效，看大片的時候臨場感強很多。

- **Bilibili（B 站）**：已經不只是二次元網站了，現在上面的學習資源多到離譜。考研、程式設計、攝影、烹飪，什麼教程都有。彈幕文化是 B 站的靈魂，看影片的時候開著彈幕，感覺像有一群人在陪你一起看。不過看學習影片的時候建議關彈幕，不然容易分心。

- **投屏到電視**：手機螢幕再大也不如電視看著舒服。同一個 WiFi 下，開啟影片 App 右上角的”TV” 投屏按鈕，就能把畫面投射到電視上。大部分智慧電視和投影儀都支援，看劇看電影體驗好太多。iPhone 使用者還可以用 AirPlay 直接映象整個螢幕。

四、電子書閱讀：手機上也能好好看書

有人說手機上讀書傷眼睛、容易分心，這些確實有道理。但手機閱讀的優勢也很明顯——隨時隨地，碎片時間利用最大化。我在地鐵上、排隊的時候、睡前，用手機讀了不少書。

- **微信讀書**：目前國內體驗最好的閱讀 App。書庫很全，出版書和網文都有。它用” 閱讀時長換書幣” 的模式，看得多就能免費換書，對學生黨很友好。它的” 劃線” 功能可以看別人在哪些段落做了標記，有時候看到別人標註的好句子，確實有共鳴。不過” 閱讀排行榜” 這個東西有點卷，容易變成” 為了刷時長而讀書”，建議關掉。

- **Kindle App**：如果你有 Kindle 裝置，手機端 Kindle App 可以同步閱讀進度。Kindle 的正版電子書排版質量比盜版 PDF 好太多，但 Kindle 中國區書城已經關了，需要切換到海外區。
- **Apple Books（圖書）**：iPhone 自帶，適合讀 EPUB 和 PDF。介面極簡，翻頁動畫很舒服。如果你從網上下載了電子書檔案，直接” 分享到圖書” 就能匯入。
- **閱讀設定建議**：不管用哪個 App，一定要把背景設成暖黃色調（護眼模式），字型調大一點，行間距拉開。晚上讀書開啟” 深色模式”——黑底白字，眼睛舒服很多。千萬別白底黑字小字型在暗光下讀太久，眼睛疲勞是積累的。

五、手機遊戲：玩得爽，也不傷手機

我對手機遊戲的態度是：玩可以，但別讓它反過來控制你。合理使用遊戲工具，玩得更舒服，也不耽誤正事。

- **遊戲模式/遊戲空間**：現在主流手機都有這個功能。開啟之後手機會自動清後臺、遮蔽通知、防止誤觸——你正在打團戰，突然一個電話彈出來，這種事情遊戲模式能幫你擋掉。部分手機還支援效能模式/E-sports 模式，強制處理器跑滿，但代價是發熱和耗電都更快。
- **畫質和幀率怎麼調**：不要無腦拉滿。先保證幀率（流暢度），再追求畫質。60 幀比 30 幀舒服太多了，但 120 幀對手機壓力很大，中端機不用強求。如果你玩的是競技類遊戲（比如王者、吃雞），幀率優先順序遠高於畫質——流暢的操作手感比好看的草皮重要一百倍。
- **遊戲時手機發燙怎麼辦**：這是正常的，但長期高溫會損傷電池壽命。幾個降溫辦法：摘掉手機殼散熱；把螢幕亮度降到 70% 左右（亮度是高發熱源）；不要一邊充電一邊打遊戲（發熱疊加，電池死得快）。如果手機燙到手握不住的程度，趕緊退出來讓它歇一會兒。
- **防沉迷小工具**：iOS 的” 螢幕使用時間” 和安卓的” 數字健康” 都可以給遊戲 App 設每日時間限額。我給自己設的是工作日最多一小時，週末兩小時。到時間了 App 圖示會變灰，點不進去。這個限制是真的有用，不是裝裝樣子的。
- **雲遊戲**：如果你手機效能不夠但又想玩大作，可以試一下雲遊戲平臺（比如騰訊 START 雲遊戲、網易雲遊戲）。遊戲在遠端伺服器上跑，畫面以影片流的方式傳到你的手機上。對網路要求高但對你手機效能幾乎沒有要求。缺點是延遲感不可避免，適合玩策略類、RPG 類，不適合 FPS 射擊類。

來動手：打造你的手機娛樂系統

1. 在音樂 App 裡下載你最喜歡的 3 張專輯或歌單，確保離線可聽。
2. 去小宇宙或蘋果播客訂閱 3 個感興趣的頻道，下載一期試聽。
3. 安裝 VLC 播放器，嘗試用它播放一段電腦複製過來的影片檔案。
4. 在微信讀書裡找一本想讀的書，設定暖色背景和護眼模式，把字型調到適合自己的大小。

5. 如果你玩遊戲，開啟遊戲模式看看有哪些設定可以最佳化（免打擾、幀率優先等）。

第十三章：工具類 App 推薦

本章精選效率、健康、出行類工具，並補充實用系統與跨裝置能力：清濁等清理與防騷擾、手機自帶攔截開關、天氣與地震預警；向日葵/*ToDesk* 遠控電腦、*Trae* 等 *AI* 辦公、人工桌面、*LocalSend* 等區域網快傳、跨裝置剪貼簿、*Aegis/Microsoft Authenticator* 動態口令、滴答清單日程與紙質掃描。

一、效率工具：管理你的時間與注意力

效率工具是數字生活的“骨架”，幫助你把零散的想法、任務、資料整理得井井有條。下面按場景分類推薦。

待辦清單（任務管理）

- 滴答清單（TickTick） 適用人群：需要把工作、生活、學習任務與日程統一記錄的人；本節重點推薦之一。核心賣點：跨手機/電腦/平板/Web 同步；清單 + 日曆 + 番茄鍾三合一；自然語言輸入「明天下午 3 點開會」「每週一交週報」自動識別時間與重複規則；支援子任務、標籤、優先順序、附件與協作清單。適合「日程安排 + 任務打卡 + 每日回顧」一體化管理。
- 滴答清單怎麼用得更順：收集箱：先全部丟進「今天/收集箱」，晚上再歸類到工作、家庭、學習清單。
- 日曆檢視：會議、截止日拖到日曆上看衝突，比只看待辦列表更直觀。
- 重複任務：吃藥、繳費、健身設成規則任務，不用每次手填。
- 番茄鍾：在任務上直接開專注，統計本週「真正做完了多少」。
- 與手機功能聯動：語音助理說「提醒我…」可進系統提醒；重要截止日同時開系統日曆雙保險。

Todoist 適用人群：追求極簡、跨平臺使用者。核心賣點：介面剋制清爽，專案層級清晰（任務子任務子子任務），karma 積分激勵習慣養成。

Microsoft To Do 適用人群：辦公黨、Outlook 使用者。核心賣點：完全免費、無廣告，”我的一天” 功能每天清空、減輕心理負擔，與微軟生態深度整合。

系統便籤/提醒事項　適用人群：不想裝第三方 App 的輕度使用者。核心賣點：原生體驗、零延遲，Siri/小愛語音直接新增，與系統 widget 聯動。

番茄鍾（專注力訓練）

- Forest　適用人群：管不住手、易刷手機的人。核心賣點：專注時種樹，提前離開 App 樹會枯死，用”虛擬生命”建立正向反饋，可與好友合種。
- 潮汐　適用人群：需要白噪音輔助專注/入睡的人。核心賣點：自然音效（雨聲、海浪、森林）+ 番茄鍾 + 冥想三合一，介面極簡治癒。
- 原則：經典 25 分鐘專注 + 5 分鐘休息，可根據自身狀態調整為 50/10 或 90/20。

日曆（時間視覺化）

- 系統日曆　適用人群：所有使用者。核心賣點：原生推送最穩、耗電極低，可與 Siri/小愛語音新增。
- Google Calendar　適用人群：多裝置使用者、需要與他人共享日程者。核心賣點：”目標”功能自動幫你安排健身/學習時間，與 Gmail 智慧識別郵件中的事件。

筆記類（知識沉澱）

- Notion　適用人群：需要把筆記、任務、資料庫、wiki 統一管理的人；學生黨、自由職業者、團隊協作。核心賣點：全能知識庫，Block 模組化編輯，可搭建個人主頁、閱讀清單、專案看板，模板生態豐富。
- Obsidian　適用人群：深度知識管理者、研究者、寫作人。核心賣點：本地 Markdown 儲存，雙向連結 + 關係圖譜，讓你的筆記形成”第二大腦”，資料完全歸你所有。
- flomo　適用人群：想記錄碎片靈感卻不想寫長文的人。核心賣點：卡片式筆記，類似發朋友圈一樣記錄想法，支援 # 標籤分類，按時間線回溯。
- 蘋果備忘錄　適用人群：iPhone/Mac 使用者、輕度記錄需求。核心賣點：啟動快、支援掃描文件/手繪/清單/iCloud 同步，免費且與系統深度整合。
- 印象筆記　適用人群：需要大量網頁剪藏、資料歸檔的資深使用者。核心賣點：網頁剪藏外掛強大，OCR 識別圖片文字，老牌穩定，但進階版需付費。

PDF 工具

- PDF Expert　適用人群：iPad/iPhone 使用者、需要批註合同的商務人士、學生黨。核心賣點：iOS 上公認最強的 PDF 編輯器，標註流暢、支援簽名、合併拆分、填寫表單。
- WPS　適用人群：需要同時處理 Word/Excel/PPT/PDF 的辦公族。核心賣點：一站式辦公套件，國內網盤/微信檔案相容好，免費版功能已足夠日常。

- Adobe Acrobat　適用人群：對 PDF 格式規範要求高的人（法律、設計、出版）。核心賣點：官方標準、轉換 PDF/A 格式無誤，跨裝置同步與電子簽名功能專業。

掃描類（紙質檔案數字化）

紙質合同、發票、作業、白板筆記、身份證影印件……手機相機已經能替代大部分掃描器。核心流程：拍清楚自動裁邊與矯正匯出 PDF/圖片 OCR 可選。

- 掃描全能王（CamScanner）　適用人群：經常需要掃描合同、票據、證件的商務人士和學生。核心賣點：邊緣自動識別、透視矯正、OCR 文字識別，可生成多頁 PDF 並雲端同步。
- 微軟 Lens　適用人群：學生黨、需要把白板/書頁掃描成 Word 的人。核心賣點：免費無廣告，可一鍵匯出到 OneNote/Word，識別文字準確率高。
- 蘋果備忘錄掃描　適用人群：iPhone 使用者、輕度掃描需求。核心賣點：無需裝 App，備忘錄內點選相機圖示即可掃描，自動去陰影、簽名標註，免費好用。
- 白描 / 掃描全能王網頁版 / 品牌相機「文件掃描」：華為、小米、OPPO、vivo 系統相機大多內建「文件/掃描」模式；安卓也可用 Google 相機或系統掃一掃。適合應急拍一頁合同。
- 拍照小技巧：光線均勻、手機與紙面平行、對準四個角；多頁檔案按順序拍完再合併 PDF；敏感證件注意水印與儲存位置，用完可從「最近刪除」徹底清理。

二、健康運動

健康類 App 是手機最被低估的“私人助理”——它們能持續追蹤你的睡眠、心率、運動量，比偶爾去一次醫院更能反映長期狀態。

運動健身

- Keep　適用人群：居家訓練者、零基礎新手、想減脂塑形的人。核心賣點：海量免費課程，從 HIIT 到瑜伽全覆蓋，AI 自動生成訓練計劃，社群氛圍濃厚。
- Nike Training Club（NTC）　適用人群：追求專業教練指導、希望系統訓練的人。核心賣點：Nike 官方出品，課程全部免費，動作示範由專業教練錄製，按肌群/器械分類清晰。
- 蘋果健身（Fitness）　適用人群：Apple Watch 使用者、想用閉環激勵自己動起來的人。核心賣點：每日三個活動圓環（移動/鍛鍊/站立），視覺化目標，與手錶心率資料無縫打通。

睡眠追蹤

- AutoSleep　適用人群：Apple Watch 使用者、想知道自己每晚深睡/淺睡比例的人。核心賣點：戴錶入睡自動追蹤，無需手動啟停，給出睡眠品質評分與就寢建議。

- Sleep Cycle　適用人群：不戴手錶睡覺、想用手機監測的人。核心賣點：透過麥克風分析翻身/呼嚕聲，在淺睡階段智慧喚醒，讓你起床不痛苦。
- 小米運動健康　適用人群：小米手環/手錶使用者。核心賣點：免費且與小米手環深度聯動，心率/血氧/睡眠一體追蹤，價效比高。

冥想放鬆

- Calm　適用人群：壓力大的上班族、入睡困難者。核心賣點：”每日平靜”冥想引導 + 睡眠故事（由名人朗讀），介面唯美，英文內容豐富。
- 潮汐　適用人群：需要白噪音專注/入睡的中文使用者。核心賣點：國產精品，自然音效 + 冥想課程 + 番茄鍾，中文引導更親切。
- 小睡眠　適用人群：需要助眠音效、記錄夢話/呼嚕聲的人。核心賣點：海量白噪音可疊加組合（雨聲 + 風扇 + 篝火），睡前定時關閉，夢話錄音有趣又實用。

女性健康

- Flo　適用人群：需要記錄經期、備孕或關注女性健康的使用者。核心賣點：AI 預測下次經期/排卵日準確度高，社群可匿名提問，介面設計精緻。
- 美柚　適用人群：國內女性使用者、希望看中文社群與科普的人。核心賣點：本地化內容豐富，按經期/備孕/孕期/育兒分階段提供對應知識。

用藥提醒

- 用藥助手　適用人群：需要長期服藥的慢性病患者、需要照顧老人用藥的子女。核心賣點：掃碼識別藥品、自動建立用藥清單，按時跳出視窗提醒，記錄服藥情況可分享給家屬。

資料中樞

- 蘋果健康/Google Fit　適用人群：所有智慧手機使用者。核心賣點：匯聚來自手機、手環及第三方健康 App 的所有資料（步數、心率、睡眠、經期等），是統一的”健康檔案中心”。

三、旅行出行

出門在外，幾款趁手的 App 能讓你少踩無數坑。下面按出行全流程推薦。

航班高鐵

- 航旅縱橫　適用人群：經常坐飛機的商務人士。核心賣點：官方民航資料，航班動態/登機口變更/延誤預警最權威，可查機型與座點陣圖。

- 12306　適用人群：所有國內火車出行者。核心賣點：官方唯一可信購票管道，支援候補購票（有人退票自動搶），學生票/團體票辦理方便。

- 飛常準　適用人群：對航班準點率敏感、需要接送機的人。核心賣點：航班延誤預測準確，可分享行程給家人實時檢視你的位置與到達時間。

地圖導航

- 高德地圖　適用人群：國內城市使用者、自駕/打車族。核心賣點：實時路況準確、避開擁堵能力強，打車/打車合乘一站搞定，公共交通換乘方案豐富。

- 百度地圖　適用人群：對地點資訊、室內導航有需求的使用者。核心賣點：全景地圖/AR實景導航，景區/商場室內地圖細緻，語音包豐富。

- Google Maps　適用人群：出國旅行者。核心賣點：全球覆蓋最全、本地評價真實，街景與離線地圖是異國他鄉的救星。

翻譯

- 谷歌翻譯　適用人群：出國旅行者、跨語言學習者。核心賣點：”對話”實時雙向翻譯、”相機”即時取詞翻譯是旅行神器，支援離線語言包下載。

匯率換算

- 極簡匯率　適用人群：出國旅行、海淘、跨境購物的人。核心賣點：介面極致清爽，支援160+ 貨幣，匯率實時更新，可同時對比多種貨幣，長按數字快速換算。

天氣

- 系統天氣 / 中國天氣網　適用人群：所有人，查預報與官方氣象預警的首選。核心賣點：廠商天氣 App 與「中國天氣」等官方管道對接氣象臺預警，暴雨、颱風、高溫等預警會以橫幅/通知推送，權威性高。

- 彩雲天氣　適用人群：需要精準到分鐘級降雨預報的人。核心賣點：”未來兩小時是否下雨”準確度業內領先，降雨帶動態地圖直觀可見，下雨前主動推送提醒。

- Carrot Weather　適用人群：追求趣味性與豐富 widget 的天氣發燒友。核心賣點：毒舌AI 語音播報天氣、超多自定義 widget 桌面小工具、雷達圖專業，付費版功能更全。

- 預警要點：天氣類 App 要允許通知才收得到預警；預警分藍、黃、橙、紅四級，紅色表示災害性天氣風險最高，應按提示調整出行。更多開啟方式見本章第七節「天氣預警 / 自然災害預警 / 地震預警」。

旅行攻略

- 馬蜂窩　適用人群：喜歡做詳細攻略、自駕遊/深度遊的人。核心賣點：使用者原創遊記內容豐富，目的地玩法/路線/避坑指南詳盡，可下載離線攻略。

- 小紅書　適用人群：想看真實體驗、找小眾打卡點/美食的年輕旅行者。核心賣點：圖文短影音形式直觀，"XX 城市 3 天攻略" 等內容更新快、時效性強，搜尋體驗好。

四、自動化工具啟蒙

自動化是“用一次配置換長期省事”的槓桿，能把你每天重複的幾十次操作壓縮成零。本節只做概念啟蒙，具體玩法與詳細教學請見第二十三章：程式設計思維與自動化啟蒙。

- iOS" 快捷指令"：蘋果官方自動化工具，無需程式設計，可一鍵完成" 到家開 WiFi+ 亮度調低 + 播報天氣" 等組合動作。

- 安卓自動化：系統級場景聯動（如小米" 自動任務"、華為" 情景智慧"）上手最簡單；Tasker 功能強大但學習曲線陡峭，適合極客。

- 核心思路：先識別" 每天都要重複做的操作"，再想" 能否用一個觸發條件 + 多個動作替代"。詳細示例與進階玩法見第 23 章。

五、瀏覽器與搜尋引擎：你的資訊入口

瀏覽器是你通往網際網路的窗口，搜尋引擎是你在資訊海洋中導航的羅盤。用好它們，能讓你少走很多彎路。

瀏覽器的工作原理：

當你在瀏覽器中輸入網址後，瀏覽器會：(1) 向 DNS 伺服器查詢域名對應的 IP 地址；(2) 向該 IP 地址的伺服器傳送 HTTP/HTTPS 請求；(3) 伺服器返回網頁程式碼（HTML、CSS、JavaScript）；(4) 瀏覽器解析程式碼並渲染成你看到的頁面。

主流瀏覽器推薦：

- 微軟 Edge：基於 Chromium 核心，速度快、相容好。內建 Copilot AI 助手，可以總結網頁、翻譯、對話。支援跨裝置同步，個人強烈推薦作為主力瀏覽器。

- Safari：蘋果裝置預設瀏覽器，與系統深度整合，省電高效。隱私保護好，但擴充套件外掛較少。

- Chrome：全球使用率最高，擴充套件豐富。但記憶體佔用大，手機端相對耗電。

搜尋引擎選擇：

- Bing：微軟旗下搜尋引擎，與 Edge 深度整合。AI 摘要功能可以直接給出答案而不只是連結，搜尋效率很高。圖片搜尋體驗也不錯。
- Yandex：俄羅斯最大的搜尋引擎，圖片搜尋和反向圖片搜尋能力很強。找相似圖片、識別物體時非常好用。
- 百度：國內資訊搜尋的主力，但廣告較多，需要一定的辨別能力。
- Google：全球最精準的搜尋引擎，但國內需要特殊網路才能使用。

高效資訊檢索指南：

- 善用 AI 搜尋：Bing 的 AI 摘要、Edge 的 Copilot 可以直接回答問題，幫你從海量結果中提煉關鍵資訊，比逐條點選連結高效得多。
- 識別虛假資訊與廣告：搜尋結果中標註” 廣告” 或” 贊助” 的條目是付費推廣，不是自然搜尋結果。注意甄別資訊來源的權威性，優先選擇官方網站、知名媒體的內容。
- 正確下載應用程式：在搜尋引擎中輸入” 應用程式名稱 + 官網”，如” 微信官網”，優先點選帶有官方域名標識的結果。不要從第三方下載站安裝軟體，它們經常捆綁流氓軟體。
- 官網鑑別技巧：下載 Windows 11 應該訪問 microsoft.com，下載 Chrome 應該訪問 google.com。看域名是否與品牌一致，是最基本的鑑別方法。

網站域名知識：

域名是網站的“門牌號”，讀懂它能幫你判斷網站的性質和來源：

- 常見字尾含義：.com —商業機構，最常見
- .cn —中國國家域名
- .org —非營利組織
- .net —網路服務機構
- .edu —教育機構
- .gov —政府機構
- .xyz/.tech/.info —新興通用域名，門檻低，需謹慎

國家/地區域名：.jp —日本

.kr —韓國

.uk —英國

.de —德國

.tw —臺灣

.hk —香港

安全提示：.gov 和.edu 字尾的網站通常更可信，因為申請門檻高。.xyz、.top 等廉價域名容易被釣魚網站利用，訪問時要多留個心眼。

廣告攔截：

- AdGuard　適用人群：被各類跳出視窗、貼片廣告、應用內廣告困擾的所有使用者。核心賣點：系統級廣告攔截（不僅限瀏覽器），可過濾 App 內橫幅廣告與開屏追蹤，省用量、加速頁面載入，付費版支援全裝置。

隱私瀏覽器：

- Brave　適用人群:注重隱私、不希望被廣告追蹤的日常瀏覽者。核心賣點:基於 Chromium 核心，開箱即用遮蔽廣告與追蹤器，自帶 Tor 隱身模式，速度比 Chrome 快、記憶體佔用更低。

- DuckDuckGo　適用人群：希望”搜一個詞不被全網追著推廣告”的使用者。核心賣點：自帶同名搜尋引擎（不記錄搜尋歷史），一鍵清除所有瀏覽痕跡（”燃燒按鈕”），跨平臺體驗統一。

瀏覽器對比一覽：

瀏覽器	核心賣點	適用人群
微軟 Edge	Chromium 核心 + 內建 Copilot AI，可總結網頁/翻譯/對話	追求效率的辦公族、AI 重度使用者（推薦主力）
Safari	與蘋果生態深度整合，省電高效，隱私保護強	iPhone/Mac 使用者，重視續航與系統聯動
Chrome	全球使用率最高，擴充套件外掛生態最豐富	需要大量擴充套件外掛的進階使用者、跨裝置同步黨
Brave	開箱即用遮蔽廣告與追蹤器，速度更快	注重隱私、不裝外掛也想遮蔽廣告的使用者
DuckDuckGo	同名搜尋引擎不記錄歷史，一鍵清除痕跡	不希望被個性化追蹤的隱私敏感使用者

試著建立一個”睡前放鬆”的自動化流程

1. 建立觸發條件：特定時間（如 22:30）。
2. 新增執行動作：調低音量 > 開啟勿擾模式 > 播放放鬆歌單 > 調低亮度。
3. 儲存並測試。

六、實用系統工具：清理、防騷擾與安全防護

除了效率與健康類 App，真正影響日常體驗的往往是「少被打擾、儲存夠用、騙子進不來」這三件事。本節先盤點常用實用軟體，再單獨展開手機自帶的騷擾攔截功能——很多人買了手機多年，卻從沒開啟過這些開關。

系統清理與儲存最佳化

- 清濁　適用人群：安卓使用者，希望輕量清理快取、殘留檔案、空資料夾，又不想被「手機管家」全家桶打擾的人。核心賣點：體積小、無廣告干擾，支援深度掃描應用殘留、大檔案定位、儲存佔用分析；比商業「一鍵加速」更剋制，適合動手能力強的使用者做定期維護。注意：清理前先確認路徑，微信、相簿、文件目錄不要盲目全選；系統級快取儘量優先用品牌自帶儲存清理，第三方工具只作補充。
- 系統自帶儲存清理　適用人群：所有人，尤其是長輩和怕折騰的使用者。核心賣點：安卓 [設定] > [儲存] > [清理加速]；iOS [設定] > [通用] > [iPhone 儲存空間]。系統更懂自家 App 快取結構，誤刪風險最低，應作為第一選擇。
- 微信/QQ 內建儲存管理　適用人群：聊天大戶。核心賣點：微信 [我] > [設定] > [通用] > [儲存空間]，可按聊天、檔案類型精細清理；比外部「清微信」類工具更安全。
- 原則：能用系統/App 自帶的，就不用第三方；能按路徑/型別勾選的，就不要「一鍵全清」。清理的目標是釋放空間，不是把重要資料一起掃掉。

防騷擾與安全類實用軟體

- 國家反詐中心　適用人群：所有人，尤其是長輩、財務相關崗位、經常網購/接陌生電話的人。核心賣點：官方出品，可開啟來電預警、簡訊風險提示、可疑網址檢測；與公安反詐資料庫聯動，是「三道防線」裡最有官方背書的一環。
- 品牌安全中心 / 手機管家　適用人群：對應品牌使用者。核心賣點：華為手機管家、小米安全中心、OPPO 手機管家、vivo 手機管家等，整合騷擾攔截、病毒查殺、權限管理、支付保護，通常免費且與系統底層協同更好，不建議再裝第二套商業安全軟體造成權限衝突。
- 電信業者防騷擾服務　適用人群：被高頻外呼、AI 語音電話轟炸的使用者。核心賣點：移動傳送 KTFSR 到 10086 開通；電信關注「天翼防騷擾」；聯通在「聯通手機管家」中設定。攔截髮生在電信業者側，手機不響鈴也能擋住一批騷擾號。
- AdGuard　適用人群：被各類跳出視窗、貼片廣告、應用內廣告困擾的所有使用者。核心賣點：系統級廣告攔截（不僅限瀏覽器），可過濾 App 內橫幅廣告與開屏追蹤，省用量、加速頁面載入，付費版支援全裝置。
- 慎裝「清理加速 / 騷擾攔截」全家桶：部分第三方管家常駐後臺、反覆請求通訊錄與簡訊權限，攔截能力未必比系統強，反而增加耗電與隱私風險。優先開啟系統自帶功能，再考慮官方反詐 App。

實用軟體選型原則

- 系統優先：攔截、清理、權限管理，先開啟手機自帶開關。
- 官方補充：國家反詐中心、電信業者防騷擾屬於官方通道，可疊加。
- 第三方剋制：同一類功能裝一個就夠，避免權限打架與後臺搶資源。
- 少即是多：工具裝得越多，通知和權限面越大，被「工具本身」打擾的機率越高。

手機自帶騷擾攔截功能：各品牌怎麼開

下面按品牌列出入口。名稱可能隨系統版本略有差異，可在設定裡直接搜尋「騷擾」「攔截」「未知來電」。與第八章「騷擾電話治理」互為補充：這裡側重開關位置，那裡側重號碼標記與誤標申訴。

平臺	入口路徑	推薦開啟項
iPhone / iOS	[設定] > [電話] / [資訊]	靜音未知來電；已阻止的聯絡人；資訊中「過濾未知發件人」
小米 / Redmi	電話 App > 設定 > 騷擾攔截；或安全中心	智慧攔截、陌生號響一聲結束通話、簡訊關鍵詞過濾
華為 / 榮耀	電話 > 騷擾攔截；手機管家 > 騷擾攔截	來電識別、黑名單、簡訊智慧攔截
OPPO / realme / 一加	電話 App > 騷擾攔截；或本機 > 騷擾攔截	來電攔截、詐騙電話識別、簡訊攔截規則
vivo / iQOO	電話 > 騷擾電話攔截；i 管家	騷擾攔截、詐騙電話預警、資訊過濾
三星	電話 > 設定 > 阻止號碼 / 來電顯示與垃圾電話	阻止未知號碼、垃圾電話識別（地區支援度不同）

三層防護疊加建議

日常推薦「系統攔截 + 電信業者防騷擾 + 國家反詐中心」三層疊加：系統管標記與黑名單，電信業者在網側擋高頻外呼，反詐中心補官方預警。三層不必都開到最嚴，避免把快遞、外賣、醫生電話也誤殺——可對通訊錄與最近聯絡人放行。

動手練習：今天就把騷擾攔截開啟

1. 開啟電話 App 設定，找到「騷擾攔截 / 靜音未知來電」，開啟智慧攔截。
2. 給爸媽手機也檢查一遍：同樣路徑 + 開通電信業者防騷擾（移動可發 KTFSR 到 10086）。
3. 應用商店安裝「國家反詐中心」，註冊並開啟來電預警。
4. 試打一個已被標記的測試號（或看攔截紀錄），確認功能生效。

七、天氣預警、自然災害預警與地震預警

騷擾攔截擋住的是「電話詐騙」，預警功能擋住的是「天災與極端天氣」。很多人只裝了天氣 App 看溫度，卻沒開通知、沒開地震預警——暴雨紅色預警、颱風、地震來臨前那幾秒到幾十秒，手機本可以幫你爭取到寶貴的避險時間。

天氣預警：暴雨、颱風、高溫怎麼收

- 官方管道優先：中國氣象局 / 中央氣象臺 / 各省氣象局發佈的預警最權威。可關注「中國天氣」微信公眾號、微博，或下載「中國天氣」等官方類 App。
- 系統天氣 App：華為天氣、小米天氣、OPPO 天氣、vivo 天氣、蘋果「天氣」等，多數會推送當地氣象臺預警；進入天氣 App 設定，確認「氣象預警 / 災害性天氣」通知已開啟。
- 彩雲天氣等分鐘級降雨：適合通勤、接送孩子前判斷「這半小時要不要帶傘」；預警能力仍以官方氣象臺為準。
- 預警顏色含義（通用）：藍色：注意防範，一般照常出行、關注更新。
- 黃色：加強防範，減少不必要戶外活動。
- 橙色：嚴重，儘量留在室內，遠離危險區域。
- 紅色：特別嚴重，聽從停課、停工、避險指引，不要硬闖積水、山洪、大風路段。

常見預警型別：暴雨、暴雪、颱風、高溫、寒潮、大風、雷電、冰雹、道路結冰、大霧、乾旱、地質災害風險。

自然災害預警：不只「下雨」

- 發佈主體：氣象臺負責氣象類；地質災害、山洪等常由自然資源、水利、應急管理部門聯合發佈；地方政府也會透過應急廣播、簡訊、12379 等管道發佈。
- 怎麼多管道接收：手機系統通知 + 天氣 App 預警。
- 微信關注當地「應急管理局」「氣象局」公眾號。
- 12379 國家預警資訊發佈相關管道（簡訊/服務，視地區開通情況）。
- 電視、應急廣播、村/社群大喇叭——山區、汛期不要只盯手機。

戶外與出行提醒：颱風天不貼廣告牌、不走樹下；暴雨不進地下車庫、隧道、下穿橋；山區旅遊遇「地質災害氣象風險」預警應立即離開陡坡、溝谷。

與急救手冊聯動：災後失聯、手機沒電、需要定位求救時，見第三十二章地震/災害場景，以及第七章衛星通訊與無網通。

地震預警：和「地震預報」不是一回事

地震預警（EEW）是在地震波（尤其是破壞力更強的橫波）到達目標區之前，利用電磁波比地震波快、以及臺站更快捕捉到縱波的原理，向可能受影響區域提前數秒到數十秒推送警報。它不能預測地震是否發生，但能搶出「就近避險、切斷火源、衝出電梯」的窗口。距離震中越近，預警時間越短，甚至可能來不及。

- 官方與行業方案：中國大陸常見接入方包括中國地震預警網相關能力，以及成都高新減災研究所等機構推動的預警服務；部分廠商手機會預裝或可聯網接收預警。
- 第三方 App：應用商店搜尋「地震預警」等官方/機構相關應用（安裝前看清開發者，避免山寨「假預警」App 要你開一堆無關權限）。
- 收到警報後怎麼做：室內就近躲避（桌下、承重牆角、堅固傢俱旁），護住頭頸；遠離窗戶、外牆、高大傢俱；不要盲目衝樓梯/跳樓；電梯裡立刻在最近樓層離開；室外到開闊地，遠離電線、廣告牌、危房。震後檢查燃氣與電路，注意餘震。
- 勿擾勿關：地震預警應允許「突破靜音/勿擾」響鈴；睡前把手機放伸手可及處，鈴聲音量別調到 0。

手機自帶地震預警：各品牌怎麼開

下列入口隨系統版本可能略有差異，可在設定中直接搜尋「地震」「預警」「安全預警」。iOS 系統本身對國內地震預警支援有限，通常需安裝可靠的第三方預警 App，並保持定位與通知權限。

平臺	常見入口	說明
小米 / Redmi	手機管家 / 安全中心 > 地震預警；或設定搜尋「地震」	部分版本系統內建預警服務，需聯網並開啟定位
華為 / 榮耀	設定 > 安全 > 應急預警通知 / 地震預警	名稱可能是「應急預警通知」，內含地震等
OPPO / realme / 一加	設定 > 安全（SOS 緊急聯絡）> 地震預警	部分機型與地區服務覆蓋有關
vivo / iQOO	天氣 App 或設定 > 安全/更多 > 地震預警	以實際系統版本選單為準
iPhone / iOS	系統設定中通常無完整內建開關；安裝可靠地震預警 App 並允許通知	App 需後臺執行與定位權限；系統勿擾勿遮蔽其警報
三星等	設定 > 安全與緊急情況 / 無線緊急警報	「緊急警報」覆蓋範圍因地區與電信業者而異

關於地震預警的常見誤區

- 預警 ≠ 預報：它不能告訴你「明天會不會地震」，只在已發生地震、波尚未到達時報警。

- 不是哪裡都能提前很多秒：震中附近可能只有數秒甚至來不及，預警主要惠及周邊區域。
- 關通知就等於沒裝：再好的預警，靜音、殺後臺、不給定位，都可能收不到。
- 山寨 App：警惕要通訊錄、簡訊權限的「地震預報」軟體，優先系統內建或正規機構方案。

預警能力怎麼搭配

日常建議：「系統天氣預警通知 + 官方氣象/應急管道 + 地震預警（系統或正規 App）」。與防騷擾一樣，不必裝一堆同類軟體；關鍵是通知權限開啟、定位允許、勿擾能突破。給長輩手機做完騷擾攔截後，順手把天氣預警和地震預警也檢查一遍。

動手練習：10 分鐘檢查預警設定

1. 開啟系統天氣 App，進入設定，開啟「氣象預警 / 災害性天氣」通知。
2. 在系統設定裡搜尋「地震」或「預警」，找到並開啟地震預警/應急預警（安卓）；iPhone 則安裝正規地震預警 App 並允許通知。
3. 關注當地氣象局或應急局的公眾號/服務號，允許其推送。
4. 確認預警聲音不會被勿擾完全靜音；給爸媽手機同樣檢查一遍。

八、跨裝置協同：遠控電腦、快傳與帳號安全

手機用得順，多半是因為它和電腦、平板「組了隊」。本節補充幾類高頻實用工具：手機遠控電腦、AI 辦公與遠端協作、動態桌布、區域網快傳、跨裝置剪貼簿、動態口令 2FA，以及紙質掃描與日程任務的銜接。更系統的多裝置聯動見第十七章。

手機遠控電腦：向日葵 / ToDesk

- 向日葵遠端控制　適用人群：需要在外出時操作家裡/公司 Windows 或 Mac 的使用者；也常用於幫長輩遠端修手機（見第二十七章）。核心賣點：國產生態成熟，手機 App 控制電腦桌面、傳檔案、開關機（部分需硬體）；免費版一般夠個人輕度使用。
- ToDesk　適用人群：追求連線穩定、低延遲遠端辦公/運維的使用者。核心賣點：裝置碼 + 密碼連線，手機控電腦、電腦控手機（視版本）、檔案傳輸與多顯示器切換較順手，國內節點體驗通常較好。
- 基本用法：電腦端安裝用戶端並登入/保持線上手機端裝 App 用裝置識別碼 + 連線密碼（或同一帳號）建立會話手機觸控板/手勢操作桌面。
- 安全紅線（務必記住）：陌生人自稱「客服/公檢法/電信業者」讓你裝向日葵、ToDesk 並遠端你手機或電腦，一律拒絕——這是典型詐騙手法（詳見第二十七章）。

- 自己用時：開啟二次驗證、修改預設密碼、不用時退出帳號或關閉無人值守；公司電腦遵守單位安全規定。

- 公共場合遠端辦公注意周圍是否有人偷窺螢幕與口令。

AI 辦公軟體：Trae / WorkBuddy / Dumate

- Trae　適用人群：程式設計與文件協作使用者，中文環境友好。核心賣點：位元組跳動推出的 AI IDE/程式設計助手，SOLO 模式可一句話生成專案；日常寫程式碼、改文件效率高。若電腦端執行，可在移動端配合遠控或官方配套能力檢視/跟進任務（以產品最新形態為準）。詳見第四十一章 AI 程式設計工具全景。

- WorkBuddy　適用人群：希望 AI 幫忙整理會議紀要、郵件、彙報、辦公文件的職場使用者。核心賣點：面向辦公場景的 AI 助手，適合「起草—潤色—總結」這類高頻文案；部分工作流可在手機端發起，重活放到電腦端完成。

- Dumate　適用人群：需要多人協作、文件與知識管理一體化的團隊/個人。核心賣點：AI 協作向辦公工具，強調資料匯聚與協同編輯；手機端可檢視與輕量編輯，複雜排版建議在電腦大屏處理。

- 「手機端遠端控制電腦」怎麼理解：很多 AI 辦公能力跑在電腦/網頁端；在手機上透過向日葵/ToDesk 連回電腦，或使用產品自身的手機 App/網頁，就能在通勤時「遙控」電腦上的 AI 任務。原則：重計算、長文件放電腦；看進度、改幾句放手機。

- 隱私提醒：上傳合同、身份證、客戶資料前看清產品隱私條款與企業合規要求；公司資料優先用單位指定工具。

人工桌面：米哈遊動態桌布（手機 + PC）

- 人工桌面　適用人群：喜歡米哈遊角色、想要免費動態桌布的使用者。核心賣點：米哈遊推出的免費動態桌布應用，支援手機端與 PC 端；角色 Live2D/動態演出可作桌面，部分機型/系統可設為鎖定畫面或桌面桌布。

- 省電與效能：動態桌布比靜態圖更耗電、佔 GPU；低電量或老機型可只在充電時使用，或降低幀率（若 App 提供選項）。

- 安裝來源：優先官網/正規應用商店，避免第三方「破解版」捆綁木馬；PC 版注意安裝路徑與開機自啟是否需要。

區域網檔案快傳：LocalSend / 飛鴿傳書 / 隔空投送

- LocalSend　適用人群：安卓/Windows/iOS/Mac/Linux 混用，想要跨平臺、不傳公有云快傳的人。核心賣點：開源、區域網直連，同一 Wi-Fi 下互傳照片影片文件，無需資料線、不經過廠商伺服器，速度取決於路由器。

- 飛鴿傳書　適用人群：辦公室 Windows 電腦之間、老區域網辦公場景。核心賣點：老牌區域網傳輸/聊天工具，適合內網傳大檔案；注意從正規管道下載，企業內網遵守 IT 策略。
- 隔空投送（AirDrop）　適用人群：iPhone、iPad、Mac 使用者。核心賣點：蘋果生態一鍵分享，藍芽發現 + Wi-Fi 直連傳輸，免用量；可設「僅聯絡人」降低陌生人騷擾。
- 其他補充：小米互傳、華為分享、vivo 互傳等品牌方案在同一品牌裝置間很快；微信「檔案傳輸助手」方便但受檔案大小與登入裝置限制，大檔案更推薦 LocalSend/隔空投送。
- 使用提示：雙方連同一 Wi-Fi；公共 Wi-Fi 傳敏感檔案有風險，優先私人熱點；傳輸完成後在接收方確認檔案是否完整。

跨裝置剪貼簿同步：KDE Connect / 手機電腦互傳

- 蘋果「通用剪貼簿」　適用人群：iPhone + Mac 使用者。核心賣點：登入同一 Apple ID、開接力後，手機複製 Mac 直接貼上（反之亦然），文字與部分檔案均可。
- KDE Connect　適用人群：安卓 + Windows/Linux/Mac，想要開源剪貼簿同步、通知映象、遠端輸入的使用者。核心賣點：區域網配對後同步剪貼簿、推送手機通知到電腦、用手機當投影片翻頁器/觸控板；資料不經過第三方帳號。
- 品牌「手機電腦互傳」：華為多屏協同/超級終端、小米妙享、vivo 互傳、OPPO 互連、榮耀互聯等，常自帶跨屏剪貼簿與檔案拖拽，同品牌體驗最好。
- Windows + 安卓官方通道：「手機連線 / Link to Windows」（Your Phone 演進）可在電腦上看手機通知、照片，並在支援的場景複製貼上；三星等機型整合更深。
- 安全提醒：剪貼簿可能殘留密碼、驗證碼；在共用電腦上慎用全域性同步，用完在密碼框避免二次貼上洩露；部分輸入法也會讀剪貼簿，注意權限。

動態口令 2FA：Aegis / Microsoft Authenticator

- 為什麼要用認證器而不是簡訊：簡訊驗證碼可能遭遇 SIM 補卡劫持；TOTP 動態口令在本地按時間演算法生成，無需依賴手機號訊號，更抗釣魚與攔截（仍需防木馬與截圖）。
- Aegis Authenticator　適用人群：安卓使用者，偏好開源、本地可控。核心賣點：開源免費，支援加密備份與匯入匯出，介面乾淨，可指紋/人臉解鎖令牌列表。
- Microsoft Authenticator　適用人群：使用微軟帳號、Office 365/企業帳號的使用者；跨 iOS/安卓。核心賣點：推送批准登入、密碼監控與帳號備份（按產品版本），與 Windows/企業身份體系結合緊密。
- Google Authenticator / 2FAS 等：生態內也常見；關鍵是有加密備份，換機不丟令牌。
- 換機必做：舊機遷移或用服務裡的「備用碼/還原金鑰」；備用碼列印在紙上存兩處，不要只存同一部手機。

- 與密碼管理器：部分密碼庫也可存 TOTP，方便但風險更集中；高價值帳號建議「密碼管理器 + 獨立認證器」分工。詳見第十四章。

推薦組合：一天裡怎麼串起來

實用工作流示例

- 早上：滴答清單看今日日程天氣/預警掃一眼電腦開 Trae/文件，手機放一旁。
- 白天：紙質材料用掃描類 App 轉 PDF，經 LocalSend/隔空投送發到電腦；長文在電腦用 AI 辦公工具起草。
- 外出：用 ToDesk/向日葵連回電腦繼續任務；重要帳號登入走 Aegis/Microsoft Authenticator 動態口令。
- 晚上：滴答清單勾選完成項；需要時人工桌面換個桌布放鬆一下。

遠控與帳號安全口訣

- 別人讓你裝遠控軟體掛電話，自己找官方客服。
- 驗證碼、動態口令不告訴任何人，包括「快遞」「客服」。
- 公司電腦遠控與安裝 AI 軟體前，先問 IT 能不能用。
- 公共網路傳檔案、遠端登入時格外警惕。

動手練習：把「跨裝置」鏈路跑通一次

1. 手機與電腦連同一 Wi-Fi，用 LocalSend（或隔空投送）互傳一張照片。
2. 在滴答清單建一條「今晚 8 點：整理掃描件」任務，確認手機與電腦都能看到。
3. 為信箱或微信開啟 2FA（若支援），用 Aegis 或 Microsoft Authenticator 繫結，並儲存備用碼。
4. 在自己電腦上安裝向日葵或 ToDesk，用自己手機成功連一次，再退出或加固密碼，熟悉流程但不留安全後門。

第十四章：密碼管理器專題

在數字時代，密碼就是你的家門鑰匙。但鑰匙太多、太複雜，怎麼辦？密碼管理器就是你的“萬能鑰匙管家”。

一、為什麼你需要密碼管理器？

現代人平均有幾十甚至上百個網路賬號。如果每個賬號都用相同的密碼，一旦一個網站被黑，所有賬號都危險。如果每個賬號都用不同的複雜密碼，根本記不住。

密碼管理器解決了這個困境：你只需要記住一個主密碼，它幫你記住所有其他密碼。

二、密碼管理器的工作原理

1. **加密儲存**：所有密碼都經過高強度加密儲存，只有你能解密。
2. **主密碼**：你設定一個主密碼，用於解鎖密碼庫。主密碼本身不儲存在伺服器上。
3. **自動填充**：在網站或 App 登入介面，密碼管理器自動填充賬號密碼。
4. **密碼生成**：可以生成隨機、複雜的密碼，提高安全性。

三、加密原理簡介

瞭解密碼管理器的加密原理，能幫你判斷一個服務商是否值得信任。下面用通俗的語言解釋幾個核心概念。

AES-256 加密

AES（高階加密標準）是美國國家標準與技術研究院（NIST）認可的對稱加密演算法，被銀行、軍方、政府廣泛使用。“256”指金鑰長度為 256 位——理論上需要超過宇宙年齡的時間才能暴力破解。主流密碼管理器（1Password、Bitwarden、KeePass 等）都用 AES-256 加密你的密碼庫。

零知識架構（Zero-Knowledge Architecture）

“零知識”不是說服務商什麼都不存，而是說**加密和解密都在你的裝置本地完成，伺服器只儲存已加密的密文**。具體流程：

1. 你在客戶端輸入主密碼。
2. 客戶端用主密碼派生出加密金鑰（這一步在你手機/電腦上完成）。
3. 只有加密後的密文才上傳到伺服器。
4. 下次登入時，伺服器把密文下發，客戶端再次用主密碼解密。

關鍵點：**主密碼本身從不離開你的裝置**，伺服器上也沒有。所以即使服務商被黑，駭客拿到的也只是一堆無法解密的密文——除非他們能猜出你的主密碼。

為什麼”零知識”意味著服務商也看不到你的密碼？

因為解密金鑰由你的主密碼派生，而主密碼只存在於你腦中、只在你本地裝置上使用。服務商的伺服器上既沒有主密碼，也沒有解密金鑰，只有密文。這就是為什麼 Bitwarden、1Password 等正規服務商都明確宣告“我們也無法看到你的密碼”——不是他們不想看，是技術上做不到。

金鑰派生函式：PBKDF2 與 Argon2

主密碼不能直接用作加密金鑰——它需要經過“金鑰派生函式”處理。這個函式的作用是**故意讓計算變慢**，讓駭客暴力破解的成本高到不可行。

- **PBKDF2**：老牌標準，被 iCloud 鑰匙串、Bitwarden、LastPass 等廣泛使用。透過反覆迭代（通常 10 萬次以上）雜湊來拖慢破解。
- **Argon2**：2015 年密碼雜湊競賽冠軍，比 PBKDF2 更現代。除了”算力消耗”還引入了”記憶體消耗”——讓 GPU/ASIC 等專用硬體也難以加速破解。Bitwarden、KeePassXC 等已支援。

迭代次數越高越安全，但解鎖速度也會變慢。正規服務商都會公開其使用的派生演算法和迭代次數，便於安全社群審計。

案例警示：LastPass 2022 年資料洩露

2022 年 8 月，老牌密碼管理器 LastPass 公告其開發環境被入侵；同年 11 月進一步披露，駭客拿走了使用者的密碼庫備份（加密後的密文）。事件的關鍵教訓：

- **零知識架構守住了底線**：由於密碼庫本身是加密的，駭客拿到密文但解不開——前提是使用者的主密碼足夠強。

- **弱主密碼使用者最危險：**駭客可以離線暴力破解弱主密碼，因此使用弱主密碼或重複主密碼的使用者面臨實質風險。
- **後設資料也洩露：**雖然密碼內容加密，但站點 URL、備註等後設資料可能未加密，會暴露使用者使用了哪些服務。

從 LastPass 事件學到的

第一，主密碼必須長且獨特（建議 16 位以上，包含大小寫、數字、符號）；第二，優先選擇支援 Argon2id、迭代次數高的服務商；第三，對高價值賬號（銀行、郵箱）務必開啟獨立的二次驗證，不要只依賴密碼管理器。

四、主流密碼管理器推薦

系統自帶：

- **iCloud 鑰匙串：**蘋果生態使用者首選。與 Safari、App 無縫整合，免費，安全。缺點是跨平臺使用不便。
- **Google 密碼管理器：**安卓使用者方便。與 Chrome、安卓系統整合，免費。
- **華為密碼保險箱：**鴻蒙系統內建，適合華為使用者。

第三方專業工具：

- **Bitwarden：**開源免費，功能強大，支援所有平臺。強烈推薦給注重隱私的使用者。
- **1Password：**介面美觀，功能豐富，適合家庭和團隊。付費。
- **LastPass：**老牌密碼管理器，免費版功能受限。
- **KeePass：**完全本地儲存，開源免費。適合極客使用者。

五、密碼強度真實評估

密碼管理器普遍會顯示“密碼強度”指示條（紅/黃/綠），但這個評估往往只看長度和字元種類，並不全面。真正判斷密碼是否安全，需要考慮“熵”和“是否已洩露”。

熵（Entropy）：密碼強度的科學度量

“熵”衡量密碼的不確定性，單位是位元。熵越高，暴力破解所需嘗試次數越多。

- **”123456”：**熵極低，瞬間破解。
- **”P@ssw0rd”：**看似複雜，但出現在所有字典攻擊列表裡，熵依然低。

- **”correct horse battery staple”**：四個隨機單詞，熵約 44 位元，破解難度遠超”P@ssw0rd”。
- **16 位隨機字元（含大小寫 + 數字 + 符號）**：熵約 100 位元，目前算力無法暴力破解。

結論：**“長度 + 隨機性”比“複雜符號”更重要**。這也是密碼管理器生成的隨機長密碼真正安全的原因。

HaveIBeenPwned：查你的密碼是否已洩露

安全研究員 Troy Hunt 維護的 haveibeenpwned.com 收錄了歷次大型資料洩露中流出的賬號和密碼。你可以：

1. 輸入郵箱，查你的賬號是否在某次洩露中被曝光。
2. 用 Pwned Passwords 功能查詢某個密碼是否在已知洩露庫中（密碼管理器和瀏覽器都已內建此查詢）。

如果一個密碼出現在洩露庫中，**無論它看起來多複雜都必須立即更換**——因為字典攻擊會優先嚐試這些已知密碼。

k-Anonymity 查詢原理

擔心在 HaveIBeenPwned 輸入密碼會再次洩露？不必擔心。現代查詢採用“k-匿名”協議：瀏覽器只把密碼雜湊的前 5 位發給伺服器，伺服器返回所有以這 5 位開頭的雜湊列表，本地再比對。也就是說，伺服器永遠看不到你的完整密碼雜湊。1Password、Bitwarden 的“已洩露密碼檢測”都是這樣工作的。

密碼管理器內建的監控功能

- **1Password Watchtower**：實時監控弱密碼、重複密碼、已洩露密碼、過期密碼、不使用 2FA 的網站，並以紅/橙/黃分級展示。
- **Bitwarden Vault Health**：免費版即可檢視弱密碼、重複密碼；付費版可檢測已洩露密碼（基於 HaveIBeenPwned）。
- **iCloud 鑰匙串”安全建議”**：iOS 設定密碼安全建議，會提示你哪些密碼出現在已知洩露中、哪些重複使用。
- **Google 密碼管理器”密碼檢查工具”**：Chrome 內建，可一鍵檢查已洩露、弱、重複密碼。

現在就查：你的密碼是否安全

1. 開啟 haveibeenpwned.com，輸入你的常用郵箱，看看出現在哪些洩露事件中。
2. iPhone 使用者：設定密碼安全建議，逐條處理紅色警告。
3. Chrome 使用者：設定密碼檢查密碼，按提示修改。

4. 1Password/Bitwarden 使用者：開啟 Watchtower/Health Reports，優先處理” 已洩露” 和” 重複” 項。

六、Passkey：無密碼登入的未來

2024-2026 年，密碼技術正在經歷一場革命——**Passkey（通行金鑰）**有望徹底淘汰傳統密碼。Apple、Google、Microsoft 三大平臺聯合推動，FIDO 聯盟和全球資訊網聯盟（W3C）背書，這是數十年來登入方式最大的一次變革。

什麼是 Passkey？

Passkey 基於 FIDO2 / WebAuthn 標準，用**裝置上的金鑰對**替代密碼。原理可以簡化為：

- 你的手機生成一對金鑰：**公鑰**（註冊給網站）+ **私鑰**（永遠存在你裝置的安全晶片中）。
- 登入時，網站發一個” 挑戰”，你的手機用私鑰簽名，再由網站用公鑰驗證。
- 簽名前需要你透過生物識別（Face ID / 指紋）或裝置 PIN 確認身份。
- 整個過程中，**沒有任何密碼在網路中傳輸**。

Passkey vs 傳統密碼

維度	傳統密碼	Passkey
抗釣魚	差，釣魚網站可竊取密碼	天然抗釣魚，私鑰只對真實域名簽名
伺服器洩露影響	密碼雜湊可能被破解	伺服器只存公鑰，洩露無影響
便捷性	需記憶/輸入/管理	Face ID/指紋一秒登入
重放攻擊	可能被中間人截獲重放	每次簽名一次性，無法重放

各平臺支援情況

- **Apple iCloud 鑰匙串 Passkey**：iOS 16+ / macOS Ventura+ 原生支援，Passkey 可在所有登入同一 Apple ID 的裝置間同步，新裝置登入網站時無需重新建立。
- **Google 密碼管理器 Passkey**：安卓 + Chrome 原生支援，Passkey 存在 Google 賬戶中，跨安卓裝置同步；2023 年起 Google 個人賬號已支援用 Passkey 登入。
- **Microsoft**：Windows 11 / Edge 已支援 Passkey；微軟賬號、Outlook、Office 等服務陸續開放 Passkey 登入，並可在 Microsoft Authenticator 中管理。
- **1Password / Bitwarden**：已支援 Passkey 的建立、儲存、跨平臺同步，讓你不受限於單一生態。

跨裝置同步是關鍵突破

早期 Passkey 的最大痛點是“繫結單臺裝置”——換手機就要重新建立。2023 年起 Apple、Google、Microsoft 實現了 Passkey 的雲端同步：iCloud 鑰匙串同步跨 Apple 裝置，Google 密碼管理器同步跨安卓裝置。1Password 等第三方則打通了跨生態同步。這意味著 Passkey 已具備替代密碼的實用性。

如何在手機上建立和使用 Passkey

iOS（iPhone）路徑：

1. 在支援 Passkey 的網站（如 Google、微軟、亞馬遜、eBay、PayPal）賬號安全頁選擇” 新增通行金鑰 / Add passkey”。
2. 系統彈出 Face ID / Touch ID 驗證視窗，透過後 Passkey 自動生成並儲存到 iCloud 鑰匙串。
3. 下次登入該網站時，輸入使用者名稱後選擇” 使用 iPhone 登入”，Face ID 一刷即可。
4. 管理入口：設定密碼找到對應網站，可見 Passkey 條目。

安卓路徑：

1. 在支援 Passkey 的網站賬號安全頁選擇” 新增通行金鑰”。
2. Google 密碼管理器彈窗選擇你的 Google 賬號螢幕鎖驗證（指紋/圖案/PIN）。
3. Passkey 自動儲存到 Google 密碼管理器，跨同賬號安卓裝置同步。
4. 下次登入時，選擇” 使用 Passkey”，螢幕鎖一解即可。
5. 管理入口：安卓設定密碼、金鑰和自動填充密碼管理器 google.com。

Passkey 跨裝置登入

如果你在 Windows 電腦上登入一個只有 iPhone 上有 Passkey 的網站，可以掃碼授權：

1. 網站登入頁選擇” 使用 Passkey”。
2. 選擇” 使用其他裝置”。
3. 電腦顯示二維碼，用 iPhone 掃碼。
4. iPhone 彈出 Face ID 驗證，確認後網站在電腦上完成登入。

這一過程透過藍芽 + 網際網路雙重加密通訊，Passkey 私鑰不會離開你的 iPhone。

現在就試：體驗無密碼登入

1. 登入 Google 賬號安全通行金鑰新增通行金鑰，體驗 Face ID/指紋登入。
2. 登入微軟賬號安全高階安全選項新增通行金鑰。
3. 在淘寶、京東、eBay、PayPal 等已支援 Passkey 的服務上嚐鮮。
4. 保留傳統密碼作為後備，逐步遷移高價值賬號到 Passkey。

過渡期建議

Passkey 並未完全取代密碼——許多網站仍處於“Passkey + 密碼”並存階段。建議：高價值賬號優先開啟 Passkey，但**不要立即刪除傳統密碼**，留作備用；同時繼續使用密碼管理器，因為不是所有網站都支援 Passkey。把 Passkey 看作“密碼管理器的強力補充”而非“立刻替換”。

七、硬體安全金鑰

如果你對安全的要求高於普通使用者——比如持有大量加密貨幣、是高管/記者/活動人士、或保護企業核心賬號——硬體安全金鑰是比簡訊驗證碼更可靠的選擇。

什麼是硬體安全金鑰？

硬體金鑰是一個物理裝置（形似 U 盤），內部安全晶片儲存你的私鑰，需要物理接觸或 NFC 才能完成驗證。代表產品：

- **YubiKey**（Yubico 公司）：最主流，支援 FIDO2/WebAuthn、U2F、OTP、PIV、OpenPGP，USB-C / USB-A / NFC 等多種介面。5 系列 50-70 美元。
- **Google Titan**：Google 自家出品，USB-C + NFC / 藍芽，約 30-40 美元。
- **OnlyKey**：開源替代品，支援多種協議。
- **國產替代**：如飛天誠信 ePasskey，價格更親民。

為什麼硬體金鑰比簡訊驗證碼安全？

- **抗釣魚**：和 Passkey 一樣，硬體金鑰只對真實域名響應，釣魚網站無法騙取驗證碼。
- **抗 SIM 劫持**：簡訊驗證碼可被 SIM-swap 攻擊截獲，硬體金鑰無需手機號。
- **抗中間人**：FIDO2 協議每次簽名都包含挑戰值，無法重放。
- **物理隔離**：私鑰永遠在硬體晶片內，惡意軟體即使攻陷你的電腦也讀不到。

適用場景

- 加密貨幣交易所賬號、錢包恢復。
- 企業 SSO、遠端辦公 VPN、SSH 登入伺服器。
- 郵箱、GitHub、社交媒體等” 主賬號”——一旦被攻破可重置所有其他賬號。
- 記者、律師、高管等高敏感人群。

如何配置硬體金鑰作為二次驗證

以 Google 賬號 + YubiKey 為例：

1. 購買 YubiKey（建議一次買兩把，一把日常用、一把備用，存放在不同地點）。
2. 登入 myaccount.google.com 安全兩步驗證。
3. 選擇” 新增安全金鑰” 把 YubiKey 插入 USB-C 口或貼近手機 NFC 感應區。
4. 按提示觸控 YubiKey 金屬觸點完成註冊。
5. 建議註冊第二把備用金鑰。
6. 在” 兩步驗證” 設定裡，把硬體金鑰設為首選驗證方式，簡訊驗證碼作為備用。

主流支援的網站：Google、GitHub、Microsoft、Apple ID、Facebook、Twitter、Dropbox、GitLab、AWS、Coinbase、Binance 等。

雙金鑰策略

硬體金鑰最大的風險是“丟失”。建議：(1) 同時註冊兩把不同的金鑰；(2) 一把隨身攜帶（鑰匙扣），一把存放在家中保險櫃或銀行保管箱；(3) 部分網站允許把“備用恢復碼”列印出來離線儲存。這樣即使主金鑰丟失，也不會被永久鎖在賬號外。

購買渠道要謹慎

務必從官方店鋪或授權代理購買，不要買二手或來源不明的金鑰——理論上預置過的硬體可能已被植入後門。YubiKey 的韌體在出廠後無法更新，但 Yubico 會持續推送安全公告，關注其官網即可。

八、家庭/團隊密碼共享

家庭場景下，Netflix、家庭 Wi-Fi、路由器後臺、智慧門鎖密碼需要全家共用；團隊場景下，伺服器、資料庫、社交媒體賬號需要多人協作。**用微信發密碼、寫在共享文件裡都是高危做法**——一旦聊天記錄或文件許可權洩露，密碼就洩露了。

1Password 的“共享保險庫”（Vault）

1Password 是家庭和團隊共享的標杆產品。它的核心機制是**多個保險庫**：

- **個人保險庫**：只有你能看，如銀行、郵箱等私人賬號。
- **共享保險庫**：家人或團隊成員都能看，如 Netflix、家庭 Wi-Fi、共享信用卡。
- **許可權分級**：團隊版可設定某些成員” 只檢視” 或” 只檢視但不編輯”。

家人或團隊成員無需共享主密碼，各自用自己的賬號登入，但可以看到被授權的共享保險庫內容。密碼修改後所有成員實時同步。

Bitwarden 的“組織”功能

Bitwarden 的免費版支援個人密碼共享（透過“傳送 Send”功能限時分享單條密碼）；付費版可建立“組織”，把若干條目共享給家庭成員或團隊同事，許可權可細分。Bitwarden 還提供自託管選項，適合對資料主權要求高的企業。

家庭場景：共享 Netflix、Wi-Fi 等密碼的安全方式

- **Netflix / 影片網站**：用 1Password 家庭版的” 共享保險庫” 存登入密碼，家庭成員各自登入即可，無需在群裡發。
- **家庭 Wi-Fi 密碼**：iOS 裝置間可透過 AirDrop 直接共享 Wi-Fi 密碼給來訪的客人，無需念出來；安卓 12+ 也有” 附近分享 Wi-Fi” 功能。
- **智慧門鎖、智慧家電**：儘量用廠商 App 的” 家庭成員” 功能為每人單獨建立賬號，而不是共享一個管理員賬號。
- **家用路由器後臺**：管理員密碼存到 1Password 共享保險庫，訪客 Wi-Fi 單獨開一個 SSID。

1Password 家庭版的價效比

1Password Families 約 4.99 美元/月，支援最多 5 位家庭成員，每人都有獨立的私人保險庫 + 4 個共享保險庫。算下來人均不到 1 美元/月，是目前家庭共享體驗最好的方案。Bitwarden 的家庭版 10 美元/年支援 6 人，價效比更高，但介面體驗略遜。

團隊場景：伺服器/資料庫密碼共享

- **1Password Teams / Business**：可按部門、專案組建立多個保險庫，細粒度許可權控制，所有訪問行為可審計。
- **Bitwarden Teams / Enterprise**：開源方案，可自託管，企業版支援 SSO 整合、目錄同步。
- **職責分離原則**：管理員密碼不應只有一人掌握——設定 2-3 名” 應急聯絡人”，確保關鍵人員離職/休假時業務不中斷。

- **離職流程**：員工離職時立即撤銷其密碼管理器訪問許可權，必要時旋轉所有他接觸過的高敏感密碼。

緊急訪問（Emergency Access）：萬一你出事

想象一個場景：你獨自管理家庭所有重要賬號，但意外住院或不幸離世，家人無法登入銀行、郵箱、保險賬號。**“緊急訪問”功能**正是為此設計——授權一位信任的家人/朋友，在你無法響應時獲取你的密碼庫。

Bitwarden 緊急訪問設定：

1. 登入 Bitwarden Web Vault 設定緊急訪問。
2. 新增信任聯絡人的郵箱，邀請對方接受。
3. 設定” 等待期”（建議 7-14 天）：從對方發起請求到你確認的時間視窗。
4. 如果你在等待期內不響應（拒絕或不操作），到期後對方自動獲得訪問權。
5. 你隨時可以手動拒絕請求或撤銷緊急訪問許可權。

1Password 緊急訪問：

1. 1Password 不直接叫” 緊急訪問”，但透過” 家庭/團隊組織” 功能可實現——把家人加入家庭版後，將部分條目放入” 共享保險庫”。
2. 對個人版使用者，建議把主密碼告知一位高度信任的家人，或寫入密封信封存放於保險櫃。
3. 1Password 還提供” 恢復碼”，開啟雙重認證時務必列印儲存。

現在就設：為家人留一把” 備用鑰匙”

1. 選定一位信任的家人作為” 緊急聯絡人”。
2. 在密碼管理器中設定緊急訪問（Bitwarden）或加入家庭共享保險庫（1Password）。
3. 把主密碼 + 恢復碼寫在紙上，封進信封，存放於家中保險櫃或告知可信家人。
4. 每半年回顧一次，確認緊急聯絡人是否仍然合適。

九、密碼管理器使用最佳實踐

- **設定強主密碼**：主密碼是最後一道防線，務必設定一個長且複雜的密碼，並牢記。
- **啟用雙重認證**：為密碼管理器賬號開啟雙重認證，增加一層保護。
- **定期檢查弱密碼**：大多數密碼管理器有” 安全審計” 功能，幫你找出弱密碼和重複密碼。

- **不要忘記主密碼**：主密碼無法找回，忘記就意味著丟失所有密碼。建議寫在紙上，放在安全的地方。
- **緊急訪問**：設定緊急聯絡人，萬一你無法訪問，信任的人可以獲取你的密碼。

重要提醒

密碼管理器雖然方便，但也有風險：如果主密碼洩露，所有密碼都暴露了。所以主密碼絕對不能和其他密碼相同，也不能寫在容易被找到的地方。另外，建議選擇有良好安全記錄的服務商。

現在就試：開始使用密碼管理器

1. 選擇一個密碼管理器（推薦 Bitwarden 或系統自帶）。
2. 註冊賬號，設定一個強主密碼。
3. 匯入現有密碼（如果有的話）或開始新增新密碼。
4. 安裝瀏覽器外掛或手機 App，體驗自動填充功能。
5. 檢查安全審計報告，找出需要改進的密碼。

三、進階篇（第 15-20 章）

第十五章：電池科學與續航最佳化

電池就是你手機的“能量心臟”。這章帶你搞懂電池的小脾氣，學會科學的充電和續航技巧，徹底告別電量焦慮。

一、電池是怎麼回事？迴圈與健康度

手機電池（鋰離子或鋰聚合物）像個能量海綿：充電就是吸水（存電），放電就是擠水（用電）。有幾個特點值得記住：

- **隨用隨充**：和舊電池不同，它沒有記憶效應，不用非得用完再充。
- **怕熱**：高溫是電池頭號殺手。
- **會老化**：用久了，最大容量就會下降，這是正常現象。

核心指標：迴圈次數與健康度

- 一次迴圈不是充一次電。比如今天用 60% 充滿，明天用 40% 充滿，總消耗 100% 電量才算一次迴圈。
- 電池健康度就是當前最大容量佔全新時的百分比。低於 80% 時，你會明顯感覺續航變短。
- **怎麼看？** iPhone：設定 > 電池 > 電池健康與充電；安卓：一般去設定裡搜電池健康。

二、充電玄學？快充、無線充與黃金法則

- **快充傷電池？** 用原裝或認證的快充配件，傷害很小。手機和充電頭有溫控，過熱會自動降功率。
- **無線充電**：方便但效率較低，發熱更明顯。適合整夜充電。
- **黃金養生法則**：儘量讓電量保持在 20%-80% 之間，別老讓它餓著（0%）或撐著（100%）。

iPhone”80% 充電限制” 功能——把黃金法則變成系統設定

iPhone 15/16 系列（iOS 17 以上）已經把“20%-80% 法則”做成了系統功能。開啟後，手機充到 80% 會自動停止充電，從而大幅延緩電池老化。設定路徑：

設定 > 電池 > 電池健康與充電 > 充電最佳化 > 80% 限制
三個選項的含義：(1) **不限**——充到 100%；(2) **最佳化電池充電**——學習你的作息，只在凌晨最後一段延遲到 100%；(3)**80% 限制**——直接停在 80%，最護電池。長期在家/辦公室有充電條件的人，強烈建議選 80% 限制；經常出差、需要滿電出門的人，可以選最佳化電池充電。

GaN 充電器——出差黨的好搭檔

你可能注意到現在的快充頭越做越小，背後功臣是 **GaN（氮化鎵）**這種新型半導體材料。相比傳統矽基充電器，GaN 充電器有幾個明顯優勢：

- **體積小**：同等功率下，GaN 充電器可以做到傳統矽基的 1/2 甚至 1/3 大小，65W 功率的 GaN 頭比蘋果老的 5W 頭還小。
- **效率高、發熱低**：氮化鎵的開關損耗遠低於矽，能量轉換效率可達 95% 以上，充電時不容易燙手。
- **適合出差**：一個 65W 或 100W 的 GaN 多口充電器，能同時給手機、平板、筆記本充電，比帶三塊" 磚頭" 輕便得多。
- **選購建議**：認準支援 PD 協議的 GaN 充電器（65W 起步可充筆電），知名品牌如倍思、安克、努比亞、小米等均可。注意核對資料線是否匹配（高功率需用 E-Marker 晶片的 Type-C 線）。

三、快充協議全解析：為什麼你的充電器充不快？

你可能遇到過這種情況：明明手機支援快充，換了充電器卻變得很慢。這是因為快充不是簡單的“功率大就行”，而是需要充電頭、資料線、手機三方“說同一種語言”——這就是快充協議。

快充的基本原理

充電功率 = 電壓 × 電流（$P = U \times I$）。要提升功率，要麼提高電壓，要麼提高電流，或者兩者都提高。不同協議選擇了不同的技術路線。

快充的工作流程：

1. **握手階段**：手機和充電頭透過資料線通訊，交換各自支援的協議和功率資訊。
2. **協商階段**：雙方確定一個共同支援的最高功率檔位。
3. **充電階段**：充電頭按協商好的電壓電流輸出，手機端進行電源管理。
4. **調整階段**：隨著電量增加，手機會逐步降低充電功率，保護電池。

主流快充協議詳解

USB PD（Power Delivery）——通用協議之王

- **特點**：USB-IF 組織制定的官方標準，覆蓋範圍最廣。PD 3.0 支援最高 100W（20V/5A），PD 3.1 擴充到 240W（48V/5A）。
- **電壓檔位**：5V、9V、15V、20V，PD 3.1 還支援 28V、36V、48V。
- **相容性**：iPhone 全系、谷歌 Pixel、部分安卓旗艦、膝上型電腦、Switch 遊戲機都支援 PD 協議。
- **優勢**：通用性強，一個充電頭可以給多種裝置充電；支援 PPS（可程式設計電源），可以精細調節電壓電流。
- **劣勢**：需要 Type-C 介面和專用晶片，成本較高。

高通 QC（Quick Charge）——安卓陣營的元老

- **QC 3.0**：支援 3.6V-20V 以 0.2V 為步進的可調電壓，最高 18W。
- **QC 4.0/4+**：相容 USB PD 協議，支援最高 27W，加入雙路充電技術。
- **QC 5.0**：支援最高 100W+，0-50% 僅需 5 分鐘，大幅改善發熱。
- **相容性**：使用高通驍龍處理器的手機大多支援，但各品牌通常會優先使用自己的私有協議。

OPPO VOOC/一加 WARP/真我 Dart——低壓大電流的代表

- **技術路線**：低壓大電流（5V/4A 起步），發熱更小，效率更高。
- **VOOC 1.0**：20W（5V/4A），2014 年釋出，開創快充時代。
- **SuperVOOC**：50W-65W，採用雙電芯串聯設計。
- **SuperVOOC 2.0**：65W，30 分鐘充滿 4000mAh 電池。
- **150W/240W 超級閃充**：目前最快的量產快充技術。
- **特點**：需要專用充電頭和資料線（有額外觸點），不相容其他協議。

華為 SuperCharge——高壓快充的演進

- **FCP（Fast Charge Protocol）**：9V/2A，18W，相容 QC 2.0。
- **SCP（Super Charge Protocol）**：支援更高功率，從 22.5W 到 66W 再到 100W。
- **技術特點**：採用電荷泵技術，將高壓轉換為低壓大電流輸入電池，效率高達 98%。

- **相容性**：華為/榮耀手機專用，部分充電器支援 PD 協議。

小米澎湃快充——自研協議的崛起

- **澎湃 P1 晶片**：小米自研的快充管理晶片，支援 120W 有線快充。
- **澎湃 G1 晶片**：電池管理晶片，與 P1 配合實現更高效的快充。
- **技術特點**：採用雙電芯串聯 + 電荷泵方案，支援多種協議相容。

蘋果快充——保守但穩定

- **協議**：僅支援 USB PD 協議。
- **功率**：iPhone 8 及以後支援快充，最高約 27W（iPhone 15 Pro Max 可達 27W+）。
- **特點**：功率相對保守，但充電曲線最佳化好，發熱控制出色。
- **配件要求**：需要 USB-C to Lightning 線（或 USB-C to USB-C 線）和 PD 充電頭。

快充協議對比表

協議	最高功率	技術路線	相容性
USB PD 3.1	240W	高壓/多檔位	最廣泛
QC 5.0	100W+	高壓可調	高通機型
VOOC/Dart	240W	低壓大電流	OPPO 系專用
SuperCharge	100W	電荷泵	華為/榮耀
蘋果快充	~27W	PD 協議	iPhone/iPad

無線充電協議

無線充電也有自己的協議標準：

- **Qi 標準**：WPC（無線充電聯盟）制定，最通用的標準。基礎 Qi 支援 5W-15W。
- **Qi2 標準**：2023 年釋出，引入磁吸定位（類似蘋果 MagSafe），支援 15W+。
- **私有協議**：各品牌有自己的高速無線充電，如小米 50W 無線充、華為 50W 無線充等，需要專用充電器。

快充選購指南

- **看協議匹配**：購買充電器時，確認它支援你手機的快充協議。第三方充電器通常會標註支援的協議。
- **看資料線**：大功率快充需要專用資料線。普通資料線可能只支援 60W 以下。

- **看認證**：蘋果使用者建議選擇 MFi 認證的資料線；安卓使用者選擇品牌原裝或知名第三方。
- **看發熱**：快充時手機發熱是正常的，但如果燙手就不正常了，建議暫停充電或更換充電器。

快充安全提示

- 避免邊充邊玩大型遊戲，會加劇發熱。
- 不要使用劣質充電器和資料線，可能導致安全隱患。
- 充電時取掉厚重的手機殼，幫助散熱。
- 如果充電器或手機異常發熱、有異味，立即停止使用。

四、電池新技術：矽碳負極電池

如果你留意 2024-2026 年的旗艦手機釋出會，“矽碳負極”這個詞幾乎成了高階電池的代名詞。這是鋰電池負極材料的一次重要升級，原理其實不復雜：

- **傳統石墨負極**：手機電池一直用的是石墨做負極，技術成熟、穩定，但能量密度已接近理論上限（約 372mAh/g）。
- **摻入矽材料**：矽的理論比容量是石墨的 10 倍左右（約 4200mAh/g）。把少量矽摻入石墨裡做成”矽碳複合負極”，就能顯著提升整塊電池的能量密度。
- **能量密度提升**：同等體積下，矽碳負極電池的容量可以提升約 **15%-20%**。這正是 2024-2026 年旗艦機能在 7-9mm 機身裡塞進 5500-6000mAh 大電池的關鍵原因。

哪些機型用上了？

- **榮耀**：Magic 系列率先商用矽碳負極，Magic6 Pro、Magic7 系列、Magic V3 摺疊屏均採用。
- **小米**：小米 14 Ultra、小米 15 系列搭載新一代矽碳負極電池。
- **vivo**：X100 Ultra、X200 Pro 系列使用高密度矽碳負極方案。
- **華為/OPPO 等**：2025-2026 年旗艦普遍跟進，矽碳負極正在從”黑科技”變成高階機的標配。

對使用者的實際影響：矽碳負極帶來的最大好處是“同體積電池容量更大”——手機可以做得更薄，同時續航不縮水；或者機身不變厚，但電池從 4500mAh 漲到 5500mAh。換句話說，你不用為續航付“厚重”的代價了。需要注意的是，矽碳負極電池在低溫和滿充管理上仍需配合快充晶片做最佳化，這也是廠商建議長期保持 80% 充電的原因之一。

五、這才是真正的省電技巧

- **螢幕（耗電王）**：調低亮度、縮短自動鎖屏時間。OLED 屏手機開深色模式能省不少電。

- **訊號差**：在電梯、地下室，手機會拼命搜訊號，耗電極快。
- **5G**：如果訊號不好，手動切回 4G，續航會好很多。
- **定位服務**：非導航類 App，定位許可權設為” 僅使用時允許”。
- **限制後臺活動**：iPhone 在設定 > 通用 > 後臺 App 重新整理；安卓在應用電池設定裡。

來檢查一下：給你的電池做個體檢

1. 檢視健康度：找到並記下你的電池最大容量。
2. 分析耗電詳情：看電池設定裡的圖表和 App 列表，找出耗電元兇。
3. 制定對策：限制後臺耗電異常的 App。

第十六章：賬號體系與雲服務

我有個朋友，去年手機被偷了。他第一時間想的不是“手機多少錢”，而是“我裡面的照片還能找回來嗎”。很遺憾，他沒開雲備份。手機是身外之物，但賬號裡的資料，是真的丟了就再也回不來的東西。這章我們就來聊聊，怎麼給你的數字生活上個真正的保險。

一、你的手機賬號，到底管著多少東西？

很多人覺得賬號就是用來下載 App 的。其實，一個手機賬號背後牽著的東西，比你想象的要多得多：

- **通訊錄**：你存了幾年甚至十幾年的電話號碼。
- **照片和影片**：從孩子出生到畢業典禮，從第一次約會到結婚旅行。
- **備忘錄**：你隨手記的靈感、日記、賬號密碼（對，很多人把密碼記在備忘錄裡）。
- **App 購買記錄**：花了幾百塊買的付費 App 和內購。
- **查詢手機**：手機丟了之後，這是唯一能幫你定位、鎖機、遠端抹資料的工具。

如果你用的是蘋果手機，以上所有東西都綁在你的 Apple ID 上。如果你用的是安卓，那就綁在你的華為/小米/OPPO/vivo 賬號上，還有你的谷歌賬號。這些賬號就是你數字生活的“戶口本”，丟了戶口本，很多東西就說不清了。

二、三個必須做的事，現在就做

1. **開啟雙重認證**：不管你用的是蘋果還是安卓，第一件事就是去賬號設定裡把” 雙重認證” 或” 兩步驗證” 開啟。這東西就像給你的門加了一把反鎖——光有密碼不夠，還得你的手機收個驗證碼。就算有人在網上猜到了你的密碼，沒有你的手機，他也進不去。
2. **繫結一個可靠的恢復方式**：雙重認證很安全，但也有個風險——你自己的手機丟了怎麼辦？所以系統會提示你設定” 恢復金鑰” 或” 備用郵箱/手機號”。這個是你在緊急情況下拿回賬號的唯一途徑，最好記在紙上，別存在手機裡。
3. **確認” 查詢手機” 已開啟**：這個功能在手機丟了之後能幫你定位、鎖定、播放聲音、甚至遠端擦除所有資料。是最後一道防線。很多人買手機的時候跳過了這一步，後面想用的時候發現沒開，就晚了。

三、iCloud：蘋果使用者的“空氣”

用蘋果手機的人，遲早要面對一個靈魂拷問：iCloud 空間滿了怎麼辦？

蘋果免費給你 5GB，說實話，這 5GB 連備份一次都不夠。所以大部分人最後都會收到那個煩人的彈窗：“iCloud 儲存空間已滿”。這時候你有三個選擇：

- **清理**：去設定你的名字 iCloud 管理儲存空間，看看是誰佔了你的空間。一般來說，照片和備份是大頭。可以先刪掉不需要的舊備份。
- **升級**：50GB 每個月 6 塊錢，200GB 每個月 21 塊錢。個人覺得 50GB 對大多數人來說夠了，主要是用來備份和存照片的縮圖。
- **曲線救國**：如果你不想花錢，可以把照片定期匯出到電腦上，然後關掉 iCloud 照片同步。但這樣的話，你就失去了照片自動備份的便利性。

一個重要的提醒：iCloud 照片相簿不是“網盤”——它是“同步”。你在手機上刪了一張照片，iCloud 上也會刪。如果你想要的是“備份到雲端然後清空手機”，那 iCloud 照片不是你要的東西，你需要的是 iCloud 雲備份，或者第三方的網盤。

iCloud+：付費使用者才能享受的隱私三件套

只要升級到任意付費 iCloud 檔位（50GB/200GB/2TB 等），就自動成為 **iCloud+** 使用者，解鎖三個隱私保護功能：

- **Private Relay（私密中繼）**：蘋果版的”VPN”。Safari 瀏覽的流量經蘋果伺服器中轉後再發往網站，沒有人（包括蘋果）能同時看到你是誰、你訪問了什麼。適合在公共 Wi-Fi 下保護隱私。設定路徑：設定 > 你的名字 > iCloud > 私密中繼。
- **Hide My Email（隱藏郵件）**：在網站/App 註冊時生成一次性隨機郵箱地址，郵件會自動轉發到你的真實郵箱。不想再收這家網站的郵件，直接關掉這個隨機地址就行，真實郵箱不暴露。設定路徑：設定 > 你的名字 > iCloud > 隱藏郵件。
- **自定義電子郵件域名**：可以用你自己的域名（比如 @ 你的姓.com）收發 iCloud 郵件，適合個人品牌或家庭郵箱管理。

提示：Private Relay 在某些國家/地區不可用，國內使用者可能需要其他方案補充。

iCloud 高階資料保護（ADP）——真正的端到端加密

2022 年底蘋果推出了 **iCloud 高階資料保護（Advanced Data Protection，ADP）**，把端到端加密（E2EE）從原來的健康、密碼等少數資料，擴充到了**絕大部分 iCloud 資料**——照片、備忘錄、iCloud 雲備份、裝置資訊等共 23 類資料。

- **什麼是 E2EE**：加密金鑰只存在你的裝置上，連蘋果伺服器都沒有。就算蘋果被駭客攻破，或者執法部門要求調取，蘋果也拿不出能解密的明文資料。
- **開啟路徑**：設定 > 你的名字 > iCloud > 高階資料保護，按提示設定恢復聯絡人/恢復金鑰（**必做**，丟了金鑰你的資料也救不回來）。

- **注意事項**：(1) 開啟後務必妥善保管恢復金鑰，否則賬號丟失時無法找回資料；(2) 共享相簿、iCloud 共享相簿等少數功能不參與 E2EE；(3) 中國大陸區 Apple ID 暫不支援 ADP，需要切換到其他地區賬號。

一句話建議：如果你的賬號裡有重要隱私資料，且賬號地區支援，強烈建議開啟 ADP——這是普通人能獲得的最強雲端資料保護。

四、同步和備份，別搞混了

這兩個詞聽起來差不多，但後果完全不同：

- **同步**是雙向的：你在手機上刪了，雲端也刪了；你在雲端刪了，手機上也跟著刪。iCloud 照片、通訊錄、備忘錄這些都是同步。好處是多裝置之間保持一致，壞處是" 刪了就真沒了"。
- **備份**是單向的：手機把當前狀態拍一張" 快照" 存到雲端。之後你在手機上刪了東西，雲端備份裡的那份還在。iCloud 雲備份、電腦本地備份，都是備份。好處是資料更安全，壞處是佔空間，而且不能實時更新。

五、安卓使用者的賬號矩陣

安卓這邊的情況比蘋果複雜一些，因為你的資料可能分散在好幾個賬號裡：

- **品牌賬號**（華為/小米/OPPO/vivo）：管著你的雲空間、查詢手機、應用商店、錢包。每個品牌都有自己的雲服務，通常是基礎免費 + 擴容收費的模式。
- **谷歌賬號**：如果你用 Gmail、Google 相簿、Google 通訊錄，這些東西都在谷歌賬號裡。谷歌免費給 15GB，比蘋果大方，但也是和 Gmail、Drive 共用的。
- **微信/QQ**：它們是獨立於手機系統的。你的聊天記錄和微信收藏，既不在品牌賬號裡也不在谷歌賬號裡，只存在微信自己的伺服器上。

所以安卓使用者做備份的時候，要分別檢查：品牌雲備份、谷歌備份、微信聊天記錄備份，三者缺一不可。

六、家庭共享：讓家人也受益

蘋果的“家人共享”可以讓你和家人共享 App 購買、音樂、電影、iCloud 空間，還能共享位置。最多可以加 6 個人。設定方法是：設定你的名字家人共享。華為和小米也有類似的功能，在賬號設定裡能找到“家庭共享”或“家庭雲”。

一個很實用的場景：給爸媽的手機上設定好家人共享，這樣你就能隨時看到他們的位置（前提是他們同意），萬一老人走丟了或者手機聯絡不上了，能有個定位。

賬號被盜的後果比你想象的要嚴重

如果駭客拿到了你的賬號密碼，他可以：遠端鎖定你的手機並勒索贖金、檢視你的照片和通訊錄、用你的支付方式買東西。更麻煩的是，如果你的賬號繫結了太多東西，找回賬號本身就是一場噩夢。所以，雙重認證，現在就去開。

實操：給你的賬號做一次全面體檢

1. 開啟手機設定，進入賬號頁面，確認雙重認證已開啟。
2. 檢查” 受信任的電話號碼” 和” 恢復金鑰” 是否已設定，金鑰記在紙上。
3. 進入 iCloud 或雲空間管理，看看哪些資料在同步，關閉不需要的。
4. 確認” 查詢我的手機” 已開啟。
5. 如果你用的是安卓，檢查品牌雲備份、谷歌備份、微信備份三者是否都覆蓋到了。

第十七章：手機與其他裝置的聯動

你手裡的手機，其實是一個超級遙控器兼資料中心。它不光能打電話，還能把電腦、電視、手錶、音箱、汽車串在一起，讓這些裝置不再是孤島。這章我們就來聊聊怎麼讓它們“組隊打怪”。

一、與電腦協同：手機和電腦不再是兩個世界

說實話，我以前最煩的就是手機拍了個照片，想放到電腦上修一下，還得找資料線、等傳輸。後來發現，其實根本不需要資料線，好的聯動比資料線快十倍。

- **蘋果生態（AirDrop + 通用剪貼簿）**：如果你用 iPhone 配 Mac，隔空投送是你第一個該學會的功能。在手機上點分享隔空投送選你的 Mac，照片瞬間就過去了，不用流量不用 Wi-Fi，用的是藍芽和 Wi-Fi 直連。還有個容易被忽略的功能叫” 通用剪貼簿”——手機上覆制一段文字，在 Mac 上直接貼上就出來了，簡直像魔法一樣。
- **安卓 +Windows（連線至 Windows）**：微軟有個叫” 連線至 Windows” 的 App，三星手機內建，其他安卓手機去應用商店下一個就行。配對之後你可以在電腦上直接看手機通知、回簡訊、翻照片，甚至接打電話。我自己用了兩年，開會的時候不用一直低頭看手機，電腦螢幕上彈個通知就夠了。
- **品牌多屏協同**：華為的” 多屏協同” 和小米的” 小米妙享” 更狠——直接把手機螢幕投到電腦上，你可以用滑鼠拖拽檔案從手機到電腦，也能在電腦上刷抖音、回微信。跨裝置複製貼上、檔案拖拽都行，體驗非常絲滑。

二、投屏到電視：全家一起看

過年回家的時候，我堂妹想在電視上放她手機裡的畢業旅行影片。折騰了半天，最後發現其實非常簡單：

- **iPhone 電視**：從右上角下拉控制中心，點那個兩個方框疊在一起的圖示（螢幕映象），找到你的電視名稱，一點就連上了。前提是電視和手機連的同一個 Wi-Fi，而且電視支援 AirPlay（大部分智慧電視都支援）。
- **安卓電視**：各家叫法不同，華為叫” 無線投屏”，小米叫” 投射”，OPPO/vivo 叫” 多屏互動”。一般在快捷開關面板裡能找到，找不到就去設定裡搜” 投屏”。

- **影片 App 直接投**：如果你只是想在電視上看愛奇藝、B 站、YouTube，其實更簡單——在播放介面找一個矩形帶 Wi-Fi 符號的小圖示，點一下，選電視，影片就過去了。這時候手機可以幹別的，影片在電視上繼續放，互不影響。
- **投屏不成功怎麼辦**：90% 的情況是沒連同一個 Wi-Fi。還有 10% 是電視的投屏功能沒開，去電視設定裡找” 多屏互動” 或”AirPlay” 開關。

三、與智慧手錶/手環：健康管家長在手腕上

我給我媽買了個手環，她一開始說“我又不運動買這個幹嘛”。結果用了兩週，她發現自己每天只睡五個多小時，開始主動調整作息了。這就是手錶/手環最實用的價值——把你看不見的健康資料變成看得見的數字。

- **通知過濾器**：手機在包裡或口袋裡震個不停的時候，抬手腕看一眼——是老闆的微信還是拼多多的促銷，一秒判斷，不值得掏手機的訊息就讓它去。
- **健康資料同步**：手錶 24 小時監測心率、血氧、睡眠、步數，這些資料會自動彙總到手機上的健康 App 裡。久而久之你能看到自己的趨勢——這個月是不是走得太少，這周睡眠質量有沒有變好。
- **交通卡/門禁卡同步**：在手機上綁好的公交卡和門禁卡，大部分品牌會自動同步到手錶上。坐地鐵的時候不用掏手機，手腕一靠閘機就過去了，真的會有種” 未來已來” 的感覺。
- **找手機神器**：手錶上一般都有” 找手機” 功能，點一下手機就會響。我自己用這個功能的頻率遠超預期——手機隨手一放就忘在哪了。

四、與智慧家居：手機是家裡的“總開關”

智慧家居聽起來很玄乎，但說穿了就三件事：用手機遠端控制、設定自動化規則、語音操控。

說說我自己的幾個實際用法，你就知道這玩意兒多實用了：

- **空調**：夏天回家路上提前用手機把空調開啟，進門就是涼快的。花一百多買個空調伴侶插座就能實現，不用換空調。
- **燈**：睡前躺床上，懶得起來關燈，手機上點一下或者喊一聲” 嘿 Siri/小愛同學，關燈”。
- **掃地機器人**：出門前手機上點一下” 開始清掃”，回來地就乾淨了。還能設每天定時掃。
- **攝像頭**：家裡有老人小孩或者寵物的，手機上隨時能看一眼家裡的情況。

選哪個生態？一句話總結：

- 用 iPhone 優先選支援 Apple HomeKit 的裝置（貴一點但隱私保護好）。
- 裝置多求價效比小米米家生態最全，燈、插座、掃地機什麼都有。
- 用華為手機優先鴻蒙智聯，和華為裝置體驗最好。

五、藍芽耳機與音箱：無線的自由

從有線耳機換成藍芽耳機，對我來說是手機體驗升級裡最明顯的一個。再也不會從包裡掏出一團亂麻，也不用擔心線掛在門把手上了。

- **配對很簡單**：耳機第一次開機一般自動進入配對模式（閃燈），手機開啟藍芽就能搜到，點一下就連上了。之後每次拿出來自動連，什麼都不用做。
- **同品牌有加成**：AirPods 配 iPhone 會自動彈窗顯示電量，還能在多個蘋果裝置之間無縫切換。華為 FreeBuds 配華為手機、小米耳機配小米手機也都有類似的彈窗和最佳化。
- **藍芽音箱**：家庭聚會、戶外露營的時候，一個藍芽音箱能讓氣氛翻倍。連上之後，手機就是你的 DJ 臺。
- **注意延遲**：打遊戲的時候如果用藍芽耳機，可能會有輕微延遲（槍響了半秒才聽到槍聲）。如果對延遲敏感，要麼用有線耳機，要麼買支援” 低延遲模式” 的藍芽耳機。

UWB 超寬頻技術——比藍芽更精準的” 空間感”

除了藍芽，新款手機裡還藏著一顆 UWB（Ultra-Wideband，超寬頻）晶片。它的能力是**精確測距和空間定位**——能算出另一個裝置離你多遠、在哪個方向，精度可達釐米級。藍芽只能告訴你“在附近”，UWB 能告訴你“在你左前方 1.2 米”。常見應用：

- **AirTag/查詢精確**：iPhone 查詢 AirTag 時顯示”7 米遠、向左走” 的箭頭導航，靠的就是 UWB。普通藍芽只能告訴” 在不在範圍內”。
- **數字車鑰匙**：搭載 UWB 的 iPhone 和安卓旗艦，可以當車鑰匙走近車輛自動解鎖、走遠自動鎖車，還能區分駕駛員和乘客位置，防中繼破解。
- **室內導航**：商場、機場裡透過 UWB 信標實現” 過道級” 導航，比 GPS 準得多。
- **支援的機型**：iPhone 11 及以後機型均帶 UWB 晶片（U1/U2）；安卓陣營小米、三星、vivo、OPPO 部分旗艦也已搭載。

LE Audio 與 Auracast——藍芽音訊的新一代標準

用了二十年的經典藍芽音訊（A2DP）正在被 **LE Audio**（低功耗音訊）取代，這是藍芽 5.2 以後的新一代音訊標準：

- **更省電、音質更好**：採用全新的 LC3 編碼，比老標準的 SBC 音質更好、功耗更低，耳機續航明顯延長。
- **雙路音訊**：左右耳塞獨立傳輸，單耳模式切換不掉線。
- **Auracast 廣播音訊**：LE Audio 的招牌功能。一個手機/電視可以同時向**無數副**耳機廣播音訊——機場候機廳、博物館、健身房、教室裡都能用一個發射源，多人用自己的耳機聽同一畫面。
- **現狀**：2024-2026 年新上市的旗艦耳機和手機普遍支援 LE Audio，老裝置需硬體升級

無法軟體解鎖。買新耳機時可以留意是否支援 LE Audio/Auracast 標識。

六、與車載系統：開車時的安全搭檔

開車看手機是違法的，也是要命的。但有時候確實需要導航、接電話、放音樂。這時候 CarPlay（蘋果）和 Android Auto（安卓）就派上用場了。

- **怎麼連**：如果你的車支援（近幾年大部分新車都支援），用資料線把手機插到車上的 USB 口，車載螢幕會自動切換成 CarPlay 或 Android Auto 介面。部分新車支援無線連線，更省事。
- **能幹嘛**：導航（比車載導航好用多了）、打電話（方向盤接聽）、聽歌（手機裡的音樂 App 直接投到車上）、語音發微信（Siri 或小愛幫忙讀訊息和回覆）。
- **老車也能用**：如果你的車比較老不支援，花幾百塊買個外掛的 CarPlay 螢幕，粘在儀表臺上，體驗差不多。

七、聯動中的安全和隱私

裝置聯的越多，隱私洩露的風險面就越大。幾點需要注意：

- 公共場合投屏之前，先看看手機通知欄有沒有私密訊息，別讓全家看到不該看的內容。
- 把裝置送給別人或者賣掉之前，先解綁——去手機設定裡找到那個裝置，選”移除”或”取消配對”。
- 智慧家居攝像頭，買有品牌的、有加密的，別買幾十塊的雜牌。聽說過太多攝像頭被駭客直播的新聞了。

來試試：把手機螢幕投到電視上，並設定一個智慧場景

1. 確保手機和電視連同一個 Wi-Fi。
2. 在電視上找到”螢幕映象”或”AirPlay”功能並開啟。
3. 在手機上：iPhone 從控制中心點「螢幕映象」；安卓下拉找「無線投屏」。
4. 選一張你最喜歡的照片投到電視上，讓家人看看。
5. 如果有智慧家居裝置，試著在手機 App 裡建立一個”離家模式”：一鍵關燈、關空調、啟動掃地機器人。

案例 1：電池續航最佳化實戰

- **場景**：使用者小米 14 續航差，半天就沒電。
- **步驟**：(1) 設定電池檢視耗電排行 (2) 識別異常後臺 App (3) 關閉 5G 切 4G (4) 限制

後臺 (5) 開啟省電模式 (6) 充電時摘殼散熱。

- **預期結果：**續航延長 30-50%，電池健康度資料可讀。

第十八章：資料備份與遷移

我一個鄰居，手機掉進水池裡撈出來已經開不了機了。她最心疼的不是手機本身，而是裡面三年沒備份過的照片——孩子從出生到會走路的全部記錄。聽她說完我心裡特別不是滋味，因為這件事完全是可以避免的。這章我們就認認真真地給資料上一道保險。

一、先搞清楚：你手機裡哪些東西丟了就沒了

不同資料的“生存狀態”不一樣，你得分清楚：

- **存在手機本地、丟了就真的沒了的：**照片影片（沒開雲同步的話）、備忘錄、簡訊、通話記錄、下載的檔案。
- **存在 App 伺服器上、換手機重新登入就有：**微信好友列表、淘寶訂單、銀行 App 裡的交易記錄。這些東西儲存在對應公司的伺服器上，不是存在你手機裡的。
- **處於灰色地帶、需要特別操作的：**微信聊天記錄（雖然微信伺服器存了，但你自己沒法從伺服器下載）、手機銀行繫結的裝置憑證、兩步驗證 App 裡的金鑰（Google Authenticator 那種）。

所以你要重點保護的，是第一類和第三類。

二、整機備份：一次搞定所有東西

整機備份就像給手機拍一張完整的“快照”——當前系統設定、桌面佈局、App 資料、照片、通訊錄，全部打包。

方案一：電腦本地備份（最可靠，推薦有電腦的人）

- **iPhone：**把手機用資料線連到電腦上，開啟 iTunes（Windows 舊版）或 Finder（Mac），在裝置頁面裡選” 立即備份”。注意要勾上” 給備份加密”，否則密碼和健康資料不會被備份進去。
- **安卓：**各品牌有自己的電腦端軟體，比如華為手機助手、小米助手、三星 Smart Switch。連上電腦按提示操作就行。

- 電腦備份的好處是不用花錢買雲空間，而且恢復速度比雲端恢復快得多。缺點是需要你記得定期去插線備份——大部分人做不到這一點。

方案二：雲備份（最省心，推薦所有人）

- **iPhone**：設定你的名字 iCloud iCloud 雲備份開啟。手機插著電源、連 Wi-Fi、鎖屏的時候會自動備份。免費的 5GB 大機率不夠，建議花 6 塊錢升級到 50GB。
- **安卓**：各家品牌雲都有備份功能。華為手機去設定搜" 雲備份"，小米去" 小米雲服務"，OPPO/vivo 也類似。另外還有一個谷歌自帶的備份（設定系統備份），這個也開啟它。
- 雲備份最大的好處是" 不用你操心"，自動搞定。最大的壞處是恢復的時候要下載很久，而且免費空間一般不夠用。

三、分類備份策略：重要的單獨另存一份

整機備份是兜底的，但有些東西太重要了，值得你單獨另存一份：

照片和影片（最珍貴的數字資產）

- **雙保險方案**：開 iCloud 照片/Google 相簿自動同步，同時每隔幾個月手動把照片複製一份到電腦或行動硬碟上。這樣雲端一份、本地一份，互為備份。
- **不想花錢的方案**：Google 相簿免費給 15GB，可以設定為" 壓縮畫質" 來省空間。或者每隔一個月用資料線把照片全部拷到電腦上。

通訊錄（容易忽略但極其重要）

- iPhone 通訊錄預設和 iCloud 同步，只要開了同步就不用額外備份。但如果你想再安全一點，可以去 iCloud.com 匯出 vCard 檔案存電腦上。
- 安卓通訊錄通常和谷歌或品牌雲同步。去通訊錄 App 的設定裡匯出.vcf 檔案，就是一份完整的備份。

微信聊天記錄（單獨處理，重中之重）

這個必須單獨拿出來說，因為整機備份裡微信聊天記錄很多時候不完整，而且換機工具根本搬不動它。

- **方法一：遷移到新手機**。微信我設定通用聊天記錄遷移與備份遷移遷移到手機/平板。新舊手機連同一個 Wi-Fi，掃碼就能傳。注意：遷移完成之前，舊手機別退出微信。
- **方法二：備份到電腦**。電腦上裝微信 PC 版，左下角三條槓遷移與備份備份與恢復備份聊天記錄到電腦。之後換手機了可以再恢復到新手機上。

- **方法三：備份到雲端**。微信也支援備份到 iCloud 或手機雲空間，但注意：這個備份只能在你當前手機上恢復，不能跨裝置還原。換手機的話，還是得用前兩種方法。

四、換機指南：從舊手機到新手機

換新手機是高興事，但如果資料沒搬過去就變成頭疼事了。三種情況分別對待：

同品牌升級（最簡單）

- **iPhone 新 iPhone**：開機啟用時把舊手機放在旁邊，會自動彈出” 快速開始”，用舊手機掃新手機上的圖案就行。所有資料、設定、App 佈局都會原封不動搬過去，包括 Wi-Fi 密碼都會自動帶過去。大概需要二三十分鐘。
- **安卓同品牌**：用品牌自帶的換機工具。華為用” 手機克隆”，小米用” 小米換機”，OPPO 用” 手機搬家”，vivo 用” 互傳”。步驟都差不多：新舊手機裝同一個 App、掃碼連線、勾選要搬的東西、開傳。

跨品牌換機（安卓換安卓，不同品牌）

用第三方換機工具或者用舊品牌自帶的換機 App（大部分品牌的換機 App 也支援換到其他安卓手機）。能搬走的有：通訊錄、照片、簡訊、通話記錄、部分 App。搬不走的：App 內的登入狀態（你得重新登入）、微信聊天記錄（用微信自己的遷移功能）。

跨平臺換機（安卓換 iPhone，或反過來）

- **安卓 iPhone**：蘋果有個” 轉移到 iOS”App，安卓手機上下載後，在新 iPhone 啟用時選擇” 從安卓裝置轉移資料”。能搬走通訊錄、照片、簡訊等基礎資料，但 App 資料搬不了。微信聊天記錄還得用微信自己的遷移功能。
- **iPhone 安卓**：各安卓品牌有對應的遷移工具，但說實話，跨平臺遷移體驗不如同平臺好。最重要的事先單獨做了：通訊錄匯出、照片備份、微信聊天記錄遷移。剩下的慢慢來。

五、備份的黃金法則：三二一原則

這是資料安全領域的一個經典原則，簡單易記：

- **3**：重要資料至少保留三份（一份原件，兩份備份）。
- **2**：備份用兩種不同的儲存介質（比如雲端一份，行動硬碟一份）。
- **1**：至少有一份備份放在不同的物理位置（防止火災、盜竊等同時毀掉所有資料）。

對普通人來說，做到“雲端一份、電腦一份”就已經算很好了。不用搞太複雜，關鍵是你真的去做了。

微信聊天記錄特別注意

微信聊天記錄無法透過任何手機換機工具遷移！它不在整機備份的覆蓋範圍內。必須用微信自帶的“聊天記錄遷移”功能，或備份到電腦版微信。很多人在換完新手機之後才發現聊天記錄全沒了，那時候已經晚了——舊手機可能已經恢復出廠設定了。

練一練：做一次完整的備份演練

1. 檢查 iCloud 雲備份或品牌雲備份是否開啟，確認上一次備份時間。
2. 手動備份照片：用資料線連線電腦，把 DCIM 和 Pictures 資料夾複製到電腦。
3. 備份通訊錄：iPhone 使用者去 iCloud.com 匯出 vCard；安卓使用者在聯絡人 App 設定裡匯出.vcf 檔案。
4. 備份微信聊天記錄：開啟微信我設定通用聊天記錄遷移與備份備份到電腦。
5. 把匯出的通訊錄 vcf 和照片資料夾複製一份到行動硬碟或隨身碟，完成”三二一”裡的”一份離線在別處”。

第十九章：故障排查與自救

手機跟人一樣，偶爾也會“生病”。卡頓、閃退、黑屏、充不進電……大部分毛病其實沒那麼可怕，你完全可以在送去維修之前自己先試試。這章就是你的“手機急診手冊”——先用低成本的方法試，不行再找專業醫生。

一、手機變卡、發熱——最常被問的問題

說實話，手機用了一年左右開始變卡，太正常了。原因不一定是手機壞了，更多時候是“積灰”——垃圾檔案、後台程式、儲存空間慢慢被填滿。

先做診斷，別急著修：

1. **查儲存空間**：去設定儲存看剩餘空間。如果不到 10% 了，系統會變得非常卡，因為它在掙扎著騰地方。我的經驗是：保持至少 15%-20% 的剩餘空間，手機才能比較順暢。

2. **查耗電排行**：去電池設定裡看過去 24 小時的耗電排行。如果某個 App 佔了大頭但你根本沒怎麼用，那就是它在後臺偷跑。直接解除安裝或者限制後臺活動。

3. **聯想最近做了什麼**：是升級了系統後變卡的？（大版本更新後前幾天變卡正常，系統在後臺重建索引）。還是安裝了某個新 App 之後變卡的？（解除安裝試試看）。

4. **看使用環境**：手機在太陽底下暴曬、套著厚厚的殼邊充邊玩遊戲，發燙是必然的。高溫還會導致 CPU 自動降頻，你就感覺手機突然變卡了。涼下來再試。

治療方法（從簡單到複雜）：重啟手機（解決 60% 的問題）清儲存空間（刪掉沒用的 App 和影片）某個 App 搞鬼就清它的快取或解除安裝重灌最後一招：備份資料後恢復出廠設定。

二、應用閃退、無響應——App 不聽話了

某個 App 點開就閃退、或者用著用著突然卡死不動了。處理流程就像醫生問診，一步步來：

1. **先關掉重開**：從後臺劃掉 App，再重新開啟。大多數臨時 bug 這一下就好了。

2. **更新 App**：去應用商店看有沒有更新。有時候是因為 App 版本太老了，跟系統不相容。

3. **檢查許可權**：有些 App 需要某個許可權才能正常工作，但你沒給。比如相機 App 沒給相機許可權打了開就閃退。去設定裡找到那個 App，把該給的許可權都給了。

4. **清快取**：去設定應用管理找到那個 App 儲存清除快取。注意是" 清除快取" 不是" 清除資料"！清除資料會把你 App 裡的登入狀態和設定全刪了，相當於恢復出廠給 App 的狀態。

5. **解除安裝重灌**：最後一招。解除安裝之前先確認這個 App 的資料會不會丟——微信的話先備份聊天記錄，遊戲的話先確認賬號有沒有繫結手機號。

三、充不進電或充電慢——可能不是電池的鍋

充不進電是特別容易讓人焦慮的問題，但很多時候原因是出在配件上，不是手機壞了。

- **先排查充電口**：用牙籤或者不導電的小工具，輕輕地把充電口裡的灰塵和棉絮掏一掏。這個我自己遇到過——口袋裡掉了一根線頭剛好堵在充電口裡，插進去以為是接觸不良，掏出來就好了。

- **換根線和充電頭試試**：線的損壞率比充電頭高得多，尤其是經常捲來捲去的線。換一條確定沒問題的線和充電頭交叉測試，很快能確定是哪個配件壞了。

- **無線充電的問題**：如果你用無線充電，檢查一下手機殼是不是太厚了，或者殼後面有沒有金屬片（磁吸支架之類）。還有，把手機準確地放到充電板的正中間——歪了就可能充不上。

- **充電慢**：確認充電頭功率夠不夠（iPhone 想要快充至少需要 20W 的 PD 充電頭），線支不支援快充（不是所有的線都能過 3A 以上電流），手機是不是太燙（太燙會自動降功率保護電池）。

四、網路連不上、Wi-Fi 老是斷

在外面用流量好好的，回家連不上 Wi-Fi；或者 Wi-Fi 連著但就是沒網。按這個順序排查：

1. **開關飛航模式**：下拉控制中心，開啟飛航模式等 5 秒再關掉。這等於讓手機重新搜一次網，非常有效。

2. **Wi-Fi" 忘記" 再重連**：去 Wi-Fi 設定裡，點那個網路名旁邊的感嘆號（安卓是齒輪），選" 忽略此網路" 或" 忘記"，然後重新輸入密碼連線。

3. **重啟路由器**：很多人家裡的路由器幾個月沒關過，時間久了會不太穩定。拔掉電源等 30 秒再插回去，大部分 Wi-Fi 問題就解決了。

4. **檢查是不是手機的問題**：別的裝置連同一個 Wi-Fi 正常嗎？如果都正常就你的手機不行，那手機可能需要重啟或者重置網路設定（設定通用傳輸或還原還原網路設定，不會刪你的資料，但會把所有 Wi-Fi 密碼都清了）。

五、螢幕問題：觸控不靈、閃屏、留印子

- **觸控不靈敏**：先撕掉鋼化膜看看是不是膜的鍋——便宜鋼化膜用久了確實會影響觸控。螢幕上有水或手上有油也會導致觸控漂移。都不是的話，重啟一下再試。
- **螢幕上有印子/殘影（燒屏）**：長時間顯示同一個畫面（比如一直亮屏刷抖音），會在 OLED 螢幕上留下” 殘影”。這不是螢幕壞了，只是暫時的。關屏休息半個小時一般能恢復。
- **螢幕閃爍**：如果是低亮度下閃爍，可能是 PWM 調光的問題，多見於 OLED 螢幕。去設定裡找”DC 調光” 或” 防頻閃” 選項，開啟後會好很多。不是所有手機都有這個功能。
- **螢幕有一塊區域完全黑了/亮了**：這個比較嚴重，大機率是螢幕物理損壞，需要換屏。非官方維修換的螢幕，色彩和亮度通常不如原裝，也影響防水。

六、無法開機、卡在 Logo 介面——先別慌

這種情況最容易讓人心跳加速，但冷靜，先試試這幾個：

- **充電半小時再說**：很多時候只是電徹底耗盡了。插上原裝充電器，等至少 30 分鐘，再嘗試開機。有些手機在電耗盡後需要充到一定電量才會有反應。
- **強制重啟**：iPhone 全面屏機型按一下音量加、一下音量減、長按側邊電源鍵不放，直到螢幕出現蘋果 Logo（可能要按住 10-15 秒）。安卓通用方法：長按電源鍵 10 秒以上，直到震動或亮屏。
- **卡在啟動介面反覆重啟**：這種情況叫” 白蘋果”（iPhone）或” 卡 Logo”（安卓）。如果強制重啟不行，可能需要進入恢復模式用電腦修復。iPhone 連電腦開啟 iTunes/Finder，進入恢復模式後選” 更新”（不是” 恢復”），會重灌系統但保留資料。安卓進 Fastboot 或 Recovery 模式，各品牌組合鍵不同，建議搜尋你手機型號 +” 進恢復模式”。
- **以上都不行**：那就只能求助專業維修了。資料如果備份過，就不用太擔心。

七、聲音問題：喇叭不響、通話聲音小

- **喇叭沒聲音**：先確定不是靜音模式（iPhone 側邊的靜音撥片）、勿擾模式、或者音量被按到最低了。聽起來很傻，但這種情況佔了 40%。然後檢查藍芽是不是連著某個耳機或音箱。
- **通話對方聽不到你說話**：檢查話筒（麥克風）孔有沒有堵住——手機底部的小孔很容易被灰塵堵住。用軟毛刷輕輕清理。別用針或牙籤硬捅，會捅壞防塵網。
- **聲音沙啞/破音**：如果是喇叭進水導致的——不要用吹風機熱風吹！用紙巾吸乾表面水分，然後放著自然晾乾。網上說用聲波排水 App 有一定效果（播放低頻聲音把水振出去），可以試試。

八、找官方維修，避開那些坑

如果上面的方法都試過了還是不行，那就得找專業維修了。但維修市場水很深，說說我自己踩過的坑和經驗：

- **找真官方，別找”看起來像官方”的：**去品牌官網（Apple.com.cn、Huawei.com.cn 等）找”授權服務商”列表。街邊掛個蘋果 Logo 的非授權店，用的可能不是原裝配件，修完你的防水就沒了一剛。

- **送修前必須做的事：**備份所有資料（別指望修的時候資料不會丟），退出所有重要賬號（Apple ID、微信、支付寶），關閉”查詢我的手機”。有些維修點會要求你提供鎖屏密碼，這事兒有隱私風險——你最好自己先退出賬號再送過去。

- **換螢幕的坑：**非官方螢幕分”原壓屏”（原裝玻璃換過）、”後壓屏”（非原裝玻璃）、”組裝屏”（全非原裝）。價格差好幾倍，但顯示效果和耐久度也差很多。如果手機還在保修期，老老實實走官方。

- **換電池的坑：**非原裝電池可能虛標容量，而且用了半年就衰減得比原裝電池兩年的損耗還大。蘋果換電池官方價格比非官方貴不了太多，建議走官方。

- **資料恢復的坑：**手機進水/摔壞後，去維修如果對方說”資料恢復要加錢”，小心——有些不良商家會故意誇大損壞程度來多收錢。正規的資料恢復是先報價再操作，而且恢復不成功不收費。

萬能第一招：重啟手機

不管你遇到什麼問題——卡頓、閃退、連不上網、聲音異常——先重啟一下手機。這句話聽起來像是敷衍，但以我自己的經驗，至少有六成的小毛病重啟之後就好了。手機跟電腦一樣，跑久了記憶體裡會積累各種亂七八糟的東西，重啟就是清空重來。所以，下次手機出問題，先別急著搜“怎麼辦”，重啟完了再看看。

現在就記：這些救命操作要爛熟於心

1. **強制重啟：**全面屏 iPhone 按音量加音量減長按電源鍵到出現 Logo。安卓長按電源鍵 10-15 秒直到震動。
2. **清理充電口：**找一個小工具（牙籤或手機取卡針），輕輕清理充電口的灰塵。
3. **Wi-Fi 重連：**去 Wi-Fi 設定裡”忘記”你當前的 Wi-Fi，重新輸入密碼連線。
4. **開/關飛航模式：**下拉控制中心，開啟飛航模式，心裡數 5 秒，關掉，看網路是否恢復。
5. 把這幾個操作的步驟截圖儲存，或者寫在備忘錄裡。遇到問題的時候不用慌慌張張去搜，直接翻備忘錄就行。

第二十章：開發者模式與高階許可權

這一章是給那些想要深入探索手機“隱藏功能”的使用者準備的。開發者模式、*Root* 許可權、*BL* 鎖……這些聽起來很極客的概念，其實離普通使用者並不遙遠。

一、開發者模式：解鎖隱藏設定

開發者模式是手機系統內建的一個隱藏選單，裡面有很多普通使用者用不到但開發者需要的功能。

如何開啟：

- **安卓**：進入設定 > 關於手機 > 連續點選” 版本號”7 次，直到提示” 您已處於開發者模式”。
- **鴻蒙**：設定 > 關於手機 > 連續點選”HarmonyOS 版本”7 次。
- **iOS**：普通使用者無法開啟完整的開發者模式，需要付費的 Apple Developer 賬號。

常用功能一覽：

- **USB 除錯**：連線電腦進行高階操作（如刷機、資料恢復、ADB 命令）的必開開關，詳見本章”ADB 實用命令” 小節。
- **顯示觸控操作**：螢幕上顯示觸控點（一個小圓點跟隨手指），適合錄製操作教程或演示。

效能與流暢度相關：

- **動畫速度調節（視窗動畫縮放 / 過渡動畫縮放 / 動畫程式時長縮放）**：這三個選項控制開啟 App、切換頁面時的動畫時長。預設是 1x，調到 0.5x 會讓手機” 看起來” 更快（實際上程式執行速度沒變，只是動畫變快了），調到關閉則直接無動畫跳轉。多數極客會設為 0.5x，既保留過渡感又顯快；完全關閉會讓操作顯得生硬，並且可能掩蓋程式真實的卡頓，反而讓你誤以為” 很流暢”。
- **強制 GPU 渲染（Force GPU Rendering）**：強制所有 2D 繪圖透過 GPU 而非 CPU 處理。在老舊機型或某些 App 最佳化不佳時能提升滾動流暢度，但也會增加 GPU 負載和耗電。現代手機預設已大量使用 GPU 渲染，這個開關對中高階機型影響有限，更多是歷史遺留選項，普通使用者保持預設即可。
- **顯示 GPU 呈現模式分析（Profile GPU Rendering）**：在螢幕上以柱狀圖實時顯示每幀的渲染耗時，綠色橫線代表 16.6ms（60fps 的幀時間閾值）。柱子超過綠線就說明掉幀了。這是開發者排查 App 卡頓的利器，普通使用者也可以用它來對比哪個 App 最佳化得差——比如某個 App 滾動時柱子經常飆到紅線以上，那基本就是它的鍋。

後臺與續航相關：

- **後台程式限制**：可以設定” 標準限制”” 最多 1 個程式”” 最多 2 個程式” 等。表面上看限制後台程式能省電，但實際上 Android 自帶的後臺管理已經做得不錯，強行限制反而會導致 App 頻繁重新載入（冷啟動反而更耗電、體驗也變差）。這個選項更適合除錯用，不建議普通使用者長期開啟，保持” 標準限制” 即可。
- **不保留活動**：每離開一個介面就立即銷燬它。極不推薦，會導致 App 頻繁白屏、重新載入，僅用於測試 App 的恢復邏輯。

注意事項

開發者模式裡的設定可能會影響系統穩定性，不懂的選項不要亂改。開啟 USB 除錯後，連線電腦時要小心，不要連線不信任的電腦。

二、OEM 解鎖：開發者選項裡的“刷機開關”

在開發者選項裡有一個不太起眼但很重要的開關——“OEM 解鎖”（部分品牌叫“OEM 解鎖允許” 或“裝置解鎖”）。這個開關本身不會解鎖 BL 鎖，但它是一個“前置許可”：只有開啟它，廠商才允許你透過 fastboot 命令或品牌工具去解鎖 BL。

- **開關的作用**：相當於向廠商宣告” 我同意放棄保修、自負風險，請允許我解鎖 BL”。如果這個開關是關閉的，任何解鎖 BL 的命令都會被直接拒絕。
- **與 BL 鎖的關係**：OEM 解鎖是” 申請資格”，BL 解鎖才是” 實際操作”。前者在手機設定裡點一下就行，後者需要去廠商官網申請解鎖碼、用電腦跑 fastboot 命令（如 `fastboot flashing unlock`），二者缺一不可。
- **恢復出廠設定會重置嗎**：會。”OEM 解鎖” 開關在恢復出廠後會回到關閉狀態，但 BL 鎖一旦解鎖就是物理級別的狀態，不會因為恢復出廠而重新上鎖——需要手動執行 `fastboot flashing lock` 才能重新上鎖。
- **什麼時候應該開啟**：除非你確定要刷機或 Root，否則保持關閉。它和”USB 除錯” 是兩個獨立開關，刷機時通常需要同時開啟。

小知識：為什麼廠商要設定這個開關

早期安卓沒有這個開關，惡意軟體只要拿到許可權就能偷偷刷入系統。Google 後來要求所有安卓廠商在開發者選項中加入“OEM 解鎖” 開關，作為一道“使用者知情同意” 的確認。這也就是為什麼即便你解了 BL 鎖，如果 OEM 解鎖開關被關閉，重新刷機仍然會失敗。

三、BL 鎖（Bootloader 鎖）：刷機的第一道門

Bootloader 是手機啟動時載入作業系統的程式。BL 鎖就是廠商為了安全而加的鎖，防止使用者刷入非官方系統。

BL 鎖的作用：

- **安全保護**：防止惡意軟體刷入系統。
- **防盜**：即使手機被刷機，沒有原賬號密碼也無法使用。
- **保修限制**：解鎖 BL 通常意味著失去官方保修。

各品牌 BL 鎖政策：

- **小米**：可以解鎖，需要繫結賬號等待期（7-30 天）。
- **華為**：2018 年後停止提供解鎖碼，基本無法官方解鎖。
- **一加**：可以解鎖，相對寬鬆。
- **OPPO/vivo**：部分機型可以解鎖，需要申請。
- **三星**：可以解鎖，但會觸發 Knox 熔斷，失去 Samsung Pay 等功能。
- **蘋果**：iOS 裝置沒有傳統意義上的 BL 鎖概念，但有類似的安全機制。

四、ADB 實用命令：透過電腦控制手機

ADB 全稱 Android Debug Bridge（安卓除錯橋），是 Google 官方提供的命令列工具，讓電腦透過 USB 或 Wi-Fi 與安卓手機通訊。它本身不需要 Root，只要手機開了“USB 除錯”就能用。可以把它理解為“用電腦給手機下指令的遙控器”——刷機、裝 App、傳檔案、查日誌，都得靠它。

如何透過 USB 連線：

1. 手機端：進入開發者選項開啟”USB 除錯”。
2. 電腦端：從 Google 官方下載 Platform Tools（含 `adb` 和 `fastboot` 兩個可執行檔案），解壓到任意目錄。
3. 用資料線連線手機和電腦，手機上會彈窗” 是否允許 USB 除錯”，勾選” 始終允許” 並確定。
4. 在電腦命令列（Windows 的 PowerShell 或 cmd）裡執行 `adb devices`，如果輸出類似 `List of devices attached /xxxxxxxx device`，就說明連上了。

驅動和授權彈窗別忽略

Windows 使用者如果 `adb devices` 顯示“unauthorized”或根本不列出裝置，多半是驅動沒裝好或者忘了在手機上點“允許”。三星/小米等品牌可以裝各自的“USB 驅動”或“手機助手”來補齊驅動；Mac 和 Linux 一般免驅。如果手機彈窗被你手滑點掉了，拔插一下資料線就會重新彈出來。

常用 ADB 命令：

- `adb install app.apk`：把電腦上的 APK 裝到手機（`-r` 表示覆蓋安裝並保留資料；`-s` 裝到 SD 卡）。
- `adb uninstall com.xxx.xxx`：按包名解除安裝 App。
- `adb push` 電腦路徑 `/sdcard/`目標路徑：把電腦檔案推送到手機。
- `adb pull /sdcard/`檔案電腦路徑：把手機檔案拉到電腦（比發給自己微信再下載快得多）。
- `adb shell`：進入手機的 Linux Shell，可以直接執行 `ls`、`pm`、`am` 等命令。
- `adb reboot bootloader`：重啟手機進入 Fastboot 模式（刷機的入口）。
- `adb reboot recovery`：重啟進入 Recovery 模式。
- `adb shell pm list packages`：列出手機上所有已安裝應用的包名（找包名必用）。加`-s` 只看系統應用，加`-3` 只看第三方應用。
- `adb shell pm uninstall --user 0` 套件名稱：為當前使用者解除安裝預裝 App，免 Root，詳見下方實戰。

免 Root 解除安裝預裝 App 實戰：

這是 ADB 最實用的功能之一：用 `pm uninstall --user 0` 命令可以為“當前使用者”解除安裝預裝 App，而不需要 Root，也不影響系統分割區。被解除安裝的 App 在系統裡“消失”，但實際還在系統分割區裡，恢復出廠設定後會回來。

1. 先用 `adb shell pm list packages -s` 列出所有系統應用，找到你想解除安裝的包名。比如小米瀏覽器是 `com.android.browser`，華為應用市場是 `com.vmall.client`。
2. 不確定包名對應哪個 App？可以在電腦上用 `adb shell pm list packages | findstr` 關鍵字搜尋，或者裝一個第三方 App ”Package Viewer” 在手機上直接檢視每個 App 的包名。
3. 執行解除安裝：`adb shell pm uninstall --user 0` 套件名稱，比如 `adb shell pm uninstall --user 0 com.android.browser`。返回 `Success` 就成功了。
4. 想恢復怎麼辦？兩種方法：(1) 恢復出廠設定（最徹底但麻煩）；(2) 重新安裝：`adb shell cmd package install-existing` 套件名稱。

千萬別亂刪系統核心包

“pm uninstall –user 0”雖然是免 Root 的安全方式，但亂刪關鍵系統應用（如 `com.android.systemui` 狀態列、`com.android.phone` 電話、`com.android.settings` 設定）會導致系統崩潰、黑屏、無法開機等嚴重問題。刪之前先 Google 一下“這個包名能不能刪”，寧可少刪不要錯刪。建議一次只刪一兩個，確認沒問題再繼續。

小知識：–user 0 是什麼意思

安卓 4.2 之後支援多使用者，每個使用者有獨立的 ID（機主是 0，附加使用者從 10 開始）。`--user 0` 表示“只對當前機主解除安裝”，系統分割區裡的 APK 檔案依然存在，只是不給機主顯示了。這就是為什麼它不需要 Root——沒動系統檔案，只是改了“使用者配置”。這也是它和 Root 後解除安裝最大的區別：Root 解除安裝是真正從系統裡刪掉，所有使用者都不再有；ADB 解除安裝只是“對當前使用者隱藏”。

五、Root 許可權：安卓的“超級管理員”

Root 許可權相當於 Linux 中的超級使用者（對/目錄擁有完全控制權，可以讀寫任意檔案），是安卓系統的最高許可權。它與 Windows 的 Administrator 有本質區別：Windows 出於安全考慮對管理員做了諸多限制（真正的最高許可權其實是 TrustedInstaller），即便是 Administrator 也無法像 Linux root 那樣直接執行 `sudo rm -rf /*`。獲得 Root 權限後，你可以修改系統檔案、解除安裝預裝應用、使用需要 Root 的高階 App。

Root 的好處：

- 解除安裝無法解除安裝的預裝應用。
- 使用高階功能 App（如鈦備份、綠色守護）。
- 修改系統介面和功能。
- 安裝第三方 Recovery 和自定義 ROM。

Root 的風險：

- **失去保修：**大多數廠商不保修 Root 過的手機。
- **安全風險：**惡意 App 可能獲得 Root 權限，造成嚴重後果。
- **系統不穩定：**誤操作可能導致系統崩潰。
- **銀行 App 不能用：**很多銀行和支付 App 檢測到 Root 會拒絕執行。
- **OTA 更新失敗：**可能無法正常接收官方系統更新。

嚴重警告

Root 操作有一定風險，可能導致手機變磚（無法開機）。如果你是新手，建議先充分了解相關知識，並確保有完整的備份。操作前請仔細閱讀教程，確認你的機型和系統版本。

六、SafetyNet 與 Play Integrity 認證：為什麼 Root 後銀行 App 不能用

很多 Root 使用者都遇到過這樣的崩潰瞬間：Root 之後興致勃勃地裝了一堆模組，結果開啟銀行 App——“您的裝置不安全，無法使用”，Google Pay 也用不了，連 Netflix 都拒絕高畫質播放。這背後是 Google 的兩套裝置完整性認證機制：舊的 SafetyNet 和新的 Play Integrity API。

什麼是 SafetyNet / Play Integrity：

- **SafetyNet Attestation（舊版，2017 年起）**：Google 早期推出的裝置完整性檢查 API，應用透過它向 Google 伺服器查詢” 這臺裝置是否透過 CTS 相容性測試、是否處於未修改的官方狀態”。返回結果分為 Basic Attestation（基礎校驗）和 Strong Attestation（強校驗，依賴硬體金鑰）。Google 已宣佈 SafetyNet 將在 2024 年後逐步停用。

- **Play Integrity API（新版，2022 年起）**：Google 推出的替代方案，比 SafetyNet 更嚴格也更全面。除了裝置完整性，還包括” 應用完整性”（App 是否被篡改）和” 帳號/授權完整性”（使用者帳號是否被 Google 信任）。新 App 應遷移到 Play Integrity。

- **誰來用**：銀行、金融、支付類 App（如 Google Pay、各銀行的網銀、支付寶/微信支付的部分場景）、流媒體（Netflix 高畫質版權保護）、部分遊戲反作弊（如騰訊/網易的部分遊戲）、企業辦公 App（MDM 裝置合規性檢查）。

Play Integrity 的三級認證結果：

- **MEETS_DEVICE_INTEGRITY（裝置完整性透過）**：裝置執行正版 Android 系統，Bootloader 未解鎖，沒有 Root，CTS 相容性測試透過。可以正常使用銀行、支付類 App。

- **MEETS_BASIC_INTEGRITY（基礎完整性透過）**：裝置基本可信，但可能存在輕微異常。某些 App 會降級使用（如 Netflix 仍可播放但限制清晰度），某些嚴格的銀行 App 會直接拒絕。

- **MEETS_STRONG_INTEGRITY（強完整性透過）**：最高階別，要求硬體金鑰背書（如 Google Pixel 系列的 Titan M 晶片），大多數第三方品牌機型即使未 Root 也達不到這一級。

- **無任何標籤（NO_INTEGRITY）**：裝置已 Root、解鎖 BL、刷了第三方 ROM 或被識別為模擬器，絕大多數支付/銀行 App 會直接拒絕執行。

為什麼 Root 後這些 App 會失效：

應用透過 Play Integrity API 向 Google 伺服器發起認證請求，Google 會檢查裝置的多項指標——Bootloader 鎖定狀態、系統分割區是否被修改、是否存在 su 二進位制檔案、SELinux 是否處於寬鬆模式等。任何一項異常都會導致返回的認證級別降低或為空，銀行 App 一看“裝置不安全”，自然就拒絕服務了。

現代隱藏方案：Magisk Hide / Shamiko / Zygisk：

為了在享受 Root 便利的同時還能用銀行 App，社群開發了一系列“隱藏 Root 狀態”的方案。核心思路都是：讓檢測程式看到的不是一個被修改過的系統，而是一個“看起來正常”的裝置。

- **Magisk**：當前最主流的 Systemless Root 方案——不修改系統分割區，而是透過掛載的方式實現 Root。這樣設計的好處是 OTA 更新失敗的機率小、解除安裝比較乾淨，是後續所有隱藏方案的基礎。但使用 Magisk 仍然需要解鎖 BL 鎖。
- **Magisk Hide（已停更）**：Magisk 早期內建的隱藏模組，能對指定 App 隱藏 Root。但 2021 年起已被 Magisk 官方移除，原因是 Google 持續升級檢測手段，雙方陷入”貓鼠遊戲”。
- **Zygisk**：Magisk 24.0+ 引入的新機制，類似 Riru，透過注入 Zygote 程式實現模組載入，比直接修改系統更隱蔽。開啟 Zygisk 是使用現代隱藏方案的前提。
- **Shamiko**：當前最主流的隱藏模組，配合 Zygisk 工作，可以為指定 App（”反向白名單”模式）隱藏 Root、隱藏 Magisk 自身、偽造 Bootloader 鎖定狀態。Shamiko 透過 LSPosed 倉庫或 GitHub 分發。
- **Play Integrity Fix（osm0sis）**：專門針對 Google Play Integrity 檢測的 Magisk 模組，會修改裝置指紋（fingerprint）讓 Google 認為你的裝置是某個未 Root 的官方機型。

貓鼠遊戲永遠不會結束

Google 每隔一段時間就會更新檢測規則，App 廠商也會跟進升級 Play Integrity 校驗。今天還能用的隱藏方案，明天可能就被識別。如果你完全依賴銀行/支付 App（比如日常轉帳、掃碼付款），最穩妥的選擇仍然是：不要 Root、不要解 BL 鎖。隱藏方案更適合“主力機不 Root + 備用機玩 Root”的玩家配置。

七、給普通使用者的建議

- **開發者模式**：可以開啟，體驗一下動畫加速、GPU 呈現分析等功能，但不要亂改其他不理解的選項。
- **OEM 解鎖與 BL 鎖**：如果你不打算刷機或 Root，兩個開關都保持預設（關閉）狀態最安全。
- **ADB**：免 Root 的強大工具，普通人也能用——比如用 `pm uninstall --user 0` 解除安裝預裝 App。但刪之前一定要查清楚包名能不能刪。
- **Root**：普通使用者不建議 Root，風險大於收益。如果確實需要某些功能，可以先了解有沒有不需要 Root 的替代方案（比如用 ADB 解除安裝預裝、用 Shizuku 替代部分 Root 功能）。

來動手：開啟開發者模式並調整動畫速度

1. 進入設定 > 關於手機。
2. 連續點選”版本號”7 次，直到提示已開啟開發者模式。
3. 返回設定，找到”開發者選項”（通常在設定底部或”系統”裡）。
4. 找到”視窗動畫縮放”、”過渡動畫縮放”、”動畫程式時長縮放”，將它們都設為 0.5x

或關閉。

5. 感受一下手機是否變快了。

第二十章附：Termux——Android 裝置上的行動式 Linux 終端環境

如果你覺得開發者模式還不夠“硬核”，想要在手機上體驗完整的 Linux 命令列，那麼 Termux 正是為你準備的。它無需 root 權限，就能把一臺普通的安卓手機變成一臺便攜的 Linux 工作站，是進階使用者探索移動端開發與運維的利器。

一、什麼是 Termux

Termux 是一款專為 Android 裝置設計的開源終端模擬器和 Linux 環境應用程式，它無需 root 權限即可在手機上提供完整的 Linux shell 體驗。作為一個輕量級應用（安裝包僅 30MB 左右），Termux 透過 APT 包管理系統支援豐富的軟體生態，允許使用者在移動裝置上執行 Python、Node.js、Git、SSH 等工具，甚至搭建伺服器或進行滲透測試。

Termux 的核心優勢在於輕量化和高度可擴充性，適合開發、學習及系統管理等多種場景。它將標準的軟體包與精確的終端模擬結合在一個開源解決方案中。

二、核心功能與特性

完整的 Linux 環境

Termux **僅內建 Bash** 這一種 Shell。如果想使用 Zsh 或 fish 等其他 Shell，需要透過包管理器手動安裝，例如：`pkg install zsh` 或 `pkg install fish`。同時 Termux 支援 Vim、Nano、Emacs 等編輯器，提供接近原生的命令列體驗。使用者可以透過“pkg install”安裝 2000+ 軟體包，包括 GCC、OpenSSH、SQLMap 等開發和安全工具。

強大的包管理

使用 APT 包管理器（類似 Debian 和 Ubuntu），可安裝數千種 Linux 軟體包，涵蓋開發工具、網路工具等。支援 Python、Ruby、Node.js、Go、Rust、Swift 等多種程式語言環境。

安全與網路功能

- **SSH 遠端連線：**支援 SSH 客戶端和伺服器功能，可遠端連線其他裝置或讓手機成為 SSH 主機。
- **加密工具：**提供 OpenSSL 等加密工具，支援資料加密和安全通訊。
- **伺服器搭建：**可配置 Nginx 或 Apache 搭建靜態網頁伺服器。

系統整合與擴充

- 訪問手機儲存權限，實現本地檔案管理。
- 透過 Termux:API 外掛呼叫感測器、攝像頭等 Android 硬體功能。
- 支援藍芽鍵盤和外部顯示器連線，具有完整的滑鼠支援。

三、外掛生態系統

Termux 的核心應用程式可以透過幾個可選的外掛應用進行擴充，提供額外的功能和整合：

外掛名稱	功能描述
Termux:API	向命令列程式公開 Android API 功能
Termux: Boot	允許在裝置啟動時執行指令碼
Termux: Float	提供浮動終端視窗
Termux: Styling	為終端啟用顏色方案和字型自定義
Termux: Tasker	將 Termux 指令碼與 Tasker 自動化應用整合
Termux: Widget	允許從主螢幕小部件執行指令碼

重要提示

Termux 及其外掛必須從同一來源安裝，因為它們共享相同的 sharedUserId。混合來自不同來源（如 F-Droid 和 GitHub）的安裝將導致安裝錯誤。

四、安裝與配置指南

安裝步驟

1. **下載應用：**從官方渠道（F-Droid 或 GitHub）下載安裝 Termux，無需 root 權限。
2. **初始化系統：**首次啟動後等待”Installing bootstrap packages” 完成。
3. **獲取儲存權限：**執行 `termux-setup-storage` 獲取手機檔案系統權限。
4. **更新軟體源：**執行 `pkg update && pkg upgrade` 更新軟體包列表。

基礎最佳化

- **換映象源：**使用 `termux-change-repo` 命令透過圖形介面更換映象源（推薦清華大學映象源）。
- **快捷鍵設定：**音量減鍵模擬 Ctrl，音量加鍵模擬 Esc，提升輸入效率。
- **介面美化：**安裝 Zsh + Oh My Zsh，配置主題和外掛。

五、典型應用場景

移動開發測試

在手機上搭建完整的開發環境，編寫並執行 Python 指令碼，使用 GCC 編譯 C 程式，透過 Git 管理程式碼倉庫，結合 SSH 與遠端伺服器同步。

伺服器運維

透過 SSH 遠端管理伺服器，執行系統維護、日誌分析等任務，隨時隨地處理伺服器問題。

學習 Linux 操作

提供真實的 Linux 環境，是學習命令列和系統管理的理想平臺，可練習 Shell 指令碼編寫、Linux 檔案權限管理（如 `chmod 755` 設定 rwxr-xr-x 權限位、`chown user: group` 修改檔案歸屬、`ls -l` 檢視 owner/group/other 三組九位權限）等基礎命令。注意這是 Unix/Linux 經典的“三組九位”權限模型（owner / group / other × 讀 / 寫 / 執行），與 Windows NTFS 的 ACL 訪問控制列表是完全不同的兩套體系。

網路安全測試

安裝 Nmap、SQLMap、Hydra、Metasploit 等安全工具，進行網路掃描和滲透測試。

六、版本與相容性

Termux 的最新穩定版本是 v0.118.2，適用於 Android ≥ 7 的裝置。需要注意的是，Termux 在 Android 12 及更高版本上可能存在不穩定性，建議關注官方更新日誌以獲取最新相容性資訊。

總結

Termux 將強大的 Linux 終端體驗帶到了 Android 裝置上，讓手機成為一臺便攜的開發和學習工具。無論是開發者、系統管理員還是技術愛好者，都可以利用 Termux 在移動裝置上享受完整的命令列體驗，實現“口袋裡的 Linux”這一理念。

來動手：在手機上跑通第一條 Linux 命令

1. 從 F-Droid 下載並安裝 Termux，首次啟動等待初始化完成。
2. 執行 `pkg update && pkg upgrade` 更新軟體包。
3. 安裝 Python：`pkg install python`。
4. 輸入 `python3 -c "print('Hello from Termux!')"`，感受在手機上執行程式碼的樂趣。

四、超越篇（第 21-26 章）

第二十一章：行動網路與資訊保安深探

咱們每天用手機上網、支付、聊天，感覺方便得很。但你有沒有想過，你的聊天記錄、支付密碼、照片……這些資訊在網上“裸奔”的時候多嗎？說實話，大部分人根本沒想過這個問題，但一旦出事——比如帳號被盜、銀行卡被刷——那時候再後悔就晚了。這一章咱們就好好聊聊，怎麼讓你的數字生活不那麼“透明”。

一、VPN：網路裡的“隱形斗篷”

VPN，全稱虛擬專用網路（Virtual Private Network）。你可以把它想象成在你家和你要去的網站之間，建起了一條專屬的、加密的地下隧道。你在隧道裡走來走去，外面的人看不見你手裡拿著什麼、要去哪。

- **普通上網**：你的手機公共網路目標網站。中間經過的路由器、交換機，隨便哪個環節都可能有人” 偷看” 你傳送的資料。尤其是在咖啡廳、機場那種公共 Wi-Fi 上，說難聽點，簡直是” 裸奔”。
- **使用 VPN**：你的手機 VPN 加密隧道 VPN 伺服器目標網站。資料被加密封裝，” 偷窺者” 只能看到一堆亂碼。另外，目標網站看到的是 VPN 伺服器的 IP 地址，而不是你的真實 IP。

那 VPN 到底能幹嘛？簡單說：第一，**保護你在公共 Wi-Fi 上的隱私**；第二，**繞過地域限制**（比如看某些只在特定國家上線的內容）；第三，**防止你的網路運營商（ISP）記錄你的瀏覽習慣**。不過我得提醒一句：**不要用免費 VPN**！免費的背後往往意味著你的資料就是他們的“商品”。我有個朋友用了某個免費 VPN，結果手機莫名其妙開始彈廣告，後來發現那個 VPN 把他的瀏覽資料打包賣給廣告商了。所以選 VPN，要麼選付費的大品牌（如 NordVPN、ExpressVPN），要麼乾脆別用。

另外，VPN 不是萬能的。它只能保護“傳輸中”的資料，如果目標網站本身不安全、或者你的手機上已經中了惡意軟體，VPN 也幫不了你。

二、公共 Wi-Fi 的“甜蜜陷阱”——比你想象的危險

你可能覺得，不就是連個免費 Wi-Fi 嘛，能有多大風險？我給你講個真事：安全研究人員曾經做過實驗，在機場擺了一個叫“Starbucks_Free”的 Wi-Fi 熱點，不到一小時就有上百人

連線。而實際上這個熱點是研究人員自己搭的，所有經過這個熱點的**未加密流量**——比如你訪問的 HTTP 網頁內容、未啟用 TLS 的 App 通訊——他們確實能截獲和檢視。

但要注意一個關鍵前提：今天你日常用的微信、網銀、支付寶，絕大多數都已經走了 HTTPS 加密。說白了，攻擊者即使截獲了資料包，看到的也只是一堆亂碼，**無法直接讀出你的聊天內容或支付密碼**。不過他們依然能知道你訪問了哪些網站（域名），如果某些老舊 App 或網頁還在用 HTTP 明文傳輸，那內容就會完全暴露。這種攻擊叫“中間人攻擊”（Man-in-the-Middle），原理是攻擊者在你和真正的 Wi-Fi 路由器之間“插了一腳”。更可怕的是，很多人的手機設定了“自動連線已知 Wi-Fi”，攻擊者只要把自己的熱點名字設成你之前連過的某個 Wi-Fi（比如“ChinaNet”或“CMCC”），你的手機就會自動連上去。

那怎麼防？幾條實用建議：

- **認準 HTTPS**：位址列開頭是 https://，多出的”s” 代表 Secure。訪問任何需要輸密碼的網站時，如果看到的是 http://（沒有 s），立刻停下來別輸！現在主流瀏覽器都會在非 HTTPS 網站前顯示” 不安全” 的警告，看到這個標記就要警惕。
- **關掉” 自動連線”**：不管是 iPhone 還是安卓，都在 Wi-Fi 設定裡把” 自動加入” 或” 自動連線” 關掉。寧可每次手動點一下，也別讓手機自己判斷。
- **用手機流量做敏感操作**：轉帳、登入網銀、輸入身份證號……這些操作老老實實用自己的 4G/5G 流量，別省那點流量錢。
- **忘記不常用的 Wi-Fi**：出差回來、旅遊回來，把之前臨時連過的酒店 Wi-Fi、機場 Wi-Fi 都” 忘記” 掉。這樣下次手機不會自動連到同名假熱點。

三、HTTPS 是怎麼保護你的？——SSL/TLS 握手

剛才提到了 HTTPS，那它到底是怎麼工作的呢？每次你訪問一個 HTTPS 網站，瀏覽器和伺服器之間會進行一次“SSL/TLS 握手”，整個過程大概就像這樣：

1. 你的瀏覽器說：” 嘿，伺服器，我想跟你安全通訊，這是我的加密能力清單（支援的加密演演算法等）。”
2. 伺服器回覆：” 好的，我從你的清單裡選了這套加密方案，這是我的數字證書，裡麵包含我的公鑰，由某某 CA 機構簽名認證。”
3. 瀏覽器驗證證書的真假：是不是由可信的 CA 簽發？有沒有過期？域名對不對得上？
4. 驗證透過後，雙方用公鑰加密協商出一個” 會話金鑰”，之後的通訊全用這個金鑰加密。

這就像你去銀行辦事：你先遞上身份證（證書），櫃員核實你的身份，然後你們倆約定一個暗號，之後的對話只有你倆能聽懂。即使有人在旁邊偷聽，也只能聽到加密後的亂碼。

但注意：HTTPS 只保證“傳輸過程”是加密的，不保證目標網站是好人。釣魚網站也可以用 HTTPS——那個小鎖圖示只能說明你連線的是“某個網站”，不代表那個網站就是你想去的那個。所以還是要自己核對網址是否正確。

四、DNS 安全：你的“網路電話本”安全嗎？

DNS（域名系統）就像是網際網路的電話本：你輸入“baidu.com”，DNS 幫你查到它對應哪個 IP 地址（比如 110.242.68.66），然後瀏覽器才能連過去。問題是，傳統的 DNS 查詢是**明文**的、**沒有加密**的！等於說，你的 ISP、Wi-Fi 提供者、或者任何中間節點，都能看到你訪問過哪些網站。

更噁心的是“DNS 劫持”：攻擊者在 DNS 環節做手腳，把“baidu.com”指向一個假的釣魚網站。你明明輸入的是正確網址，卻被帶到了一個一模一樣但實際是假的登入頁面，一輸入帳號密碼就完蛋了。

好在現在有了加密 DNS 方案：

- **DNS over HTTPS（DoH）**：把 DNS 查詢藏在 HTTPS 流量裡，混在正常的網頁瀏覽流量中，根本分不出來哪些是 DNS 查詢。
- **DNS over TLS（DoT）**：用 TLS 加密協議專門保護 DNS 查詢，埠獨立，網管能知道你用了 DoT 但看不到內容。

我個人推薦用 Cloudflare 的 1.1.1.1 或者 Google 的 8.8.8.8 作為 DNS 伺服器，它們都支援加密 DNS。在安卓的“私人 DNS”設定裡直接填“one.one.one.one”或者“dns.google”就行，超簡單。iPhone 稍微麻煩點，需要裝個描述檔案或者透過 App 設定，但也值得搞一下。

五、“無痕瀏覽”真的能隱身嗎？

先說結論：**無痕模式是你家窗簾，不是隱身斗篷。**

它做了什麼：不在本地留下痕跡——關閉無痕視窗後，瀏覽器不儲存你的瀏覽歷史、Cookies、表單填寫記錄、搜尋記錄。借別人電腦查個東西、或者買禮物不想被家人從瀏覽記錄發現，用無痕模式是合適的。

它做不到什麼：

- 你訪問的網站**照樣能看到你來了**，你的 IP 地址、裝置資訊、瀏覽行為，網站端一清二楚。
- 你的網路運營商**照樣能看到你訪問了哪些網站**，只是看不到具體內容（如果用了 HTTPS）。
- 公司/學校的網路管理員**一樣能監控**你的上網行為。
- 你下載的檔案、新增的書籤會**保留下來**，不會隨著關閉視窗而消失。

所以記住：無痕模式是“本地擦除器”，不是“網路隱身衣”。如果你真的需要隱藏上網痕跡，得用 VPN+Tor 瀏覽器這樣的組合拳，不過對於日常使用來說，瞭解這些就夠用了。

六、瀏覽器隱私：別讓瀏覽器把你賣了

你可能沒意識到，瀏覽器本身就是一個巨大的資訊收集器。Cookies、瀏覽歷史、快取、自動填充、位置權限……每一樣都可能洩露你的隱私。幾個建議：

- **定期清理瀏覽資料**：每隔一段時間，去設定裡把 Cookies、快取、歷史記錄清一清。不是為了防止”被追蹤”——那個靠清 Cookies 已經不太管用了——而是為了防止手機越來越卡。瀏覽器快取能佔好幾個 G 的空間。
- **關掉”請勿追蹤”的幻想**：瀏覽器裡的”傳送’請勿追蹤’請求”選項，本質上是”請”網站不要追蹤你。但網站聽不聽完全是另一回事，大部分網站直接忽略。別指望它。
- **善用隱私保護瀏覽器**：Firefox Focus、DuckDuckGo 瀏覽器、Brave 這些瀏覽器預設攔截追蹤器和廣告，比 Chrome 和 Safari 的預設設定要乾淨得多。
- **搜尋引擎也重要**：用 DuckDuckGo 代替百度/Google 搜尋，它不記錄你的搜尋歷史，不建立使用者畫像。說實話搜尋結果確實不如 Google 精準，但日常查資料夠用了。
- **檢查 App 權限**：去手機設定裡看看你的瀏覽器被授予了什麼權限。它真的需要訪問你的位置、麥克風、通訊錄嗎？大機率不需要。

七、實用安全清單：保護自己從現在開始

說了這麼多，最後給你一個簡單粗暴的安全清單，花半小時搞定，你的手機安全性會有質的飛躍：

1. **關掉 Wi-Fi 自動連線**：iPhone 在設定-無線區域網裡，安卓在 Wi-Fi 設定裡。
2. **換一個加密 DNS**：安卓直接設定私人 DNS 為”dns.google”，iPhone 用 App 或描述檔案設定 1.1.1.1 的 DoH。
3. **開啟雙因素認證（2FA）**：微信、支付寶、Apple ID、Google 帳號，能開 2FA 的都開。簡訊驗證碼已經是最低階別的 2FA 了，能用 TOTP（如 Google Authenticator、Microsoft Authenticator）更好。
4. **查一遍 App 權限**：把每個 App 的權限過一遍，特別是位置、通訊錄、簡訊、麥克風、相機這五項。你會發現有些手電筒 App 居然要你的通訊錄——趕緊解除安裝。
5. **定期清理瀏覽器資料**：設定個日曆提醒，每月一次，清快取、Cookies。
6. **不要用公共充電樁的資料線**：USB 口既能充電也能傳資料，惡意充電樁（叫”Juice Jacking”）可以趁你充電時偷資料。出門帶自己的充電頭和線。

現在就做：給你的手機做一次”安全體檢”

1. **DNS 升級**：安卓：設定 > 連線與共享 > 私人 DNS > 填入”one.one.one.one”或”dns.google”。iPhone：用 Safari 搜尋”1.1.1.1 DNS profile”安裝描述檔案，或下載 Cloudflare 的 1.1.1.1 App。
2. **Wi-Fi 審計**：開啟 Wi-Fi 列表，把不認識的、不常用的、名字怪怪的熱點都”忘記”掉。
3. **權限大掃除**：進入設定 > 隱私/權限管理，逐個 App 審查權限。標記出那些權限要求

不合理（比如計算器要通訊錄）的 App，考慮解除安裝或用替代品。

4. **檢查帳號安全**：開啟 Apple ID 或 Google 帳號的安全頁面，看看有沒有不明裝置登入，確認 2FA 已開啟，檢查恢復聯絡人是否可信。

第二十二章：AI 如何在手機中工作

歡迎來到手機最酷的“魔術現場”，主角是：人工智慧。你每天在用的“一拍即得”的美照、張口就來的語音助手、越用越懂你的輸入法，背後都站著這位隱形大腦。

一、AI 相機：你的“後期暗房”搬進了手機

現在的 AI 計算攝影，更像是把一個專業修圖師 + 攝影師團隊直接塞進了快門鍵裡。在你按下快門前，AI 就已經開始工作了：掃描場景，識別出“這是藍天”、“這是人臉”、“這是夜景”。

二、語音助手：你的“聲音秘書”如何聽懂你？

1. **聽清（語音識別）**：將聲波轉變成文字。

2. **聽懂（自然語言理解）**：理解意圖，提取關鍵實體。

3. **執行與回答**：呼叫對應 App 或服務，再透過語音合成” 說” 給你聽。

三、端側 AI vs. 雲端 AI

端側 AI 把計算留在手機裡，速度快、隱私高、可離線。雲端 AI 需要聯網，但能處理更復雜的任務。

四、AI 大模型與手機：你的隨身智慧助手

2025-2026 年，AI 大模型（如 DeepSeek R1、ChatGPT-5、文心一言 5.0、通義千問 3 等）的爆發深刻改變了手機的使用方式。手機不再只是執行指令的工具，而是能理解、推理和創作的智慧夥伴。端側 AI 晶片（NPU 算力突破 80 TOPS）讓大模型推理首次在手機本地跑通，無需聯網也能用 AI。

什麼是 AI 大模型？

AI 大模型（Large Language Model，LLM）是透過海量文字資料訓練出來的 AI 系統，能理解和生成自然語言。你可以把它想象成一個讀過網際網路上幾乎所有文字的“超級學霸”，能回答問題、寫文章、翻譯、程式設計、分析資料。

手機上的 AI 大模型應用：

- **對話助手**：DeepSeek、ChatGPT、豆包、Kimi 等 App，可以隨時提問、翻譯、寫文案、總結文章。比傳統搜尋引擎更智慧——它不只給你連結，而是直接給你答案。
- **寫作與創作**：讓 AI 幫你寫郵件、總結會議紀要、潤色文章、生成創意文案。
- **影象生成與編輯**：AI 一鍵消除照片中的路人、擴充圖片邊界、風格轉換、甚至根據文字描述生成圖片。
- **實時翻譯**：通話實時翻譯、面對面對話翻譯、拍照翻譯，出國旅行不再有語言障礙。
- **學習輔導**：AI 可以解釋概念、出題練習、批改作業，是隨身家教。

端側大模型：手機自己跑 AI

最新的旗艦手機已經能在本地執行小型 AI 大模型（通常 7B 引數左右），帶來的變化是：

- **不需要網路**：斷網也能用 AI 功能，飛機上、地鐵裡都行。
- **隱私更安全**：你的對話和資料不會上傳到雲端，完全在手機本地處理。
- **響應更快**：沒有網路延遲，AI 回覆幾乎是即時的。
- **當前限制**：端側模型的能力還比不上雲端大模型，複雜任務仍需聯網。但隨著晶片 AI 算力的提升，端側模型會越來越強。

系統級 AI 整合：

- **蘋果 Apple Intelligence**：iOS 18 引入的 AI 系統，整合寫作工具、影象魔法棒、Siri 增強、通知摘要等功能。端側處理優先，複雜任務才上雲端。
- **華為盤古大模型**：整合到鴻蒙系統中，小藝助手獲得大模型能力，支援檔案摘要、智慧識屏、AI 修圖等。
- **小米澎湃 AI**：系統級 AI 能力，支援 AI 搜圖、AI 摘要、AI 實時字幕、端側大模型對話等。
- **三星 Galaxy AI**：通話實時翻譯、筆記 AI 摘要、圖片生成編輯、搜尋圈選等功能。

五、幾個典型應用場景與實操

為了讓你更直觀地感受手機 AI 的“超能力”，下面我們用五個具體場景，展示 AI 是如何在日常生活中“出手相救”的：

場景 1：實時翻譯——出國旅行的隨身翻譯官

實際表現：在巴黎點餐不會法語？iPhone 的“翻譯”App 支援面對面雙人對話模式，雙方各看自己語言的字幕，甚至可以同聲傳譯。安卓陣營的三星 Galaxy AI 通話實時翻譯能讓你在電話裡直接說中文，對方聽到的是英文，反之亦然。

背後原理：語音識別（ASR）機器翻譯（NMT）語音合成（TTS）三步流水線，端到端延遲已可控制在 1 秒內。

場景 2：AI 消除——拯救被路人“毀掉”的遊客照

實際表現：在景點拍了一張絕美的自拍，但背景裡擠滿了其他遊客？用“魔法消除”圈選路人，手機會自動識別並把路人“塗掉”，再智慧填充背景，效果常常令人驚豔。

背後原理：影象分割（Inpainting）模型先識別並刪除選中區域，然後根據周圍畫素和語義理解“腦補”出合理的填充內容。這背後的 AI 模型訓練資料量高達數百萬張圖片。

場景 3：AI 修圖——一鍵變“攝影大師”

實際表現：拍了一張陰天的灰濛濛風景照？AI“魔法天空”可以一鍵把天空換成晴空、晚霞甚至星空。AI 擴圖可以讓豎構圖變成橫構圖，手機“腦補”出畫面外的場景。

背後原理：基於生成式 AI（Generative AI）的擴散模型（Diffusion Model），理解影象的語義內容後生成新畫素。這是當前 AI 影象處理的“最前沿”。

場景 4：檔案處理——辦公族的效率倍增器

實際表現：會議結束後，把會議錄音丟給 AI，它能自動整理出結構化的會議紀要、提取待辦事項、生成摘要。一份 10 頁的英文 PDF 報告，AI 可以在 10 秒內給你提煉出核心結論。

背後原理：長上下文大模型（Long Context LLM）可以“讀完”數萬字的檔案，進行多輪推理和資訊抽取。Apple Intelligence、華為小藝、WPS AI、騰訊元寶等都具備此能力。

場景 5：AI 搜圖與識物——你的“視覺搜尋引擎”

實際表現：在街上看到一種不認識的花，相機一開，AI 立刻告訴你“這是木槿花”。相簿裡翻找去年在海邊拍的照片，搜尋“大海”，AI 自動把所有含海的照片找出來，不用手動翻。

背後原理：計算機視覺（CV）模型對圖片內容進行向量化（Embedding），再與文字查詢進行相似度匹配。這種“多模態搜尋”是手機相簿管理的一大飛躍。

六、提示詞最佳化：讓 AI 聽你指揮的“溝通公式”

AI 的回答質量，很大程度上取決於你如何“提問”。同樣一個問題，問法不同，AI 給的答案天差地別。下面我們介紹一套**最實用、最容易上手的“角色-背景-任務-限制”溝通思路**，並配以大量例項。

為什麼需要框架？

很多人用 AI 像用百度一樣——敲幾個關鍵字進去。但 AI 不是搜尋引擎，它是一個“閱讀理解後回答”的語言模型。**給它更多上下文、說清你要什麼、限制它別跑偏**，它就能給你精準的結果。框架就是把你腦子裡的“好答案長什麼樣”翻譯成 AI 能理解的指令。

角色（Role）：告訴 AI 它是誰

給 AI 一個具體的“身份”，它就會呼叫對應領域的專業知識和表達方式。**角色設定得越具體，輸出越專業**。

實用示例：

- ”你是一位**有 10 年經驗的小米手機維修工程師**。”（技術問題）
- ”你是一位**擅長把複雜概念講簡單的中學物理老師**。”（科普問題）
- ”你是一位**審美好、讀得多、有自己風格的圖書編輯**。”（寫作問題）
- ”你是一位**每天給上班族做早餐的烹飪博主**。”（生活問題）
- ”你是一位**耐心、溫和、永遠不嘲笑新手的中醫**。”（健康問題）

為什麼有效？角色設定了 AI 的**知識範圍、表達風格**和**專業深度**。沒有角色的 AI 是個“全才但啥都不精”的通才；有了角色，它能“進入”那個身份，像那位專業人士一樣思考。

背景（Background）：告訴 AI 你的情況

把“你是誰、你的現狀、你的約束條件”提前說清楚，AI 就能給出**真正適合你**的答案，而不是泛泛而談。

實用示例：

- ”我的手機是**小米 14，買了半年，最近玩遊戲時發燙嚴重**。”（裝置情況）
- ”我是**一個 60 歲的退休阿姨，眼睛不太好，不太會用智慧手機**。”（使用者畫像）
- ”我正在**準備 2026 年 6 月的英語六級考試，基礎一般**。”（學習背景）
- ”我開了一家**三線城市的小咖啡店，月流水 3 萬，僱了兩個員工**。”（商業背景）
- ”我**剛生完寶寶在家坐月子，睡眠嚴重不足**。”（生活階段）

為什麼有效？背景資訊讓 AI 知道**“為誰服務”**和**“在什麼條件下”**。同一道題，給學生和給家長講法不同；給城裡人和給鄉鎮人建議也不同。背景缺失，AI 只能給“最大公約數”的答案，往往不痛不癢。

任務（Task）：明確告訴 AI 做什麼

這一步要用**動詞開頭**（寫、列、對比、翻譯、解釋、總結……），把任務描述得**具體、可執行**。含糊的任務只會換來含糊的答案。

實用示例：

- ”請**列出 5 個我應該立刻關閉的權限**，並說明原因。”（動作明確）
- ”請**把下面這段英文郵件翻譯成中文**，保持商務口吻。”（任務具體）
- ”請**對比這四款 3000 元價位的手機**，給出一張優缺點對比表。”（格式明確）
- ”請**用小學生能懂的話解釋’什麼是 5G’**。”（風格明確）
- ”請**給我的早餐店寫三條短影片文案**，分別針對上班族、情侶、帶孩子的家長。”（細分明確）

為什麼有效？任務部分是 AI 行動的“指令中樞”。**動詞越清晰、要求越具體，AI 越知道“交付什麼”**。模糊的“弄一下”、“搞個”、“看看”是 AI 大忌——它會揣測你的意思，結果常常跑偏。

限制（Constraints）：告訴 AI 不要做什麼

“限制”聽起來像“約束”，其實是**讓 AI 收斂到最適合你的輸出**的利器。常見維度有：字數、格式、語氣、避免項、必含項。

實用示例：

- ”**不超過 200 字，分點列出**。”（長度 + 格式）
- ”**語氣要親切自然，像朋友聊天**，不要用’首先、其次、再次’這種官話。”（語氣）
- ”**不要使用專業術語**，遇到專業名詞要順便解釋。”（用詞限制）
- ”**每段不超過 3 行**，方便手機閱讀。”（排版）
- ”**必須包含：使用場景、優缺點、我的預算匹配度**。”（必含項）
- ”**不要給具體投資建議**，只解釋原理。”（邊界）

為什麼有效？沒有限制，AI 容易“自由發揮”——要麼太學術、要麼太長、要麼跑題。明確的限制 = 明確的“交付標準”，雙方對齊預期，結果才可控。

提示詞質量評估：好提示詞的 5 個特徵

寫好提示詞不是一次性的事，**寫完先自查這五條**，再發給 AI：

1. **具體而非抽象**：不是” 寫篇文章”，而是” 寫一篇 800 字、3 個小節、面向新手媽媽的手機選購指南”。
2. **包含背景而非孤立**：告訴 AI” 我是誰、為什麼問這個”，它才能給出” 為你定製” 的答案。
3. **有明確輸出格式**：是表格、列表、段落、還是 JSON？提前說。
4. **限制到位**：字數、風格、避項，三選一就有明顯提升。
5. **可被 AI 驗證**：問完之後，讓 AI 複述一遍你的要求；說不清楚就是沒說清楚。

常見問題與解法

症狀	解法
回答太泛、不實用	加背景（你是誰/什麼場景）
回答太學術/太官方	加語氣限制 + 拒絕” 官話” 清單
回答跑題/答非所問	把任務用動詞重新說一遍，並要求” 只回答這個”
回答太長看不完	硬性限制字數（200 字內）
AI 一本正經胡說	加” 如不確定請標註’ 我不確定’” + 重要事實自查
第一版不滿意	不要重寫，在原提示詞後追加” 再…一些”

三個完整場景案例

場景一：生活助手（早餐搭配建議）

提示詞：

你是一位擅長營養搭配的早餐博主（**角色**）。我是一位上班族媽媽，每天早上只有 15 分鐘做早餐，家裡有 6 歲的孩子和 60 歲的婆婆（**背景**）。請**列出 7 天不重樣的早餐搭配**（**任務**）。每份早餐要求：(1) 準備時間不超過 15 分鐘 (2) 營養均衡（蛋白 + 碳水 + 蔬果）(3) 老少皆宜 (4) 食材在普通超市能買到（**限制**）。

為什麼有效：角色定專業度，背景給三類人共性，任務明確交付物，4 條限制確保可執行。

場景二：學習輔助（英語六級備考）

提示詞：

你是一位**經驗豐富的英語六級老師**（角色）。我是一名普通本科生，**四級 480 分，六級目標 500，還有 3 個月**（背景）。請**為我制定一份可執行的 12 周備考計劃**（任務）。要求：(1) 每週明確聽說讀寫的時間分配 (2) 給出每個月的重點突破方向 (3) 推薦 3 本合適的複習資料（中文說明理由）(4) 表格形式輸出（限制）。

為什麼有效：角色定專業背景，背景給出分數和時間錨點，任務明確“分階段計劃”，限制

要求“表格化”和“可執行”。

場景三：工作效率（會議紀要整理）

提示詞：
你是一位**資深行政助理**（角色）。我剛開完一個 1 小時的部門會議，**討論 Q2 營銷推廣方案**（背景）。**請把以下會議記錄整理成結構化紀要**（任務）。要求：(1) 分“決議事項 / 待辦事項 / 風險與異議”三部分 (2) 待辦事項標出負責人和截止時間（如未提到則寫“待確認”）(3) 總字數控制在 600 字內 (4) 用 markdown 格式輸出（限制）。
為什麼有效：角色定行政視角，背景給會議主題，任務明確結構化輸出，4 條限制讓結果直接可複用。

進階玩法：讓 AI 成為你的“私人助理”

試試這些進階用法，體驗 AI 的真正實力：

- **多輪對話最佳化：**對 AI 的回答不滿意時，可以說” 這個太專業了，請用小學生能懂的話再解釋一次”。
- **鏈式思考（CoT）：**讓 AI” 一步步思考”，回答質量通常更高：” 請一步步分析這道數學題”。
- **多 AI 協作：**用 A 寫初稿，B 潤色，C 翻譯——組合使用效果更佳。
- **系統提示詞：**在 AI 設定中預設你的身份、偏好、忌諱等，讓回答更貼合你的需求。

七、AI 的侷限與挑戰：保持清醒的判斷力

AI 雖然強大，但絕非“無所不能”。清醒地認識它的侷限，才能更好地使用它：

- **” 幻覺” 問題：**AI 可能一本正經地胡說八道（專業術語叫 Hallucination）。它會編造不存在的歷史事件、錯誤的引言、虛假的統計資料。重要資訊必須人工核實。
- **知識截止：**大多數 AI 的訓練資料有截止日期（如 2024 年初），它不知道之後發生的事。
- **推理能力有限：**複雜的多步推理、數學證明、邏輯陷阱，AI 仍然容易出錯。
- **隱私風險：**在雲端 AI 服務中，你的對話可能被用於模型訓練。敏感資訊應優先使用端側 AI。
- **偏見與價值觀：**AI 可能帶有訓練資料中的偏見，需要保持獨立思考。

AI 使用小貼士

- AI 生成的內容可能不準確（” 幻覺” 問題），重要資訊務必核實。
- 不要向 AI 輸入個人隱私資訊（身份證號、密碼、銀行資訊等）。
- 善用提示詞（Prompt）：越具體、越清晰的描述，AI 的回答質量越高。

- AI 是工具不是權威，保持獨立思考和判斷力。
- 遇到關鍵決策（醫療、法律、財務），諮詢專業人士而非依賴 AI。

來挑戰：三道 AI 實戰題

1. **挑戰一：複雜指令：**對語音助手說：” 明早 7 點提醒我帶傘，並把今天下午 3 點的日曆改到 4 點。” 看看它能否一次完成多步操作。
2. **挑戰二：AI 修圖：**拍一張有路人的遊客照，使用” 魔法消除” 或”AI 橡皮擦” 功能，把路人清除乾淨。觀察 AI 對背景的” 腦補” 效果。
3. **挑戰三：提示詞最佳化：**先用簡單提示詞” 寫一段旅遊攻略” 問 AI，再用 CRISPE 框架寫一個詳細提示詞。比較兩次回答的質量差異。

案例 2：換機資料遷移完整流程

- **場景：**iPhone 13 小米 15 跨品牌換機。
- **步驟：**(1) 舊機 iCloud/微信自帶備份 (2) 退出 Apple ID + 關閉” 查詢” (3) 新機用” 小米換機”App 掃碼 (4) 選擇遷移項 (5) 微信單獨用自帶” 聊天記錄遷移” (6) 驗證關鍵資料（通訊錄/相簿/微信）(7) 舊機加密擦除。
- **預期結果：**資料零丟失，舊機可安全轉賣。

第二十三章：程式設計思維與自動化啟蒙

前兩章，咱們給手機築起了安全的“邊界牆”，也見識了它在牆內日益聰明的“*AI* 大腦”。現在，讓我們解鎖手機的終極潛力之一：讓它像程式設計師一樣“思考”，並自動完成你設定的連續任務。你可能會說：“我又不會寫程式碼，這跟我有什麼關係？”放心，本章講的根本不需要你寫一行程式碼——現代手機已經把“程式設計”的門檻降到了拖拖拽拽就能搞定的程度。

一、自動化到底是個啥？

用大白話說，自動化就是：**讓手機在某個條件滿足時，自動幫你做一連串事**。你不需要每次都手動操作，也不用記住該做什麼——手機會自己判斷時機，自己執行。

舉個例子：每天早上你醒來，要手動關掉勿擾模式、開啟微信看看訊息、開啟天氣 App 看今天穿什麼、開啟網易雲放首歌。如果把這些步驟打包成一個自動化流程，設定成“每天早上 7 點自動執行”，那你醒了只需要拿起手機，一切都已經準備好了。是不是想想就很爽？

自動化本質上就是“如果……那麼……”的邏輯鏈條，程式設計裡的 if-else。學明白了這個思維方式，哪怕你一輩子不寫程式碼，處理問題的條理性也會強很多。核心要素：

1. **觸發條件**：什麼時候開始？可以是” 每天早上 7 點”、” 當我連線到家裡的 Wi-Fi”、” 當我到達某個地點”、” 當手機電量低於 20%”。

2. **執行動作**：具體做什麼？可以是發一條微信、開啟某個 App、調整音量、切換桌布、播放音樂、傳送通知……幾乎所有你能在手機上手動做的事，自動化都能做。

3. **中間邏輯**（可選但很強大）：如果情況 A 就做 X，否則做 Y。比如” 如果是工作日就播放新聞播客，週末就播放音樂”，這叫條件分支。

二、iOS“快捷指令”——蘋果使用者的自動化神器

蘋果的“快捷指令”（Shortcuts）App 是自帶的應用，不需要下載。它的核心概念就是把各種操作像搭積木一樣拼起來。先說幾個關鍵概念：

- **” 快捷指令”**：一個完整的自動化流程，可以手動點選執行，也可以透過 Siri 語音喚醒。比如你說” 嘿 Siri，回家模式”，它就自動執行你設定好的一連串操作。

- **” 自動化”**：由觸發條件自動執行的指令，不需要你手動觸發。比如” 當手機連線到車載藍芽時，自動開啟地圖 App”。自動化在” 自動化” 標籤頁下設定，和” 快捷指令” 是分開的。
- **” 動作”**：最小的執行單元。開啟 App 是一個動作，傳送訊息是一個動作，獲取天氣是一個動作，調整亮度是一個動作。你可以把成百上千個動作串在一起。

給你看幾個實實在在好用的例子，你跟著做一遍就明白了：

例項一：一鍵“早安流程”

每天早上點一下（或對 Siri 說“早上好”），手機會自動：獲取今天的天氣朗讀天氣和氣溫顯示今天的日曆安排開啟你常用的音樂 App 開始播放。操作步驟：新建快捷指令新增動作“獲取當前天氣”新增“朗讀文字”新增“獲取日曆事件”新增“顯示提醒”新增“開啟 App”選網易雲音樂。就這麼簡單，全程拖拽。

例項二：自動“省電模式”

在“自動化”標籤頁，新建自動化選“電池電量”設“低於 30%”新增動作“設定低電量模式為開啟”、“降低螢幕亮度到 30%”、“關閉藍芽”。從此手機電量低於 30% 就自動進入省電狀態，不用你操心。

例項三：掃碼自動記帳

每次你用微信/支付寶掃碼支付後，手機會讀取支付通知，自動把金額和商家記錄到一個備忘錄或 Numbers 表格裡。這需要用到快捷指令的“自動化”裡的“收到簡訊/通知”觸發條件（iOS 17+ 支援讀取錢包交易），配合“追加到 Numbers”動作。對於想記帳又懶得手動記的人來說，簡直是救星。

還有一個很多人不知道的巧用：你可以把快捷指令新增到主螢幕，變成一個 App 圖示。比如做一個“打車回家”的快捷指令，圖示放桌面上，點一下就直接開啟滴滴並把目的地設成家，比你自己開啟 App 慢慢輸入快多了。

三、安卓自動化——Tasker 和它的朋友們

安卓這邊，情況稍微複雜一點，但天花板也更高。先說簡單的：

入門級：自動點選器

很多安卓手機自帶“自動點選器”或者可以在應用商店下載。它的原理超簡單：你手動操作一遍，它把你點選的座標和時間間隔錄下來，然後可以高速回放。比如某個 App 每天要領簽到獎勵，你錄一遍點選流程，以後每天自動跑一遍就完事了。缺點是：它只認座標，如果介面佈局變了（比如彈了個廣告），它就點歪了。

進階級：Tasker——安卓自動化之王

Tasker 是安卓平臺上公認最強的自動化工具，常年霸榜。說實話它的學習曲線有點陡，介面不算好看，但一旦上手，你會覺得手機突然變得無所不能。Tasker 的核心也是“觸發任務”的模式：

- **配置檔案（Profile）**：定義” 什麼時候觸發”。觸發條件可以是時間、地點、Wi-Fi 連線狀態、藍芽連線、收到特定通知、電池狀態、螢幕朝向……Tasker 支援的觸發條件多到離譜，有幾十種。
- **任務（Task）**：定義” 觸發後做什麼”。一個任務可以包含多個動作，按順序執行。動作型別包括：調整系統設定（音量、亮度、Wi-Fi、藍芽、飛航模式）、傳送簡訊、HTTP 請求、執行 Shell 命令、控制媒體播放、彈出對話方塊等等。

給你幾個 Tasker 的經典實用場景：

- **到家自動開 Wi-Fi**：配置觸發條件為” 連線到特定基站”（因為你在家附近會連到固定的基站）任務” 開啟 Wi-Fi”。不用 GPS 也不耗電，手機一進小區範圍就自動開 Wi-Fi。
- **會議模式**：觸發條件為” 日曆事件標題包含’ 會議’” 任務” 設定靜音模式、關閉媒體音量、自動回覆微信（需要外掛配合）”。再也不用擔心開會時手機突然響了。
- **深夜護眼**：觸發條件為” 時間 22:00-06:00” 任務” 開啟護眼模式、降低亮度到 20%、將螢幕設為黑白（灰度模式）”。黑白螢幕對刷短影片的慾望有奇效——畫面一灰，你就不想刷了。
- **耳機插入自動播客**：觸發條件為” 有線耳機插入/藍芽耳機連線” 任務” 開啟小宇宙 App 並開始播放最新一集”。戴上耳機就能聽，不用解鎖手機找 App。

如果你覺得 Tasker 上手太難，可以試試它的簡化版兄弟：**MacroDroid**。介面友好很多，模板豐富，適合不想折騰的使用者。另外現在很多國產安卓系統（小米的 MIUI、華為的鴻蒙、OPPO 的 ColorOS）都自帶了“場景”或“智慧助手”功能，雖然沒有 Tasker 那麼強大，但日常自動化完全夠用。

四、自動化思維的日常應用

學會自動化最大的收穫，其實不是省那點時間，而是**改變你思考問題的方式**。一旦你習慣了“如果發生 X，就自動做 Y”的思維模式，你會發現生活中一大堆重複勞動都可以被解決。比如：

- 每次拍完照片，自動備份到網盤然後把本地照片壓縮成小尺寸節省空間。
- 每天早上自動從日曆讀取今天的安排，生成一段文字透過微信發給家人。
- 收到快遞簽收簡訊後，自動在備忘錄裡新增一條待辦” 取快遞”。

- 手機電量充滿時自動發出語音提醒拔掉充電器（避免過充傷電池）。
- 每天下班時間到了，如果還在公司附近（透過基站定位），自動提醒”該下班了”。

這些聽起來很高階，但其實你只要掌握了“觸發 + 動作”這兩個基本概念，配合快捷指令或 Tasker，都能做出來。關鍵是**你要先意識到“這件事可以自動化”**——很多人不是不會，是根本沒往那方面想。

五、踩坑指南：自動化的常見問題

搞自動化也不是一帆風順的，有幾個坑我幫你先踩過了：

- **位置觸發可能不靈**：iOS 的”到達/離開位置”觸發用的是地理圍欄技術，準確度取決於 GPS 訊號和系統最佳化。有時候你都到家十分鐘了才觸發。建議用 Wi-Fi 連線或基站觸發代替位置觸發，更可靠也更省電。
- **快捷指令的”自動化”需要授權**：iOS 裡，”自動化”標籤下的觸發大多數需要你手動確認才能執行（iOS 的安全機制）。但 Wi-Fi 連線、藍芽連線、鬧鐘停止這幾個觸發是可以真正全自動執行的，不需要確認。
- **Tasker 需要開啟無障礙權限**：很多高階功能（比如模擬點選）需要無障礙服務權限，這會讓手機在設定裡顯示一個常駐通知。這是安卓的安全設計，沒法去掉。
- **別把自動化搞得太複雜**：剛開始做自動化，從”2-3 個動作”的小流程開始。別一上來就搞一個 20 步的超級流程——除錯起來會瘋掉的。

小貼士：自動化入門的正確姿勢

先花 5 分鐘想一下：你每天在手機上重複最多的 3 件事是什麼？比如每天早上的例行操作、每次連藍芽耳機後的習慣、每次出門前的設定……從這些高頻小事入手，各做一個簡單的自動化。哪怕每個自動化只省你 5 秒鐘，日積月累也是可觀的。而且做出來之後的滿足感，真的會上癮。

練一練：三個入門自動化挑戰

選一個最適合你的完成，做完你就入門了：

1. **挑戰一：早安自動化**：iOS：在快捷指令 App 裡新建快捷指令新增”獲取當前天氣”” 朗讀文字”” 開啟 App 選網易雲”。完成後你可以對 Siri 說”早上好”來觸發。安卓：在 Tasker 裡新建任務新增”獲取天氣”” 朗讀”” 開啟 App”設定觸發時間為每天 7:00。
2. **挑戰二：低電量自動省電**：iOS：自動化標籤新建”電池電量”低於 25% 新增”設定低電量模式開”+”設定亮度 25%”。安卓：Tasker 新建 Profile 狀態電源電量 0-25% 任務”省電模式開”+”亮度調低”。
3. **挑戰三：連線車載藍芽自動導航**：iOS：自動化新建”藍芽”選擇你的車載藍芽新增”開啟 App 選高德地圖”+”媒體音量調至 80%”。安卓：TaskerProfile 狀態藍芽已連線選擇車載藍芽任務”開啟高德”+”媒體音量調高”。

案例 3：自動化” 早安流程”

- **場景：**每天 7:00 自動播報天氣 + 關閉靜音 + 開啟新聞。
- **步驟：**iOS: 快捷指令自動化時間觸發新增” 獲取天氣”+” 設定音量”+” 開啟 App”;Android: MacroDroid 觸發器 (時間 7:00) 動作 (媒體音量/開啟 URL/TTS 朗讀天氣)。
- **預期結果：**免安裝即可實現，降低上手門檻。

第二十三章附：GitHub 入門指南——從程式碼託管到協作平臺

上一章我們講了“程式設計思維”，而真正要把程式設計思維落地、和他人一起做專案，就繞不開一個名字——*GitHub*。無論你是計算機系學生要參加比賽、想儲存自己的程式碼作品，還是單純想看看全球開發者在折騰什麼，*GitHub* 都是你必須認識的平臺。這一節我們從最基礎的概念講起，幫你建立一個清晰的認知。

一、GitHub 到底是什麼？

簡單來說，GitHub 是一個基於 Git 的**程式碼託管與協作平臺**。它於 2008 年上線，2018 年被微軟收購，至今仍是全球最大的開原始碼託管平臺，託管著數億個倉庫，覆蓋幾乎所有主流程式語言。

如果用一個類比來理解：**Git 是你電腦上的日記本，GitHub 就是一個公開的部落格平臺**——你可以釋出自己的作品，關注別人的專案，參與討論和協作。它不僅是“程式設計師的後臺倉庫”，更像是全球開發者共建的一座“技術城市”——專案在這裡誕生、迭代、被討論、被複用。

對普通人來說，GitHub 也並非遙不可及。很多非程式設計師會用它來託管筆記、電子書、個人部落格，甚至用來備份配置檔案。只要你有一點點“想把自己的東西整理好並分享出去”的需求，GitHub 都值得你瞭解。

二、Git 和 GitHub 的區別（關鍵概念！）

很多人第一次接觸時會混淆這兩個詞，這裡幫你梳理清楚：

對比項	Git	GitHub
本質	版本控制工具（軟體）	程式碼託管平臺（網站）
執行位置	本地電腦上	雲端（瀏覽器訪問）
核心職責	記錄檔案的每次變更（怎麼改的、誰改的、何時改的）	把 Git 倉庫放到雲端，方便儲存、展示、協作
是否需要網路	不需要，離線可用	需要聯網訪問
歸屬	開源軟體，由 Linus Torvalds（Linux 之父）建立	微軟旗下產品
同類競品	SVN、Mercurial	GitLab、Gitee（碼雲）、Bitbucket

簡單說：**Git 管“怎麼記錄變更”，GitHub 管“變更放在哪、怎麼協作”**。Git 是工具，在本地執行；GitHub 是平臺，把 Git 倉庫放到雲端，方便協作和展示。你可以只用 Git 不用 GitHub（比如自己寫程式碼做版本管理），但用 GitHub 一定要用 Git（因為 GitHub 就是基於 Git 的）。

三、GitHub 的核心概念與功能

基礎概念

在動手之前，先認識幾個高頻詞彙，否則看 GitHub 的介面會一頭霧水：

- **倉庫（Repository，簡稱 Repo）**：一個專案對應一個倉庫，裡面存放著這個專案的所有程式碼檔案、歷史版本和檔案。可以理解為一個” 專案資料夾”。
- **提交（Commit）**：對程式碼的一次修改記錄。每次 Commit 都附帶說明資訊，記錄” 這次改了什麼、為什麼改”。這是 Git/GitHub 能” 時光倒流” 的基礎。
- **分支（Branch）**：從主線上分出來的獨立開發線。比如你想加個新功能，可以開一個分支單獨開發，開發完再合併回主線，不影響其他人。
- **克隆（Clone）**：把雲端倉庫完整複製一份到本地電腦，之後就可以在本地修改程式碼。
- **推送（Push）/拉取（Pull）**：Push 是把原生程式碼上傳到 GitHub，Pull 是從 GitHub 下載最新程式碼到本地。這是團隊協作時同步程式碼的兩個核心動作。
- **合併請求（Pull Request，簡稱 PR）**：當你在一個分支上改完程式碼，想合併到主分支時，發起一個 PR，讓團隊成員審查程式碼後再合併。這是 GitHub 協作的核心機制。
- **議題（Issue）**：用來記錄 bug、功能需求、討論話題。每個 Issue 像一個討論帖，可以指派負責人、打標籤。
- **Fork**：把別人的倉庫複製一份到自己帳號下，你可以在自己的副本上隨意修改，不影響原倉庫。改好後可以給原作者發 PR，請求合併你的改進。

核心功能

GitHub 提供的不只是程式碼儲存，還包括：

- **版本控制**：記錄每次程式碼更改的歷史，改壞程式碼隨時回到上一個正常版本。
- **團隊協作**：支援多使用者對同一專案的並行開發，多人同時開發不會覆蓋、不會衝突。
- **問題跟蹤**：用 Issue 記錄 bug 和功能請求，方便團隊跟進。
- **專案管理**：整合了待辦事項、專案看板（Projects）等管理工具，相當於內建了一個輕量級 Trello。
- **自動化**：透過 **GitHub Actions** 實現程式碼提交後自動編譯、測試、打包、部署，是現代"CI/CD（持續整合/持續部署）" 的核心工具。
- **檔案管理**：README 介紹專案，Wiki 存放詳細架構和開發規範，Pages 甚至可以直接託管靜態網站。
- **程式碼託管與展示**：每個倉庫都有漂亮的網頁展示，支援語法高亮，是你技術作品的" 門面"。
- **開源社群**：你可以 Star（收藏）喜歡的專案、Watch（關注）專案動態、Follow（關注）開發者，參與全球開源協作。

四、註冊與上手：五分鐘建立你的第一個倉庫

註冊帳號

1. 訪問 github.com，點選"Sign up"。
2. 填寫郵箱、設定密碼和使用者名稱（使用者名稱會出現在你的個人主頁地址裡，建議用英文，比如 `github.com/yourname`）。
3. 完成郵箱驗證和人機驗證。
4. 選擇免費方案（Free）即可，對個人使用者完全夠用。

建立第一個倉庫

1. 登入後點選右上角"+" 號，選"New repository"。
2. 填寫倉庫名（Repository name），比如 `my-first-project`。
3. 寫一段簡短描述（Description），讓別人知道這個倉庫是幹嘛的。
4. 選擇公開（Public）或私有（Private）。公開倉庫所有人可見，私有隻有自己和協作者可見。免費帳號也支援無限私有倉庫。

5. 勾選”Add a README file”——README 是專案的” 說明書”，每個倉庫都建議有。

6. 點選”Create repository”，你的第一個 GitHub 倉庫就誕生了！

小知識：README 是什麼

README.md 是一個 Markdown 格式的檔案，是別人開啟你倉庫時第一眼看到的內容。一個好的 README 通常包含：專案簡介、安裝方法、使用示例、許可證、聯絡方式。寫好 README 是程式設計師的基本素養，也是專案能否被他人使用的第一道門檻。

五、基本工作流：從本地到雲端

理解 GitHub 的工作流，核心是記住這個迴圈：**本地修改暫存提交推送雲端更新**。下面用大白話解釋每一步：

1. **修改檔案**：在本地電腦上用編輯器寫程式碼或改檔案。

2. **git add**：把修改的檔案加入” 暫存區”，相當於告訴 Git” 這些改動我準備記錄下來”。

3. **git commit**：把暫存區的改動正式記錄成一個” 提交”，附上一句說明，比如” 修復登入 bug”。這一步只在本地完成。

4. **git push**：把本地的提交上傳到 GitHub 雲端，這樣別人就能看到了。

5. **git pull**：在開始工作前，先從雲端拉取最新程式碼，確保本地和團隊其他人保持同步。

對新手來說，記住這個口訣：**“先 pull 後改，改完 add，commit 帶說明，最後 push 上去”**。養成這個習慣，團隊協作就不會亂。

六、團隊協作流程：Pull Request 是怎麼工作的

GitHub 最強大的地方在於它的協作機制。一個典型的團隊協作流程是這樣的：

1. **開分支**：從主分支（通常叫 main 或 master）開一個新分支，比如 `feature/login`，在這個分支上開發新功能。

2. **開發並提交**：在新分支上寫程式碼，多次 commit 記錄進度。

3. **推送到 GitHub**：把分支 push 到雲端倉庫。

4. **發起 Pull Request**：在 GitHub 網頁上發起 PR，請求把你的分支合併到主分支。寫清楚這個 PR 做了什麼改動、為什麼改。

5. **程式碼審查（Code Review）**：團隊成員檢視你的程式碼，提出意見或建議修改。你可以根據反饋繼續 commit，PR 會自動更新。

6. **合併**：審查透過後，點選”Merge” 把程式碼合併到主分支。你的貢獻就正式進入專案了！

這套流程保證了**任何程式碼進入主分支前都經過審查**，大大降低了出錯機率。開源專案也是用這套機制接受全世界陌生人的貢獻——你完全可以在某個知名開源專案上提交 PR，被採納後你就是這個專案的貢獻者了。

七、GitHub 的典型應用場景

學生參加比賽與課程專案

計算機系學生參加 ACM、駭客松、畢業設計，幾乎都離不開 GitHub。它既能作為團隊協作平臺，又能作為作品展示視窗——比賽評委可以直接看你的倉庫，瞭解程式碼質量和協作過程。很多公司招聘時也會要求附上 GitHub 主頁，作為技術能力的證明。

個人知識庫與筆記託管

非程式設計師也能用 GitHub。很多人用它託管學習筆記、技術部落格（配合 GitHub Pages 可以免費建站）、電子書。Markdown 格式寫作 + Git 版本管理 + 雲端備份，三合一。

參與開源專案

GitHub 是全球開源運動的核心陣地。你可以 Fork 喜歡的專案，修改後發 PR 回饋社群；也可以單純 Star 收藏好專案，跟蹤更新。參與開源是提升技術、積累聲譽的絕佳途徑。

自動化部署與 CI/CD

藉助 GitHub Actions，可以實現“程式碼一 push，自動測試 + 自動部署到伺服器”的全自動流程。很多個人部落格、檔案站點都是這麼搭建的，寫完文章 push 一下，網站就自動更新了。

程式碼備份與多裝置同步

把程式碼倉庫當作雲盤用，換電腦、多裝置辦公時，git pull 一下就能拿到最新程式碼，再也不用隨身碟拷來拷去。

八、給新手的幾點建議

- **提交說明要寫清楚**：別寫”update” 這種廢話說明。好的 commit message 寫” 修復使用者登入後跳轉錯誤頁面的問題”，半年後回看也知道當時幹了啥。
- **勤提交，小步快跑**：每完成一個小功能就 commit 一次，而不是寫了一整天才提交一次。這樣出問題更容易定位。
- **用分支開發，別直接改主分支**：哪怕是個人專案，養成開分支的習慣，未來團隊協作時就不會手忙腳亂。

- **善用.gitignore**：這個檔案用來告訴 Git 哪些檔案不要納入版本管理（比如密碼配置、編譯產物、依賴資料夾），避免把垃圾檔案傳上去。
- **多看優秀開源專案**：逛逛 GitHub Trending，看看高星專案的程式碼結構、README 寫法、協作流程，是提升最快的免費方式。
- **申請 GitHub 學生包**：學生認證後可免費獲得大量付費工具和服務的額度（Copilot、域名、雲服務等），價值上千美元。

安全提醒

- 千萬不要把密碼、API 金鑰、資料庫連線串等敏感資訊提交到 GitHub，尤其是公開倉庫——一旦洩露，會被爬蟲秒抓，後果嚴重。
- 不小心提交了敏感資訊，光刪除檔案不夠（歷史記錄裡還有），要用 `git filter-branch` 或專業工具清理歷史，並立即更換洩露的金鑰。
- 私有倉庫也不是絕對安全，敏感資訊最好用環境變數管理，不程序式碼倉庫。

國內訪問 GitHub 的注意事項

GitHub 在國內訪問有時不穩定，可能出現打不開、克隆慢的情況。可以嘗試：(1) 使用國內映象站（如 GitClone、ghproxy）加速克隆；(2) 配置 Hosts 或使用代理；(3) 如果主要面向國內協作，也可以考慮國產替代品 **Gitee（碼雲）**，功能類似且訪問速度快。很多團隊會同時在 GitHub 和 Gitee 上託管，兼顧國際化和國內速度。

進階玩法：用 GitHub Pages 搭建免費個人部落格

GitHub Pages 允許你免費託管一個靜態網站，配合 Jekyll、Hexo、Hugo 等靜態站點生成器，可以輕鬆搭建個人部落格。寫完 Markdown 文章 push 到倉庫，部落格自動更新，還自帶 HTTPS 和自定義域名。無數技術博主的部落格都是這麼搭的，零成本、無廣告、版本可控。

九、總結：為什麼每個人都該瞭解 GitHub

GitHub 早已不只是“程式設計師的工具”，它更像是一個**數字時代的工作方式**：版本管理讓一切可追溯，協作機制讓團隊高效運轉，開源精神讓知識自由流動。哪怕你不寫程式碼，理解“提交、分支、合併”這些概念，也能幫你更好地管理任何需要“反覆修改 + 多人協作”的內容——從論文到檔案，從筆記到方案。

對計算機系的學生來說，GitHub 更是必備技能：它是你的作品集、協作平臺、學習資源庫，也是未來求職時最有說服力的“簡歷”。早點註冊帳號，把自己的課程作業、小專案傳上去，慢慢積累，幾年後你會擁有一份讓面試官眼前一亮的技術履歷。

來動手：完成你的 GitHub 初體驗

1. 註冊一個 GitHub 帳號，完善個人資料（頭像、簡介、感興趣的技術領域）。
2. 建立一個名為 `hello-github` 的公開倉庫，勾選”Add a README file”。
3. 進入倉庫，點選 README.md 的編輯按鈕（鉛筆圖示），用 Markdown 寫一段自我

介紹，提交更改。

4. 在 GitHub 搜尋欄搜尋一個你感興趣的關鍵字（比如 `python-tutorial`），Star 兩個你喜歡的專案。

5.（進階）在電腦上安裝 Git，把倉庫 Clone 到本地，修改 README 後 push 回雲端，完成一次完整的” 本地雲端” 工作流。

第二十四章：Root 權限與 BL 鎖深度解析

這一章是對第二十章內容的深入擴充，適合那些想要真正理解手機底層機制、甚至嘗試刷機的使用者。請注意，這些操作有一定風險，請謹慎行事。

一、深入理解 Bootloader

Bootloader 是手機啟動時第一個執行的程式，它的任務是初始化硬體、載入作業系統核心。可以把 Bootloader 理解為手機的“守門人”。

BL 鎖的工作原理：

- **驗證簽名：**BL 鎖會驗證啟動映象的數字簽名，只有廠商簽名的系統才能啟動。
- **防止降級：**防止使用者刷入舊版本系統來利用已知漏洞。
- **保護資料：**即使刷入新系統，沒有原帳號密碼也無法訪問使用者資料。

硬體級儲存與驗證

BL 鎖的鎖定狀態（Locked/Unlocked）及廠商的公鑰，通常儲存在受保護的硬體區域（如高通的 devinfo 分割區、eFuse 或 RPMB 分割區），防止被軟體篡改。每次啟動時，Bootloader 會從此安全區域讀取狀態，並據此決定是否執行嚴格的簽名驗證。

數字簽名驗證

廠商在釋出系統更新或映象時，會用自己的私鑰對其進行簽名。裝置啟動時，Bootloader 會使用內建的公鑰對系統映象（如 boot.img、recovery.img）進行驗證。它的工作方式是：

1. 對系統映象重新計算雜湊值。
2. 用公鑰解密映象自帶的簽名，得到一個雜湊值。
3. 比較這兩個雜湊值，若一致則透過驗證。這確保了系統映象未被第三方篡改。

安全啟動鏈

BL 鎖是整個安全啟動鏈的第一環。它驗證透過後，才會載入並執行下一階段的載入程式（如核心），從而層層確保系統啟動環境的可信。

鎖定 vs 解鎖：BL 鎖的雙面影響

為了更直觀地理解 BL 鎖的利弊，我們用一個對比表來展示它在 5 個核心維度上的差異：

核心作用	鎖定狀態（預設）	解鎖後狀態
系統安全	防止刷入未簽名的非官方系統，保護系統完整性與底層安全 高安全：僅載入官方簽名映象，有效抵禦惡意韌體篡改	安全降低：允許刷入第三方 ROM，系統完整性校驗機制失效，惡意軟體若獲取 Root 權限可深度篡改系統
系統穩定性	確保手機執行經過廠商測試和最佳化的官方系統，保障穩定流暢 穩定：使用官方韌體，功能完整，OTA 升級可靠	風險增加：第三方 ROM 可能存在相容性問題，可能導致系統崩潰、變磚、硬體功能異常（如攝像頭、快充），OTA 更新可能失敗
保修權益	廠商透過鎖定 BL 提供穩定的官方系統，以提供保修服務 保修有效：享受官方免費保修服務（非人為損壞）	保修失效：絕大多數廠商將解鎖 BL 視為主動放棄保修的行為，即使重新上鎖，也可能被檢測出歷史解鎖記錄而拒保
資料安全	解鎖過程通常強制清除資料，防止舊資料在新系統環境下被惡意利用 資料安全：日常使用中，BL 鎖限制了惡意程式對系統分割區的修改能力	資料風險：解鎖時會強制清除所有使用者資料；若解鎖後未正確加密，資料儲存可能不安全；丟失後更易被刷機提取資料
功能限制	確保依賴安全環境的功能（如支付、銀行 App）正常執行 功能完整：所有官方功能均可正常使用	功能受限：部分安全敏感應用（如 Google Pay、銀行 App）可能因檢測到 BL 解鎖狀態而拒絕執行；DRM 服務可能降級，影響高畫質影片播放

二、解鎖 BL 鎖的詳細流程（以小米為例）

1. **申請解鎖權限：**在小米解鎖工具網站申請解鎖。

2. **繫結帳號：**在手機設定中繫結小米帳號。

3. **等待期**：根據帳號等級，需要等待 7-30 天。
4. **解鎖操作**：使用小米解鎖工具連線電腦解鎖。

三、Root 的多種方案

- **Magisk**：現代 Root 方案，Systemless 設計，可隱藏 Root 狀態。
- **Kernelsu**：基於核心的 Root 方案，更底層，隱藏性更好。
- **SuperSU**：老牌 Root 方案，目前已停止更新。

四、刷機入門

刷機是指給手機安裝新的作業系統。常見的刷機型別：

- **刷官方 ROM**：安裝官方系統，用於救磚或降級。
- **刷第三方 Recovery**：如 TWRP，用於備份恢復、刷入第三方 ROM。
- **刷第三方 ROM**：如 LineageOS、Pixel Experience 等，體驗不同的系統。

五、三星 Knox：BL 解鎖的不可逆代價

與其他安卓廠商不同，三星手機的 BL 解鎖機制有一個獨特的“熔斷”設計：**對於三星手機來說，解鎖會觸發 Knox 熔斷，永久失去部分安全功能（如 Samsung Pay）；會清除所有受保護資料；影響保修**。具體來說：

- **熔斷器是一次性的**：手機內部有一顆 eFuse 熔絲，一旦檢測到 BL 被解鎖就會永久燒斷，無法恢復。
- **受影響的功能**：Samsung Pay（公交卡/銀行卡）、Knox 安全資料夾、企業 MDM 管控、部分健康保險類 App 都會失效。
- **強制清空資料**：熔斷觸發後手機會強制恢復出廠設定，所有本地資料被擦除（雲端資料不受影響）。
- **保修影響**：官方明確表示 Knox 熔斷後不再享受保修服務。

三思而後行

三星的 Knox 熔斷是**不可逆**的，一旦觸發就回不去原廠狀態。如果你只是想用一些原本需要 Root 的功能，建議先查查有沒有不需要 Root 的替代方案。

重要警告

刷機有風險，操作需謹慎。可能導致：

- 手機變磚（無法開機）
- 失去保修
- 資料丟失
- 支付功能異常

刷機前務必做好完整備份，仔細閱讀教程，確認機型和版本匹配。

推薦資源

- **XDA Developers**：全球最大的安卓開發者社群，有豐富的刷機教程和 ROM 資源。
- **酷安**：國內知名的數碼社群，有很多中文教程和討論。
- **各機型論壇**：如小米社群、一加社群等，有官方和民間教程。

六、常見問題解答（Q& A）

Q1：解鎖 BL 鎖後還能恢復嗎？

A：可以重新上鎖。通常需要刷入官方完整的線刷包，並使用特定的刷機指令碼（如小米的 `flash_all_lock.bat`）即可重新上鎖。但注意，重新上鎖後，因解鎖而觸發的一些安全熔斷（如三星的 Knox）可能無法逆轉。

Q2：解鎖後如何 Root？

A：解鎖 BL 鎖只是第一步。Root 通常需要：

1. 刷入自定義 Recovery（如 TWRP）。
2. 透過 Recovery 刷入 Magisk 等 Root 管理器來修補 boot 映象，從而獲取 Root 權限。

第二十五章：二手手機處理與數字遺產

嘿，夥計們，經過前面二十多章的“修煉”，相信你已經從一個“數字小白”成長為一名相當從容的手機使用者了。但科技生活總有輪迴——新歡終會變舊愛。當你準備讓老夥計“退役”時，你知道該如何體面、安全地告別嗎？

一、舊手機的“安樂死”：安全出售或轉贈指南

核心原則：資料清理不是“刪除”，而是“覆蓋”。許多人以為把照片拖進回收站、清空相簿就萬事大吉，結果手機轉手後，新主人用資料恢復軟體一掃，老照片、聊天記錄、身份證照片全回來了——這種“社死現場”每年都在上演。

1. **自我” 大掃除”**：完成最後一次完整備份，退出所有帳號（iCloud、華為/小米/三星帳號、Google 帳號），關閉” 查詢我的手機” 功能——否則下一位使用者會被啟用鎖攔在門外。

2. **執行終極” 格式化”**：現代手機的” 恢復出廠設定” 通常包含加密擦除——它銷燬的是加密金鑰本身，金鑰一沒，整盤資料瞬間變為亂碼。

3. **最後的” 障眼法”（僅老裝置需要）**：現代手機（iOS 8+、Android 6+）已預設全盤加密（FBE），恢復出廠設定會銷燬加密金鑰，資料理論上不可恢復，**無需此步驟**。僅對極老舊的非加密裝置（2015 年前），才需要在恢復出廠設定後用垃圾檔案填滿儲存，再執行一次恢復出廠設定。

各品牌資料擦除路徑速查表：儘管原理相通，但每個品牌把“恢復出廠設定”藏的位置略有不同，關鍵差異如下。

品牌	擦除入口	關鍵差異與注意事項
華為	設定 > 系統和更新 > 重置 > 恢復出廠設定	需先退出華為帳號並關閉”查詢我的手機”，否則會觸發啟用鎖；純淨模式下可能需要密碼驗證。
小米 / Redmi	設定 > 我的裝置 > 恢復出廠設定	底部有”格式化虛擬 SD 卡”和”清除全部內容與設定”選項，二者都要勾選；退出小米帳號前先關閉”查詢裝置”。
OPPO / 一加	設定 > 其他設定 > 還原手機 > 徹底清除全部資料	ColorOS 提供”徹底清除”模式，會多次覆蓋儲存；需輸入鎖屏密碼確認。
vivo	設定 > 系統管理 > 備份與重置 > 清除所有資料	勾選”格式化手機儲存”才會清理 SD 卡區；需退出 vivo 帳號，關閉”查詢手機”。
三星	設定 > 通用管理 > 重置 > 恢復出廠設定	需先移除 Samsung 帳號並關閉”查詢我的手機”；部分機型有”自動覆蓋”選項，建議開啟。
iPhone	設定 > 通用 > 傳輸或還原 iPhone > 抹掉所有內容和設定	必須先退出 Apple ID，否則下一位使用者被 iCloud 啟用鎖攔住；iOS 15+ 支援”退出時保留 iCloud 資料”以備後查。

最容易被忽略的三件事

- **未退出帳號**：手機賣給下家後被遠端鎖定，自己也下不來，最後只能持身份證去官方售後解鎖。
- **SD 卡未取出**：恢復出廠設定不會格式化外接 SD 卡裡的照片，記得手動取出或單獨格式化。
- **隨身碟/OTG 外設未斷開**：擦除過程中如果連著外設，可能誤清外設資料，也可能導致擦除中斷。

二、數字遺產：你的社交靈魂如何安放？

手機裡的資料，早就不是“虛擬”的了。它承載著你的資金、社交關係、創作成果，乃至你的“數字身份”。**一個人去世後，這些資產該怎麼處理？**這不再是一個抽象問題，而是越來越多家庭真實面臨的難題——親屬不知道密碼、平臺不讓繼承、加密貨幣永遠丟失。

法律依據：個人資訊保護法第 49 條

2021 年施行的《中華人民共和國個人資訊保護法》第 49 條明確規定：**自然人死亡的，其近親屬為了自身的合法、正當利益，可以對死者相關個人資訊行使本章規定的查閱、複製、更正、刪除等權利；死者生前另有安排的除外。**
這意味著：

- **近親屬有法律依據**向平臺申請處理死者的帳號與個人資訊。
- ”合法、正當利益”通常包括辦理遺產繼承、關閉帳號、刪除敏感隱私等。
- 死者生前若有遺囑或協議另行安排（如指定遺產聯絡人），從其約定。
- 平臺不能以”使用者隱私”為由一概拒絕配合，但具體流程各家不同。

微信與支付寶：帳號不能繼承，但資金可以

這兩個中國人最常用的 App，規則各不相同：

- **微信帳號**：依據《騰訊微信軟體許可及服務協議》，帳號所有權歸騰訊，使用者僅享有使用權，**帳號本身不能繼承、不能轉讓**。但帳號內的**零錢、零錢通資金**屬於個人財產，近親屬可憑死亡證明、親屬關係證明、繼承公證書向財付通申請提現或轉移。
- **支付寶**：帳號同樣不能直接繼承，但支付寶餘額、餘額寶、理財通等**資金屬於可繼承財產**。繼承人可透過支付寶”客服熱線”或線下提交死亡證明、繼承公證書辦理；螞蟻財富類理財產品需走基金公司流程。
- **聊天記錄**：法律上屬個人隱私，平臺不會主動提供給家屬。如涉及遺產線索，可透過訴訟或公證申請調取。

千萬別做的事

- **不要把死者帳號密碼直接交給家人”接著用”**——這違反平臺協議，資金被凍結甚至觸發風控時舉證更難。
- **不要嘗試用死者手機收驗證碼”提現”**——可能被認定為盜用，反而觸犯法律。
- **不要拖延**：部分帳號長期未登入會被自動登出，資金找回會更麻煩。

Apple“遺產聯絡人”：提前指定，事半而功倍

iOS 15.2 起蘋果推出“遺產聯絡人”功能，允許使用者提前指定信任的聯絡人，去世後該聯絡人可憑**死亡證明 + 遺產聯絡人訪問金鑰**申請訪問死者的 iCloud 資料（照片、備忘錄、通訊錄、備忘錄等）。操作步驟：

1. 在 iPhone 上進入 [設定] > [你的姓名] > [登入與安全] > [遺產聯絡人]。
2. 點選”新增遺產聯絡人”，使用面容 ID 確認身份。

3. 選擇聯絡人（從通訊錄挑選），系統會生成一份**訪問金鑰**。
4. 將訪問金鑰透過簡訊、郵件或列印紙質件交給對方，並提醒對方妥善儲存。
5. 可新增多個聯絡人作為備份，避免單點失效。

被指定人無需 Apple 裝置，憑金鑰和死亡證明即可在蘋果官網申請訪問。整個流程通常在 7 天內完成。

Google“不活躍帳戶管理器”：自動處理身後事

Google 提供“不活躍帳戶管理器（Inactive Account Manager）”，能在使用者長期未登入時自動執行預設操作——這是目前最完善的數字遺產機制之一。

1. 訪問 **myaccount.google.com/inactive**，登入 Google 帳號。
2. 設定超時期限：3 / 6 / 12 / 18 個月未登入後觸發。
3. 新增受信任聯絡人（最多 10 人），並設定每位可訪問哪些資料（Gmail、相簿、雲端硬碟等）。
4. 可選：設定自動回覆郵件，告知來信者你已無法處理。
5. 最後選擇：到期後是**分享資料給聯絡人**，還是**直接刪除帳戶**。
6. 設定手機號作為兜底驗證，避免誤觸發。

國內社交帳號：紀念化與登出流程

國內主流社交平臺對逝者帳號的處理分兩種思路：**“紀念化”（保留帳號讓親友緬懷）**和**“登出”（徹底刪除）**。家屬可憑死亡證明、親屬關係證明、申請人身份證向平臺客服申請。

- **微博**：支援帳號” 紀念化”，由親屬提供死亡證明申請，帳號會被鎖定無法登入但內容保留，主頁會顯示” 該帳號已設定為紀念帳號”。也可申請登出，需提供繼承公證。
- **抖音**：透過” 我 - 設定 - 反饋與幫助 - 帳號與安全 - 帳號登出” 或人工客服，提交死亡證明、親屬關係證明、申請人身份證，稽核透過後可登出或轉為紀念狀態。
- **B 站**：客服渠道提交逝者證明及親屬證明，可申請帳號登出或保留為紀念頁；UP 主的歷史稿件處理需結合版權宣告。
- **小紅書**：透過官方客服申訴，提交死亡證明、親屬關係證明，可申請登出帳號並刪除內容。

關鍵提醒

- 各平臺規則會更新，辦理前先查閱官方客服最新指引。
- 所有證明檔案建議先掃描備份再提交，原件妥善保管。
- 跨平臺一次性辦理時，可統一申請一份**繼承公證書**，省去反覆跑手續。

數字資產的繼承難題

除了社交帳號，越來越多的人持有**真正有價值的數字資產**，其繼承問題更加複雜：

- **加密貨幣（比特幣、以太坊等）**：基於” 私鑰即所有權” 原則，**沒有私鑰就等於資產消失**。繼承人無法透過法律檔案向鏈上” 申訴”。建議：將助記詞/私鑰分片存放在多個物理安全位置（如銀行保險箱 + 家裡保險櫃），並在遺囑中註明訪問方式。
- **交易所帳戶**：和銀行帳戶類似，可憑繼承公證書向交易所（如幣安、OKX）申請處置，但跨境交易所流程漫長。
- **域名**：屬於可繼承財產，繼承人在域名註冊商（如阿里雲、GoDaddy）提交繼承公證後可辦理過戶。
- **智慧財產權（原創文章、影片、專利、軟體著作權）**：著作權中的財產權利可繼承（人身權不繼承），收益權可由繼承人享有至作者死後 50 年。
- **雲端儲存與網盤**：內容資料歸屬個人，可繼承；但部分平臺協議規定帳號本身不可轉讓，需先下載再登出。

加密貨幣“消失”的真實代價

據 Chainalysis 報告，全球已有**超過 300 萬枚比特幣**因私鑰丟失而永遠無法找回，相當於現有總量的六分之一。如果你持有加密資產，請務必讓至少一位你絕對信任的人知道**恢復路徑**——否則這些財富將永遠沉睡在區塊鏈上。

一份“數字遺囑”模板

傳統的財產遺囑管不了數字資產。建議你每年更新一份“數字遺囑”附件，與正式遺囑一併存放。以下模板供參考：

《個人數字資產處置附件》（模板）

第一部分：帳號清單（僅記錄帳號名與所屬平臺，**不寫密碼**）

- 社交類：微信（帳號 xxx）、微博、抖音、B 站、小紅書等
- 支付類：支付寶、微信支付、各銀行卡 App
- 雲端類：iCloud、華為雲、小米雲、Google 帳號、微軟帳號
- 郵箱類：QQ 郵箱、網易郵箱、Gmail 等
- 其他：網盤、訂閱服務、遊戲帳號等

第二部分：密碼託管方案

- 主密碼存放方式：使用密碼管理器（如 Bitwarden、1Password），主密碼寫在紙質件中，封存於家中保險櫃。

- 恢復金鑰/助記詞：分片存放，至少兩處物理位置（如銀行保管箱 + 家中保險櫃）。
- 授權查詢人：姓名、身份證號、與本人關係、聯絡方式。

第三部分：處置意願

- 各類帳號希望” 紀念化” 還是” 登出”。
- iCloud / Google 資料是否指定” 遺產聯絡人” 或” 不活躍帳戶管理器”。
- 加密資產由誰繼承，私鑰獲取路徑說明。
- 版權類作品收益由誰享有。

第四部分：聯絡人與平臺

- 信任執行人姓名、聯絡方式、備用聯絡人。
- 常駐平臺客服入口與所需證明材料清單。
- 本附件更新日期：_____ 年 ___ 月 ___ 日，下次複核日期：_____ 年 ___ 月 ___ 日。

關於” 密碼” 的法律提示

將密碼直接寫進遺囑存在洩露風險，且遺囑公示後所有繼承人都能看到。**更穩妥的做法是：遺囑裡只寫 “帳號清單 + 處置意願 + 密碼託管位置”，真正的密碼透過密碼管理器主密碼單獨傳給信任執行人**。這樣既符合法律要求，也降低了密碼洩露風險。

三、環保的最後一程

一部手機裡含金、銀、銅、稀土等數十種金屬，隨意丟棄既汙染土壤，也浪費資源。正確的回收渠道有三類，各有側重：

渠道	回收價	資料安全	適用場景
官方以舊換新（華為/蘋果/小米等）	中等，常疊加補貼券	最高，官方有規範流程	正打算換同品牌新機時最劃算
第三方回收平臺（愛回收、轉轉、回收寶等）	略高，可比價	較高，有資料清除報告	想直接變現、不繫結品牌
公益捐贈（如鄉村學校、愛心機構）	無經濟回報	需自行徹底擦除	裝置仍可用、想做善事
發揮餘熱（自留改造）	省下新裝置錢	原機主本人繼續用，無風險	裝置效能尚可、家裡有改造需求

- **官方回收/以舊換新**：最推薦的方式，流程規範，資料安全有保障。蘋果、華為、小米等品牌官網都有” 以舊換新” 入口，評估後可抵扣新機款。

- **正規回收平臺**：選擇信譽良好的大型回收平臺，下單後順豐上門取件，檢測後打款。務必選擇**提供”資料清除報告”的**平臺，並保留憑證。
- **公益捐贈**：可聯絡當地鄉村學校、社群公益組織，舊手機可改裝成教學平板或老人機使用。**捐贈前務必執行完整的擦除流程，並卸除所有 SIM/SD 卡**。
- **發揮餘熱**：改成車載導航儀、智慧家居中控、監控攝像頭、寶寶監視器、電子相框。一臺舊手機可以再戰好幾年。

電子垃圾的真相

一部手機的電路板裡含有**金、銀、鈀、銅**等貴金屬，每噸廢舊電路板含金量是金礦石的數十倍。但若隨意丟棄，其中的鉛、汞、鎘會滲入土壤與地下水，汙染持續數十年。**正確回收，既保護環境，也能讓稀缺金屬迴圈再生**。

實操：為你的舊手機執行一場”標準葬禮”

1. 最後一次完整備份：照片上傳雲端、重要檔案複製到電腦、聊天記錄匯出存檔。
2. 退出所有帳號：iCloud / 華為 / 小米 / Google 帳號，關閉”查詢我的手機”。
3. 取出 SIM 卡與 SD 卡，斷開所有 OTG 外設。
4. 執行加密恢復出廠設定，必要時手動勾選”格式化儲存”。
5.（僅老裝置）用垃圾檔案填滿儲存，再恢復出廠設定一次。
6. 選擇它的下一站：官方以舊換新、正規回收平臺、公益捐贈，或轉贈家人。
7. 同步更新你的”數字遺產清單”，把已淘汰的裝置從清單中劃掉。

本月任務清單

1. 為你目前的主力手機指定 Apple”遺產聯絡人”或設定 Google”不活躍帳戶管理器”。
2. 用本節模板起草一份《個人數字資產處置附件》，與家人溝通存放位置。
3. 檢查家中是否有 2 臺以上閒置舊手機，預約一次官方或正規平臺回收。

第二十六章：當下手機技術前沿

聊完了舊手機的“身後事”，讓我們把目光拉回當下。*2024-2026* 年是手機技術的一個集中爆發期：*AI* 從雲端塞進了口袋，衛星通訊從旗艦下放到了中端機，摺疊屏從“嚐鮮”變成了“常用”，電池技術也迎來了多年未見的材料級突破。這一章，我們就來盤點這些正在發生的技術前沿，幫你判斷“現在該不該換機”以及“換什麼機”。

一、AI 手機：是營銷噱頭，還是真變革？

2024 年起，幾乎所有安卓廠商都在旗艦機上打出“AI 手機”的旗號，蘋果也在 iOS 18 中上線了 Apple Intelligence。但到底什麼樣的手機才算“AI 手機”？業內大致形成了三條判斷標準：

- **端側大模型**：手機本地能跑引數量在 7B 左右的語言模型（如 Qwen、Llama、DeepSeek 的輕量版本），不依賴雲端就能完成文字摘要、翻譯、文案生成等任務。判斷方法：飛航模式下若還能用 AI 功能，則說明是端側模型。
- **NPU 算力（TOPS）**：NPU（神經網路處理單元）的算力以 TOPS（每秒萬億次操作）為單位。2024 年旗艦晶片（如驍龍 8 Gen3、天璣 9300）的 NPU 算力在 40-70 TOPS 區間，2025-2026 年的新一代晶片普遍突破 100 TOPS，這是端側跑大模型的硬體門檻。
- **AI Agent 場景**：不僅會” 問答”，還能跨 App 執行任務鏈——例如” 幫我把昨天開會的照片整理成一個相簿並分享給張三”，系統能自動呼叫相簿、檔案、微信等多個 App 完成。這是 AI 手機區別於” 帶 AI 助手的手機” 的核心標誌。

二、各廠商 AI 戰略對比：誰在認真做，誰在貼牌？

各家的 AI 路線差異明顯，選購時可以重點關注：

- **蘋果 Apple Intelligence（iOS 18+）**：強調整合系統級應用——郵件摘要、通知優先順序、寫作潤色、相簿 AI 修圖。端側模型為主，複雜任務上雲到 Private Cloud Compute。優勢是隱私保護和系統流暢度，劣勢是中文場景上線較慢，部分功能 2025 年才逐步支援。
- **谷歌 Gemini Nano**：Pixel 8/9 系列內建，支援” 畫圈搜尋”、Magic Compose、錄音摘要等。Gemini Nano 是真正意義上的端側小模型，開源生態友好，是安卓 AI 手機的參考實現。

- **華為盤古大模型**：接入鴻蒙小藝，支援端側文案生成、AI 修圖、AI 消除（去路人）、通話摘要。鴻蒙 NEXT 5.0 後進一步系統級整合，AI 能力與系統深度融合。
- **小米澎湃大模型（MiLM）**：HyperOS 內建，支援 AI 寫真、AI 摘要、AI 畫師、超解析度。小米走" 端雲協同" 路線，端側跑輕量模型，重任務上雲。
- **OPPO（安第斯大模型）與 vivo（藍心大模型）**：ColorOS 15 與 OriginOS 5 深度接入，主打 AI 消除、AI 檔案摘要、AI 識圖問答、自然語言搜照片（" 找去年在海邊拍的照片"）。

選購建議：如果你經常需要 AI 處理工作檔案、整理資訊，優先選端側算力強、AI 功能系統級整合的機型；如果只是偶爾用 AI 修圖，主流中端機已經夠用。

三、衛星通訊：從旗艦“噱頭”到中端標配

衛星通訊是這兩年最實在的技術下沉之一。2022 年華為 Mate 60 和 iPhone 14 首發時還只是旗艦專屬，2025 年已經下放到 2000-3000 元檔。

- **北斗短報文下放中端機**：北斗衛星的短報文功能（可傳送 1000 漢字以內的簡訊 + 位置）已下放到 Redmi、realme、iQOO 等中端機型，無需月租，免費使用。適合戶外徒步、自駕遊等場景。
- **天通衛星通話**：華為 Mate 60/70 系列、榮耀 Magic6 系列等支援天通一號衛星通話功能，可撥打/接聽衛星電話、傳送衛星訊息。需要開通運營商衛星套餐（約每月 10-200 元），適合經常進無人區的使用者。
- **iPhone 衛星訊息（SOS）**：iPhone 14 及以後機型支援衛星 SOS，可向緊急服務機構傳送文字訊息並共享位置。免費提供 2 年服務，部分地區續費可用。
- **實用建議**：城市使用者基本用不上，但戶外愛好者、出差到偏遠地區的商務人士強烈建議選帶衛星功能的機型——關鍵時刻能救命。

四、摺疊屏生態成熟度：能當主力機了嗎？

摺疊屏已從“科技嚐鮮”走向“主力機可用”。判斷摺疊屏是否成熟，看三點：硬體耐用性、App 適配度、互動生態。

- **硬體耐用性**：鉸鏈壽命普遍達 50 萬次以上（每天開合 100 次可用 13 年），水滴鉸鏈減少螢幕摺痕，IPX8 級防水已成標配。內屏 UTG 玻璃 + 保護膜方案讓螢幕不再" 嬌貴"。
- **App 適配**：主流 App（微信、淘寶、抖音、B 站、Office、WPS）已針對大屏做適配，但仍有約 30% 的長尾 App 未適配，顯示為拉伸或黑邊。建議購機前先試用你常用的幾個 App。
- **懸停態互動**：摺疊到 90°-120° 時，部分 App 支援" 上半屏顯示內容、下半屏操作" 的懸停模式，適合視訊通話、自拍、看片。相機懸停自拍是摺疊屏獨有的體驗。

- **分屏與多工:**大屏下可同時開兩個 App 分屏,甚至三個 App 小窗並存。HyperOS、ColorOS、OriginOS 的分屏邏輯已比較成熟，但 iOS（iPadOS 風格）的分屏在 iPhone 上仍未開放。
- **價格:** 橫折（書本式）從萬元級下探到 5000-7000 元檔，豎折（翻蓋式）已下探到 4000-5000 元檔。2025 年部分廠商推出 3000 元檔豎折，摺疊屏真正進入大眾市場。

選購建議：如果你是重度閱讀、檔案處理、影片使用者，橫折屏值得考慮；如果你追求便攜和社交屬性，豎折屏更適合。建議至少去線下店把玩 30 分鐘再決定。

五、6G 與 Wi-Fi 7：下一代連線的速度與邊界

雖然 6G 還沒真正落地，但相關技術已經在路上了：

- **6G 進展：**3GPP 計劃 2027 年完成 6G 標準制定，2030 年左右商用。6G 的理論峰值速率可達 1 Tbps（5G 的 50 倍），延遲低至 0.1ms，支援" 通感一體化"（通訊 + 感知），手機甚至能當低精度雷達用。目前各廠商還在原型驗證階段，離消費級產品尚遠。
- **Wi-Fi 7（802.11be）：**2024 年起成為旗艦機標配，理論峰值速率達 46 Gbps（Wi-Fi 6 的 4 倍），支援 320MHz 頻寬、4K-QAM、MLO（多鏈路同傳）。實測在 Wi-Fi 7 路由器下，旗艦機傳輸速率可達 2-5 Gbps，比 Wi-Fi 6E 快 2-3 倍。如果你換了 Wi-Fi 7 手機，記得配一臺 Wi-Fi 7 路由器才能發揮全部效能。
- **藍芽 5.4 與 LE Audio：**2024-2026 年新機普遍支援藍芽 5.4 及 LE Audio（低功耗音訊），可同時連兩副耳機聽同一首歌，Auracast 廣播音訊讓多人在公共場合同步聽同一音源。

六、矽碳負極電池：續航終於迎來材料級突破

手機續航多年來被詬病“一天一充”，2024 年起矽碳負極電池技術的普及讓情況有了實質改善。

- **技術原理：**傳統石墨負極的理論容量約 372 mAh/g，矽的理論容量高達 4200 mAh/g（約 10 倍），但矽充放電時體積膨脹達 300%，容易粉化。矽碳負極透過奈米矽-碳複合結構，把矽的膨脹問題控制在可接受範圍，讓電池能量密度提升 15%-30%。
- **實際表現：**2024-2025 年採用矽碳負極的旗艦機（如小米 14 Ultra、榮耀 Magic 6 至臻版、vivo X100 Pro 等），在 7-8mm 機身內塞進 5000-6000 mAh 電池，比上代多出約 800-1000 mAh，重度使用可撐一天半。
- **百瓦快充配合：**矽碳負極對快充相容性良好，100W/120W 快充 17 分鐘充滿已不是新聞。2025 年部分機型支援 150W-240W，10 分鐘內充至 80%。
- **選購建議：**如果你是重度使用者，優先選採用" 矽碳負極 + 百瓦快充" 組合的 2024 年後機型，續航焦慮會大幅緩解。

七、本章速覽：一句話記住這些前沿

- AI 手機三標準：端側大模型、NPU 算力 TOPS、AI Agent 跨 App 場景。
- 衛星通訊：北斗短報文下放中端，天通通話仍屬旗艦。
- 摺疊屏：硬體成熟、App 適配七成、懸停/分屏是獨有體驗。
- 6G 還在路上，Wi-Fi 7 已落地，藍芽 LE Audio 開始普及。
- 矽碳負極 + 百瓦快充：續航與充電的雙突破。

來試試：給你的下一臺手機做一份”技術需求清單”

拿出一張紙，按“必備/加分/不需要”三檔，列出你對 AI、衛星、摺疊屏、Wi-Fi 7、快充、電池容量的需求。這樣下次去線下店時，導購推薦的“賣點”你能迅速判斷是真需求還是被推銷。例如：

- 必備：5000mAh 以上電池、80W 以上快充、NFC、IP68 防水。
- 加分：端側 AI 摘要、Wi-Fi 7、北斗短報文。
- 不需要：摺疊屏（我習慣直板）、衛星通話（不出遠門）、240W 極限快充。

本章小結

- AI 手機三大判斷標準：端側大模型、NPU TOPS、AI Agent 場景
- 各廠商 AI 戰略：蘋果重隱私整合、谷歌端側開源、華為/小米/OPPO/vivo 各有所長
- 衛星通訊：北斗短報文下放中端機，天通通話仍屬旗艦專屬
- 摺疊屏：硬體已成熟，App 適配約 70%，懸停態與分屏是獨有體驗
- 6G 標準 2027 年制定，Wi-Fi 7 已普及，藍芽 LE Audio 開始落地
- 矽碳負極電池讓 5000-6000mAh 成為旗艦標配，續航焦慮大幅緩解

五、實戰篇（第 27-32 章）

第二十七章：爸媽也能輕鬆用——適老化手機完全指南

說句實話，寫這一章的時候，我想到了我爸媽。我媽每次手機彈出一個“系統更新”，她都不敢點，怕把手機弄壞了。我爸的微信字型比螞蟻還小，但他不知道怎麼調大，就這麼眯著眼睛看了兩年。這不是他們“笨”，是手機預設把所有人都當成了二十多歲的年輕人。這一章，我用最直白的話，把長輩們手機使用中最頭疼的問題一個個解開。

零、先把心態擺正：這不是“老人用不來”，是“手機沒做好”

在做任何設定之前，有一個觀念要先說清楚：**現在的智慧手機介面，從設計之初就沒怎麼考慮老年使用者**。彈窗小得看不清、操作手勢需要精準觸碰、圖示長得都差不多——這些不是使用者的問題，是產品的問題。所以帶長輩用手機，第一件事不是教，是幫他們**把環境調好**。環境調好了，大半的問題就消失了。

帶長輩用手機有三條鐵律：(1) **別催**——你用了十年的智慧機，他們可能才用了三個月。(2) **用類比**——“Wi-Fi 就像家裡的水管，開了水龍頭才有水”，這種比喻比專業術語好理解一萬倍。(3) **寫下來**——口頭教一遍他們可能忘了，你拿張紙把關鍵步驟寫下來貼在他手機殼背面，比什麼都管用。

一、第一步：開啟“簡易模式”——手機變清爽了

現在的手機基本都內建了一個“簡易模式”（有的叫“關懷模式”“老年模式”“長輩模式”），開啟之後，整個手機都會大變樣：

- **字型和圖示變得很大**——再也不用戴老花鏡看手機了。
- **桌面圖示大幅精簡**——只保留電話、簡訊、相機、相簿、微信等最常用的幾個 App，其他全收起來。
- **操作變簡單了**——很多花裡胡哨的功能（負一屏、智慧助手、浮窗等）預設關閉，介面主打一個清爽。
- **聲音變大了**——來電鈴聲、通知音量、通話聲音都自動調到最大。

各品牌簡易模式開啟方法（直接照著操作就行）：

品牌	模式名稱	開啟方法
iPhone	輔助使用（沒有單獨簡易模式，但可逐項調大）	設定 > 顯示與亮度 > 文字大小，拉到最大。再進” 輔助功能 > 顯示與文字大小” 開” 粗體文字”” 按鈕形狀”
華為/榮耀	簡易模式	設定 > 系統和更新 > 簡易模式 > 開啟
小米/紅米	極簡模式	設定 > 更多設定 > 極簡模式 > 開啟
OPPO/一加	簡易模式	設定 > 特色功能 > 簡易模式
vivo/iQOO	簡易桌面	設定 > 桌面、鎖屏與桌布 > 簡易桌面
三星	簡易模式	設定 > 顯示 > 簡易模式

如果你用的是其他品牌，在手機“設定”頂部的搜尋框裡直接搜“簡易”或“長輩”兩個字，一般都能找到。找不到的話，按下面的“手動調大”方案一條條設定，效果也一樣。

二、如果不開簡易模式，手動調也可以——“大字 + 大圖示”五步走

有些長輩不想用簡易模式，覺得介面太“老氣”。那就手動調以下五項，效果一樣好：

1. **字型調到最大：**設定 > 顯示 > 字型大小，拖到最右邊。順便把” 顯示大小”（有的叫” 顯示縮放”）也調到最大，這樣所有介面元素都跟著變大。
2. **圖示變大：**在主螢幕雙指捏合（像縮小照片那樣）進入桌面設定 > 桌面佈局（或圖示大小），選格子最少的佈局（比如 3×5），圖示自然就大了。
3. **開啟” 高對比度文字”：**讓文字邊緣更銳利，在” 輔助功能” 裡可以找到這個開關。
4. **關閉不必要的通知：**設定 > 通知，把除了電話、簡訊、微信之外的 App 通知全關掉。這樣手機不會一會兒響一下，不會讓人覺得煩。
5. **把常用聯絡人放到桌面：**長按桌面空白處 > 新增小工具 > 聯絡人，把老伴、兒女、醫生、緊急聯絡人的快捷撥號放到桌面上。點一下就撥出去，不用翻通訊錄。

動手：幫長輩” 裝修” 一下手機

1. 開啟簡易模式（按上面的品牌表格操作）。
2. 把電量數字顯示開啟（設定 > 電池 > 電量百分比），讓他們隨時知道還剩多少電。
3. 調出通知欄，把藍芽、NFC、飛航模式這些長輩用不著的快捷開關移除或藏起來。
4. 在桌面上放四個快捷撥號：老伴、兒女、家庭醫生（如果有）、社群/物業。

三、防詐騙——這章請一定帶著長輩一起看

老年人是電信詐騙的重災區。不是他們警惕性低，而是騙子的話術經過了精心設計，專門針對老年人的心理弱點。以下是我整理的最常見騙局型別和應對方法，建議你直接念給長輩聽，或者列印出來貼在電話旁邊。

六大常見騙局（看到了立刻掛電話）：

- **(1) ” 你孫子出事了，趕緊匯錢”**——騙子會說” 你孫子/孫女在外面闖禍了，被警察抓了，需要保證金”。記住：**任何要求轉帳才能” 放人” 的電話都是詐騙**。公檢法不會電話要錢。
- **(2) ” 你的醫保卡被停用了”**——簡訊裡說醫保卡異常，讓你點連結” 啟用”。記住：**醫保局、社保局不會發簡訊讓你點連結**。遇到這種情況直接打 12333（社保官方熱線）核實。
- **(3) ” 高收益理財，穩賺不賠”**——微信群、簡訊裡推薦各種” 養老理財”” 國家扶持專案”，承諾高額回報。記住：**承諾年化收益超過 5% 還” 保本” 的，99.9% 是騙局**。
- **(4) ” 我是你領導/老朋友，換號碼了”**——冒充熟人借錢。記住：**打電話核實比什麼都管用**，說” 不方便接電話” 的八成是騙子。
- **(5) ” 免費領雞蛋/保健品講座”**——線下免費活動誘導老人聽” 健康講座”，然後賣天價保健品。記住：**天上不掉餡餅，免費活動背後都是生意**。
- **(6) ” 你的手機中毒了，需要遠端修復”**——騙子讓你下載一個遠端控制 App（如向日葵、TeamViewer），然後遠端操控你的手機轉走錢。記住：**絕對不要讓任何人遠端操控你的手機！**手機廠商、通訊運營商、銀行都不會給你打電話讓你裝遠端控制軟體。

三道防線，立好之後能擋掉 90% 的詐騙：

1. **第一道：手機自帶攔截**——安卓手機開啟” 騷擾攔截”（在撥號 App 或手機管家 App 裡），iPhone 開啟” 靜音未知來電”（設定 > 電話 > 靜音未知來電）。
2. **第二道：運營商攔截**——傳送簡訊”KTFSR” 到 10086，免費開通移動的高頻騷擾電話攔截；電信聯通也有類似服務。打運營商客服電話讓人工幫你開通就行。
3. **第三道：國家反詐中心 App**——在應用商店下載” 國家反詐中心”App，註冊並開啟來電預警。這個 App 和公安的資料庫聯動，騙子號碼一撥過來就能識別。

給長輩的” 防騙口訣”（列印出來貼電話旁邊）

- **陌生電話要轉帳掛掉！**
- **簡訊連結要你點刪掉！**
- **說高收益保本的拉黑！**
- **讓你裝遠端控制軟體直接掛！**
- **真警察不會電話辦案更不會要錢！**

- **拿不準的事，先給兒女打個電話問問。**

四、遠端協助——你在外地也能幫爸媽修手機

有時候長輩遇到的問題，你在電話裡描述半天他們也找不到對應的按鈕。這時候“遠端協助”功能就派上用場了——你可以直接在自己的手機上看到他們手機的螢幕，幫他們操作。各品牌的方案不一樣：

- **華為/榮耀**：自帶”暢連”App，裡面有”螢幕共享”功能。你和爸媽都裝暢連，發起視訊通話後點選”共享螢幕”，你就可以在他們螢幕上畫圈標註，告訴你媽媽”點這裡”。
- **小米/紅米**：小米視訊通話 App 裡有”遠端協助”，支援螢幕共享 + 語音通話同步。
- **OPPO/vivo**：內建”家庭空間”或”遠端守護”，在設定裡搜尋”遠端”就能找到。
- **通用方案（所有品牌都可用）**：下載”向日葵遠端控制”App（免費版夠用），你們各裝一個。註冊同一個帳號或加好友，讓爸媽把手機上顯示的驗證碼念給你，你就能遠端看到和操作他們的螢幕了。
- **iPhone 使用者**：FaceTime 通話中支援”共享螢幕”。在 FaceTime 通話時，點選螢幕共享按鈕，對方就能看到你的螢幕。但 iOS 安全限制無法遠端操作，只能”指揮式教學”。

五、微信的“適老改造”——跟長輩說這幾句就夠了

微信是長輩們用得最多的 App，但介面太密太雜。以下是幾個立竿見影的調整：

- **調大微信自己的字型**：微信 > 我 > 設定 > 通用 > 字型大小，拖到最大。注意，這個隻影響微信內部的字型，不影響手機系統字型。
- **開啟”關懷模式”**：微信 > 我 > 設定 > 關懷模式 > 開啟。開啟後字型和按鈕都會變大，色彩對比度加強，還能一鍵”聽文字訊息”（點一下文字訊息就能朗讀出來）。
- **幫他們備註好重要聯絡人**：把兒女、親戚的名字在通訊錄裡改成容易認的——比如”大兒子-小明””二女兒-小紅”，讓他們一開啟微信就知道誰是誰。
- **教他們用”語音輸入”而不是打字**：聊天框旁邊的鍵盤上，按住空格鍵（安卓）或點麥克風圖示（iPhone），直接說話就能變成文字傳送。識別準確率已經非常高了，對不會拼音打字的老人來說是”質變級”的便利。
- **關閉”發現頁”裡不用的功能**：微信 > 我 > 設定 > 通用 > 發現頁管理，把”朋友圈””影片號””直播””購物””遊戲”裡不用的全關掉，減少幹擾。

六、手機丟了怎麼辦？幫長輩做個“應急預案”

預則立，不預則廢。趁手機還沒丟，先把這幾件事做好：

1. **確認” 查詢我的手機” 已開啟**：iPhone 在設定 > 你的名字 > 查詢 > 查詢我的 iPhone；安卓在設定搜” 查詢裝置”。務必確認這個開關是亮著的。
2. **把重要的密碼和帳號記在一張紙上**：Apple ID / 華為帳號 / 小米帳號的密碼，鎖在家裡抽屜裡一份，給信得過的兒女一份。丟了手機之後要登入這些帳號來遠端鎖定和定位，如果忘了密碼就麻煩了。
3. **教長輩一句話：” 手機丟了，第一時間給家人打電話。”** 家人可以立刻登入 iCloud 或安卓的查詢裝置網頁，遠端標記為” 丟失模式”，這樣撿到的人也打不開、用不了。

七、無線耳機和助聽器相容——被忽視的好功能

很多長輩有聽力問題，但不知道手機可以跟助聽器協同工作。iPhone 和部分安卓手機支援 MFi（Made for iPhone）助聽器，可以直接把電話聲音傳到助聽器裡，比舉著手機聽筒清楚太多了。

- **連線方法**：iPhone 設定 > 輔助功能 > 助聽裝置，開啟藍芽，按照助聽器的配對說明操作。
- **如果不用助聽器**：在” 輔助功能 > 音訊/視覺” 裡開啟” 電話降噪” 和” 單聲道音訊”，能改善通話清晰度。
- **安卓**：在” 輔助功能 > 聽力增強” 裡有類似設定，各品牌位置略有差異，直接搜” 助聽” 兩個字。

八、健康相關功能——手機不只是通訊工具

- **SOS 緊急求助**：連續按電源鍵五次（大多數手機），手機會自動撥打緊急電話並給預設的緊急聯絡人傳送定位。幫長輩提前設定好——iPhone：” 設定 > 緊急 SOS”，安卓：” 設定 > 安全 > 緊急求助”。教長輩用之前先演習一遍，免得到時候緊張按不準。
- **跌倒檢測**：Apple Watch 有這個功能，檢測到你摔倒且一段時間不動，會自動撥打急救電話並通知家人。有預算的話給長輩配一個，關鍵時刻能救命。
- **用藥提醒**：iPhone 的” 健康”App 內建用藥提醒功能，設好藥名、時間和劑量，到點就彈出提醒。安卓可以用” 定時鬧鐘” 或下載” 吃藥提醒” 類 App。

本章小結

- 先開簡易模式或手動調大字型/圖示，環境調好大半問題就沒了
- 帶長輩用手機三鐵律：別催、用類比、寫下來

- 六大常見騙局要反覆提醒，三道防線立好能擋掉 90% 詐騙
- 遠端協助讓你人在外地也能幫爸媽修手機
- 微信開” 關懷模式”+” 語音輸入” 是長輩最實用的兩個功能
- SOS 緊急求助和跌倒檢測，關鍵時刻救命的配置值得花五分鐘設好

第二十八章：花對錢買到心儀機——手機選購終極指南

買手機這件事，很多人會提前看一週評測、比一個月價格，結果最後還是稀裡糊塗花五六千買了臺自己其實用不上的旗艦。這章不列引數列——那些你在官網都能看到。我想做的，是給你一套“選機思維”，以後不管市場上出了什麼新手機，你都能自己判斷它適不適合你。

零、三步選機法：需求預算機型

很多人在網上問“三千塊買什麼手機好”，這個問題其實沒法回答——因為不知道你買手機主要用來幹什麼。買手機的正確思考順序應該是：

選機三板斧

1. **先想清楚你最在乎什麼：**拍照？打遊戲？續航？螢幕？系統流暢？還是只需要能打電話發微信就行？把需求排個序。
2. **再定預算上限：**不是”大概多少錢”，是”最多能接受花多少錢”。定一個硬上限，選機的時候不會越看越貴。
3. **最後才挑機型：**拿著你的需求排序和預算，去匹配機型。

一、核心引數怎麼看——不用全懂，看懂這五項就夠了

引數列動輒幾十項資料，其實普通使用者只需要關注這五個維度：

維度	怎麼看	一句話總結
處理器（SoC）	決定了手機能流暢用幾年。2026 年的標準：驍龍 8 Elite Gen5/天璣 9500 以上 = 頂級；驍龍 8s Gen4/天璣 8400= 中高階；驍龍 7+ Gen4/天璣 7350= 夠用三年。低於驍龍 6 系的建議慎重。	買新不買舊，SoC 決定了你手機能用幾年不卡
螢幕	OLED 比 LCD 色彩好、更省電；重新整理率 120Hz 比 60Hz 滑屏流暢很多；解析度至少 1080P 以上。高頻 PWM 調光（1920Hz 以上）對眼睛更友好。	OLED+120Hz+ 高頻調光 = 好螢幕的及格線
攝像頭	**不看畫素數看感測器尺寸**。底大一級壓死人，感測器尺寸比畫素重要得多。超廣角實用；長焦（3x 以上光學變焦）拍遠處的東西；微距鏡頭多半是湊數的。	大底主攝 + 實用長焦 > 多顆湊數鏡頭
電池和充電	4500mAh 是基礎及格線，5000mAh 以上一天一充很穩。快充看瓦數：60W 以上算快，100W 以上飛快。無線充電方便但慢。	5000mAh+60W 快充 = 不焦慮組合
系統更新承諾	決定了手機能安全用多久。蘋果一般給 5-6 年系統更新；三星旗艦給 4 年系統 +5 年安全更新；谷歌 Pixel 給 7 年；國產中端機更新承諾普遍偏短（2-3 年）。	想用久一點，選系統更新承諾長的品牌

二、按預算分檔推薦（以 2026 年中市場為基準）

以下推薦基於 2026 年中國內地市場行情，價格為官方起售價。具體價格會有浮動，以你入手時的實際價格為準。

1000 元以下：能打電話，能刷微信，夠用就行

適合誰：學生、備用機、對手機要求不高、主要用於打電話和微信。

能買到什麼：Redmi Note 系列、榮耀 X 系列、realme 數字系列的中低配。處理器大概驍

龍 6 系或天璣 6000 系列，螢幕是 LCD 為主，拍照能拍清楚但對焦慢，系統流暢度一般。

選購建議：這個價位看兩點：電池夠不夠大（選 5000mAh 以上的），記憶體夠不夠（選 6GB+128GB 起步，別買 4GB 的會卡）。紅米在這個價位優勢最明顯。

1000-2000 元：價效比最高，能用三四年不卡的甜點區

適合誰：學生、預算有限但想用得久、對拍照和遊戲沒有極致追求。

代表性機型：Redmi Turbo 系列、iQOO Z 系列、榮耀 X 系列高配、一加 Ace 系列入門。

選購建議：選搭載驍龍 7+ Gen3 或天璣 7300 以上處理器的機型，配 8GB+256GB 記憶體。這個配置日常使用 + 偶爾打王者榮耀完全夠用，正常能用 3-4 年不卡。

2000-3500 元：中堅力量，效能和拍照都有明顯提升

適合誰：想要不錯的拍照、打遊戲不卡、希望用得久的普通使用者。這是最多人買的價格區間。

代表性機型：小米數字系列（如小米 17 標準版降價後）、vivo S 系列高配、OPPO Reno 系列、榮耀數字系列、Redmi K 系列。

選購建議：這個價位最大的特點是“水桶機”——沒有明顯短板。比起 2000 元以下的機子，主要提升在：相機明顯變好（有光學防抖了）、螢幕上了高素質 OLED、快充更快（80W 以上常見）、做工質感也更精緻。選 12GB+256GB 或 12GB+512GB，用四五年沒問題。

3500-5000 元：準旗艦，除了極客玩家其他人完全夠用

適合誰：對拍照有追求的使用者、想體驗頂級質感的使用者。

代表性機型：小米數字系列標準版、vivo X 系列、OPPO Find X 系列標準版、iPhone 老款旗艦（如 iPhone 16）。

選購建議：這個價位已經能買到去年的旗艦機型。我的建議是：**買去年的旗艦，不買今年的中端**。去年旗艦的用料、做工、攝像頭硬體通常比今年的中端好一個檔次。前提是你能接受它不是“最新款”。

5000 元以上：旗艦區，不差錢或對某項體驗有極致追求

適合誰：攝影發燒友、重度遊戲玩家、追求頂級螢幕和工藝的使用者、用手機拍影片做內容的創作者。

代表性機型：iPhone 17 Pro/Pro Max、三星 S26 Ultra、華為 Pura 80 Ultra、小米 17 Ultra、vivo X300 Pro。

選購建議：這個價位各家的差距越來越小，其實主要看你喜歡什麼系統。喜歡拍照選 vivo 華為，喜歡拍影片選 iPhone，喜歡螢幕和 SPen 選三星，喜歡均衡選小米。說實話到了這個價位，**你可以任性，選你看著順眼的那臺就行**。

三、按場景選機——你的需求是什麼？

以下是幾個最常見的“使用畫像”，對號入座就行：

使用場景	重點看什麼	推薦方向
拍照達人	大底主攝（1 英寸最佳）、光學防抖、長焦（3x 以上光學變焦）、影像晶片	vivo X 系列、華為 Pura/Mate 系列、小米 Ultra 系列
遊戲玩家	頂級處理器、高觸控取樣率、散熱系統、大電池 + 快充	iQOO 數字系列、ROG Phone、紅魔、Redmi K 系列
續航優先	5500mAh 以上電池、能效好的處理器、LTPO 螢幕	榮耀 Magic 系列、一加、Redmi Note 大電池版
長輩使用	簡易模式、大字型、大音量、大電池、售後網點多	華為暢享、榮耀 Play、OPPO A 系列、Redmi Note
學生用機	價效比高、耐用、電池大、有家長控制功能	Redmi Note/Turbo、realme GT、iQOO Neo
蘋果生態使用者	已有 Mac/iPad/AirPods，買 iPhone 是唯一解	iPhone 17 標準版（價效比）或 Pro 版（滿血體驗）
出國旅行	eSIM 支援、全球頻段覆蓋完整、Google 服務	iPhone、三星 S 系列、Pixel、一加

四、五個容易踩的坑——買之前對照看一眼

1. **坑一：四攝五攝不等於拍照好**——廠商喜歡在引數上堆鏡頭數量，裡面經常有兩顆是 200 萬畫素的” 湊數鏡頭”（微距 + 景深），實際完全沒用。看手機拍照能力，**只看主攝 + 超廣角 + 長焦**這三顆就夠。其他多出來的可以當它不存在。

2. **坑二：不要被 1 億畫素唬住**——畫素高不代表畫質好。感測器尺寸、光圈、演演算法調教比畫素重要得多。很多旗艦機用 5000 萬畫素的大底感測器，拍出來的照片比某些 1 億畫素的千元機好太多。

3. **坑三：別買運營商定製機**——” 存電電話費送手機” 看起來很劃算，但運營商定製機系統更新慢、可能有刪除不了的預裝 App、而且通常是運營商鎖定的（只能用這家運營商的卡）。看似省了錢，實則” 鎖定” 了你。

4. **坑四：” 曲面屏很高階”——是營銷，不是真理**——曲面屏曾經是旗艦機的標緻，但現在越來越多人發現它的問題：貼膜難、邊緣誤觸、碎了維修貴。2026 年的趨勢是迴歸” 微曲” 甚至” 純直屏”。不要因為” 曲面的就是高階的” 而選曲面屏。

5. **坑五：不是所有”AI 手機” 都真的有用**——2026 年所有廠商都在喊”AI 手機”，但很多隻是把已有的語音助手換了個名字。真正有用的 AI 功能是：實時翻譯、AI 消除（拍照把人去掉）、AI 修圖、端側大模型推理、本地化的智慧摘要。其他的花哨功能基本都是噱頭。

五、什麼時候買最劃算？

- **618（5 月底到 6 月中旬）**：一年中手機降價幅度最大的時段。很多手機廠商為了沖銷量，旗艦機能降價 800-1500 元。是上半年買手機的最佳視窗。
- **雙 11（10 月底到 11 月中旬）**：下半年的大促，力度和 618 差不多。這個時間段很多品牌新一代旗艦剛釋出，上一代旗艦降價幅度最大。
- **新機發布後一個月**：剛釋出時原價，一個月後渠道價（第三方店鋪）一般會便宜 200-500。
- **春節前兩週到元宵節**：線下實體店清庫存的”年貨節”，可以和店員談價。
- **不推薦的時間**：新品釋出會後當天衝首發——除非你真的特別想要當第一批使用者，否則首發價是最貴的。

六、渠道怎麼選——在哪買靠譜？

- **品牌官方商城/天貓旗艦店/京東自營**：最保險，價格沒優勢但不用擔心翻新和假貨，售後最好。
- **拼多多”百億補貼”**：價格最低，但需要選標註了”百億補貼”的店鋪。沒標註的不要碰。收到貨記住錄開箱影片，有問題能退貨。
- **線下實體店**：可以親手摸、試手感。注意：導購會推利潤高的機型（而不是最適合你的），建議進店之前先在網上鎖定 2-3 款目標機型，去了就說”我只想看這幾款”。
- **二手平臺（閒魚/轉轉）**：便宜但有風險。如果要買二手，記住三條：要求提供購買發票或訂單截圖、要求提供 IMEI 號查驗、**只走平臺交易，絕不私下轉帳**。

動手：用這套方法選一臺手機

1. 拿一張紙，寫下你的兩個核心需求（比如：續航第一、拍照第二）。
2. 定一個預算上限（比如 3500 元）。
3. 開啟京東或天貓，按價格篩選，圈出 3-5 款備選機型。
4. 去 B 站搜”XX 機型一個月使用體驗”（不是第一天評測，是用了至少一個月之後的真實感受），看兩三個影片。
5. 確認渠道靠譜後下單，收貨時錄開箱影片。

本章小結

- 選機三板斧：先想需求再定預算最後挑機型
- 核心引數只看五項：處理器、螢幕、攝像頭、電池充電、系統更新承諾
- 1500-3500 元是價效比最高、最推薦的甜點區間

- 買去年的旗艦比買今年的中端更劃算
- 五大坑：多攝騙局、高畫素陷阱、運營商定製、曲面屏迷信、AI 噱頭
- 618 和雙 11 是一年中最適合買手機的兩個時間視窗

第二十九章：數字健康——和手機建立健康關係

你有沒有這樣的體驗：本來只想刷 5 分鐘短影片，結果一抬頭已經過去了兩小時？或者晚上躺在床上刷手機，越刷越精神，最後凌晨兩點才睡著？手機改變了我們的生活，但也在悄悄吞噬我們的時間和健康。這一章不是讓你“戒掉手機”，而是幫你重新奪回對手機的主動權。

一、先看看你的“螢幕時間”——數字讓人清醒

在改變之前，先搞清楚現狀。手機裡有一個很多人從未開啟過的功能，叫“螢幕使用時間”（iPhone）或“數字健康”（安卓），它能告訴你每天到底在手機上花了多少時間，以及這些時間花在了哪些 App 上。

- **iPhone**：設定 > 螢幕使用時間 > 檢視所有活動。你會看到每天的解鎖次數、拿起手機的次數、每個 App 的使用時長。
- **安卓**：設定 > 數字健康與家長控制。華為叫” 健康使用手機”，小米叫” 螢幕時間管理”。功能類似，都能看到每日使用資料。

第一次看這個資料的人，80% 會被嚇到。我有個朋友看完之後發現自己每天在抖音上花了 4 個多小時，一週就是 28 小時——相當於一份兼職的時間。這種“數字衝擊”是改變的第一步。

二、給 App 設定“使用限額”——給自己裝個限速器

知道問題在哪之後，就可以有針對性地設定限制。不需要一刀切把 App 刪了，只需要給它設個“每日額度”：

1. **iPhone**：設定 > 螢幕使用時間 > App 限額 > 新增限額。選中你要限制的 App（比如抖音），設定每天 30 分鐘。到時間後 App 圖示會變灰，開啟需要額外輸入密碼。
2. **安卓**：設定 > 數字健康 > 應用計時器。選中 App 後設定每日使用上限，到時間後 App 圖示會變灰。
3. **小技巧**：把限額密碼設成一個你不常用的、不太好記的密碼。太容易輸的話，你會忍不住每次都” 續命”，限額就形同虛設了。

三、“勿擾模式”的正確用法——不是關掉通知，是關掉焦慮

很多人的手機一天響幾十次通知，每響一次注意力就被打斷一次。你以為你只是“看一眼通知”，實際上每次打斷後重新集中注意力需要 15-20 分鐘。勿擾模式不是讓你“失聯”，而是幫你重新掌控注意力。

- **定時開啟：**設定勿擾模式在晚上 10 點到早上 7 點自動開啟。這樣睡覺時手機不會因為一條垃圾簡訊把你吵醒。
- **允許重要聯絡穿透：**在勿擾模式設定裡，把” 允許來電” 設為” 收藏聯絡人” 或” 重複來電”（3 分鐘內第二次來電會正常響鈴）。這樣你不會錯過家人的緊急電話，但也不會被推銷電話打擾。
- **工作專注模式：**iPhone 的” 專注模式” 可以設自定義場景——比如” 工作” 模式下只允許工作相關 App 通知，” 閱讀” 模式下所有通知全靜音。

四、護眼不靠 App 靠習慣——20-20-20 法則

長時間看手機螢幕，眼睛乾澀、酸脹是必然的。護眼模式（藍光過濾）有一定幫助，但真正有效的是改變用眼習慣：

20-20-20 法則

每看螢幕 **20 分鐘**，把視線移到 **20 英尺（約 6 米）**以外的地方，保持 **20 秒**。這個簡單的動作能有效緩解睫狀肌疲勞。

- **開啟護眼模式：**設定 > 顯示 > 護眼模式（或夜覽模式），開啟後螢幕色溫變暖，減少藍光。建議設為日落到日出自動開啟。
- **調整螢幕亮度：**環境暗時螢幕也要暗。很多人在黑暗中看手機亮度調太高，這是傷眼最嚴重的行為。開啟” 自動亮度” 並確保它正常工作。
- **保持距離：**手機距離眼睛至少 30 釐米。很多人喜歡把手機拿得特別近看，這樣睫狀肌一直在緊張收縮狀態，近視加深很快。

五、“低頭族”的頸椎自救指南

低頭看手機時，你的脖子承受的壓力是正常站立時的 5-6 倍。長期低頭會導致頸椎曲度變直、骨質增生、頭暈手麻。以下是幾個簡單的改善方法：

- **把手機舉高：**看手機時儘量把手機舉到與眼睛平齊的高度，而不是低頭看放在腿上的手機。雖然手臂會累，但脖子會感謝你。
- **每 30 分鐘活動一下脖子：**慢慢向左轉頭到極限，停 3 秒，回正；再向右轉，停 3 秒。重複 5 次。然後做” 米字操”——用下巴在空中寫一個” 米” 字。

- **靠牆站立 3 分鐘**：後腦勺、肩胛骨、臀部、腳後跟四點貼牆，每天站 3 分鐘，能有效糾正含胸低頭的體態。

六、睡眠與手機——臥室裡最該改的習慣

手機螢幕的藍光會抑制褪黑素分泌，讓你“越看越精神”。很多人覺得“我刷著刷著就困了”，實際上是被藍光欺騙了大腦，等你放下手機反而不容易入睡。以下是改善建議：

- **睡前 1 小時不看手機**：這是最有效也最難做到的一條。如果做不到，至少開” 夜覽/護眼模式” 把藍光降到最低。
- **手機放在伸手夠不到的地方**：把充電器放在臥室另一頭，而不是床頭櫃。這樣你既不會忍不住拿起來刷，鬧鐘響了你也得起床才能關掉——一舉兩得。
- **開啟” 睡眠模式”**：iPhone 的” 睡眠” 專注模式可以自動在睡眠時段鎖定螢幕、簡化鎖屏；安卓各品牌也有類似功能。
- **用鬧鐘代替手機**：買一個十幾塊錢的實體鬧鐘，不用手機當鬧鐘，你會發現少了” 看一眼時間然後忍不住刷一下” 的陷阱。

七、資訊焦慮症——學會“選擇性失明”

我們每天接收的資訊量，是古人一輩子接收的量。但人的大腦不是為處理這麼多資訊設計的，資訊過載會導致焦慮、注意力渙散、決策疲勞。解決方法不是“獲取更多資訊”，而是“主動遮蔽”：

- **取消關注**：開啟你關注的所有公眾號、UP 主、博主列表，問自己一個問題：” 過去一個月我從中獲得過實際價值嗎？” 如果沒有，取消關注。資訊渠道貴精不貴多。
- **關閉非必要 App 推送**：除了微信、電話、簡訊、日曆之外，其他 App 的通知全關掉。新聞不需要推送——你主動去看的時候自然會看到。
- **設立” 資訊齋戒日”**：每週選一天（比如週日），全天不看手機新聞、不刷短影片。你會發現天不會塌下來，而且週一精神狀態明顯更好。

動手：一週數字健康挑戰

1. 今天：開啟螢幕使用時間/數字健康，檢視你上週的使用資料。
2. 明天：給耗時最長的娛樂 App 設定每日限額（建議 30-60 分鐘）。
3. 後天：設定勿擾模式定時開啟（22:00-07:00），允許收藏聯絡人來電。
4. 第四天：睡前 1 小時不看手機，手機充電器放到臥室另一頭。
5. 第五天：取消關注 5 個你不再看的公眾號或博主。
6. 第六天：嘗試一整天的” 資訊齋戒”——不看新聞不刷短影片。

7. 第七天：檢視本週的螢幕使用時間報告，和上週對比。

本章小結

- 先看螢幕使用時間資料，認清現狀是改變的第一步
- 給娛樂 App 設限額，給勿擾模式設定時，重新掌控注意力
- 20-20-20 法則是最簡單有效的護眼習慣
- 低頭看手機時脖子承重 5-6 倍，舉高手機 + 定期活動脖子
- 睡前 1 小時不看手機，充電器放到伸手夠不到的地方
- 資訊焦慮的解藥是” 主動遮蔽”，不是” 獲取更多”

第三十章：投屏、演示與車載互聯

手機螢幕再大也就 *7* 英寸，但你的電視有 *55* 英寸、你的投影儀有 *100* 英寸。把手機畫面“扔”到大螢幕上，體驗是完全不同維度的。這一章講清楚投屏、演示和車載互聯這三個高頻場景，幫你把手機的能力延伸到更大的螢幕上。

一、無線投屏三大陣營——別搞混了

無線投屏看起來都是“手機畫面到電視上”，但底層技術有三種，互不通用：

技術	誰用	特點
AirPlay	iPhone/iPad/Mac	蘋果生態，體驗最流暢。需要 Apple TV 或支援 AirPlay 的電視
Miracast	安卓手機	Wi-Fi Direct 直連，不需要路由器。安卓手機”無線投屏”預設用這個
DLNA	所有平臺	只推送媒體檔案（影片/音樂/圖片），不映象螢幕。適合推影片到電視上播放

各平臺操作方法：

- **iPhone**：控制中心 > 螢幕映象 > 選擇你的電視/Apple TV。如果電視不支援 AirPlay，可以買一個 Apple TV 盒子或支援 AirPlay 的電視棒。
- **安卓**：下拉通知欄 > 找到”無線投屏”/”智慧投屏”/”多屏互動”按鈕 > 選擇電視。需要手機和電視連同一個 Wi-Fi。
- **萬能方案**：如果電視太老不支援任何無線投屏，買一個 Chromecast 或小米盒子/華為盒子插到電視 HDMI 口上，就能接收投屏了。

二、有線投屏——更穩定、零延遲

無線投屏方便但有延遲，玩遊戲和做演示時體驗不好。有線投屏能解決這個問題：

- **iPhone**：需要一根 Lightning/USB-C 轉 HDMI 的轉換線（約 100-200 元）。插上後手機畫面直接輸出到電視/投影儀的 HDMI 口。
- **安卓**：部分手機支援 USB-C 轉 HDMI 直連（需要手機支援 DP Alt Mode，大部分旗艦都支援）。買一根 USB-C to HDMI 線即可。
- **會議室場景**：如果你經常需要用手機做演示，建議在包裡常備一根轉 HDMI 線。會議室的電視/投影儀幾乎都有 HDMI 口，插上就能用，比折騰無線投屏靠譜多了。

三、手機變電腦——桌面模式

部分安卓手機支援“桌面模式”——連上顯示器後，手機會輸出一個類似電腦桌面的介面，可以用滑鼠鍵盤操作：

- **Samsung DeX**：三星旗艦的招牌功能。連線顯示器後彈出桌面介面，可以多視窗操作，體驗最接近真正的電腦。配合藍芽鍵鼠，出差可以不帶筆記本。
- **華為桌面模式**：華為 Mate 系列/Pura 系列連線顯示器後也有類似功能。支援多視窗、檔案管理、辦公 App。
- **摩托羅拉 Ready For**：Moto 旗艦也支援，功能類似 DeX。
- **適用場景**：輕辦公（回郵件、改檔案）、演示 PPT、大屏看影片。不適合重度 Excel 操作或專業設計。

四、車載互聯——開車用手機的正確方式

開車時操作手機是違法的，但導航、聽音樂、接電話又是剛需。車載互聯絡統讓你在車機螢幕上使用手機的核心功能，手機可以放在一邊不碰：

- **Apple CarPlay**：iPhone 使用者的首選。透過 USB 或無線連線車機，車機螢幕上出現 iPhone 的精簡介面——導航（高德/百度）、音樂、電話、資訊、播客。Siri 語音控制全程免手。
- **Android Auto**：安卓使用者對應方案。功能與 CarPlay 類似，支援 Google Maps、Spotify 等。國內因 Google 服務限制，部分功能不可用。
- **華為 HiCar**：華為自研方案，國內車型支援率高。除了導航音樂，還能用華為視訊通話、泊車影像融合等特色功能。
- **百度 CarLife**：跨平臺方案，iOS 和安卓都能用。功能較少但相容性廣，很多老車型只支援 CarLife。

安全提醒

無論哪種車載互聯方案，開車時都應儘量使用語音控制。導航提前設好目的地，音樂提前選好歌單，不要在行駛中操作車機螢幕。

五、會議室投屏實戰——不再手忙腳亂

開會是投屏最高頻的場景，也是最容易出問題的場景。以下是幾條實戰經驗：

- **提前到場測試**：永遠不要等到會議開始才連投屏。提前 10 分鐘到場，測試好連線方式。Wi-Fi 投屏失敗的機率比你想的高得多。
- **準備有線方案做 Plan B**：在包裡放一根轉 HDMI 線。無線投屏連不上的時候，有線就是救命稻草。
- **關閉通知預覽**：投屏時你的微信訊息會顯示在會議室大屏上。投屏前在設定裡關閉”鎖屏通知內容預覽”，或者開啟”演示模式”（部分手機支援，投屏時自動隱藏通知）。
- **用”演示檔案”而不是”操作手機”**：如果你要展示 PPT，直接把 PPT 存到手機裡用 WPS 開啟全屏播放，而不是投屏整個手機桌面。這樣更專業、更流暢。

動手：完成一次手機到電視的投屏

1. 確保手機和電視連同一個 Wi-Fi。
2. 用手機開啟一張照片或一個影片。
3. 嘗試無線投屏（iPhone 用螢幕映象，安卓用無線投屏按鈕）。
4. 如果失敗，檢查電視是否支援對應協議，或嘗試 DLNA 推送（在相簿/影片 App 裡點”投屏”按鈕）。
5. 投屏成功後，試試在手機上做其他操作，看看大屏是否同步顯示。

本章小結

- 無線投屏三陣營：AirPlay（蘋果）、Miracast（安卓）、DLNA（跨平臺推媒體）
- 有線投屏更穩定零延遲，會議室場景必備轉 HDMI 線
- Samsung DeX/華為桌面模式讓手機變身輕辦公電腦
- 車載互聯：CarPlay/Android Auto/HiCar/CarLife 四大方案
- 會議室投屏鐵律：提前測試、準備 Plan B、關閉通知預覽

第三十一章：短影片拍攝與內容創作入門

2026 年，短影片已經不是“年輕人的玩具”了，它變成了所有人獲取資訊和表達自我的基礎方式。不管你是想記錄生活、分享知識，還是做點副業，手機就是你最好的創作工具——不需要買相機、不需要學 *Premiere*，一部手機就能開始。這一章幫你從零起步。

一、拍之前先想清楚：你要拍給誰看？

很多人拍影片的第一個錯誤就是“拿起手機就拍”，拍完不知道給誰看。在開拍之前，先回答三個問題：

1. **我的觀眾是誰？**——拍給孩子看、拍給同齡人看、拍給長輩看，內容風格完全不同。

2. **我能提供什麼價值？**——讓人笑（娛樂）、讓人學（知識）、讓人饞（美食）、讓人暖（情感）。至少佔一樣。

3. **我能在哪個平臺持續更新？**——抖音/快手（泛娛樂）、小紅書（生活方式）、B 站（深度內容）、影片號（社交圈）。選一個主陣地深耕，不要一上來就全平臺分發。

二、手機拍攝基礎設定——開拍前必做的 5 件事

1. **解析度設 1080P，幀率設 30fps：**這是短影片平臺的標準。4K 檔案太大上傳慢，60fps 在社交平臺上沒有優勢反而檔案更大。

2. **擦乾淨鏡頭：**第一條也是最重要的一條。拍之前用衣服擦一下鏡頭，畫面立刻清晰一個檔次。

3. **鎖定對焦和曝光：**在取景框長按畫面主體 2 秒，會出現”AE/AF 鎖定” 提示。這樣拍的時候不會因為移動導致畫面忽明忽暗。

4. **橫拍還是豎拍？**——發抖音/快手/影片號/小紅書**豎著拍**（9:16），發 B 站/YouTube **橫著拍**（16:9）。想清楚平臺再決定方向。

5. **開網格線：**設定 > 相機 > 網格線。三分法構圖的基礎，幫你快速找到畫面重心。

三、運鏡五招——讓畫面“動”起來

新手拍影片最大的問題是畫面太“靜”——全程站著不動拍。學會下面五個基本運鏡，畫面立刻有專業感：

- **推（Push）**：手機慢慢向被拍物件靠近。製造”靠近”的緊張感或親密感。
- **拉（Pull）**：手機慢慢遠離被拍物件。從細節拉到全景，交代環境。
- **搖（Pan）**：手機位置不動，左右轉動。適合拍風景、展示空間。
- **移（Track）**：手機平行移動，跟著被拍物件走。需要手持穩定，最好配合手機穩定器。
- **跟（Follow）**：手機跟在被拍物件後面或前面走。Vlog 最常用的運鏡。

穩是第一要務：手抖的畫面看起來很不專業。雙手持機比單手穩；走路時膝蓋微屈，用“忍者步”減少上下顛簸；如果有預算，買一個手機穩定器（大疆 OM 系列約 500-800 元），效果質變。

四、剪輯入門——剪映就夠了

剪映（CapCut 的國內版）是目前最適合新手的手機剪輯工具，免費、功能全、模板多。以下是基礎工作流：

1. **匯入素材**：開啟剪映 > 開始創作 > 選擇拍好的影片片段匯入。
2. **裁剪**：拖動時間軸，把每個片段開頭結尾的廢片剪掉。原則：每一段只保留有資訊量的部分，寧短勿長。
3. **加轉場**：兩段影片之間點”轉場”按鈕，選一個簡單的轉場效果。新手建議用”疊化”或”黑場”，不要用花哨的 3D 轉場。
4. **加字幕**：剪映有”識別字幕”功能，一鍵自動生成字幕。準確率約 90%，手動校對一遍即可。
5. **配音樂**：剪映音樂庫裡有大量版權免費的音樂。選一首節奏和內容匹配的 BGM，音量調到影片原聲的 1/3 左右。
6. **匯出**：1080P 匯出，幀率 30fps，位元速率選”推薦”即可。

五、封面的重要性——70% 的人只看封面決定看不看

短影片的封面就是“門面”，決定別人要不要點進來。幾個做封面的要點：

- **封面要有一句話**：用大字寫明影片的核心內容，比如”3 分鐘學會手機修圖”。字型要大、顏色要亮、和背景有對比。

- **封面和內容要一致**：不要做” 標題黨”——封面說一件事，點進去是另一回事，觀眾會立刻劃走，演演算法會降權。
- **人物表情要生動**：如果封面有人，選一個表情生動或誇張的瞬間。人臉比文字更能吸引點選。
- **剪映做封面**：在剪映裡選一幀最好的畫面，加文字疊加，匯出為圖片作為封面。

六、內容創作的三條心法

- **前 3 秒定生死**：短影片平臺，觀眾 3 秒內決定劃走還是繼續看。前 3 秒必須給出” 鉤子” ——一個問題、一個懸念、一個意外畫面。
- **一條影片只講一件事**：新手最容易犯的錯是一條影片塞太多內容。一條影片只解決一個問題，講透比講全重要。
- **堅持比天賦重要**：演演算法推薦需要” 養號”。前 20 條影片幾乎不會有播放量，這是正常的。堅持每週更新 2-3 條，讓演演算法認識你的內容方向，流量會慢慢起來。

動手：拍一條 30 秒的短影片

1. 選一個你擅長的小知識（比如” 怎麼把手機字型調大”）。
2. 用手機豎屏拍攝 3-5 段素材，每段 5-10 秒。
3. 用剪映拼接、裁剪、加字幕、配 BGM。
4. 做一張帶大字的封面圖。
5. 匯出後先發給朋友看，收集反饋再發布。

本章小結

- 拍之前想清楚：觀眾是誰、提供什麼價值、在哪個平臺發
- 開拍 5 件事：1080P/30fps、擦鏡頭、鎖對焦曝光、確定橫豎、開網格線
- 運鏡五招：推、拉、搖、移、跟——穩是第一要務
- 剪輯用剪映：裁剪轉場字幕 BGM 匯出，六步走
- 前 3 秒定生死，一條影片只講一件事，堅持比天賦重要

第三十二章：手機安全急救手冊

這一章是一份“應急預案”。希望你永遠用不上，但如果真的遇到手機丟失、被盜、被詐騙的情況，這份手冊能幫你把損失降到最低；地震、暴雨等災害來臨時，手機同樣是接收預警、定位求救、保持聯絡的工具。建議你現在就通讀一遍，然後把關鍵步驟截圖儲存，或者列印出來放在家裡——因為出事的時候你的手機可能已經不在手上了，災害時則可能沒電或沒訊號。

場景零：地震 / 極端天氣——手機怎麼幫你保命

防的是人身安全，不是帳號資金。預警設定詳見第十三章「天氣預警、自然災害預警與地震預警」；這裡只講警報響起之後該怎麼做。

地震警報響了：先躲後撤，別只顧著拍影片

1. 就近避險：趴下、掩護、抓牢（桌下、承重牆角、堅固傢俱旁），護住頭頸；遠離玻璃、外牆、吊燈。
2. 不要盲目衝樓：主震往往只有十幾秒到幾十秒，樓梯間可能更危險；電梯絕對不能用。
3. 震停後再有序撤離：走樓梯、注意餘震與掉落物；到開闊地，遠離電線杆、廣告牌、危房。
4. 手機用在刀刃上：給緊急聯絡人發簡短訊息（「我在 XX，安全/受傷，需要幫助」）比打長電話更省電；能發簡訊/微信時優先文字。
5. 沒訊號時：見第七章衛星 SOS / 衛星訊息（機型支援時）；或到高處、開闊地再試；部分場景下無網藍芽對講亦有幫助。
6. 關閉非必要功能：應急時開飛航模式再定時開、關藍芽掃描、降亮度，能多撐幾小時；把最後一格電留給報平安與救援資訊。

暴雨 / 颱風 / 山洪：預警階段就要行動

- 看到紅色預警：儘量減少外出，遠離地下空間、下穿橋、河道、陡坡。
- 在戶外被困：往堅固建築物高處走，勿強行涉水（積水可能有井蓋缺失與漏電）；打 119/110 時說清地標與水位。
- 出行前：用天氣 App 與當地預警公眾號確認風險；長途自駕避開山區臨水臨崖路段。
- 家人互報：約定「若失聯，以最後微信/簡訊定位為準」，避免各自瘋狂撥號耗光電量。

平時就做好的 4 件事

1. 開啟系統地震預警 / 正規預警 App，並允許其突破靜音。
2. 在通訊錄置頂 2-3 位「緊急聯絡人」，鎖定畫面介面可顯示（iOS/安卓醫療急救卡或鎖定畫面資訊）。
3. 家裡和包裡備行動電源；長途出行前充滿電並帶線。
4. 記住至少一個「不用通訊錄也能打」的號碼（家人手機號），寫在紙上放錢包。

場景一：手機丟了/被偷了——黃金 30 分鐘

手機丟失後的前 30 分鐘是“黃金時間”，處理得當可以保住你的錢財和隱私。按以下順序操作，一步都不要跳：

丟失後 30 分鐘標準操作流程

1. 借別人手機撥打自己號碼（第 1 分鐘）：可能只是掉在附近，好心人接了就能拿回來。
2. 遠端鎖定手機（第 2-5 分鐘）：用別人的手機或電腦登入查詢服務：iPhone：iCloud.com > 查詢 > 選你的裝置 > 標記為丟失
3. 安卓各品牌：華為 cloud.huawei.com / 小米 i.mi.com / OPPO cloud.oppo.com 等
4. 掛失 SIM 卡（第 5-10 分鐘）：打電信業者客服電話（移動 10086/聯通 10010/電信 10000），報手機丟失要求暫停 SIM 卡服務。這一步最關鍵——騙子拿到你的 SIM 卡就能接收驗證碼，重置你所有密碼。
5. 凍結支付帳戶（第 10-20 分鐘）：支付寶：95188 掛失，或在別人手機上登入支付寶 > 我的 > 設定 > 安全中心 > 緊急凍結
6. 微信：110 報警後聯絡微信客服，或在微信安全中心 > 凍結帳號
7. 銀行 App：撥打銀行卡背面的客服電話，要求凍結網銀
8. 通知親友（第 20-25 分鐘）：在朋友圈或讓家人幫忙發訊息，通知親友手機丟失，防止騙子用你的身份騙別人。
9. 報警（第 25-30 分鐘）：撥打 110 或去最近的派出所報案，拿到報案回執。後續保險理賠、銀行糾紛都需要這個回執。

場景二：被騙了——72 小時黃金處理期

發現自己被騙後，最關鍵是冷靜。慌亂中做錯誤決定會導致二次損失。按以下步驟操作：

1. 立即停止一切轉帳操作：不管騙子說什麼，不要再轉一分錢。” 再轉一筆保證金就能退回之前的錢” 是經典二次詐騙話術。

2. 保留所有證據：不要刪聊天紀錄！不要刪通話紀錄！截圖儲存聊天紀錄、轉帳紀錄、對方帳號資訊。這些都是報案和追回資金的關鍵證據。

3. 立即撥打 96110：這是全國反詐專線。接線員會指導你做緊急止付——如果轉帳時間不長（通常 24 小時內），銀行有可能凍結對方帳戶攔截資金。

4. 撥打 110 報警：帶上手機、身份證、轉帳憑證去最近的派出所做筆錄。

5. 聯絡銀行/支付平臺：告知被騙情況，要求凍結涉案帳戶。如果是信用卡被盜刷，要求銀行啟動拒付流程。

6. 修改所有密碼：在另一臺安全裝置上修改你的 Apple ID/品牌帳號、微信、支付寶、銀行 App 的密碼，開啟雙重認證。

場景三：手機中了病毒/木馬——怎麼辦？

如果你發現手機出現以下症狀——莫名其妙跳出廣告、自動安裝奇怪 App、通電話費暴增、用量異常消耗——很可能是中了惡意軟體：

- 第一步：進入安全模式——安卓手機長按電源鍵關機，再長按電源鍵開機，開機震動後長按音量減鍵不放，直到進入安全模式（左下角會顯示” 安全模式” 字樣）。安全模式下所有第三方 App 被停用，方便排查。

- 第二步：解除安裝可疑 App——在安全模式下進入設定 > 應用管理，按時間排序，找到最近安裝的不認識的 App 解除安裝掉。

- 第三步：安裝安全 App 掃描——下載” 騰訊手機管家” 或”360 手機衛士” 做一次全盤掃描。

- 第四步：恢復原廠設定（終極方案）——如果以上方法無效，備份重要資料後恢復原廠設定。注意：恢復原廠設定會刪除所有資料，一定要先備份照片、通訊錄等重要檔案。

- iPhone 基本不會中毒——iOS 的沙盒機制讓惡意軟體很難存活。如果你的 iPhone 出現異常，大機率是某個 App 的 bug 或設定問題，不是病毒。恢復原廠設定能解決 99% 的問題。

場景四：帳號被盜——快速奪回控制權

1. 立即修改密碼：如果你還能登入帳號，第一時間改密碼。新密碼要和舊密碼完全不同。

2. 踢掉其他裝置：在帳號安全設定裡找到” 登入裝置管理”，把不認識的裝置全部踢下線。

3. 檢查登入記錄：檢視最近 7 天的登入記錄，如果有異地登入，說明帳號確實被盜了。

4. 開啟雙重認證：如果你還沒開雙重認證，現在立刻開。這是防止帳號再次被盜最有效的方法。

5. 檢查繫結資訊：檢查帳號繫結的手機號、信箱有沒有被改。如果被改了，用帳號申訴功能找回。

6. 檢查支付設定：如果被盜的是微信/支付寶，檢查免密支付、自動扣款有沒有被篡改，銀行卡有沒有被新增到陌生帳戶。

場景五：收到勒索資訊——不要怕，不要付

常見的勒索套路有兩種：(1)“我有你的不雅影片/照片，不給錢就發給你通訊錄裡所有人”；(2)“你的手機被駭客控制了，不給錢就鎖死”。遇到這種情況：

- 99.9% 是詐騙：他們手裡根本沒有你的任何隱私資料。群發同樣的勒索資訊給幾萬人，總有人害怕付錢。
- 不要回復，不要付錢：回覆了等於告訴對方” 這個號碼有人用”，你會被標記為” 可勒索目標”，後續騷擾更多。
- 截圖儲存證據：然後拉黑對方。
- 修改密碼：如果擔心，修改一下你主要帳號的密碼，圖個安心。
- 如果真的有隱私洩露風險：報警。警方有處理這類案件的經驗。

場景六：AI 換臉/AI 擬聲詐騙——眼見不再為實

2024 年起，AI 換臉和 AI 擬聲技術已經能讓騙子在視訊通話裡“變成”你的家人、領導、朋友的臉和聲音。“媽，我出事了，快轉錢過來”——影片裡的人長得、聽起來都跟你兒子一模一樣，但其實是假的。這是當前最難防、也最危險的詐騙手法之一。

第一步：識別破綻——AI 偽造的四個常見漏洞

- 眨眼頻率異常：真人每分鐘眨眼 15-20 次，AI 換臉往往眨眼過少或節奏僵硬。
- 嘴型與音訊不同步：仔細看口型和聽到的聲音之間，會有零點幾秒的錯位。
- 面部邊緣模糊：臉的輪廓、耳朵附近、頭髮邊緣容易出現” 毛邊” 或閃爍。
- 光影方向不一致：臉上的高光、陰影和背景的光照方向對不上，或者側臉時五官” 跟著轉” 得不對。

第二步：三招驗證對方真假

- 要求做特定動作：讓對方轉頭、捂臉、用手擋住嘴巴再做開合動作。AI 換臉在遮擋和大角度轉頭時最容易” 露餡”——臉會變形或跟不上動作。
- 問只有對方知道的問題：編一個假問題，比如” 上次咱家狗送去哪家醫院看的？”——如果是騙子，他答不上來或者支支吾吾。

- 回撥已知號碼：結束通話影片，用你通訊錄裡存的號碼直接回撥過去。騙子控制不了真實的手機號，一撥就露餡。

第三步：確認被騙後的報警流程

1. 立即停止轉帳：再”急”也先停一下，凡是催你”快轉帳、別掛電話”的，幾乎都是騙子。
2. 保留全部證據：錄屏、截圖儲存視訊通話畫面、聊天紀錄、對方帳號、轉帳憑證——這些是技術鑑定的關鍵材料。
3. 撥打 96110 反詐專線：告知是”AI 換臉/擬聲”類詐騙，接線員會啟動緊急止付。
4. 撥打 110 報警：帶齊證據去派出所做筆錄，特別說明使用了 AI 偽造技術，便於警方立案定性。
5. 通知親友：騙子可能用你的偽造影片去騙別人，提前在朋友圈或家庭群裡發個提醒。

場景七：SIM swap 攻擊（SIM 劫持）——你的號碼被“偷”走了

這是一種很多人沒聽過、但危害極大的攻擊。SIM swap（SIM 卡劫持）指的是攻擊者透過偽造你的身份資訊，冒充你到營業廳補辦一張新的 SIM 卡。一旦補辦成功，你手機的訊號會突然消失，而攻擊者的手機插著那張新卡，能收到你所有的簡訊驗證碼——支付寶、銀行、微信，全部門戶大開。

怎麼發現？SIM swap 的三個預警訊號

- 手機突然顯示”無服務”/”無 SIM 卡”，但你明明沒動過卡，重啟也沒用——這往往是被劫持的瞬間。
- 收到電信業者發來的”補卡成功”或”業務辦理成功”簡訊——但你本人沒辦過。
- 突然收不到驗證碼，但其他簡訊也收不到——因為簡訊被劫持到了攻擊者的卡上。

防護措施——把 SIM 卡也”上鎖”

- 設定 SIM 卡 PIN 碼：路徑設定 > 電話 > SIM 卡 PIN 碼。開啟後，每次手機重啟或換卡，都要輸 PIN 才能用這張卡。攻擊者就算拿到卡也用不了。
- 開啟電信業者二次驗證：登入中國移動/聯通/電信 App，找到”安全中心”或”業務辦理驗證”，開啟非本人補卡簡訊提醒和業務辦理二次密碼驗證。這樣別人去補你的卡時，營業員會要求額外的身份核驗。
- 重要帳號別隻用簡訊驗證：銀行、Apple ID 等高價值帳號，優先用認證器 App（Google Authenticator、微軟 Authenticator）或硬體金鑰，而不是簡訊驗證碼。
- 身份證影印件加水印：辦理業務需要的影印件寫明”僅供 XX 業務使用”——攻擊者往往靠洩露的身份證資訊冒充你。

被劫持後的應急處理——速度就是一切

1. 立即撥打電信業者客服掛失：移動 10086 / 聯通 10010 / 電信 10000，要求立即停機並凍結補卡業務，阻止攻擊者繼續使用你的號碼。
2. 用其他裝置登入關鍵帳號：用電腦或家人手機，立即修改銀行、支付寶、微信、Apple ID 的密碼，並踢掉陌生裝置。
3. 凍結支付帳戶：支付寶 95188 掛失，微信安全中心凍結，銀行客服凍結網銀。
4. 帶身份證去營業廳：本人持身份證到自營廳補辦新卡，還原號碼使用權。
5. 報警：SIM swap 已構成”侵犯公民個人資料罪”，帶齊證據（停機時間記錄、資金損失、補卡記錄）報案。
6. 全面排查：還原號碼後，逐一檢查近期登入記錄和資金流水，確認有沒有被冒用過的痕跡。

預防勝於治療——日常 5 件事

1. 鎖定畫面密碼 + 雙重認證，兩個都開
2. ”查詢我的手機”確認開啟
3. SIM 卡設定 PIN 碼（設定 > 電話 > SIM 卡 PIN 碼），丟了手機別人不能用你的卡
4. 不安裝來歷不明的 App，不點陌生連結
5. 定期備份重要資料（照片、通訊錄、聊天紀錄）

動手：做一份屬於你的”應急聯絡卡”

1. 找一張紙，寫下以下號碼：電信業者客服（移動 10086/聯通 10010/電信 10000）、銀行客服、96110 反詐專線、110 報警電話、家人電話。
2. 寫下你的 Apple ID/品牌帳號的註冊信箱和備用聯絡方式。
3. 把這張紙放在錢包裡或貼在冰箱上——不要存在手機裡，因為出事時手機可能不在身邊。
4. 現在就檢查：”查詢我的手機”是否已開啟，SIM 卡 PIN 碼是否已設定。

本章小結

- 手機丟失黃金 30 分鐘：藉手機撥打遠端鎖定掛失 SIM 卡凍結支付通知親友報警
- 被騙後 72 小時：停止轉帳保留證據撥 96110 報警聯絡銀行改密碼
- 手機中毒：安全模式解除安裝可疑 App 安全掃描恢復原廠設定（終極方案）
- 帳號被盜：改密碼踢裝置查登入記錄開雙重認證查繫結查支付
- 勒索資訊 99.9% 是詐騙，不回覆不付錢，截圖拉黑

- AI 換臉/擬聲詐騙：看眨眼/嘴型/邊緣/光影破綻，讓對方轉頭捂臉、問私密問題、回撥已知號碼
- SIM swap 劫持：手機突然” 無服務” 即警覺；設 SIM 卡 PIN、開電信業者二次驗證、重要帳號改用認證器 App
- 日常預防五件事：鎖定畫面 +2FA、查詢手機、SIM 卡 PIN、不裝來歷不明 App、定期備份

六、展望篇與車聯篇（第 33-34 章）

第三十三章：未來展望——手機的下一程

手機不會一夜之間消失，但它的“獨霸地位”可能被新一代裝置分流。這一章我們聊三件事：下一代裝置長什麼樣、手機如何“隱身”為新身份基石、以及那些我們必須提前思考的倫理議題。

一、AR/VR 與空間計算：手機之外的“第二桌面”

下一代裝置的競爭已經打響，最有可能分流手機時間的，是 AR/VR 與空間計算裝置：

- **Vision Pro 後續**：蘋果 Vision Pro 的釋出標誌著” 空間計算” 概念的正式落地，但第一代產品價格（29999 元起）和重量仍是門檻。預計 2026-2027 年會推出更輕、更便宜的平價版 Vision，讓空間計算從” 開發者玩具” 走向” 早期大眾”。屆時 iPhone 可能成為 Vision 眼鏡的算力伴侶與資料中樞。
- **AR 眼鏡**：給你的眼前開個” 第二桌面”，數字資訊變得像空氣一樣無處不在。Meta 與 Ray-Ban 合作的智慧眼鏡已賣出數百萬副，國內 Rokid、XREAL、INMO 等廠商也在推出消費級 AR 眼鏡。2025-2026 年的 AR 眼鏡已能做實時翻譯、導航 HUD、AI 助手對話，但仍受限於光波導顯示和續航，距離” 取代手機” 還有 3-5 年。
- **智慧戒指與可穿戴**：Samsung Galaxy Ring、Oura Ring 等智慧戒指開始流行，作為手機的” 貼身感測器” 補充健康監測。手機的角色正從” 萬能終端” 轉向” 中樞 + 多個輕量化配件” 的組合。

二、腦機介面：思想成為互動媒介

思想本身將成為互動的媒介，但這條路比想象中漫長：

- **Neuralink 最新進展**：馬斯克的 Neuralink 已於 2024 年完成首例人體植入，首位受試者 Noland Arbaugh 透過意念控制電腦滑鼠下國際象棋。目前植入式 BCI（腦機介面）主要面向癱瘓患者輔助場景，目標是幫助他們重新” 操作” 數字世界。
- **非侵入式 BCI**：多家公司開發非侵入式頭環（如 Kernel、OpenBCI），無需手術即可讀取腦電訊號，但精度遠低於侵入式。距離” 用意念打字” 還有相當距離。

- **消費級展望**：腦機介面距離消費級手機互動還有 10-20 年，但短期可能先用於無障礙輔助、專注力監測等場景。倫理與隱私問題（”我的腦電波算誰的隱私”）將先於技術成熟成為爭論焦點。

三、手機的“隱身”與新身份基石

手機的另一個未來進化方向或許是“隱身”——化身為無處不在的“幕後推手”，成為你數字身份的唯一基石：

- **雲端雙因素認證**：瞬間完成” 你是你” 的認證，登入任何服務無需手動輸驗證碼。
- **數字遺產成為出廠設定**：新機啟用時預設數字資產繼承方案，讓” 身後事” 成為出廠預設。
- **從” 裝置中心” 到” 身份中心”**：你的雲端個人資料保險櫃才是中心，硬體只是訪問入口。換機、換裝置，身份與資料始終跟著你。

四、倫理思考：我們，還是手機的主人嗎？

一個終極拷問始終存在：我們是被技術賦能的“神”，還是被演演算法餵養的“缸中之腦”？

- **AI 倫理**：當端側 AI 比你自己更懂你的偏好，你的” 選擇” 是自主意志還是被引導的結果？AI Agent 代你下單、回覆訊息時，責任與邊界在哪裡？
- **資料主權**：你的健康資料、腦電資料、生物特徵資料，是屬於你、屬於裝置廠商還是屬於雲服務商？《個人資訊保護法》和歐盟 GDPR 只是起點，” 資料可攜帶權” 和” 被遺忘權” 的真正落地還有很長的路。
- **數字鴻溝**：AI 手機、摺疊屏、衛星通訊都在拉大” 用得起” 和” 用不起” 之間的差距。數字基礎設施的公平接入，將成為新的社會議題——就像 20 年前” 會不會上網” 一樣關鍵。
- 當演演算法比你更懂你想看什麼，你的” 喜歡” 有多少是出於自發？
- 當你每隔幾分鐘就不自覺地點亮螢幕，這是你的習慣還是被設計出來的” 上癮行為”？

五、保持連線，也保持獨立——你的數字生活哲學

無論技術如何進化，下面這幾條數字生活哲學，10 年後依然適用：

1. 做資源的分配者，而非被推送的接收者。
2. 讓自動化成為你忠實的” 副駕駛”。
3. 建立你穩固的” 數字後方”。
4. 定期執行” 數字斷舍離” 與” 社交齋戒”。
5. 永遠把安全和隱私作為一切便利的前提。

來試試：制定你的《個人數字裝置使用服務條款》

為你和你的手機（以及未來的 AR 眼鏡、AI 助手）寫一份簡單的“服務條款”，來定義你們的關係。例如：

- 第 1 條：我的注意力時間屬於我。你在我工作/陪伴家人時應主動進入” 勿擾” 狀態。
- 第 2 條：我的個人資料是我的財產。你的首要職責是安全加密與便捷同步。
- 第 3 條：我們的共同目標是提升生活效率與體驗。若你透過誘導性設計讓我沉迷，我有權將你解除安裝。

本章小結

- Vision Pro 與 AR 眼鏡是手機之外的新計算入口，但短期難以取代手機
- 腦機介面距離消費級還有 10-20 年，倫理問題先於技術成熟
- 手機正向” 隱身” 方向進化，從裝置中心走向身份中心
- AI 倫理、資料主權、數字鴻溝是未來 10 年的核心議題
- 保持連線也保持獨立，是數字生活不變的哲學

第三十四章：車機互聯深度指南——手機與汽車的融合

汽車正在從“四個輪子加沙發”進化為“四個輪子加智慧手機”。無論你開的是燃油車還是新能源，手機與車機的關係都決定了你的駕駛體驗。這一章帶你釐清車機互聯的三大路線、各派系車機系統的特點，以及那些讓開車更順手的小技巧。

一、車機互聯的三大路線

當前手機與汽車“聯網”主要有三種思路，理解了它們就理解了整個生態：

- **手機投屏到車機**：車機只當一塊” 顯示器” 和” 音箱”，運算和地圖資料都來自手機。代表方案：Apple CarPlay、Google Android Auto、華為 HiCar、百度 CarLife。優點是手機生態直接搬到車上，不用重新裝 App；缺點是車機本身的硬體能力被閒置。

- **手機與車機深度融合**：車機是獨立系統，但與手機共享帳號、應用、資料，甚至能” 接力” 任務。代表方案：鴻蒙座艙（問界/智界）、比亞迪 DiLink、蔚來 NIO OS、理想車機系統。優點是體驗一體化；缺點是通常繫結同一品牌生態。

- **CarPlay 2.0 深度整合版**：2025 年蘋果釋出的新一代 CarPlay，不再只是” 投屏”，而是接管車機的儀表盤、空調控制、座椅調節等核心功能。法拉利、阿斯頓 • 馬丁、保時捷等品牌率先搭載，標誌車機系統進入” 手機廠商主導” 的新階段。

二、CarPlay / Android Auto / HiCar 對比

三大主流投屏方案各有側重，選車或選手機時可以對照下表：

方案	支援手機系統	支援車型	功能豐富度	國內可用性
Apple CarPlay	僅 iPhone	極廣，絕大多數車型支援	高，地圖/音樂/電話/訊息均可	好，原生可用
Android Auto	Android（國內手機需裝服務框架）	較廣，合資/進口車居多	高，與 CarPlay 相當	一般，需科學上網啟用
華為 HiCar	華為手機為主，部分支援其他安卓	國產車與部分合資車型	較高，可與鴻蒙座艙聯動	很好，本土生態
百度 CarLife	iOS 與 Android 均可	較廣，老車型後裝常見	中等，更新較慢	很好，本土生態

三、鴻蒙座艙：手機與車機的無縫流轉

如果說 CarPlay 是“手機投到車機”，鴻蒙座艙則是“手機和車機本就是一家”。華為手機與問界/智界車機的協同，代表了當前國內車機互聯的最高水準：

- **超級桌面**：車機直接呼叫手機上的 App，無需在車機重新安裝。手機裡在聽的音樂、在導航的路線，上車自動續接到車機螢幕，下車再回到手機，全程無縫。
- **應用接力**：手機上設好的導航，關車門時車機自動接管；到達停車場，下車後手機繼續步行導航到最終目的地。” 最後一公里” 不再斷檔。
- **手機車機共用帳號**：華為帳號一登，車機自動同步你的音樂會員、地圖收藏、日程、聯絡人。車機與手機是同一身份的不同終端，而非兩套獨立系統。

四、造車新勢力的車機系統

國產新能源車的車機已經從“導航 + 音樂”進化為移動智慧終端，各家的玩法各有特色：

- **蔚來 NIO OS**：NOMI 語音助手是行業標杆，能識別多指令、上下文連續對話，還帶情感表情。NIO Phone 則把手機做成” 車鑰匙 + 第二屏”，深度繫結蔚來生態。
- **理想車機**：多屏互動是亮點，副駕屏可獨立看劇、孩子屏可看動畫片。” 任務大師” 讓你自定義場景，比如” 開門自動暫停音樂、關門繼續播放”。
- **小米澎湃車機**：與小米手機、手錶、家電深度聯動。手機上設的鬧鐘會在車機提醒；車上導航回家，家裡空調提前開啟。HyperOS 打通了” 人車家全生態”。
- **比亞迪 DiLink**：標誌性的旋轉屏，豎屏看短影片、橫屏看地圖，自適應 App 方向。自帶應用市場，生態豐富度在傳統車企中領先。

五、手機當車機——能用嗎？

如果你的車機太老或太難用，能否直接用手機“頂上”？答案是：可以，但要看場景。

- **摩托羅拉 Ready For / 桌面模式：**部分摩托羅拉、三星、華為手機支援” 桌面模式”，連上顯示器就是一臺簡易電腦。理論上接車機屏也能用，但實際相容性參差不齊，且缺少汽車專屬互動（如方向盤按鍵）。
- **手機支架 vs 原車車機：**如果原車車機卡頓、地圖不更新，一個穩固的手機支架 + 手機導航是務實之選。但要注意：(1) 手機螢幕小，駕駛時看不清；(2) 夏天暴曬會讓手機過熱降頻甚至自動關機；(3) 長時間導航耗電快，需配車載充電器。
- **建議：**短途或臨時用車，手機支架夠用；日常通勤長途，建議升級支援 CarPlay/HiCar 的車機或加裝後裝車機，體驗和安全都更好。

安全提醒

無論用哪種方案，駕駛時操作手機都違反交規。手機支架只能用於“看導航”，不能用於“操作手機”。設定目的地、切換歌曲等操作，請務必停車後再做，或使用語音控制。

六、實用技巧

- **無線 CarPlay 連線不穩定：**(1) 關閉車機藍芽再重新配對；(2) 刪除車機端已配對裝置記錄後重連；(3) 檢查手機是否同時連了多個藍芽裝置造成衝突；(4) 升級車機韌體和手機系統到最新版；(5) 實在不行，改用有線 CarPlay，更穩定。
- **車機流量用完了：**(1) 用手機開熱點，車機連手機熱點上網；(2) 在車機設定中把” 高耗流量” 應用（如影片、線上音樂）切到 Wi-Fi；(3) 定期清理車機離線地圖快取。
- **駕駛模式自動觸發：**(1)iOS” 專注模式” 設定” 連線車載藍芽時自動開啟”，自動遮蔽非緊急通知；(2) 安卓” 駕駛模式” 或第三方 App（如 Tasker）可實現連藍芽自動靜音、自動回覆訊息；(3) 華為手機可設定” 智感支付”，上車自動切到車機播放音樂。

選車時的小建議

如果在乎車機體驗，選車時務必去 4S 店實測：(1) 帶自己的手機試 CarPlay/HiCar 連線是否穩定；(2) 試語音助手的反應速度和指令識別率；(3) 看應用市場有沒有你常用的 App；(4) 關注後續 OTA 升級頻率——好車機是用軟體迭代出來的。

動手：最佳化你的車機互聯體驗

1. 確認你的手機和車機支援的互聯方案（CarPlay / HiCar / Android Auto / CarLife）。
2. 清理一次車機的藍芽配對列表，只保留常用手機，減少連線衝突。
3. 設定一個” 駕駛專注模式”，連線車機藍芽時自動開啟。
4. 下載車機品牌 App（如鴻蒙智行、理想汽車、蔚來 App），看是否能遠端控制空調、檢

視車況。

本章小結

- 車機互聯三大路線：投屏（CarPlay 等）、深度融合（鴻蒙座艙等）、CarPlay 2.0 深度整合
- CarPlay 生態最廣；HiCar 國內體驗好；Android Auto 國內需折騰；CarLife 適合老車後裝
- 鴻蒙座艙的超級桌面、應用接力、共用帳號代表了深度融合方向
- 蔚來 NOMI、理想多屏、小米人車家、比亞迪旋轉屏各有特色
- 手機當車機僅適合臨時場景，長期建議升級或後裝車機
- 實用技巧：CarPlay 重連、手機熱點補流量、藍芽觸發駕駛模式

兒童與手機專題（第 35 章）

第三十五章：兒童與手機——數字時代的教育指南

本章不講“手機是洪水猛獸”，也不鼓吹“越早接觸越聰明”。我們按年齡分段給裝置、設規則、教素養、護健康，一步步來。

一、幾歲給孩子手機？

沒有標準答案，但有年齡基線。過早給手機，孩子大腦前額葉尚未發育成熟，自控力跟不上演演算法的“誘惑設計”；過晚給，又可能讓他在同伴中“社交脫節”。下面是分齡建議：

- **3-6 歲**：儘量不碰手機。如需使用，單次不超過 15 分鐘，內容以家長精選的教育動畫為主，且必須**陪同觀看**。這一階段螢幕對視力與注意力發育影響最大。
- **7-9 歲**：可在家長裝置上”借用”，用於網課、聽故事、查資料。開始建立”使用前申請、用完歸還”的規則，不配置專屬手機。
- **10-12 歲**：可考慮給一臺”功能機”或兒童手錶，解決上下學聯絡。如確需智慧機，應開啟嚴格的家長控制。
- **13 歲以上**：進入初中，社交需求增強，可以給智慧機，但需配合”使用公約”和定期覆盤。

> **”夠用就好”原則**
>
> 孩子的第一臺手機不需要旗艦。處理器的“過剩效能”往往被用來跑遊戲。選一臺電池夠大、效能中等、螢幕護眼的千元機或官方翻新機，反而更利於管控。預算參考：1500-2500 元區間即可滿足絕大多數學生需求。

智慧手錶作為過渡方案

如果只是想解決“放學後能不能聯絡上”的問題，兒童手錶比手機更合適。它只能打電話、發語音、定位，沒有瀏覽器和短影片，天然限制了“沉迷入口”：

- **小天才電話手錶**：生態最完整，同齡人社交”碰一碰加好友”是其核心賣點，但專屬 App 生態較封閉。

- **華為兒童手錶**：與華為手機聯動好，定位精準，支援視訊通話，價效比高。
- **小米兒童手錶**：價格親民，基礎功能齊全，適合預算有限的家庭。

買手錶時注意：選擇支援 **4G 全網通**的型號（2G/3G 正逐步退網），並確認運營商兒童套餐資費。

二、家長控制系統全攻略

裝置給到孩子之前，第一件事不是開機，而是設定家長控制。兩大平臺都有原生方案：

iOS：螢幕使用時間

- **App 限額**：設定 > 螢幕使用時間 > App 限額，可為每個 App 類別設定每日可用時長（如遊戲 30 分鐘）。
- **停用時間**：設定每天”熄燈時段”（如 22:00-7:00），期間所有 App 變灰禁用，僅電話可用。
- **內容限制**：可按年齡段過濾影視、網頁、App 安裝，並禁止 App 內購買。
- **家庭共享**：用家長的 iPhone 遠端管理孩子的裝置，批准下載請求、檢視使用報告。

安卓：Google Family Link / 品牌家長控制

- **Google Family Link**：谷歌官方方案，可遠端管理裝置使用時長、鎖定裝置、檢視活動報告。國內安卓需視機型可用性。
- **小米兒童空間**：設定 > 兒童空間，可設定使用時長、白名單 App，退出需家長密碼。
- **華為學生模式**：設定 > 健康使用手機 > 學生模式，限制使用時段與 App，支援遠端管理。
- **OPPO/vivo 兒童模式**：在”設定”或專屬”兒童空間”App 中開啟，原理類似。

功能	iOS 螢幕使用時間	安卓 Family Link / 品牌方案
App 限額	支援，按類別	支援，按 App
停用時間	支援	支援（品牌方案略有差異）
內容過濾	系統級，較徹底	依賴品牌，參差不齊
遠端管理	家庭共享	Family Link / 品牌雲
安裝 App 審批	需家長批准	Family Link 支援

家長控制不是”一勞永逸”

孩子總能找到繞過的方法（網上搜“如何破解螢幕使用時間”的教程一大把）。技術手段只是底線，真正的防線是親子溝通和信任。建議每月與孩子一起復盤使用報告，而不是單方面監控。

三、青少年模式

各大內容平臺都內建了“青少年模式”，開啟後會限制使用時長、過濾不良內容、關閉充值打賞。這是國家法規要求的強制功能，但**需要家長主動開啟並設定密碼**：

- **抖音/快手青少年模式**：每天限 40 分鐘（22:00-6:00 禁用），內容只展示精選科普、教育類影片，無法直播、無法打賞、無法充值。
- **微信青少年模式**：限制微信支付（需家長授權額度），限制公眾號、影片號內容，搖一搖、附近的人等功能預設關閉。
- **遊戲防沉迷系統**：國家層面的實名認證 + 時間限制。未成年人僅可在週五、週六、週日及法定節假日的 20:00-21:00 登入遊戲，每日 1 小時。

實名認證別用家長身份證“借號”

很多孩子用家長身份證註冊遊戲帳號繞過防沉迷，這既違反使用者協議，也讓孩子失去保護。家長應保管好自己的支付密碼與身份資訊，不要為圖省事“借號”給孩子。

四、網路素養教育

技術能攔截 90% 的風險，剩下 10% 靠孩子的判斷力。比起“不許上網”，更重要的是教他們“怎麼上網”：

- **隱私保護**：不向陌生人透露真實姓名、學校、家庭住址；不傳送個人照片、身份證、家庭合影；定位功能預設關閉，僅在使用時開啟。
- **網路欺凌識別與應對**：被辱罵、孤立、惡意 P 圖都屬於欺凌。告訴孩子遇到時不要對罵、不要刪除證據、第一時間告訴家長。微信可長按訊息投訴，平臺會處理施暴帳號。
- **內容批判**：教孩子分辨” 事實” 和” 觀點”，識別標題黨、震驚體；理解演演算法只會推你想看的，不代表世界的全貌；遇到” 天上掉餡餅” 的廣告一律不信。
- **網路安全**：不點選陌生連結、不掃碼來歷不明的二維碼、不下載非官方商店的 App；密碼不分享給任何人（包括” 好朋友”）；遇到索要驗證碼的一律視為騙子。

動手：和孩子一起籤一份《手機使用公約》

把規則寫下來、雙方簽字貼在顯眼處，比口頭約定有效得多。建議包含：

- 使用時段：如” 作業完成後 19:30-20:30 可用”。
- 禁用區域：餐桌上、臥室睡前、課堂上手機一律收起。
- 內容邊界：不安裝未告知家長的 App，不與陌生人私聊。
- 違約後果：如” 超時一次，次日使用時長減半”。
- 家長承諾：不在孩子面前” 刷手機” 超過約定時長，以身作則。

五、健康使用習慣

孩子的眼睛、頸椎、睡眠都在發育期，手機對身體的傷害比成年人更大：

- **螢幕時間建議**：3-6 歲每日不超過 1 小時；6-12 歲學習類不限制，娛樂類不超過 1 小時；12 歲以上與孩子協商，但娛樂類不超過 2 小時。
- **護眼習慣**：遵守”20-20-20 法則”——每看螢幕 20 分鐘，抬頭看 20 英尺（約 6 米）外的遠處 20 秒；睡前 1 小時不看手機，藍光會抑制褪黑素影響睡眠。
- **姿勢管理**：手機螢幕與眼睛平齊，不要低頭含胸；連續使用不超過 30 分鐘就要起身活動。
- **親子共同規則**：餐桌、臥室、書房設” 無手機區”；全家一起看電影、戶外活動時手機統一收起，讓” 放下手機” 成為家庭文化而非對孩子的懲罰。

家長是最好的” 防火牆”

研究表明，父母自身手機使用時長與孩子呈強正相關。要求孩子不看手機，自己卻刷短影片到深夜，規則就會失效。改變孩子，先從放下自己的手機開始。

六、數字遺產與兒童

這一節容易被忽略，但很重要。孩子的數字資產（帳號、照片、充值餘額、遊戲裝備）同樣面臨繼承與隱私問題：

- **兒童數字資產的繼承**：孩子不幸離世時，其社交帳號、遊戲帳號、雲盤照片的繼承流程與成年人相同，需提供死亡證明、親屬關係證明。建議家長平時就為孩子的重要帳號做” 遺產聯絡人” 或” 緊急聯絡人” 設定（如蘋果遺產聯絡人、微信帳號繼承）。
- **未成年人的數字隱私**：《個人資訊保護法》規定，處理不滿 14 週歲未成年人個人資訊需取得監護人同意，且應制定專門的隱私規則。家長應定期審查孩子常用 App 的隱私政策，關閉” 個性化推薦” 和” 資料共享” 開關。
- **帳號註冊建議**：為孩子註冊的帳號應使用家長可控的郵箱/手機號，便於在未成年階段統一管理；孩子成年後可再做帳號移交。

警惕” 兒童數字遺產” 的另一面

孩子從出生起就在被“記錄”——家長曬的成長照、幼兒園發的影片、智慧手錶的健康資料。這些資料一旦上傳就難以徹底刪除。釋出前想一想：十年後孩子會不會因為這張照片感到尷尬？尊重孩子的數字人格，從剋制“曬娃”開始。

本章小結

- 給手機的年齡沒有標準答案，但 3-6 歲儘量不碰、13 歲前用兒童手錶或功能機過渡
- ” 夠用就好”：第一臺手機選千元機而非旗艦，效能過剩等於誘惑過剩

- iOS 螢幕使用時間與安卓 Family Link/品牌家長控制是兩套技術防線，配合使用更穩
- 青少年模式與遊戲防沉迷是法規要求，但需家長主動開啟並守護密碼
- 網路素養比技術攔截更重要：教孩子識別欺凌、保護隱私、批判內容
- 健康習慣要從家長做起——你放下手機，孩子才會放下
- 兒童數字遺產與隱私是新興議題，平時就應做”遺產聯絡人”設定並剋制曬娃

第三十六章：手機與智慧家居——讓家變聰明

手機還能幹一件越來越重要的事——當整個家的“遙控器”。從一盞燈、一個插座到一把門鎖，智慧家居正在從“極客玩具”變成“家常配置”。這一章幫你理清協議、生態、中樞和入門套件，再用一份安全清單收尾，讓你不被“智慧”反噬。

一、智慧家居協議：選誰很重要

買智慧裝置前先看它支援什麼協議，這決定了它能不能被你的手機和生態“使喚”。目前主流協議有四種，各自的脾氣如下：

- Matter 協議：2022 年由 CSA（連線標準聯盟）釋出，背後是蘋果、谷歌、亞馬遜、三星等巨頭聯手。它的目標只有一個——讓所有品牌的智慧裝置說同一種語言。2023-2026 年正快速成為統一標準，支援它的裝置可同時接入 Apple HomeKit、Google Home、Amazon Alexa 和米家，不再被單一生態綁架。
- Thread 協議：Matter 的底層網路”管道”。低功耗、自組網（某個節點斷電，訊號會自動繞路），響應快、不卡頓。買帶 Thread 邊界路由器的裝置（如 HomePod mini、Apple TV 4K、小米中樞閘道器），體驗明顯更穩。
- Wi-Fi：頻寬大、能直連路由器，但功耗高、裝置多了易擠佔通道，適合相機、掃地機這類需要傳大資料的裝置。
- 藍芽（BLE）：配網簡單、省電，但距離短、不能自組網，適合門鎖、溫溼度計這類小資料裝置。
- Zigbee：老牌低功耗自組網協議，需要專用閘道器，裝置便宜穩定，但跨品牌互通差，正被 Matter+Thread 逐步替代。

一句話原則

2026 年買智慧裝置，優先看是否有“Matter over Thread”標識——這是當下最經得起時間檢驗的選擇。老裝置若只支援 Wi-Fi 或藍芽也不必急著淘汰，先用著，新購時再轉向 Matter。

二、三大生態對比

協議是“語言”，生態是“圈子”。即使都支援 Matter，你日常仍要選一個主生態來搭自動化和語音控制。三大主流生態對比如下：

生態	優勢	短板	適用人群
Apple HomeKit	隱私最強（本地處理為主）、與 iPhone/Siri/快捷指令聯動順滑	裝置偏少偏貴、需要 HomePod 或 Apple TV 做中樞	全家蘋果裝置、重視隱私與體驗的使用者
米家（小米）	裝置最豐富、價效比最高、小愛同學好用	生態相對封閉、資料多上雲、廣告偏多	預算有限、想一站式配齊全屋的入門使用者
華為智聯（鴻蒙）	與鴻蒙手機/平板/手錶無縫流轉、裝置數量快速增長	海外裝置支援少、生態仍在追趕	華為手機使用者、看重”超級終端”多裝置協同

選生態的決策建議

看手機選生態：iPhone 使用者優先 HomeKit，華為使用者優先智聯，其他安卓使用者米家最省心。靠 Matter 協議，不同生態的裝置也能被同一部手機控制，不必一次性“站隊”。

三、手機當家居中樞

在多數入門場景裡，手機就是你的“家居中樞”——透過 App 控制燈光、空調、窗簾、安防，還能搭建自動化場景，讓家“自己動起來”：

- 裝置控制：開啟 HomeKit/米家/智慧生活 App，可遠端或本地開關燈、調色溫、設定空調溫度、拉合窗簾、檢視相機畫面。
- 自動化場景：把”條件動作”組合起來，常用的三大場景：回家模式：定位到家 + 天黑開玄關燈 + 開空調 + 拉窗簾 + 關門鎖。
- 離家模式：最後一個出門（定位離家或門鎖上鎖）關燈 + 關空調 + 關插座 + 啟動相機 + 佈防。
- 睡眠模式：每晚 23:00 關全屋主燈 + 拉窗簾 + 空調切 26 度 + 開夜燈。

NFC 碰一碰：在門貼一張 NFC 標籤，手機一碰即觸發對應場景（碰一下開門燈、再碰一下切睡眠）。iPhone 用”快捷指令”自動化，安卓用”錢包”或廠商 App 均可寫入。

來試試：搭一個”回家模式”

在手機智慧家居 App 的“自動化”裡新增一條：觸發條件選“到達某地點”（家），動作依次加“開玄關燈”“空調設 26 度”“窗簾關閉”。儲存後出門逛一圈，回家時看它是否自己

亮燈——第一次成功的成就感，比下單新裝置還過癮。

四、入門套件推薦

不必一次配齊，從這四件套開始，就能覆蓋八成日常需求：

- 智慧音箱：小愛同學（米家生態，價效比高）/ HomePod mini（蘋果生態，兼作 Thread 中樞）/ Alexa（海外生態廣）。注意：音箱決定了你的語音生態，先定生態再買音箱。
- 智慧燈泡：Yeelight（米家/Matter，色溫亮度可調，便宜）/ Philips Hue（色彩好、穩定，但需 Hue 閘道器，偏貴）。注意：買前確認燈座口徑（E27/B22）和是否需要調色。
- 智慧插座：小米智慧插座（便宜、電量統計）/ TP-Link Kasa（海外 Matter 支援好）。注意：用在大功率電器（熱水器、電暖器）時務必看額定功率，別過載。
- 智慧門鎖：鹿客/小米（價效比、米家聯動）/ 凱迪仕（傳統鎖具大廠、機械結構紮實）。注意：確認是否支援 Matter 或你所在生態，留意電池續航和應急鑰匙孔位置。

選購順序建議

先買智慧音箱（定生態）再買智慧燈泡和插座（低門檻、立刻有反饋）最後買門鎖和相機（涉及安全，謹慎選）。一次只加一類裝置，熟悉後再擴，避免“買回來不會用”。

五、安全注意事項

智慧裝置越多的家，攻擊面也越大。下面四條是底線，請逐條對照：

1. 改預設密碼：相機、路由器、門鎖出廠預設密碼在網上全是公開的，買到手第一件事就是改成” 長且不重複” 的強密碼。
2. 及時更新韌體：廠商推送的韌體往往修復了安全漏洞，在 App 裡開啟” 自動更新”，或至少每月手動檢查一次。
3. 網路隔離：把 IoT 裝置單獨放到一個 Wi-Fi（或訪客網路）上，和手機、電腦的主網路分開。萬一某個裝置被攻破，也難以橫向威脅你的重要資料。
4. 隱私保護：相機開啟” 隱私區域” 遮蔽（如臥室、衛生間不拍），關閉雲端 24 小時錄影；不在家的相機畫面別長期開給第三方 App。

這幾個坑別踩

- 別用門鎖的” 遠端開鎖” 功能給陌生人——優先用臨時密碼，用完即失效。
- 別把相機直接對著床、對著窗外——洩露的是你自己的隱私。
- 別圖便宜買” 白牌” 無品牌裝置——多半不更新韌體，等於給家裡安了個後門。

本章小結

- Matter+Thread 是 2023-2026 年的統一方向，買新裝置優先看這個標識
- 三大生態各有所長：蘋果重隱私體驗、米家重價效比、華為重多裝置協同
- 手機可當中樞，搭好” 回家/離家/睡眠” 三大場景，家就能自己動起來
- 入門四件套：智慧音箱、燈泡、插座、門鎖，按這個順序逐步添置
- 改預設密碼、更新韌體、網路隔離、隱私遮蔽——四條安全底線不能省

七、AI 程式設計實戰篇（第 37-47 章）

第三十七章：正規化革命——從“寫程式碼”到“指揮 AI 寫程式碼”

恭喜你，翻開這本書的你，正站在一場職業正規化革命的正中央。這場革命的起點，是 *2025* 年一個程式設計師在社交媒體上發的一句話。聽起來有點戲劇性？但事實就是這樣。

一、Vibe Coding 的誕生：一句話引發的運動

2025 年 2 月，Andrej Karpathy——這個名字請先記住，後面還會見到他好幾次——在社交媒體上發了一條後來被反覆引用的推文：“有一種新程式設計方式叫 Vibe Coding，你完全憑感覺（vibes）來，忽略程式碼細節，讓 AI 包攬一切。你看到什麼就說什麼，複製貼上，基本上能用就行。”

這條看起來隨口一說的發言，其實精準命名了一場已經悄然發生的正規化轉移：人類不再逐行編寫程式碼，而是用自然語言描述意圖，由 AI 完成實現。Karpathy 的措辭帶著一絲戲謔，但背後是個深刻的洞察：當 AI 的程式碼生成能力跨過某個閾值之後，程式設計的瓶頸就不再是“這行程式碼我能不能寫出來”，而是“這件事我到底想不想得清楚”。

小貼士：Vibe 是什麼？

Vibe 直譯是“氛圍、感覺”。Vibe Coding 就是“跟著感覺指揮 AI 程式設計”——你描述想要什麼，AI 負責實現。記住這個詞，它是這場革命的“出生證明”。

從那一刻起，“編碼”這個詞的含義開始分裂：傳統的編碼是把設計翻譯成語法；新的編碼是把意圖翻譯成指令。前者考驗手指，後者考驗腦子。

Vibe Coding 這個詞很快從戲謔變成了嚴肅的工程概念。到 2026 年中期，它已經從一個社交媒體上的梗，演化為一套完整的方法論體系——涵蓋需求表達、上下文管理、迭代驗證到品質保障的全流程。人們也意識到：“憑感覺”只是入門階段的狀態，真正高效的 AI 程式設計需要一套系統化的思維方式。而這套思維方式，就是本書要給你的東西。

二、2026 年現狀：Agentic Engineering 時代

到了 2026 年 8 月，AI 程式設計領域已經經歷了幾次重要的正規化迭代。Vibe Coding 是最初的火花，但很快大家發現：光“憑感覺”和 AI 聊天，產出的程式碼品質極不穩定——有時

驚豔，有時翻車。於是正規化開始進化：從簡單的對話式程式設計，走向結構化的 Agent 工程。

又是 Karpathy，2026 年初提出了 Agentic Engineering 的概念。與 Vibe Coding 的“說了就生成”不同，Agentic Engineering 強調：給 AI 一個目標，讓它自主規劃步驟、呼叫工具、驗證結果、迭代修正。AI 不再只是一個程式碼生成器，而是一個能感知環境、做出決策、執行動作的智慧體（Agent）。

這個轉變帶來了三個關鍵變化：

1. 互動模式變了：從”一問一答”變成”設定目標後放權”。
2. 工作產物變了：從”一段程式碼”變成”一個完整的專案”。
3. 人的角色變了：從”逐行稽核程式碼”變成”定義邊界、驗收結果”。

數字不會說謊：Cursor 在 2026 年 4 月釋出的 Cursor 3 版本中，Agent 模式的使用率首次反超 Tab 補全模式，標誌著開發者已經大規模轉向這種工作方式；而 Claude Code 這個終端原生的 AI 程式設計工具，到 2026 年中期已達到年化收入 25 億美元的規模。下面的趨勢圖能讓你直觀感受這個交叉時刻：

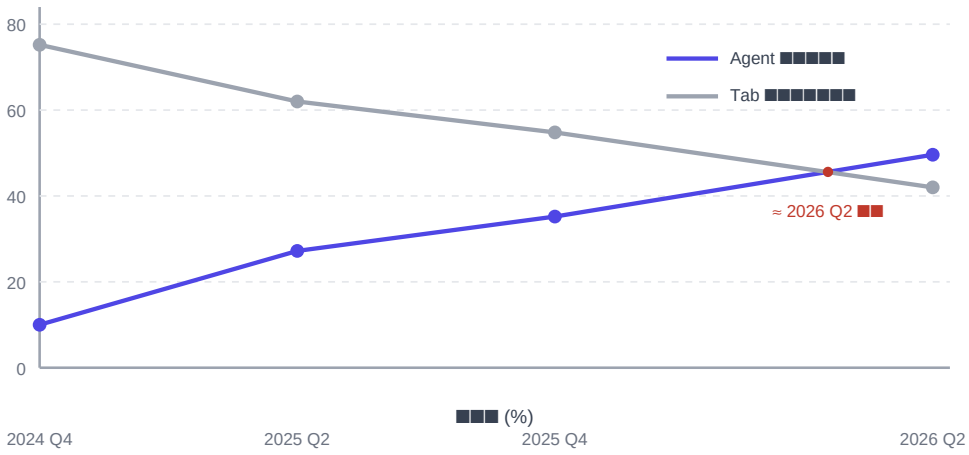


圖 1：AI 程式設計互動模式使用率變化趨勢（基於行業調研資料估算，2026 Q2 Agent 模式反超 Tab 補全）

但說句實在話：工具和模型的變化只是表象。無論你用 Cursor 還是 Claude Code，無論底層模型是 GPT-5 還是 Claude 4.5，成功的 AI 程式設計實踐者都在遵循同一套底層方法論。這就是本指南要教給你的東西。

三、核心認知轉變：從 Coder 到 Conductor

傳統程式設計教育教你如何成為一個 Coder——理解語法、記憶 API、除錯邏輯錯誤。AI 程式設計時代，這個角色正在被重新定義。你不再需要親手把每一行程式碼寫出來，但你需要成為一個“指揮官”（Conductor）：定義戰場、制定規則、分配任務、驗收成果。

這個轉變的核心不在於“你會不會寫程式碼”，而在於“你會不會想清楚要做什麼”。一個優秀的指揮官不一定是最好計程車兵，但他必須最清楚戰役的目標、地形和兵力部署。同理，一個優秀的 AI 程式設計實踐者不一定精通每一種程式語言的語法細節，但他必須最清楚專案的目標、架構約束和品質標準。

這張表值得你多看兩遍——它概括了這次角色轉變的全部內涵：

維度	傳統 Coder	AI 指揮官（Conductor）
核心技能	語法記憶、API 呼叫、除錯	需求定義、上下文管理、驗收判斷
工作方式	逐行編寫程式碼	描述意圖，審查產出
品質把控	自己寫的，自己心裡有數	透過測試和審查驗證 AI 產出
知識要求	深度掌握特定語言/框架	廣度理解技術全貌，深度理解業務邏輯
瓶頸所在	編碼速度和語法記憶	思考清晰度和表達能力
錯誤來源	語法錯誤、邏輯遺漏	需求模糊、上下文不足

傳統 Coder 與 AI 指揮官的能力維度對比

這絕不是說程式設計知識不重要了。恰恰相反：當你能看懂 AI 寫的程式碼時，你才能有效地審查和修正它；當你理解資料結構和演演算法時，你才能判斷 AI 的方案是否合理。知識儲備決定了你的“指揮上限”——但知識的角色從“執行工具”變成了“判斷依據”。

四、為什麼方法論比工具更重要

工具每三個月迭代一次，方法論每三年才進化一次。學會一個工具的操作，三個月後可能就過時了；掌握一套思維方式，三年後依然適用。

很多人在入門 AI 程式設計時，第一件事就是追問“哪個工具最好”“哪個模型最強”。這些問題不是不重要，但它們問錯了優先順序。2026 年 8 月的工具格局和 2025 年 8 月已經完全不同——Trae、Qoder 等工具興衰起伏，Cursor 從外掛變成了獨立 IDE 又進化到 Cursor 3，Claude Code 從實驗性專案變成了終端程式設計的主流選擇。但在這劇烈變化背後，成功實踐者遵循的思維原則幾乎沒有改變。

這就是為什麼本書不會把篇幅花在教你某個工具的具體操作上——那些內容寫完就會過時。相反，本書的全部重心放在一套經過驗證的思維方法論上：如何思考需求、如何與 AI 溝通、如何驗證產出、如何持續迭代。掌握了這套方法論，你換任何工具都能快速上手；只學了工具操作，換一個工具就要從零開始。

給零基礎讀者的定心丸

如果你是零基礎，別被“方法論”三個字嚇到。方法論不是抽象的哲學，它就是一套“遇到問題該怎麼想”的操作手冊。就像學做菜：你可以照著菜譜一步步來（工具操作），但真正的大廚理解的是“為什麼要這樣調味”“火候怎麼控制”（方法論）。本書會從最基礎的概念講起，每個原則都配具體的對比案例，確保你不僅理解“是什麼”，更能掌握“怎麼做”。

來動手：給現在的你拍張”角色照”

拿出一張紙，寫下三個問題的答案：(1) 你現在是怎麼用 AI 幫你寫程式碼的（如果用過的話）? (2) 你更像上表中的“Coder”還是“指揮官”? (3) 你希望三個月後的自己會怎樣描述自己的工作方式？寫完後夾在書裡，讀完第四十七章再回來看——你會發現變化比想象中快。

本章小結

- 2025 年 Karpathy 用”Vibe Coding” 命名了 AI 程式設計革命：從逐行寫程式碼，變成用自然語言描述意圖
- 2026 年進化到 Agentic Engineering：給目標、放權、AI 自主規劃執行，人負責驗收
- Cursor 3 的 Agent 模式使用率首次反超 Tab 補全；Claude Code 年化收入達 25 億美元
- 你的新角色是 Conductor（指揮官）：定義戰場、制定規則、分配任務、驗收成果
- 工具每三個月迭代一次，方法論每三年才進化一次——學方法論，不學操作

第三十八章：AI 程式設計五大思維原則

如果這本書只能讓你記住一章，那必須是這一章。五大思維原則是全書的心臟，後面所有章節——上下文、需求、對話、品質——都是這五條原則在不同場景下的展開。放心，每一條都不難懂，但每一條都值得你用幾個專案去“長進肌肉裡”。

故事的起點很有意思：Andrej Karpathy 在開源他的專案時，附了一份僅 70 行的 CLAUDE.md 檔案，總結了與 AI 協作的四條核心原則。這份檔案在 GitHub 上獲得了超過 70,000 個 Star，成為全球 AI 程式設計實踐者最廣泛引用的準則。這四條原則不是空洞的口號，而是從大量實踐中提煉出的、可操作的思維習慣。本書在其基礎上補充了第五條——迭代驗證——構成完整的五大原則體系。

70 行檔案為什麼這麼值錢？

70,000 個 Star 是什麼概念？GitHub 上絕大多數專案一輩子都到不了這個數字。一個 70 行的文字檔案能獲得它，恰恰證明瞭一件事：AI 時代最稀缺的不是程式碼，而是“想清楚”的能力。這 70 行，就是“想清楚”的濃縮版。

一、原則一：Think Before Coding（先思考，再編碼）

這是五條原則中最反直覺的一條。既然 AI 能幫我們寫程式碼，為什麼還要“先思考”？答案很簡單：AI 的產出品質與你的輸入品質正相關。你自己都沒想清楚要做什麼，AI 給出的答案只會讓你陷入無休止的“改了又改”迴圈——俗稱“AI 陪你一起迷路”。

“先思考”意味著在給 AI 下達指令之前，你需要回答三個問題，它們構成一個缺一不可的“思考三角”：

1. 我要解決什麼問題？（目標）

2. 我期望的結果長什麼樣？（驗收標準）

3. 有哪些約束條件需要遵守？（邊界）

思考維度	模糊輸入（差）	清晰輸入（好）
問題定義	幫我做個網站	幫我做一個個人工具站，展示 3 個 AI 小工具的入口卡片
期望結果	好看一點	響應式佈局，暗色主題，卡片有懸停動畫，移動端可用
約束條件	（未說明）	不使用框架，純 HTML+CSS+JS，單檔案部署，不需要後端

同一需求的模糊輸入與清晰輸入對比

注意：先思考 ≠ 先寫完美檔案

“先思考”不等於“寫一份完美的 PRD 再開始”。它是一個快速但有條理的思考過程——花 30 秒想清楚上面三個問題，遠比花 30 分鐘在對話中反覆修改來得高效。小任務三問可以很短；大專案可以把它們擴充套件成完整的需求檔案（詳見第四十章）。

二、原則二：Simplicity First（簡單至上）

AI 程式設計最大的陷阱不是“做不出來”，而是“做得太多”。當你對 AI 說“做一個使用者管理系統”，它可能會生成認證、權限、角色、審計日誌、資料匯出等一整套功能——但你現在可能只需要一個簡單的登入頁面。過度生成，是 AI 程式碼膨脹的主要原因。

“簡單至上”要求你在每次下達指令時都遵循“最小可行”原則：用最少的程式碼實現核心功能，用最簡單的方案解決問題，用最少的功能驗證想法。Karpathy 對此有個精闢的比喻：“如果你要計算一個折扣價格，你不需要一個電商引擎。”

在實踐中，“簡單至上”意味著幾個具體的操作習慣：

- 每次只讓 AI 實現一個功能點，而不是一口氣描述整個系統；
- 優先選擇標準庫和簡單方案，而不是引入複雜的相依性；
- 能用 50 行程式碼解決的問題，不要讓 AI 寫 500 行；
- 當 AI 的產出超出預期範圍時，勇敢地告訴它”砍掉多餘的部分”。

場景	過度方案（避免）	簡單方案（推薦）
需要儲存資料	搭建 MySQL 資料庫 + ORM	先用 JSON 檔案或 SQLite
需要使用者登入	OAuth2.0 + JWT + 角色權限	先做簡單的密碼驗證
需要前端框架	React + Redux + Router	先試純 HTML + 原生 JS
需要部署上線	Docker + CI/CD + K8s	先在本地跑通再說

常見場景的過度方案與簡單方案對比

三、原則三：Surgical Changes（精準修改）

當你在迭代一個已有專案時，每次修改都應該像外科手術一樣精準——只動該動的地方，不碰不該碰的。這一點在 AI 程式設計中尤其重要，因為 AI 有“順手改一下”的傾向：你讓它修一個 bug，它可能順便重構了半個檔案。

“精準修改”的核心要求是：每次給 AI 的指令，都明確指定修改的範圍和邊界。告訴它“只修改 calculate_discount 函式”“不要動其他部分”“保持現有的程式碼風格”。當 AI 的修改超出了你要求的範圍時，立即要求它回退多餘的部分。

Karpathy 用一個 diff（差異對比）的例子說明瞭這一點。假設你讓 AI 給購物車的折扣計算函式新增“會員價”邏輯：好的產出是隻修改這一個函式、增加幾行程式碼；壞的產出是把整個檔案重構了一遍——雖然新程式碼可能更“優雅”，但你無法確定它有沒有引入新 bug。在 AI 程式設計中，“看起來更好”永遠不能成為偏離指令的理由。

差的指令（範圍模糊）

```
幫我最佳化這個專案的 折扣計算邏輯
```

好的指令（精準指定範圍）

```
只修改 cart.py 中的 calculate\_discount 函式： 1. 在現有折扣基礎上增加會員價邏輯 2.
    會員使用者額外享受 10\% 折扣 3. 不要修改函式簽名和其他函式 4. 保持現有的變數命名風
    格
```

四、原則四：Goal-Driven Execution（目標驅動執行）

這條原則回答了一個關鍵問題：你應該給 AI 下達“怎麼做”的指令，還是“做什麼”的指令？Karpathy 的答案是後者。與其告訴 AI“先建立一個類，然後新增方法，然後寫測試”，不如告訴它“我需要一個能解析 CSV 檔案並統計每列唯一值數量的工具”。讓 AI 自己決定實現路徑，你負責驗收結果。

“目標驅動”的好處是雙重的。對 AI 來說，它獲得了更大的自主空間，可以選擇最優的實現路徑，而不是被你的中間步驟束縛；對你來說，你把注意力從“怎麼做”轉移到了“做出來對不對”——這正是指揮官該做的事。

注意：目標驅動 ≠ 放任不管

它需要你在 AI 執行後進行嚴格的驗收：結果是否滿足需求？邊界情況是否處理了？是否有明顯的過度設計？如果結果不符合預期，提供具體的反饋——不是“重寫”，而是“這裡的問題在於空列表輸入會報 IndexError，請修改為新增空值檢查”。

指令風格	示例	結果特點
命令式（怎麼做）	”先建立一個 Python 類，叫 DataParser，加一個 parse 方法，接收 filename 引數，用 csv 模組讀取…”	AI 變成打字員，你的中間步驟可能不是最優路徑
目標式（做什麼）	”我需要一個工具：讀取 CSV 檔案，統計每列的唯一值數量，輸出為 JSON 格式。請給出完整實現。”	AI 自主選擇最優路徑，你負責驗收結果

命令式指令與目標式指令的對比

五、原則五：Iterate and Verify（迭代驗證）

這是本書在 Karpathy 四原則基礎上補充的第五條。它的核心信念是：AI 的第一次產出幾乎永遠不是最終版本。有效的 AI 程式設計是一個“生成-驗證-修正”的迭代迴圈，而不是一次性的“指令-產出”。

“迭代驗證”不是被動的“不對就再試”，而是一個有結構的驗證流程。每次 AI 給出產出後，按三個層次進行驗證：

- 第一層・功能驗證：它能不能跑？輸入輸出對不對？
- 第二層・邊界驗證：空輸入、超長輸入、異常輸入會怎樣？
- 第三層・架構驗證：程式碼結構是否合理？有沒有硬編碼？是否便於後續擴充套件？

定義目標
AI 生成程式碼
功能驗證
透過
邊界驗證
透過
架構審查
透過，進入下一步
不透過
提供具體反饋
重新生成

圖 2：AI 程式設計的迭代驗證迴圈

這個迴圈的關鍵在於“提供具體反饋”這一步。當驗證不透過時，不要說“不對，重做”，而要說“函式在輸入空列表時報 IndexError，請在第 15 行新增空值檢查”。具體的反饋讓 AI 能精準修正；模糊的反饋只會讓它重新猜測你的意圖。

六、Harness Engineering：Agent = Model + Harness

2026 年中期，一個叫 Harness Engineering 的概念開始在 AI 程式設計社群流行。它的核心公式很簡單：

Agent 的能力 = 模型能力 × Harness 質量

模型能力是 AI 大腦的智力上限；Harness 是圍繞模型搭建的“工作環境”——包括上下文管理、工具整合、流程編排、記憶系統等。

這個概念解釋了一個常見的困惑：為什麼同樣用 GPT-4 或 Claude，有的人能做出驚豔的專案，有的人只能得到平庸的程式碼？區別不在模型，而在 Harness。一個好的 Harness 包括：清晰的專案規範檔案、合理的工具整合、有條理的對話上下文、完善的驗證流程。模型決定了智力上限，Harness 決定了你能多大程度地發揮這個上限。

天才實習生比喻

你可以把模型想象成一個天才實習生——智商很高，但如果不給他專案背景檔案、不告訴他用哪些工具、不給他設定期限和驗收標準，他也做不出好東西。Harness Engineering 就是“如何給天才實習生搭好工作環境”的工程。

本書的後續章節，本質上都在教你如何構建一個好的 Harness：第三十九章教你管理上下文（專案記憶系統），第四十章教你定義需求（PRD），第四十三章教你對話技術（Prompt 分層），第四十五章教你品質保障（驗證流程）。這些組合起來，就是你的 Harness。

來動手：用” 思考三角” 拆解你的下一個想法

挑一個你一直想做但沒動手的小東西（比如一個倒數日網頁、一個喝水提醒指令碼）。花 3 分鐘回答：(1) 它解決什麼問題？(2) 做出來長什麼樣？(3) 有什麼約束（時間、技術、環境）？然後把答案寫成一段話。恭喜你，你已經完成了人生第一次“Think Before Coding”——這段話就是一條合格的高品質指令。

本章小結

- 五大原則源自 Karpathy 70 行的 CLAUDE.md（70K+ Star），本書補充第五條” 迭代驗證”
- Think Before Coding：開口前回答三問——做什麼？結果長啥樣？有什麼約束？
- Simplicity First：AI 最大的坑是” 做得太多”——算折扣不需要電商引擎
- Surgical Changes：像外科手術一樣精準修改，拒絕 AI 的” 順手改一下”
- Goal-Driven Execution：說” 做什麼”，不說” 怎麼做”，讓 AI 自主選路徑
- Iterate and Verify：三層驗證（功能/邊界/架構）+ 具體反饋，迴圈迭代

- Harness Engineering：能力 = 模型 × Harness，把 AI 當” 天才實習生” 來管理環境

第三十九章：上下文工程——讓 AI 理解你的意圖

你有沒有過這種體驗：換個新對話視窗，*AI* 就像失憶了一樣，把你專案的基本情況全忘了？這不是 *AI* 故意氣你，而是它天生“健忘”。這一章教你給 *AI* 裝上一套“長期記憶系統”，順便認識 *2026* 年最火的開發正規化之一——規範驅動開發。

一、專案記憶檔案：AI 的“長期記憶”

AI 模型有一個根本性限制：它的記憶是“短時”的。每次對話開始時，AI 對你專案的瞭解為零。你需要在對話中不斷告訴它“我們用的是 React”“後端是 Node.js”“資料庫是 PostgreSQL”“程式碼風格用雙引號”。這不僅低效，而且容易遺漏——就像每天都要重新給新來的同事做入職訓練。

解決方案是“專案記憶檔案”——一個放在專案根目錄的純文字檔案，AI 工具會在每次對話開始時自動讀取它。不同的工具有不同的命名約定：

工具	檔名
Cursor	`.cursorrules`
Claude Code	`CLAUDE.md`
GitHub Copilot	`copilot-instructions.md`
通用方案	`AGENTS.md`

各工具的專案記憶檔案命名約定

雖然命名不同，但原理一樣：把你希望 AI“始終記住”的資訊寫在這個檔案裡。一個好的專案記憶檔案應該包含：專案概述（這是什麼專案）、技術棧（用了什麼技術）、程式碼規範（命名、縮排、註釋風格）、目錄結構（程式碼怎麼組織的）、開發約定（分支策略、提交規範、測試要求）。Karpathy 的 CLAUDE.md 只有 70 行，但覆蓋了所有關鍵資訊——簡潔但完備。

```
# 專案：個人記帳應用
## 概述
這是一個全棧個人記帳應用，幫助使用者記錄日常收支。
## 技術棧
- 前端：React + TypeScript + Tailwind CSS
```

```
- 後端：Node.js + Express + TypeScript
- 資料庫：SQLite（開發）/ PostgreSQL（生產）
- 測試：Vitest
## 程式碼規範
- 使用雙引號
- 縮排2個空格
- 函式名用camelCase
- 型別定義用PascalCase
- 每個函式必須有JSDoc註釋
## 目錄結構
/src
  /frontend  - 前端程式碼
  /backend   - 後端程式碼
  /shared    - 前後端共享的型別
  /tests     - 測試檔案
## 開發約定
- 提交資訊用中文
- 新功能必須先寫測試
- API返回統一用 { success, data, error } 格式
```

注意：別寫成”第二份需求檔案”

注意這個檔案的長度——不要寫成幾十頁的需求檔案。專案記憶檔案的目的是“讓 AI 快速建立背景認知”，不是“詳細描述每個功能”。保持簡潔、結構化、只寫“AI 需要知道的”。

二、規範驅動開發（SDD）

Spec-Driven Development（SDD，規範驅動開發）是 2026 年逐漸成熟的一種開發正規化，核心理念一句話：先寫規範，再寫程式碼。與傳統 TDD（測試驅動開發）不同的是，SDD 的“規範”不僅是測試用例，還包括需求描述、介面定義、資料模型等。AI 根據這些規範來生成程式碼，人類的職責是編寫和維護規範。

SDD 的工作流程五步走：

1. 用自然語言描述需求規範（寫在一個.md 檔案中）；
2. 定義介面和資料結構（寫在規範檔案中）；
3. 讓 AI 根據規範生成程式碼實現；
4. 執行測試驗證產出是否符合規範；
5. 如果不符合，修正規範或讓 AI 修正程式碼。

在這個流程中，規範檔案是“唯一真理源”——程式碼可以重新生成，但規範必須先正確。

編寫規範檔案

審查規範是否完備？

完備

AI 根據規範生成程式碼

執行測試驗證產出

不完備 / 測試不過

程式碼問題修正程式碼；規範問題修正規範

回到驗證

全部透過交付

圖 3：規範驅動開發（SDD）的工作流程

SDD 的優勢在於：規範檔案比程式碼更穩定。當底層模型或工具變化時，你的規範檔案依然有效，只需要讓新的 AI 工具重新生成程式碼即可。這也是為什麼本書強調“方法論比工具重要”——規範檔案就是你方法論的具體載體。

三、上下文視窗管理

即使有了專案記憶檔案，你仍然會在對話中遇到“上下文不夠用”的問題。AI 模型的上下文視窗（能同時處理的資訊量）是有限的。當對話太長、引用的檔案太多時，AI 會“忘記”早期的資訊，導致產出品質下降——就像你和一個朋友聊天聊了六個小時，他早忘了你開頭說的第一件事。

管理上下文視窗的核心策略是“分層引用”：

上下文層次	內容	管理方式
第一層：專案記憶	技術棧、規範、目錄結構	寫在 CLAUDE.md 等檔案中，自動載入，無需重複
第二層：檔案引用	當前任務涉及的程式碼檔案	用 @ 符號精確引用，不貼上全文到對話
第三層：對話上下文	當前任務的對話歷史	完成一個任務後開新對話，避免上下文膨脹
第四層：即時指令	當前這一步的具體要求	清晰、簡潔、有針對性，避免冗長描述

上下文視窗的分層管理策略

四、上下文工程的實踐要點

上下文工程不是一次性的設定，而是一個持續最佳化的過程。以下是幾個在實踐中被反覆驗證的要點：

1. 記憶檔案隨專案演進同步更新。引入新技術或改變架構時，第一時間更新記憶檔案，而不是在對話中臨時說明。

2. 善用 @ 引用而非全文貼上。大多數 AI 程式設計工具支援 @file 或 @folder 語法，它會在後臺將檔案內容注入 AI 的上下文，比手動貼上更高效且不會汙染對話。

3. 對話要” 短小精悍”。一個對話只處理一個任務或一個功能點。完成後開新對話處理下一個。長對話不僅消耗上下文視窗，還會讓 AI” 分心”。

4. 給 AI” 看” 結果而非” 描述” 結果。遇到 bug 時，把錯誤日誌直接 @ 引用給 AI，而不是用自己的話描述” 好像有個什麼錯誤”。AI 看到原始資訊比看到你的轉述更有效。

5. 定期清理和歸檔。已完成的對話和臨時檔案會佔用工作空間。定期清理無關內容，保持專案目錄整潔，也有助於 AI 更高效地理解專案結構。

來動手：給你的專案寫一份 70 行記憶檔案

挑一個你正在做（或打算做）的專案，寫一份 CLAUDE.md：專案概述（2-3 句話）、技術棧（列出來）、程式碼規範（3-5 條）、目錄結構（畫個樹）、開發約定（2-3 條）。控制在 70 行以內——寫完你會發現自己對專案的理解都清晰了不少。

本章小結

- AI 的記憶是” 短時” 的：每次新對話，它對專案的瞭解為零
- 專案記憶檔案（CLAUDE.md 等）是 AI 的” 長期記憶”，每次對話自動載入
- 好的記憶檔案 = 概述 + 技術棧 + 規範 + 目錄 + 約定，70 行足矣
- SDD 規範驅動開發：先寫規範再寫程式碼，規範是” 唯一真理源”，比程式碼更穩定
- 上下文分四層管理：專案記憶 @ 檔案引用對話歷史即時指令
- 五個實踐要點：同步更新、@ 引用、短對話、給原資訊、定期清理

第四十章：需求工程——把想法變成 AI 可執行的指令

如果說上下文工程解決的是“讓 *AI* 瞭解背景”，那需求工程解決的是“讓 *AI* 知道這次具體要做什麼”。這是 *AI* 程式設計中最能體現人與人差距的環節——同樣的工具、同樣的模型，有人能做出令人驚豔的專案，有人只能做出平庸的玩具。區別往往不在技術能力，而在需求表達能力。

一、從模糊想法到結構化需求

每個專案都始於一個模糊的想法——“我想做一個記帳 App”“我想做個個人網站”“我想做個待辦清單工具”。這些想法本身沒有問題，但如果你直接把它丟給 AI，產出的品質完全取決於運氣。需求工程的目標，就是把這個模糊的想法，變成一個 AI 能精確執行的結構化需求。

好訊息是：結構化需求不需要你寫一份幾十頁的 PRD。它只需要你回答五個問題，我們稱之為“需求五要素”：

要素	問題	示例：記帳 App
Who	給誰用？	個人使用，記錄日常收支
What	做什麼？	一個能記錄每筆收支、按月彙總、視覺化的應用
Why	為什麼做？	想了解自己的消費結構，找出不必要的支出
How	大致怎麼做？	前端用 React，資料存瀏覽器本地，先不做後端和登入
Constraint	有什麼限制？	單人開發，2 周內完成，不需要上線部署

需求五要素及示例

這五個要素不需要寫成正式檔案，甚至在腦海中快速過一遍就足夠了。關鍵是養成“先想清楚再開口”的習慣。你會發現，當你能清晰地回答這五個問題時，你的需求描述自然就變得結構化了。

二、需求拆解的分層藝術

對於較大的專案，一個需求描述是不夠的。你需要把大需求拆解成小的、可獨立驗證的子需求。這個過程叫做“需求拆解”，核心原則是：每個子需求都應該能獨立完成、獨立驗證、獨立交付。

以“做一個個人記帳 App”為例，一個合理的拆解方式是：

1. 實現基礎的收支記錄功能（能新增、檢視、刪除記錄）；
2. 新增分類功能（給每筆記錄打上餐飲、交通等標籤）；
3. 實現月度彙總功能（按月統計收支情況）；
4. 新增資料視覺化（用圖表展示消費結構）。

每一步都是一個可以獨立執行和驗證的功能增量。

需求拆解的黃金法則：每一步完成後，專案都應該是一個” 能用的東西”——哪怕功能很簡陋。永遠不要讓專案停留在” 什麼都做了一半但什麼都用不了” 的狀態。

這種拆解方式有兩個好處。首先，它讓你始終能看到進展——每完成一步，你的 App 就多了一個功能，這種正反饋對於長期專案至關重要（人都是需要成就感的嘛）。其次，它讓 AI 的每次任務都足夠小，降低了“AI 理解偏差”的風險——任務越小，AI 的理解越準確，產出品質越高。

三、PRD 寫作模板與實踐

對於較大的專案（比如一個有前後端、多個功能模組的全棧應用），僅憑需求五要素可能不夠。這時你需要寫一份更正式的專案需求檔案（PRD）。一個容易誤解的點是：PRD 不是寫給 AI 看的——它是寫給你自己的，幫你理清思路，然後你才能給 AI 提供更精確的指令。

一份好的 PRD 不需要很長，但應該包含六個部分：專案概述（一段話說清楚做什麼）、功能列表（每個功能點是什麼）、技術方案（用什麼技術棧、大致架構）、資料模型（核心資料結構）、介面描述（主要頁面長什麼樣）、開發計劃（分幾步完成）。

```
# 個人記帳應用 PRD
## 專案概述
一個輕量級個人記帳應用，支援收支記錄、分類、月度彙總和資料視覺化。
## 功能列表
1. 記錄收支：金額、型別(收入/支出)、分類、備註、日期
2. 分類管理：預設餐飲、交通、購物、娛樂、其他
3. 月度彙總：按月統計總收入、總支出、結餘
4. 消費視覺化：餅圖展示各分類佔比
5. 資料匯出：匯出為CSV
## 技術方案
- 前端：React + TypeScript + Tailwind CSS
- 資料儲存：localStorage（無需後端）
```

```
- 圖表：使用Chart.js
- 構建：Vite
## 資料模型
Record {
  id: string
  amount: number
  type: 'income' | 'expense'
  category: string
  note: string
  date: string  // YYYY-MM-DD
}
## 開發計劃
Step 1: 基礎記錄功能（CRUD）
Step 2: 分類管理
Step 3: 月度彙總
Step 4: 資料視覺化
Step 5: 資料匯出
```

注意這份 PRD 的寫作風格：用簡潔的列表和結構化的格式，而不是長篇大論。每個功能點都有明確的定義，資料模型用簡單的型別描述。這種風格的 PRD 不僅你自己看得清楚，當你把它提供給 AI 時，AI 也能精確理解每個需求點。

四、實戰：從想法到方案的完整推演

紙上談兵沒意思，咱們用一個完整的例子走一遍需求工程的全過程。假設你的想法是：“我想做一個能追蹤我每天學了什麼的學習追蹤器”。

第一步，過一遍需求五要素。

- Who：個人使用。
- What：一個能記錄每天學習內容、追蹤學習時長、回顧學習進度的工具。
- Why：想養成持續學習的習慣，看到自己的積累。
- How：先做一個網頁版，資料存在瀏覽器本地。
- Constraint：零基礎開發，1 周內完成核心功能。

第二步，拆解需求。

1. Step 1：實現” 記錄學習條目” 功能（日期、內容、時長）；
2. Step 2：新增” 學習時長統計”（今日、本週累計）；
3. Step 3：實現” 學習日曆” 檢視（像 GitHub 貢獻圖一樣展示）；
4. Step 4：新增” 學習回顧” 功能（隨機展示過去某天的學習記錄，幫助複習）。

第三步，寫一份簡短 PRD。不需要長——上面的拆解本身就構成了 PRD 的主體。把它整理成一個.md 檔案，放在專案根目錄，作為你和 AI 的共同參考。

第四步，開始第一步的開發。給 AI 的指令可以是：

```
我需要一個學習追蹤器的第一個功能——記錄學習條目。
使用者可以新增一條學習記錄，包含日期、學習內容、學習時長（分鐘）。
資料存在localStorage。
前端用React + TypeScript。
請給出完整實現。
```

注意這個指令的特點：它只描述當前這一步的目標（記錄功能），不摻雜其他功能；它明確了技術約束（React、TypeScript、localStorage）；它給了足夠的資訊讓 AI 知道“做什麼”和“怎麼做”，但具體實現由 AI 決定。這就是一個經過需求工程加工後的、高品質的 AI 程式設計指令。

來動手：給你的”心頭好”寫一份五要素卡

選一個你醞釀已久的小專案，花 5 分鐘填一張卡：Who / What / Why / How / Constraint，各一句話。然後把它拆成 3-5 個“每一步都能用”的步驟。你會發現：把想法寫下來之後，它就從“白日夢”變成了“待辦清單”。

本章小結

- 需求五要素：Who / What / Why / How / Constraint——想清楚再開口
- 需求拆解黃金法則：每一步完成後專案都是”能用的東西”
- PRD 是寫給自己的：概述、功能列表、技術方案、資料模型、介面、計劃
- 好指令 = 只描述當前一步 + 明確約束 + 說”做什麼”不說”怎麼做”
- 完整推演模板：五要素拆解 PRD 給指令，四步走

第四十一章：2026 年 AI 程式設計工具全景

先打個預防針：本章是 *2026* 年 *8* 月工具生態的一張“快照”。半年後它就會過時——這恰恰是本章想教會你的第一課。本章的目的不是教你“用哪個工具”，而是幫你建立“如何選擇工具”的判斷框架。當你理解了不同工具的設計哲學，你就能在任何時候做出適合自己的選擇。

一、三大流派與選擇框架

2026 年的 AI 程式設計工具大致分為三大流派，每種流派背後是一種不同的設計哲學。先看這張全景圖：

流派	代表工具	設計哲學	適合場景
IDE 內建 AI	Cursor 3, Trae, Windsurf, CodeBuddy	把 AI 能力深度整合到編輯器中	日常開發、程式碼審查、漸進式增強
終端 Agent	Claude Code, Codex CLI, Aider	以終端為介面，AI 直接操作檔案系統和命令	專案級操作、批次修改、CI/CD 整合
雲端 Builder	v0, Bolt, Replit Agent	在瀏覽器中從零建立完整專案	快速原型、零配置上手、即時預覽

AI 程式設計工具的三大流派

選擇工具的關鍵不是“哪個最好”，而是“哪個最適合你當前的場景”。三個實用的判斷維度：

- 你的程式設計基礎如何？零基礎選雲端 Builder，有基礎選 IDE 內建 AI，進階使用者選終端 Agent。
- 你的專案階段在哪裡？從零開始選雲端 Builder，迭代開發選 IDE 內建 AI，維護重構選終端 Agent。
- 你的部署需求是什麼？快速驗證選雲端 Builder，嚴肅專案選 IDE 內建 AI，自動化流程選終端 Agent。

二、IDE 內建 AI 流派

IDE 內建 AI 是最主流的流派：把 AI 能力嵌入到你的程式碼編輯器裡，在保留傳統程式設計體驗的同時疊加 AI 增強。2026 年這個流派的代表是 Cursor 3、Trae、Windsurf 和 CodeBuddy。

Cursor 3

Cursor 是目前全球使用最廣泛的 AI 程式設計 IDE。2026 年 4 月釋出的 Cursor 3 標誌著 AI 程式設計工具的一個重要轉折點：Agent 模式的使用率首次超過 Tab 補全模式——開發者從“AI 幫我補全程式碼”轉向了“AI 幫我完成任務”。Cursor 3 的核心特性包括：多檔案編輯（AI 可以同時修改多個檔案來完成任務）、上下文感知（自動理解專案結構和相依關係）、與終端的深度整合（AI 可以在編輯器內直接執行命令和檢視輸出）。

Trae

Trae 是位元組跳動推出的 AI 程式設計 IDE，在國內市場有較大的使用者基數。它的 SOLO 模式（也稱為 Builder 模式）是典型的“一句話建立專案”能力——你描述需求，AI 自主完成從專案初始化到程式碼生成的全流程。Trae 的優勢在於中文支援和國內網路環境適配，適合國內使用者快速上手。

其他工具

Windsurf（原 Codeium）以流暢的程式碼補全體驗著稱，適合追求“打字如飛”體驗的開發者。騰訊雲 CodeBuddy 作為 VS Code 外掛，擅長與雲生態整合，適合計劃部署到騰訊雲或開發微信小程式的專案。阿里通義靈碼和百度 Comate 也提供了類似的 IDE 整合體驗，各自在中文場景最佳化上有特色。

三、終端 Agent 流派

終端 Agent 是 2026 年增長最快的流派。核心理念：AI 不應該只是一個編輯器外掛，它應該能像開發者一樣操作整個開發環境——讀寫檔案、執行命令、檢視輸出、安裝依賴。Claude Code 是這個流派的代表。

Claude Code

Claude Code 是 Anthropic 推出的終端 AI 程式設計工具，到 2026 年中期已達到年化收入 25 億美元的規模。它的工作方式與傳統 IDE 完全不同：你在終端中用自然語言描述任務，Claude Code 會自主讀取相關檔案、修改程式碼、執行測試、檢視結果、迭代修正——整個過程不需要你開啟編輯器。

Claude Code 的優勢在於“全專案感知”：它不受編輯器視窗限制，可以自由地瀏覽整個專案目錄，理解檔案間的相依關係。它也支援 CLAUDE.md 專案記憶檔案，讓 AI 在每次會話開

始時就瞭解專案背景。對於大型專案的重構、批次修改、測試編寫等任務，終端 Agent 比 IDE 外掛更高效。

Codex CLI 與 Aider

OpenAI 的 Codex CLI 提供了類似的終端程式設計體驗，並支援 /import 功能——可以匯入現有的程式碼庫讓 AI 理解。Aider 是一個開源的終端 AI 程式設計工具，支援多種模型（GPT、Claude、DeepSeek 等），適合想要靈活切換模型的使用者。這些工具雖然各有特色，但核心理念一致：讓 AI 在終端中擁有完整的開發環境操作能力。

四、雲端 Builder 流派

雲端 Builder 是最適合零基礎使用者的流派。你不需要安裝任何軟體，只需要開啟瀏覽器，描述你的需求，AI 就會在雲端建立完整的專案，並提供即時預覽。

- v0（Vercel 出品）：擅長生成前端 UI 元件和頁面，適合快速製作網頁原型。
- Bolt：支援從一句話建立全棧應用，包含前端、後端和資料庫。
- Replit Agent：在 Replit 的雲端 IDE 中工作，不僅能建立專案，還能直接在雲端執行和部署。

這些工具的共同優勢是“零配置上手”——開啟瀏覽器就能開始。侷限在於：當專案變複雜後，雲端編輯環境的靈活性不如本地 IDE。因此它們最適合“從 0 到 1”的原型階段：用雲端 Builder 快速驗證想法，驗證透過後再遷移到本地 IDE 進行深度開發。

五、工具會變，方法論不變

如果明天所有 AI 程式設計工具都消失了，你仍然應該保持的三個習慣：先想清楚再動手、每次只做一小步、做完就驗證。工具是放大器，方法論是地基。放大器可以替換，地基不能動搖。

回顧 2024 到 2026 年的工具變遷：2024 年 GitHub Copilot 是王者，2025 年 Cursor 取而代之，2026 年 Claude Code 異軍突起。每一個新工具出現時，都有人宣稱“這次不一樣”，但最終發現：用好新工具的人，恰恰是那些方法論紮實的人。他們換工具只需要幾天適應期，而只學了工具操作的人換一次就要重新學起。

因此，本書建議的學習路徑是：先用一個工具（任何一個都行）入門，在實踐中體會五大思維原則；當你的方法論足夠紮實後，再嘗試不同流派的工具，感受它們的設計哲學差異；最終，你會形成自己的工具組合——比如用 Cursor 做日常開發，用 Claude Code 做重構和測試，用 v0 做快速原型。工具組合因人而異，但方法論是共通的。

來動手：給你自己配一套”兵器組合”

用三個判斷維度（程式設計基礎、專案階段、部署需求）給自己定位：如果你是零基礎試試 Bolt 或 v0 做一個頁面；有基礎用 Cursor 或 Trae 做一個迭代任務；想挑戰在終端裡跑一次 Claude Code。然後回答：哪種流派最符合你當前的階段？

本章小結

- 三大流派：IDE 內建 AI（日常開發）、終端 Agent（專案級操作）、雲端 Builder（零基礎原型）
- 選工具看三問：程式設計基礎？專案階段？部署需求？
- Cursor 3 的 Agent 模式首次反超 Tab 補全；Claude Code 年化收入 25 億美元
- 雲端 Builder 適合”從 0 到 1”，驗證後遷回本地 IDE 深度開發
- 工具每三個月迭代一次：學方法論的人換工具只需幾天適應期

第四十二章：大型語言模型全景與 API 實戰

前面幾章講的都是“怎麼和 *AI* 協作”，本章解決一個更基礎的問題：“用哪個 *AI*？怎麼用？”*2026* 年 *8* 月的大型語言模型格局已經極其豐富——國際有 *GPT*、*Claude*、*Gemini* 三強爭霸，國內有 *DeepSeek*、通義千問、*GLM*、*Kimi*、豆包百花齊放。每個模型都有自己的性格和特長：選對了事半功倍，選錯了事倍功半。本章不僅帶你認識這些模型，還會教你如何透過 *API* 將它們接入自己的應用，以及如何根據自己的需求選擇合適的 *Token Plan*（用量方案）。

一、國際主流大型語言模型：三強格局

2026 年 8 月的國際大型語言模型市場，形成了 OpenAI（GPT 系列）、Anthropic（Claude 系列）、Google（Gemini 系列）三強鼎立的格局。每家都有自己鮮明的特色——咱們一個一個認識。

OpenAI GPT-5.6 系列：全能均衡的六邊形戰士

OpenAI 是大型語言模型浪潮的開啟者，GPT 系列至今仍是全球使用最廣泛的模型。2026 年 7 月釋出的 GPT-5.6 系列採用三檔定位策略：旗艦款 GPT-5.6 Sol 面向進階推理需求，均衡款 GPT-5.6 Terra 面向日常開發，輕量款 GPT-5.6 Luna 面向高併發低成本場景。三款模型共享 105 萬 Token 的上下文視窗，都支援影像輸入、函式呼叫和結構化輸出。

型號	定位	輸入 (/百萬 Token)	輸出 (/百萬 Token)	上下文視窗
GPT-5.6 Sol	旗艦推理	US$5.00 (約 36 元)	US$30.00 (約 217 元)	105 萬 Token
GPT-5.6 Terra	均衡日常	US$2.50 (約 18 元)	US$15.00 (約 109 元)	105 萬 Token
GPT-5.6 Luna	輕量低成本	US$1.00 (約 7 元)	US$6.00 (約 44 元)	105 萬 Token

OpenAI GPT-5.6 系列定價（2026 年 8 月）

GPT 系列的核心優勢在於“均衡”——推理、程式設計、寫作、多模態，沒有明顯短板。加上 OpenAI 龐大的外掛生態和開發者社群，適配檔案和工具鏈最為完善。對於“不知道選什麼”的場景，GPT 是“閉眼選都不會錯”的預設答案。但它的短板是：國內訪問需要特殊網路，中文表達偶爾有“翻譯腔”，API 價格處於中高水準。

好訊息

OpenAI 在 2026 年 8 月宣佈免費使用者預設模型升級為 GPT-5.6 Luna，並開放無限量文字聊天。這意味著你可以零成本體驗 GPT 的基礎能力，只有在需要更強推理或 API 呼叫時才需要付費。

Anthropic Claude 系列：程式設計與嚴謹的天花板

如果說 GPT 是全能選手，那 Claude 就是程式設計領域的“天花板”。Anthropic 的 Claude 系列在 SWE-bench（軟體工程基準測試）上長期斷層領先，程式碼生成品質和工程級專案落地能力業界第一。更難得的是，Claude 的幻覺率（生成虛假資訊的機率）全網最低，僅約 1%——這使它特別適合學術、法務等高精度場景。

Claude 的另一個殺手鐧是超長上下文：支援 100 萬 Token 的上下文視窗，能一次性“讀完”幾百頁的合同或整個程式碼庫不漏細節。這在程式碼重構、長檔案分析等場景中幾乎無可替代。Claude Code 作為終端 AI 程式設計工具，到 2026 年中期年化收入已達 25 億美元——這是 Claude 程式設計能力最受認可的證據。

型號	定位	輸入 (/百萬 Token)	輸出 (/百萬 Token)	上下文視窗
Claude Fable 5	旗艦款	US$10.00 (約 72 元)	US$50.00 (約 362 元)	100 萬 Token
Claude Opus 4.8	專業款	US$5.00 (約 36 元)	US$25.00 (約 181 元)	100 萬 Token
Claude Sonnet 5	價效比款	US$2.00 (約 14 元)	US$10.00 (約 72 元)	100 萬 Token

Anthropic Claude 系列定價（2026 年 8 月，Sonnet 5 為優惠價）

Claude 的短板是價格——Fable 5 的輸出價格是所有模型中最高的，每百萬 Token 要 50 美元。多模態能力（影像、影片理解）也相對較弱。對於預算有限的個人開發者，Sonnet 5 在優惠期內的價效比非常突出。

Google Gemini 系列：多模態的王者

Google 的 Gemini 系列在 2026 年已迭代至 3.1 Pro 版本。Gemini 的最大殺手鐧是原生多模態——圖文解析精準、OCR 表格識別無誤、影片幀提取高效，還支援音影片深度解析。它與 Google 生態（搜尋、信箱、雲服務）深度繫結，在實時資訊檢索和跨模態理解上表現突出。Gemini 還支援 200 萬 Token 的超長上下文輸入，是目前上下文視窗最大的模型。

Gemini 的定位是“科學推理和多模態場景的首選”。如果你的專案涉及大量影像分析、影片理解或科學計算，Gemini 幾乎沒有對手。但它在純文字程式碼生成上略遜於 Claude，中文表達也不如國產模型自然。

對比項	GPT-5.6	Claude Fable 5	Gemini 3.1 Pro
綜合得分	94.8	95.0	92.1
程式設計能力 (SWE-bench)	74.9%	64.3% (Pro)	中等
上下文視窗	105 萬 Token	100 萬 Token	200 萬 Token
多模態能力	均衡	較弱	最強
幻覺率	較低	最低 (約 1%)	較低
國內可用性	需特殊網路	需特殊網路	需特殊網路
API 生態	最完善	成長中	Google 生態繫結

國際三大型語言模型橫向對比（2026 年 8 月）

注意：國內使用的現實門檻

對於國內開發者來說，GPT、Claude、Gemini 的核心障礙是網路訪問。雖然可以透過 Azure 代理、API 聚合平臺等方式間接使用，但穩定性和合規性需要自行評估。如果你的專案對資料安全有嚴格要求，建議優先考慮國產模型。

二、國產主流大型語言模型：百花齊放

2026 年的國產大型語言模型已經從“追趕者”變成了“並跑者”，在某些領域甚至實現了“領跑”。與 2023 年“套殼”爭議不同，2026 年的國產旗艦模型在邏輯推理、程式碼生成、中文理解等方面已經與國際一線模型旗鼓相當，差距縮減到 5% 以內。更重要的是，國產模型在中文場景、價格成本和合規部署上有不可替代的優勢。

DeepSeek V4：價效比之王

DeepSeek（深度求索）是國產大型語言模型中現象級的存在。2025 年初，DeepSeek-V3 以“比 GPT 便宜十分之一但能力相當”的定位震撼市場，被業內稱為“通用 AI 大型語言模型斬殺線”——比它便宜的沒它好用，比它貴的沒它量大管飽。到 2026 年 8 月，DeepSeek V4-Pro 正式版釋出：1.6 萬億引數 MoE 架構，啟用約 49B，支援 100 萬無損上下文，程式碼價效比被廣泛認為是全球天花板。

DeepSeek V4 在 Agent 能力上實現了大幅躍升——DeepSWE 基準從 12.8 跳至 62.7（+390%），NL2Repo 從 38.5 升至 61.5。API 新增了 Responses API 與 Anthropic API 雙協議支援，已相容 Codex 和 Claude Code 等主流程式設計工具。這意味著你可以用 DeepSeek 的 API 替代 Claude 的 API，在 Claude Code 中使用——成本降低數倍，體驗幾乎一致。

型號	定位	輸入 (/百萬 Token)	輸出 (/百萬 Token)	上下文視窗
DeepSeek V4 Pro	旗艦推理	3 元 (約 US$0.43)	6 元 (約 US$0.87)	100 萬 Token
DeepSeek V4 Flash	輕量快速	1 元 (約 US$0.14)	2 元 (約 US$0.28)	100 萬 Token

DeepSeek V4 系列定價（2026 年 8 月，促銷價）

價格對比：同樣 100 萬 Token 的輸入 + 100 萬 Token 的輸出，DeepSeek V4 Flash 花費約 0.42 美元，GPT-5.6 Sol 花費約 35 美元，Claude Fable 5 花費約 60 美元。差距高達 80-140 倍。

避坑提醒：DeepSeek 要漲價了

DeepSeek 官方已於 2026 年 8 月 6 日公告“計劃近期大幅上調 API 定價”，促銷價可能不會持續太久。部分媒體傳聞漲幅可能達 30 倍，截至 8 月 14 日官方尚未公佈最終調價方案。此外，DeepSeek V4 正式版引入了峰谷定價機制——高峰時段呼叫價格直接翻倍。建議關注官方公告獲取最新定價。

通義千問 Qwen3.7 Max：阿里雲的企業級選擇

阿里雲的通義千問（Qwen）系列依託阿里雲生態，在企業級部署上有天然優勢。2026 年 7 月的 Qwen3.7 Max 是當前旗艦：235B 引數稠密模型，256K 原生上下文，可擴充套件至 1M。Qwen 的核心優勢在於：開源生態成熟（從 7B 到 235B 全系列開源）、中文場景理解精準、與阿里雲基礎設施深度整合。

型號	輸出價格 (/百萬 Token)	適用場景
qwen-turbo	0.6 元	極簡問答、文字分類、高併發
qwen-long	2.0 元	10M 超長上下文處理
qwen3.7-flash	7.2 元	通用對話、內容生成
qwen3.7-plus	12.0 元	多模態（圖片 + 影片）、中文創作
qwen3.7-max	36.0 元	旗艦中文能力、複雜任務

通義千問 Qwen 系列階梯定價（2026 年 8 月）

Qwen 的定價採用階梯策略——從極低成本的 qwen-turbo（輸出 0.6 元/百萬 Token）到旗艦 qwen3.7-max（輸出 36 元/百萬 Token），覆蓋從輕量問答到複雜推理的全場景需求。對於已經在使用阿里雲的企業，Qwen 是最順手的整合選擇。

智譜 GLM-5.1：國產 SWE-bench 第一

智譜 AI 的 GLM 系列在 2026 年表現亮眼。GLM-5.1 在 SWE-bench 上拿下國產第一的成績：744B MoE 稠密混合架構，支援 100 萬上下文。它的特點是中文語義理解精準，支援國

內合規私有化部署——這對資料安全要求高的金融、政務場景非常重要。GLM-5.1 的定價為輸入 6 元、輸出 24 元每百萬 Token，處於國產模型中游水準。

Kimi K3：超長文字之王

月之暗面的 Kimi 系列以超長文書處理能力著稱。2026 年 7 月釋出的 Kimi K3 擁有 2.8 萬億引數規模，在多項複雜任務評測中進入全球第一梯隊。Kimi 的核心場景是長程式碼庫分析和長檔案檢索增強——它能處理整本書的文字或整個專案的程式碼，在“大海撈針”式檢索任務中表現卓越。Kimi 的定價相對較高（輸出約 12-20 元/百萬 Token），但在長文字場景的價效比無人能及。

豆包 Doubao：位元組跳動的全能選手

位元組跳動的豆包系列在 2026 年表現穩定。Doubao Seed 2.1 Pro 是火山引擎旗艦模型，256K 上下文，在實測中綜合發揮穩定，中文場景體驗突出。豆包的優勢在於：註冊即送大量免費 Token，火山引擎生態整合完善，定價處於國內最低梯隊之一。對於想要快速體驗 AI 程式設計的初學者，豆包是零門檻的入門選擇。

對比項	DeepSeek V4	Qwen3.7 Max	GLM-5.1	Kimi K3	豆包 2.1 Pro
綜合得分	87.5	89.7	90.5	88+	85+
程式設計能力	極強	強	國產 SWE 第一	強	中上
中文理解	優秀	優秀	精準	優秀	優秀
上下文視窗	100 萬	256K-1M	100 萬	超長	256K
輸出價格 (元/M)	6	36	24	12-20	30
開源	MIT 協議	全系列開源	部分開源	閉源	閉源
私有化部署	支援	支援	支援	不支援	不支援

國產五大型語言模型橫向對比（2026 年 8 月）

一個重要的趨勢是：國產模型與 GPT/Claude 的“代差感”已經消失。在邏輯推理和數學競賽上，國產旗艦與 GPT-5 的分差基本在個位數。在系統化架構思維和複雜工程推理上，Claude 仍然領先——它能給出更有“靈氣”的架構方案，而國產模型有時顯得“教條”。但在中文場景、成本控制和合規部署上，國產模型有壓倒性優勢。

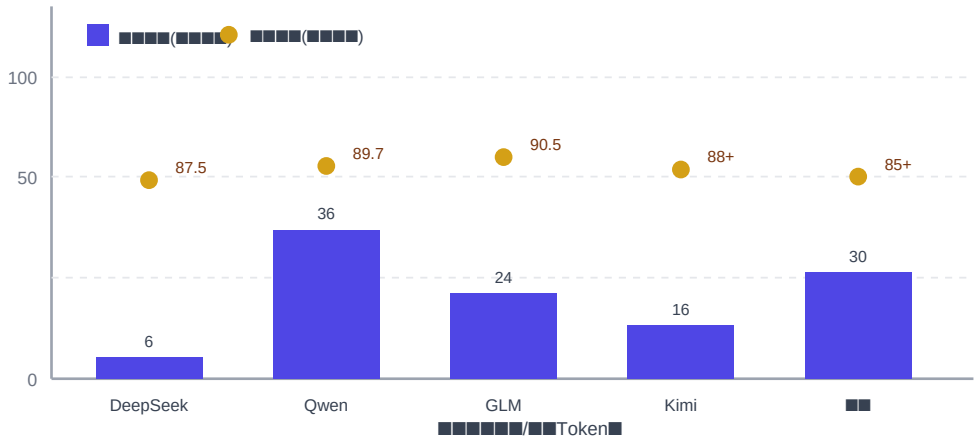


圖 4：五款模型的價格與效能對比（價格越低越好，得分越高越好）

三、模型選擇決策框架

瞭解了各模型的特點後，如何為自己的專案選模型？這裡送你一套“四問決策法”——回答四個問題，就能鎖定最適合你的模型。

你的專案在國內還是國際部署？

國內選國產模型

|

國際可選海外模型

核心任務是程式設計還是其他？

程式設計 Claude 或 DeepSeek

|

其他繼續判斷

需要超長上下文 (>50 萬 Token)？

需要 Gemini 或 Kimi

|

不需要繼續判斷

預算敏感嗎？

敏感 DeepSeek 或 Qwen-turbo

|

不敏感 GPT-5.6 或 Claude

圖 5：大型語言模型選擇四問決策法

小貼士：組合策略

這個決策法不是教條——很多場景需要組合使用多個模型。一個常見的組合策略：用 DeepSeek 或 Qwen 做日常開發（成本低、中文好），用 Claude 做關鍵架構設計（推理強、幻覺低），用 Gemini 做多模態分析（影像影片處理）。按需切換模型，而不是“一個模型用到死”。

四、API 呼叫入門：從零開始接入大型語言模型

很多 AI 程式設計教學只教你怎麼用對話介面，但真正的“AI 程式設計”——讓你的應用呼叫大型語言模型——需要透過 API。API（Application Programming Interface，應用程式設計介面）是程式之間的通訊協議。透過 API，你的程式碼可以像呼叫一個函式一樣，呼叫大型語言模型的能力。

什麼是 Token

在瞭解 API 之前，必須先理解“Token”這個概念。Token 是大型語言模型計費的基本單位，可以理解為“AI 的閱讀和寫作單位”。一個 Token 大約對應：英文中 0.75 個單詞（4 個字元），中文中 0.5-1 個漢字。一段 1000 字的中文文字，大約消耗 1500-2000 個 Token。

Token 不是字數，也不是字元數。大型語言模型使用專門的分詞器（Tokenizer）將文字切分成 Token。同一個詞在不同模型中的 Token 數量可能不同——比如“人工智慧”在 GPT 中可能是 3 個 Token，在 DeepSeek 中可能是 2 個。大多數模型的 API 會返回實際消耗的 Token 數量，你可以在 API 響應中檢視。

Token 成本快速估算

100 萬字中文約等於 150 萬 Token。用 DeepSeek V4 Flash 處理 100 萬字輸入 + 100 萬字輸出，成本約 0.42 美元（約 3 元人民幣）；用 Claude Fable 5 處理同樣的量，成本約 60 美元（約 434 元人民幣）。這就是為什麼模型選擇直接影響你的專案成本。

獲取 API Key

使用大型語言模型 API 的第一步是獲取 API Key——一串用於身份驗證的金鑰。每個模型提供商都有自己的 API 平臺，你需要註冊帳號並建立 API Key。以下是主流模型的 API 平臺入口和獲取流程：

模型	API 平臺	獲取流程	免費額度
DeepSeek	platform.deepseek.com	註冊實名認證建立 API Key	新使用者贈送額度
通義千問	bailian.aliyun.com（百鍊平臺）	阿里雲帳號登入開通服務建立 Key	新使用者免費試用
智譜 GLM	open.bigmodel.cn	註冊實名認證建立 API Key	新使用者贈送額度
Kimi	platform.moonshot.cn	註冊實名認證建立 API Key	有限免費額度
豆包	volcengine.com（火山引擎）	註冊開通方舟建立 API Key	註冊送大量 Token
OpenAI GPT	platform.openai.com	註冊繫結信用卡建立 API Key	無免費額度 (需付費)
Anthropic Claude	console.anthropic.com	註冊綁卡建立 API Key	新使用者少量額度

主流模型 API 平臺及 Key 獲取方式（2026 年 8 月）

安全提示：API Key = 錢包密碼

API Key 等同於你的錢包密碼——任何拿到 Key 的人都可以用你的額度呼叫模型。因此：永遠不要把 API Key 寫在程式碼中（用環境變數管理）；永遠不要把含 Key 的程式碼提交到公開的 Git 倉庫；永遠不要在社交媒體分享含 Key 的截圖。如果 Key 洩露了，立即在平臺上刪除並重新增立。

第一次 API 呼叫

獲取 API Key 後，你可以用任何程式語言呼叫大型語言模型 API。以下用 Python 示範一個最簡單的呼叫——向 DeepSeek 傳送一條訊息並獲取回覆。

```
# 安裝SDK：pip install openai
# DeepSeek相容OpenAI SDK格式，可以直接用
from openai import OpenAI
import os

# 1. 建立客戶端（API Key從環境變數讀取）
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

# 2. 傳送請求
response = client.chat.completions.create(
    model="deepseek-chat",  # 指定模型
    messages=[
        {"role": "system", "content": "你是一個Python程式設計助手。"},
```

```
        {"role": "user", "content": "寫一個快速排序函式。"}
    ]
)

# 3. 獲取回覆
print(response.choices[0].message.content)

# 4. 檢視Token消耗
print(f"輸入Token: {response.usage.prompt_tokens}")
print(f"輸出Token: {response.usage.completion_tokens}")
print(f"總Token: {response.usage.total_tokens}")
```

這段程式碼的核心邏輯是：建立使用者端（攜帶 API Key 和模型地址）構造訊息列表（system 設定角色，user 是使用者輸入）傳送請求獲取回覆。這個模式幾乎適用於所有相容 OpenAI 格式的模型——只需要更換 api_key 和 base_url 就能切換模型。這就是為什麼很多國產模型選擇相容 OpenAI SDK 格式：開發者只需改兩行程式碼就能切換模型，遷移成本極低。

以下是切換到其他模型的程式碼差異——注意只需要改兩行：

```
# DeepSeek
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)
# model="deepseek-chat"

# 通義千問（百鍊平臺）
client = OpenAI(
    api_key=os.environ.get("DASHSCOPE_API_KEY"),
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)
# model="qwen-plus"

# 智譜GLM
client = OpenAI(
    api_key=os.environ.get("GLM_API_KEY"),
    base_url="https://open.bigmodel.cn/api/paas/v4"
)
# model="glm-4"

# OpenAI GPT
client = OpenAI(
    api_key=os.environ.get("OPENAI_API_KEY"),
    # base_url使用預設值即可
)
# model="gpt-5.6-terra"
```

流式響應處理

在實際應用中，你通常不希望等 AI 寫完所有內容才一次性顯示——那會讓使用者等太久。流式響應（Streaming）可以讓你在 AI 生成的同時逐字顯示，大幅提升使用者體驗。就像打字機一樣，字一個一個蹦出來，使用者的感覺是完全不同的。

```
# 流式響應：逐字輸出
stream = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "user", "content": "寫一個Python的氣泡排序教程"}
    ],
    stream=True  # 開啟流式響應
)

# 逐塊接收並列印
for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="", flush=True)
# 輸出會像打字機一樣逐字顯示
```

五、Token Plan 購買指南

大多數模型提供商有兩種計費方式：按量計費（Pay-as-you-go）和資費方案訂閱（Token Plan / Subscription）。理解這兩種方式的區別，能幫你大幅降低使用成本。

按量計費 vs 資費方案訂閱

維度	按量計費 (Pay-as-you-go)	資費方案訂閱 (Subscription)
計費方式	用多少 Token 付多少錢	預付固定費用，獲得額度
適合場景	使用量不穩定、偶爾呼叫	使用量穩定、高頻呼叫
價格優勢	標準價	比按量低 20%-50%
靈活性	隨時可用，無需承諾	需要預付，額度過期可能作廢
超額處理	自動按量計費	超出額度後轉為按量計費
代表平臺	各模型官方 API 平臺	OpenAI Plus/Pro, Claude Pro, Cursor Pro

兩種計費方式的對比

對於初學者，建議從按量計費開始——不需要預付費用，用多少花多少。當你的月使用量穩定在一個水準後（比如每月超過 50 元），再考慮切換到資費方案訂閱以獲得更低單價。

主流訂閱資費方案一覽

資費方案	月費	包含內容	適合人群
ChatGPT Free	免費	GPT-5.6 Luna 無限文字對話	零基礎體驗
ChatGPT Plus	US$20/月	GPT-5.6 Sol、影像生成、資料分析、外掛訪問	中度使用者
ChatGPT Pro	US$200/月	無限制使用所有模型、優先訪問新功能	重度專業使用者
Claude Free	免費	有限次數 Claude 對話	零基礎體驗
Claude Pro	US$20/月	無限制 Claude 使用、優先訪問新模型	程式設計開發者
Cursor Free	免費	有限次數 AI 補全和對話	初學者體驗
Cursor Pro	US$20/月	無限 AI 補全、500 次/月進階請求	日常開發者
Cursor Business	US$40/月/人	團隊協作、管理後臺、隱私模式	團隊使用

主流 AI 工具訂閱資費方案（2026 年 8 月）

國內模型的訂閱方案略有不同。DeepSeek 主要按量計費，沒有月度訂閱資費方案——你充值後按實際使用量扣費。通義千問在百鍊平臺提供預付費資源包（購買一定量的 Token 額度，享受折扣價）。豆包在火山引擎上也有類似的預付費方案。對於國內開發者，最經濟的組合通常是：DeepSeek 按量計費（日常開發）+ 豆包免費額度（體驗測試）。

API 聚合平臺

如果你需要同時使用多個模型，API 聚合平臺是一個省心的選擇。聚合平臺提供一個統一的 API Key，你就能呼叫全球幾十種模型，無需在每個平臺分別註冊。

平臺	特點	價格優勢	適合場景
OpenRouter (openrouter.ai)	海外聚合平臺，一個 Key 呼叫全球模型	價格透明，支援實時比價	需要靈活切換多個海外模型
SiliconFlow (siliconflow.cn)	國內聚合平臺，提供多種國產模型 API	比官方低 20%-30%	國內開發者，多模型比價
Azure OpenAI (微軟代理)	微軟 Azure 代理的 OpenAI API	Batch API 可打 5 折	企業級 OpenAI 呼叫，大批次非同步任務
百度千帆 (qianfan.baidubce.com)	百度 AI 雲平臺，提供文心等模型	新使用者贈送額度	百度生態使用者

主流 API 聚合平臺（2026 年 8 月）

避坑提醒：別買“共享 Key”

不要在淘寶、閒魚等平臺購買所謂的“共享 API Key”——這些 Key 通常來路不明，可能隨時被封號，還有資料洩露風險。始終透過官方平臺或正規聚合平臺獲取 API Key。

成本最佳化策略

選對模型只是第一步，如何“用得省錢”同樣重要。以下是幾個經過實踐驗證的成本最佳化策略：

1. 善用快取命中。很多模型提供” 快取” 功能——當你的請求中包含之前已經處理過的內容（如系統提示詞），這部分只按極低價格計費。DeepSeek 的快取命中價格僅為普通價格的 1/60。把固定的系統提示放在訊息列表開頭，可以最大化快取命中。

2. 分級路由——用便宜模型處理簡單任務，貴模型處理複雜任務。比如用 DeepSeek V4 Flash 做文字分類（輸出 2 元/百萬 Token），用 Claude 做架構設計（輸出 181 元/百萬 Token）。一個專案裡 80% 的呼叫可以用便宜模型。

3. 限制輸出長度。API 按輸出 Token 計費，輸出越長越貴。在不需要長篇大論的場景中，設定 max_tokens 引數限制輸出長度，避免 AI” 廢話太多”。

4. 使用 Batch API 處理非實時任務。如果你有大量不需要即時響應的任務（如批次資料標註、檔案摘要），使用 Batch API 可以享受 50% 折扣。OpenAI 和 Azure 都提供這個功能。

5. 監控 Token 用量。大多數 API 平臺提供用量統計面板。定期檢視用量分佈，找出消耗 Token 最多的呼叫，針對性最佳化。一個小小的 Prompt 調整（比如把” 請詳細解釋” 改成” 請用 3 句話解釋”）可能節省 50% 的 Token。

最佳化策略	節省幅度	實施難度	說明
快取命中	最高 98%	低	把固定內容放在訊息開頭
分級路由	50%-80%	中	簡單任務用便宜模型
限制輸出長度	30%-60%	低	設定 max_tokens 引數
Batch API	50%	低	非實時任務用批處理
Prompt 最佳化	20%-50%	中	精簡 Prompt，減少冗餘

成本最佳化策略及效果（2026 年 8 月）

六、實戰：在專案中接入大型語言模型 API

讓我們用一個完整的實戰來結束本章——建立一個“AI 摘要生成器”：使用者輸入一段長文字，呼叫大型語言模型 API 生成摘要。這個實戰會展示 API 呼叫的完整流程：環境配置、API 呼叫、錯誤處理、成本控制。

```
# ai_summarizer.py - AI摘要生成器
import os
from openai import OpenAI

# 1. 配置（API Key從環境變數讀取，絕不硬編碼）
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

def summarize(text: str, max_length: int = 200) -> dict:
    """生成文字摘要

    Args:
        text: 要摘要的原文
        max_length: 摘要最大長度（Token數）

    Returns:
        dict: 包含摘要和Token消耗資訊
    """
    try:
        response = client.chat.completions.create(
            model="deepseek-chat",
            messages=[
                {
                    "role": "system",
                    "content": "你是一個摘要生成器。"
                               "請用簡潔的中文總結使用者提供的文字。"
                               "摘要不超過3句話。"
                },
                {
                    "role": "user",
                    "content": f"請摘要以下文字:\n\n{text}"
                }
            ],
            max_tokens=max_length,  # 限制輸出長度
            temperature=0.3  # 低溫度=更穩定
        )

        return {
            "success": True,
```

```
            "summary": response.choices[0].message.content,
            "input_tokens": response.usage.prompt_tokens,
            "output_tokens": response.usage.completion_tokens,
            "total_tokens": response.usage.total_tokens
        }

    except Exception as e:
        return {
            "success": False,
            "error": str(e)
        }

# 2. 使用示例
if __name__ == "__main__":
    long_text = """
    （貼上一段長文字...）
    """

    result = summarize(long_text)

    if result["success"]:
        print(f"摘要: {result['summary']}")
        print(f"消耗Token: {result['total_tokens']}")
        # 估算成本：DeepSeek V4 Flash
        cost = result['total_tokens'] * 0.000004  # 約0.4分/千Token
        print(f"預估成本: {cost:.4f}元")
    else:
        print(f"出錯了: {result['error']}")
```

這段程式碼體現了 API 呼叫的幾個最佳實踐：API Key 透過環境變數讀取，絕不硬編碼在程式碼中；函式有清晰的檔案和型別提示；設定了 max_tokens 限制輸出長度；temperature 設為低值（0.3）以獲得更穩定的輸出；有完整的錯誤處理；返回 Token 消耗資訊用於成本追蹤。當你用 AI 來寫這段程式碼時，可以告訴它這些要求——它完全有能力幫你實現。

本章的最終啟示：模型是”彈藥”，方法論是”戰術”。選對彈藥能讓戰鬥更順利，但沒有好的戰術，再好的彈藥也打不贏仗。瞭解了各模型的特點和 API 使用方法後，你就有了一個充足的”彈藥庫”——但怎麼打這場仗，還是要靠前幾章學到的思維方法論。

來動手：跑通你的第一次 API 呼叫

註冊一個國產模型平臺（推薦 DeepSeek 或智譜，都送新使用者額度），建立 API Key，把它寫進系統環境變數（而不是程式碼裡！），然後執行上面“第一次 API 呼叫”的程式碼。看到 AI 的回覆列印出來的那一刻，你就跨過了“用 AI”和“造 AI 應用”之間的分界線。

本章小結

- 國際三強：GPT-5.6（均衡全能）、Claude（程式設計天花板 + 幻覺率僅 1%）、Gemini（多模態之王 +200 萬上下文）
- 國產五強：DeepSeek V4（價效比之王，注意漲價公告）、Qwen3.7（企業級）、GLM-5.1（SWE 國產第一）、Kimi K3（超長文字）、豆包（免費入門）
- 四問決策法：部署地點？程式設計還是其他？超長上下文？預算敏感？
- Token 是計費單位：中文 1 字約 1.5-2 Token；100 萬字約 150 萬 Token
- API Key = 錢包密碼：環境變數管理，絕不硬編碼、不上傳 Git
- 國產模型普遍相容 OpenAI SDK：換模型只需改 api_key 和 base_url 兩行
- 省錢五招：快取命中、分級路由、限制輸出、Batch API、監控用量

第四十三章：對話工程——與 AI 高效協作的對話技術

對話是 AI 程式設計的核心互動方式。但請注意：“對話”不等於“聊天”——高效的 AI 程式設計對話是一種有結構的溝通技術，我們稱之為“對話工程”。它研究的是：如何用最少的對話輪次，獲得最高品質的程式碼產出。說白了，這一章就是教你怎麼和 AI“把話說清楚”。

一、Prompt 分層策略

與 AI 的對話可以分為三個層次，每層承擔不同的功能。理解這三個層次，你就能像搭積木一樣組合出高效的 Prompt。

層次	功能	示例	頻率
系統層	設定 AI 的角色和全域性約束	” 你是一個 React 專家，程式碼必須用 TypeScript”	專案開始時設定一次
任務層	描述當前要做的具體任務	” 實現一個表單元件，包含使用者名稱和密碼欄位”	每個任務開始時給出
反饋層	對 AI 產出進行評價和修正	” 密碼欄位缺少顯示/隱藏切換，請新增”	根據產出品質給出

對話 Prompt 的三個層次

系統層的 Prompt 通常寫在專案記憶檔案中（如 CLAUDE.md），不需要每次對話重複。任務層是你在對話中最常使用的——它應該包含“做什麼”“用什麼技術”“有什麼約束”三個要素。反饋層是最考驗功力的——好的反饋能讓 AI 精準修正，壞的反饋只會讓它重新猜測。

小貼士：反饋三板斧

一個實用的反饋模板是“指出問題 + 說明期望 + 提供上下文”。比如：“第 25 行的錯誤處理只用了 console.log（指出問題），生產環境應該用統一的錯誤處理器並返回使用者友好提示（說明期望），專案裡有一個 handleError 函式可以用（提供上下文）”。這種反饋比“改

一下錯誤處理”有效十倍。

二、除錯對話：如何描述錯誤讓 AI 快速定位

除錯是 AI 程式設計中最頻繁的場景之一。當你遇到 bug 時，如何向 AI 描述問題，直接決定了它能否快速幫你定位和修復。以下是除錯對話的四個要點：

1. 給 AI”看”原始資訊，而不是你的轉述。把錯誤日誌、異常堆疊、相關程式碼直接 @ 引用給 AI，而不是用”好像有個什麼報錯”來描述。AI 看到原始資訊比看到你的理解更準確。
2. 描述”期望行為”和”實際行為”的差異。”我期望點選按鈕後跳轉到首頁（期望），但實際點選後頁面沒有反應（實際）”——這種描述比”按鈕壞了”有用得多。
3. 提供復現步驟。”先輸入使用者名稱，然後點選登入按鈕，就會出現這個錯誤”——復現步驟讓 AI 能模擬你的操作路徑，更容易定位問題。
4. 說清楚你已經嘗試了什麼。”我已經檢查了網路請求，返回 200，資料正常”——這能幫 AI 排除已驗證的方向，不浪費時間重複排查。

差的除錯描述

```
按鈕點選沒反應，幫我看看
```

好的除錯描述

```
點選登入按鈕後頁面無反應。 期望行為：點選後跳轉到 /home 頁面 實際行為：點選後無任何反
    應，控制檯無報錯 復現步驟： 1. 在登入頁輸入使用者名稱admin和密碼123456 2. 點選"登
    入"按鈕 3. 頁面無反應 已排查： - 網路請求正常，返回200 - onClick事件已繫結 - 檢查
    了LoginButton元件的程式碼 相關檔案：@src/components/LoginButton.tsx @src/pages/
    Login.tsx
```

三、迭代策略：小步快跑 vs 大步重構

AI 程式設計中的迭代有兩種基本策略，適用於不同場景。理解它們的區別，能幫你在不同情況下做出正確選擇。

“小步快跑”策略適用於功能開發和 bug 修復。它的核心是：每次讓 AI 只做一個小的修改，驗證透過後再做下一個。優勢是每一步都可驗證、可回退，風險可控；劣勢是速度較慢，需要頻繁的對話互動。適合：新功能開發、不確定最優方案時的探索、對 AI 產出不太放心時的謹慎迭代。

“大步重構”策略適用於明確知道目標的大規模修改。它的核心是：一次性描述完整的目標，讓 AI 自主規劃和執行整個修改。優勢是速度快，AI 能看到全域性圖景做出更優的架構決策；劣勢是風險高，如果 AI 理解偏差，修正成本大。適合：明確的架構升級、大規模程式碼遷移、有充分測試保障的重構。

維度	小步快跑	大步重構
每次修改量	一個功能點或一個 bug	一個模組或一個架構層
驗證頻率	每次修改後立即驗證	全部修改完成後統一驗證
對話輪次	多（5-10 輪/功能）	少（1-3 輪/模組）
風險	低，每步可回退	高，偏差修正成本大
適用場景	新功能、探索性開發	明確目標的重構、遷移
前提條件	無特殊要求	需要有測試覆蓋

兩種迭代策略的對比

實踐中，兩種策略常常組合使用。比如重構一個模組時，先用“大步重構”讓 AI 給出整體方案，審查方案後，再用“小步快跑”逐步執行。這樣既保證了全域性視角，又控制了執行風險。

四、多輪對話的上下文管理

隨著對話輪次增加，你需要主動管理對話上下文，避免“AI 忘記了早期資訊”的問題。以下是幾個實用的上下文管理技巧。

第一，“錨定”關鍵資訊。在對話開始時，把關鍵約束寫清楚，並在後續對話中適時重申。比如“記住，我們的資料存在 localStorage，不能用資料庫”——在相關的修改請求中重申這個約束，防止 AI 遺忘。

第二，善用“總結-續接”技巧。當對話變得很長時，可以讓 AI“總結一下目前的進展和待辦事項”，然後開一個新對話，把這個總結作為起點。這樣既保留了關鍵上下文，又避免了長對話的退化。

第三，“一個對話一個任務”。這是最簡單也最有效的上下文管理策略。每個對話只處理一個任務或一個功能點，完成後開新對話處理下一個。這天然限制了對話長度，保持上下文清晰。

第四，利用工具的上下文管理功能。大多數 AI 程式設計工具都有“清除上下文”或“新會話”按鈕——當你感覺 AI 的回答品質下降時，不要猶豫，開一個新會話。丟失一些對話歷史比繼續在退化的上下文中掙扎要好得多。

來動手：給你的 AI 當一次”病人”

故意在瀏覽器裡做一個小頁面，製造一個 bug（比如按鈕沒反應）。然後按本章“好的除錯描述”的格式，向 AI 完整描述一次：原始資訊、期望 vs 實際、復現步驟、已排查項。感受一下 AI 定位問題的速度差異——你會理解為什麼“描述問題”本身是門手藝。

本章小結

- Prompt 三層：系統層（寫在記憶檔案）、任務層（做什麼 + 技術 + 約束）、反饋層（指出問題 + 期望 + 上下文）
- 除錯四要點：給原始資訊、期望 vs 實際、復現步驟、已排查項
- 小步快跑：低風險高輪次，適合新功能；大步重構：高風險低輪次，需要測試保障

- 上下文管理四招：錨定關鍵資訊、總結-續接、一個對話一個任務、果斷開新會話

第四十四章：從 0 到 1 實戰——完整專案全流程

理論學完了，現在是動手的時候。本章不是簡單的“照著敲”教學，而是一套“怎麼想、怎麼問、怎麼驗”的實戰方法論。我們會用三個不同難度的專案，展示從需求到成品的完整思考過程。無論你用哪個工具，這套流程都適用。

一、專案一：極簡個人工具站（入門級）

需求思考

目標：建立一個個人工具站網頁，展示 3-4 個實用小工具的入口。這是一個純前端專案，不需要後端和資料庫。重點不是工具本身有多複雜，而是體驗“從需求到成品”的完整流程。

按照需求五要素：Who——個人使用。What——一個展示工具入口的網頁。Why——把常用工具集中在一個頁面方便訪問。How——單檔案 HTML+CSS+JS，不使用框架。Constraint——30 分鐘內完成，不需要部署。

下達指令

基於 Think Before Coding 原則，你的指令應該包含足夠的資訊讓 AI 理解目標，但不需要指定每個實現細節：

```
請建立一個個人工具站網頁，要求：
1. 單檔案HTML（內聯CSS和JS），不使用框架
2. 暗色主題，現代極簡風格
3. 展示4個工具入口卡片：
   - 文字摘要工具
   - 字數統計工具
   - 顏色選擇器
   - 二維碼生成器
4. 每個卡片有圖示、標題、簡短描述
5. 卡片有懸停動畫效果
6. 響應式佈局，手機上可用
7. 頂部有一個標題欄
請直接給出完整的HTML檔案。
```

驗收與迭代

AI 給出程式碼後，按照迭代驗證的三層檢查進行驗收。第一層功能驗證：檔案能否在瀏覽器中開啟？4 個卡片是否都顯示了？懸停動畫是否正常？第二層邊界驗證：在手機螢幕尺寸下佈局是否正常？視窗很窄時卡片是否會換行？第三層架構驗證：程式碼是否可讀？CSS 是否內聯在一個 style 標籤中？有沒有明顯的冗餘？

如果某個工具卡片的功能也想實現（而不只是入口），可以逐步新增——每次新增一個工具的功能，驗證後再新增下一個。這就是“小步快跑”策略的實際應用。

二、專案二：全棧記帳系統（進階級）

需求思考與 PRD

目標：一個有前端、後端和資料庫的全棧記帳應用。這個專案比第一個複雜得多，需要用到第四十章的需求工程方法。

先寫 PRD。按照第四十章第三節的模板，寫出專案概述、功能列表、技術方案、資料模型和開發計劃。技術方案的選擇要考慮 Simplicity First 原則——前端用 React 但不引入 Redux 等複雜狀態管理，後端用 Node.js 但不引入微服務，資料庫用 SQLite 而非 PostgreSQL（開發階段）。

開發計劃拆解為 5 步：Step 1 後端 API 骨架（Express 伺服器 + SQLite + CRUD 介面）；Step 2 前端基礎頁面（記錄列表 + 新增表單）；Step 3 前後端聯調（API 呼叫 + 資料展示）；Step 4 分類和篩選功能；Step 5 月度彙總和簡單圖表。每步完成後都是“能用的東西”。

設定專案記憶檔案

在開始編碼之前，先建立專案記憶檔案。這是上下文工程的核心實踐——讓 AI 在每次對話開始時都瞭解專案背景。

```
# 專案：全棧記帳應用
## 概述
個人記帳應用，支援收支記錄、分類、月度彙總。
## 技術棧
- 前端：React + TypeScript + Vite
- 後端：Node.js + Express + TypeScript
- 資料庫：SQLite (透過 better-sqlite3)
- 前後端共享型別定義
## API約定
- 基礎路徑：/api
- 返回格式：{ success: boolean, data: any, error: string }
- RESTful風格
## 資料模型
Record { id, amount, type, category, note, date }
## 程式碼規範
- TypeScript嚴格模式
```

```
- 縮排2空格
- 函式有JSDoc註釋
- 中文註釋和提交資訊
```

逐步實現

按照開發計劃，每一步用一個獨立的對話來完成。以下是第一步的指令示例：

```
我需要建立後端API骨架。
要求：
1. Express伺服器，埠3001
2. SQLite資料庫，表名records
3. records表欄位：
   id (TEXT PRIMARY KEY)
   amount (REAL)
   type (TEXT: 'income'|'expense')
   category (TEXT)
   note (TEXT)
   date (TEXT: YYYY-MM-DD)
4. CRUD介面：
   GET /api/records - 獲取所有記錄
   POST /api/records - 新增記錄
   PUT /api/records/:id - 更新記錄
   DELETE /api/records/:id - 刪除記錄
5. 錯誤處理中介軟體
請給出完整的後端程式碼結構。
```

注意這個指令的特點：它只描述當前這一步（後端骨架），不摻雜前端需求；它明確列出了欄位和介面（給 AI 足夠的約束），但具體實現由 AI 決定；它遵循了 Goal-Driven Execution 原則——說“做什麼”而非“怎麼做”。

AI 給出程式碼後，立即驗證：執行伺服器，用 curl 或 Postman 測試每個介面是否正常工作。確認無誤後，開新對話處理 Step 2（前端基礎頁面）。每一步都遵循“指令-驗證-新對話”的迴圈。

三、專案三：AI 增強應用（進階）

專案構思

目標：一個接入大型語言模型 API 的應用——比如一個“AI 學習助手”，使用者輸入學習內容，AI 生成摘要、關聯知識點、推薦學習資源。這個專案的特殊之處在於：你的應用本身就在呼叫 AI，你需要處理 API 呼叫、prompt 工程、錯誤處理等新問題。

這個專案教會你的最重要的一課是：當你“用 AI 來寫呼叫 AI 的程式碼”時，你需要格外注意 prompt 工程。你的應用呼叫大型語言模型 API 時使用的 prompt，也需要遵循本指南討論的原則——清晰、簡潔、目標驅動。

關鍵技術點

這個專案涉及幾個你在前兩個專案中沒遇到的技術點：

- API 金鑰管理：金鑰絕不能硬編碼在程式碼中，用環境變數管理；
- 流式響應處理：大型語言模型的回覆通常是流式的，需要逐字顯示；
- Token 用量控制：大型語言模型按 token 計費，需要在應用層面控制用量；
- Prompt 模板管理：不同功能用不同的 prompt 模板，需要結構化管理。

這些技術點都可以讓 AI 幫你實現，但你需要理解每個技術點的“為什麼”——為什麼金鑰不能硬編碼？為什麼需要流式響應？為什麼 token 用量需要控制？理解了“為什麼”，你才能判斷 AI 的方案是否合理。

四、通用實戰流程清單

無論專案大小，以下是一個通用的 AI 程式設計實戰流程清單。每次開始新專案時，過一遍這個清單，確保沒有遺漏關鍵步驟。

階段	做什麼	對應原則
1. 需求思考	過一遍需求五要素，想清楚要做什麼	Think Before Coding
2. 需求拆解	把大需求拆成可獨立驗證的小步驟	Simplicity First
3. 寫 PRD	整理成結構化檔案，作為參考	Goal-Driven
4. 設定記憶檔案	建立 CLAUDE.md 或同類檔案	Harness Engineering
5. 逐步實現	每步一個對話，指令-驗證-下一步	Surgical Changes
6. 測試驗證	每步完成後立即測試功能正確性	Iterate & Verify
7. 程式碼審查	檢查架構合理性、安全性、可維護性	Iterate & Verify
8. 迭代最佳化	根據測試和審查結果迭代改進	Iterate & Verify

AI 程式設計通用實戰流程清單

來動手：本週完成”專案一”

把第四十四章第一節的“個人工具站”當成你的第一個 AI 程式設計作業：花 5 分鐘過五要素，把指令發給 AI（任何工具都行），再按三層驗證驗收。30 分鐘內完成它——你的第

一個 AI 程式設計作品就誕生了。做完記得把它存好，這是你從“讀者”變成“實踐者”的紀念品。

本章小結

- 專案一（入門）：純前端工具站——體驗完整流程，30 分鐘能完成
- 專案二（進階）：全棧記帳系統——PRD + 記憶檔案 + 5 步逐步實現，每步” 能用”
- 專案三（進階）：AI 學習助手——用 AI 寫呼叫 AI 的程式碼，注意 prompt 工程
- 通用流程八步：需求思考拆解 PRD 記憶檔案逐步實現測試審查迭代
- 每一步都遵循” 指令-驗證-新對話” 迴圈

第四十五章：品質保障——讓 AI 寫的程式碼可靠

AI 寫的程式碼有一個特點：它通常“看起來很對”——變數命名規範、註釋齊全、邏輯清晰。但“看起來對”不等於“真的對”。*AI* 程式碼中隱藏的問題往往比手寫程式碼更難發現，因為它的表面品質太高了，容易讓人放鬆警惕。本章教你如何為 *AI* 程式碼建立有效的品質保障體系——用測試、審查、版本控制三道防線，把“看起來對”變成“真的對”。

一、測試驅動：讓 AI 寫測試

測試是 AI 程式設計品質保障的第一道防線。好訊息是：讓 AI 寫測試比讓它寫功能程式碼更容易、更可靠——因為測試的預期行為是明確的，不太容易產生“看起來對但實際不對”的程式碼。

測試驅動的 AI 程式設計流程是：先讓 AI 寫功能程式碼，驗證基本功能透過後，立即讓它寫測試用例。測試用例應覆蓋正常路徑、邊界情況和異常情況。然後再讓 AI 根據測試反饋修正功能程式碼。這個流程的好處是：測試一旦寫好，就是持續的驗證工具——後續任何修改都可以透過執行測試來確認沒有引入迴歸問題。

```
請為 calculate_discount 函式編寫單元測試。
測試用例應覆蓋：
1. 正常路徑：
   - 100元商品打8折 -> 80
   - 200元商品打5折 -> 100
2. 邊界情況：
   - 0元商品打8折 -> 0
   - 100元商品打0折 -> 100（不打折）
   - 100元商品打10折 -> 100（全價）
3. 異常情況：
   - 負數價格 -> 應丟擲錯誤
   - 折扣率 > 1 -> 應丟擲錯誤
   - 非數字輸入 -> 應丟擲錯誤
使用Vitest框架，給出完整測試檔案。
```

注意這個指令的特點：它明確列出了三類測試用例（正常、邊界、異常），不給 AI 自由發揮的空間去“選擇性測試”。AI 有時候會只寫正常路徑的測試來“快速交差”，明確列出所有類

別可以避免這個問題。

二、程式碼審查技巧

即使有測試覆蓋，你仍然需要對 AI 產出的程式碼進行人工審查。程式碼審查不是逐行閱讀——那太耗時也太依賴你的程式設計水準。有效的 AI 程式碼審查應該關注以下幾個關鍵維度：

審查維度	關注什麼	常見問題
安全性	輸入驗證、金鑰處理、SQL 隱碼攻擊防護	AI 經常忘記驗證輸入，金鑰硬編碼在程式碼中
錯誤處理	異常是否被捕獲，錯誤資訊是否合理	AI 喜歡用空 catch 塊或只 console.log
效能	是否有明顯的效能問題，如 N+1 查詢、無限迴圈	AI 有時會寫出 $O(n^2)$ 的演演算法
可維護性	程式碼結構是否清晰，命名是否有意義	AI 偶爾會生成過度抽象的程式碼
相依性管理	引入的依賴是否必要，版本是否明確	AI 傾向於引入不必要的函式庫

AI 程式碼審查的關鍵維度

審查時不必自己逐一檢查——可以讓 AI 幫你做第一輪審查。指令如：“請審查這段程式碼的安全性、錯誤處理和效能，指出潛在問題。”AI 的自我審查能力在不斷提升，雖然不能完全替代人工審查，但能幫你過濾掉大部分明顯問題。

三、版本控制：Git 在 AI 程式設計中的角色

Git 在 AI 程式設計中的角色比在傳統程式設計中更加重要。因為 AI 的每次修改都可能引入不可預知的變化，你需要一個可靠的“後悔藥”——版本控制就是那個後悔藥。

AI 程式設計中 Git 的最佳實踐是“頻繁提交、小步快進”。每完成一個可驗證的功能點，就提交一次。這樣當 AI 的某次修改出了問題時，你可以精準回退到上一個可用狀態，而不需要推翻所有工作。

另一個實用技巧是“分支隔離”。對於不確定結果的 AI 修改（比如大步重構），先建立一個新分支來做。如果結果滿意，合併回主分支；如果不滿意，直接刪除分支，不影響主分支。這比在主分支上反覆修改安全得多。

1. 每次 AI 修改前，確保工作區是乾淨的（沒有未提交的更改）。這樣如果 AI 的修改不滿意，可以用 git checkout . 快速回退。

2. 每次 AI 修改後、驗證透過後，立即提交。提交資訊描述這次修改做了什麼，比如”feat: 新增使用者註冊功能”。

3. 對於大步重構，先建立分支：git checkout -b refactor/auth-system。在分支上完成修改，測試透過後再合併。

4. 養成檢視 diff 的習慣。每次 AI 修改後，用 git diff 檢視具體改了什麼。這不僅能幫你發現問題，還能幫你學習程式碼。

四、常見陷阱與避坑指南

AI 程式設計中有幾個反覆出現的陷阱，幾乎所有初學者都會踩。提前知道它們，能幫你少走很多彎路。

陷阱一：信任陷阱

“AI 寫的程式碼看起來很專業，一定沒問題吧？”——這是最危險的陷阱。AI 生成的程式碼在表面上非常整潔——命名規範、註釋齊全、結構清晰——這種“表面專業感”會讓人放鬆警惕。但表面整潔不代表邏輯正確。解藥：永遠透過測試驗證，而非透過“看起來對不對”判斷。

陷阱二：膨脹陷阱

“既然 AI 能幫我寫程式碼，那不如把功能做得全面一點？”——這會導致程式碼膨脹。AI 很擅長“多給”：你讓它做一個登入頁面，它會給你一套完整的使用者管理系統。多餘的功能不僅增加複雜度，還增加了出 bug 的機率。解藥：嚴格遵循 Simplicity First 原則，明確告訴 AI“只做這個功能，不要額外新增”。

陷阱三：理解陷阱

“AI 應該理解我的意思吧？”——不一定。AI 的理解能力雖然很強，但對模糊的需求仍然會“猜”。當它猜錯了，產出的程式碼就是錯的。更麻煩的是，它猜錯的程式碼通常“看起來很合理”，讓你誤以為它理解了。解藥：在指令中提供足夠的具體細節，減少 AI 需要“猜”的空間。

陷阱四：上下文陷阱

“我在之前的對話裡說過了，AI 應該還記得吧？”——很可能不記得。隨著對話變長或開新會話，AI 會丟失早期上下文。如果某個關鍵約束沒有被寫入專案記憶檔案，它很可能在後續修改中被忽略。解藥：關鍵約束寫入專案記憶檔案，不要依賴對話記憶。

四大陷阱速記

信任陷阱——別信“看起來對”；膨脹陷阱——AI 愛“多給”；理解陷阱——模糊需求它會“猜”；上下文陷阱——對話記憶會“忘”。對應的解藥分別是：測試驗證、砍掉多餘、給足細節、寫入記憶檔案。

來動手：給你的專案建三道防線

如果你已經開始做專案一（工具站）：(1) 讓 AI 給核心函式寫 3 類測試用例；(2) 按五個維度審查一遍程式碼（安全性/錯誤處理/效能/可維護性/依賴）；(3) 用 git 初始化專案，做第一次提交。三道防線齊了，你的專案才真正屬於你。

本章小結

- AI 程式碼” 看起來對”≠” 真的對”——表面品質太高反而容易放鬆警惕
- 讓 AI 寫測試：明確列出正常/邊界/異常三類用例，防止” 選擇性測試”
- 審查五維度：安全性、錯誤處理、效能、可維護性、相依性管理
- Git 是” 後悔藥”：頻繁提交、小步快進、分支隔離、常看 diff
- 四大陷阱：信任（測試驗證）、膨脹（砍掉多餘）、理解（給足細節）、上下文（寫入記憶檔案）

第四十六章：進階之路——從使用者到架構師

當你掌握了第三十七至四十四章的方法論，能熟練地用 *AI* 完成專案開發後，下一步是什麼？本章探討從“*AI* 工具使用者”到“*AI* 系統架構師”的進階之路。這個階段的關注點不再是“怎麼讓 *AI* 寫好程式碼”，而是“怎麼讓 *AI* 更好地融入整個開發流程”。

一、MCP 協議與工具擴充套件

MCP（Model Context Protocol）是 2025 年出現、2026 年逐漸普及的一個開放協議，它定義了 AI 模型與外部工具之間的通訊標準。簡單來說，MCP 讓 AI 能夠“使用工具”——不只生成程式碼，還能執行命令、查詢資料庫、呼叫 API、操作檔案系統。

在 MCP 之前，每個 AI 工具都有自己的工具整合方式，互不相容。MCP 的出現意味著：一個符合 MCP 標準的工具可以被任何支援 MCP 的 AI 工具使用。這大大降低了工具整合的成本，也讓 AI 的能力邊界從“生成文字”擴充套件到了“執行動作”。

對於進階使用者來說，MCP 的意義在於：你可以為 AI 構建定製化的工具能力。比如，如果你的團隊有一套內部的部署系統，你可以寫一個 MCP 工具讓 AI 直接呼叫部署系統，而不需要人工切換到終端操作。這種“AI 呼叫工具”的能力，是 Agentic Engineering 的基礎設施。

二、自定義工作流與 Skills

當你反覆執行某些固定的 AI 程式設計流程時，可以把它們固化成“自定義工作流”或“Skills”。比如，你的團隊可能有一套標準的程式碼審查流程——先檢查安全性，再檢查效能，再檢查可維護性。你可以把這個流程寫成一個 Skill，讓 AI 在每次程式碼審查時自動按這個流程執行。

自定義工作流的核心價值是“一致性”——它確保每次執行都遵循同樣的標準，不會因為對話長短或上下文不同而產生偏差。這對於團隊協作尤其重要：每個成員使用同樣的工作流，產出的程式碼風格和品質標準就會趨於一致。

構建自定義工作流的方法因工具而異。Cursor 支援 Rules 和 Custom Instructions，Claude Code 支援自定義命令，Replit 支援 Runme。但核心思路一樣：把你的最佳實踐“編碼”成 AI 可執行的指令模板。

三、多 Agent 協作模式

到 2026 年，多 Agent 協作開始成為一個熱門話題。它的核心理念是：不只用一個 AI 來完成任務，而是用多個分工不同的 AI 來協作。比如，一個 AI 負責寫程式碼，一個 AI 負責寫測試，一個 AI 負責程式碼審查——它們各司其職，互相檢查。

多 Agent 協作的優勢在於“分工”和“交叉驗證”：每個 Agent 專注於自己的領域，產出品質更高；Agent 之間的交叉檢查可以發現自己可能忽略的問題。但劣勢也很明顯：協調成本高，通訊開銷大，除錯更復雜。

小貼士：一個人也能玩”多 Agent”

對於個人開發者來說，你不需要搭建複雜的多 Agent 系統——你可以“扮演”不同的角色來模擬多 Agent 協作。比如：先用一個對話讓 AI 寫功能程式碼；然後開一個新對話（提供 CLAUDE.md 上下文），讓 AI“作為程式碼審查者審查這段程式碼”；最後再開一個對話讓 AI“根據審查意見修正程式碼”。這種“一人多角”的方式，能獲得多 Agent 協作的大部分好處，而不需要複雜的基礎設施。

四、持續學習路徑

AI 程式設計是一個快速變化的領域。今天的最佳實踐，明天可能就需要更新。以下是幾條持續學習的建議：

第一，關注源頭而非二手解讀。Andrej Karpathy 的原始推文和 CLAUDE.md、Anthropic 和 OpenAI 的官方檔案、各大工具的更新日誌——這些是第一手資訊。社群文章和教學是對這些資訊的解讀和總結，有價值但可能失真。養成讀原始資料的習慣。

第二，實踐驅動學習。不要“學完再做”，而要“邊做邊學”。每次遇到新的問題型別（比如第一次處理檔案上傳、第一次接入第三方 API），就以此為機會學習相關知識。實踐中的學習效率遠高於脫離場景的理論學習。

第三，保持工具敏感度但不過度追新。每隔一兩個月，花半天時間瞭解最新的工具和趨勢——但不要每次有新工具就切換。判斷標準是：新工具是否能顯著提升你的某個瓶頸？如果不能，當前的組合就夠了。

第四，構建個人知識庫。把你每次解決複雜問題的對話、每次踩坑的教訓、每次有效的方法記錄下來。不需要公開發布——這是你自己的“方法論筆記”。隨著時間積累，它會成為你最寶貴的資產。

來動手：開一個”踩坑筆記本”

今天就建一個檔案（筆記軟體或.md 檔案都行），寫下你到目前為止最想記住的三條 AI 程式設計心得——哪怕是“記得先想清楚再問”。以後每次踩坑、每次學到新招，都往裡面記一條。半年後回看，你會感謝今天的自己。

本章小結

- MCP 是 AI 與工具之間的通訊標準：讓 AI 能執行命令、查資料庫、調 API
- 自定義工作流/Skills 的核心價值是” 一致性”：把最佳實踐編碼成指令模板
- 多 Agent = 分工 + 交叉驗證；個人開發者可用” 一人多角” 模擬
- 持續學習四招：讀源頭、邊做邊學、不過度追新、建個人知識庫

第四十七章：速查手冊與資源

本章是一份可以獨立使用的速查手冊。當你記不清某個原則或需要快速決策時，直接翻到這裡。建議你把它當成“口袋卡片”：五大原則速查、工具決策樹、常見問題、學習資源，一頁不落。

一、思維模型速查卡

原則	一句話	操作要點
Think Before Coding	先想清楚再開口	回答三問：做什麼？結果長啥樣？有什麼約束？
Simplicity First	用最少程式碼解決問題	每次只做一個功能，砍掉不必要的複雜度
Surgical Changes	精準修改不越界	明確指定修改範圍，拒絕 AI 的” 順手改一下”
Goal-Driven Execution	說” 做什麼” 不說” 怎麼做”	給目標不給步驟，讓 AI 自主選擇路徑
Iterate & Verify	生成-驗證-修正迴圈	三層驗證：功能、邊界、架構

五大思維原則速查卡

二、工具選擇決策樹

當你需要選擇 AI 程式設計工具時，按照以下決策路徑選擇。注意：這不是“哪個最好”的排名，而是“哪個最適合你”的判斷框架。

你要做什麼？

零基礎，想快速體驗選雲端 Builder (v0/Bolt/Replit)

日常開發，需要程式碼補全選 IDE 內建 AI (Cursor/Trae)

大型專案，需要批次操作選終端 Agent (Claude Code)

快速原型驗證想法選雲端 Builder 或 Trae SOLO

圖 6：AI 程式設計工具選擇決策樹

三、常見問題 FAQ

Q1：我完全不會程式設計，能用 AI 程式設計嗎？

能。2026 年的 AI 程式設計工具已經足夠強大，你完全可以用自然語言描述需求，讓 AI 生成程式碼。但有兩點建議：第一，從雲端 Builder 工具（如 v0、Bolt）開始，它們不需要配置開發環境；第二，邊用邊學基礎概念——不需要精通語法，但你需要能“看懂”AI 寫的程式碼的大意，這樣才能有效驗收。

Q2：我應該學哪個程式語言？

取決於你要做什麼。做網站前端——JavaScript/TypeScript；做資料處理或 AI——Python；做企業級後端——Java 或 Go。但更重要的是：在 AI 程式設計時代，“語言”的重要性在下降。AI 能幫你處理語法細節，你需要理解的是“程式的結構”——什麼是函式、什麼是迴圈、什麼是資料結構。這些概念是跨語言的，一旦理解，換什麼語言都快。

Q3：AI 寫的程式碼有版權問題嗎？

截至 2026 年 8 月，這仍是一個有爭議的法律問題。一般來說，AI 生成的程式碼在大多數司法管轄區不被視為有明確的版權所有者。但如果你用於商業專案，建議：(1) 瞭解你使用的 AI 工具的服務條款中關於智慧財產權的條款；(2) 對關鍵程式碼進行人工審查和修改，使其具有你的創造性投入；(3) 避免讓 AI 直接複製可能有版權的開原始碼。

Q4：AI 會取代程式設計師嗎？

短期內不會。AI 取代的不是“程式設計師”這個職業，而是“逐行編碼”這項工作。就像計算機沒有取代數學家，而是讓數學家從繁瑣的手算中解放出來一樣。AI 程式設計工具讓程式設計師從“寫程式碼”轉向“想問題”——這實際上提高了程式設計師的價值上限。未來的競爭力不在於“能寫多少行程式碼”，而在於“能解決多複雜的問題”。

Q5：如何判斷 AI 的產出是否正確？

三層驗證法。第一層：功能驗證——它能不能跑？輸入輸出對不對？這是最基本的。第二層：邊界驗證——空輸入、超長輸入、異常輸入會怎樣？這能發現隱藏的問題。第三層：架構驗證——程式碼結構是否合理？有沒有硬編碼？是否便於擴充套件？這需要你有一定的技術判斷力。如果你無法判斷第三層，至少做到前兩層——能跑、邊界正常，就比大多數初學者強了。

四、推薦學習資源

以下是截至 2026 年 8 月的推薦學習資源。請注意，網際網路資源連結可能變化，如果某個連結失效，搜尋其標題通常能找到最新版本。

型別	資源	推薦理由
原始文獻	Andrej Karpathy 的 CLAUDE.md	五大原則的源頭，70 行但字字珠璣
原始文獻	Anthropic 官方檔案 (Claude Code)	終端 Agent 的最佳實踐指南
教學	Vibe Coding 社群整理的 Skills 合集	社群實踐者總結的實戰技巧集合
教學	Cursor 官方檔案和 YouTube 頻道	IDE 內建 AI 的權威使用指南
社群	GitHub 上的 awesome-claude-code	Claude Code 的最佳實踐和外掛集
社群	Reddit 的 r/cursor 社群	全球 Cursor 使用者的經驗分享和討論
基礎	MDN Web Docs (developer.mozilla.org)	前端開發最權威的參考檔案
基礎	Python 官方教學 (docs.python.org)	Python 學習的官方起點

推薦學習資源清單

五、結語：工具在手，方法論在心

回到本書的核心信念：工具每三個月迭代一次，方法論每三年才進化一次。在 2026 年 8 月的今天，AI 程式設計工具的能力已經強大到令人驚歎——你能用一句話建立一個網站，用一段描述生成一個全棧應用，用一條指令完成一個複雜重構。但工具越強大，方法論的重要性反而越突出——因為工具的強大放大了你的每一個決策。

如果你的需求清晰，AI 能幫你高效實現；如果你的需求模糊，AI 也會高效地做出錯誤的東西。如果你遵循簡單至上，AI 能幫你快速驗證想法；如果你貪多求全，AI 也會幫你快速堆積技術債務。如果你精準修改，AI 能幫你外科手術般地最佳化程式碼；如果你指令模糊，AI 也會幫你大刀闊斧地引入 bug。

這就是為什麼本書花了大部分篇幅講方法論而非工具操作。不是因為工具不重要，而是因為方法論是工具的“乘數”——方法論越好，工具的價值越大；方法論為零，工具再強也只是零乘以任何數。

你不再是一個人在程式設計。你是一個指揮官，AI 是你的軍團。你的思考質量決定了軍團的戰鬥力。從今天開始，用方法論武裝自己的思維，用工具放大自己的能力——這就是 AI 程式設計的全部秘密。

參考文獻

1. Andrej Karpathy. (2025). Vibe Coding. Social media post introducing the concept of vibe-driven programming with AI.

2. Andrej Karpathy. (2025). CLAUDE.md - Project guidelines for AI-assisted development. GitHub repository, 70+K stars.

3. Anthropic. (2026). Claude Code: Terminal-native AI programming. Official documentation and best practices.

4. Cursor Team. (2026). Cursor 3: The next generation of AI-first IDE. Release notes and documentation.

5. OpenAI. (2026). Codex CLI: Bringing AI to the terminal. Technical documentation.

6. Model Context Protocol (MCP) Specification. (2025-2026). Open protocol for AI-tool integration.

7. Vibe Coding Community. (2026). Community-curated skills and best practices for AI-assisted programming.

8. Spec-Driven Development (SDD) methodology. (2026). Emerging practices in specification-first AI programming.

八、安卓開發實戰篇（第 48-58 章）

第四十八章：認識安卓——從作業系統到開發生態

恭喜你做出這個決定——從“玩手機的人”變成“做手機應用的人”。在動手之前，咱們先把地圖攤開：安卓從哪裡來？為什麼全世界的開發者都在給它寫應用？一個 *App* 從你的手指敲鍵盤到使用者手機上，中間都發生了什麼？

一、安卓的身世：從“小機器人”到“全球最大系統”

安卓（Android）的故事始於 2003 年——一個叫 Andy Rubin 的工程師創辦了 Android 公司，目標很樸素：做一套“更懂相機”的智慧作業系統。後來谷歌在 2005 年收購了它，2007 年聯合幾十家廠商成立開放手機聯盟（OHA），2008 年 9 月，第一款安卓手機 HTC Dream（G1）正式發售。

後面的故事大家都知道了：到 2026 年，安卓是全球市場份額最大的移動作業系統，活躍裝置數以十億計。你的手機、電視盒子、車載大屏、智慧手錶，很可能都在跑安卓。這意味著什麼？意味著你寫的每一行程式碼，都有機會被世界上最多的人用到。

小貼士：安卓的開源基因

安卓的核心程式碼（AOSP，Android Open Source Project）是開源的——任何廠商都能拿去修改定製。所以小米有 MIUI（現在叫澎湃 OS）、華為有鴻蒙相容層、三星有 One UI。這對開發者是好事：你學的是“安卓開發”，不是“某個品牌的開發”，一套功夫，全網通吃。

版本號的進化也很有意思：從 1.5 的紙杯蛋糕（Cupcake）開始，每個大版本都以甜點命名——甜甜圈、閃電泡芙、凍酸奶、薑餅……一直到 2019 年的安卓 10 才改成了純數字。到 2026 年 8 月，最新正式版是 Android 17，而開發時你真正需要關心的“最低支援版本”，主流 App 一般定在 Android 8（API 26）到 Android 10（API 29）之間——支援到幾年前的裝置，你的使用者盤子才夠大。

版本	代號/年份	對開發者重要的事
Android 8 (API 26)	Oreo • 2017	通知管道、後臺限制——很多老機型的最低線
Android 10 (API 29)	2019	分割區儲存登場，檔案訪問規矩大改
Android 12 (API 31)	2021	Material You 動態取色，介面觀感大升級
Android 13 (API 33)	2022	執行時通知權限，跳出視窗問你” 準不準發通知”
Android 14-15 (API 34-35)	2023-2024	預測性返回、16KB 頁、隱私進一步收緊
Android 16-17 (API 36-37)	2025-2026	更快的小版本節奏、AI 能力整合（如 Gemini 擴充套件）

安卓版本簡史（截至 2026 年 8 月，開發重點關注項）

二、用什麼語言寫：Java 還是 Kotlin？

安卓開發歷史上出現過兩種主流語言。Java 是“開國元老”——安卓誕生之初就用它；Kotlin 是 2017 年穀歌欽點的 “一等公民”，2019 年起官方明確 “Kotlin 優先”（Kotlin-first），到 2026 年，新專案幾乎全部用 Kotlin，舊專案也大多在陸續遷移。

說句實在話：如果你是完全零基礎，直接學 Kotlin，別回頭碰 Java。原因有三：一是 Kotlin 程式碼量比 Java 少 30%-40%（同樣的功能，Kotlin 更短更清晰）；二是它天生防 “空指標” 這種新手噩夢；三是所有現代教學、官方檔案、AI 程式設計工具（還記得咱們的 AI 程式設計書嗎？）都在以 Kotlin 為準。Java 只在維護老專案、讀老程式碼時會遇到。

Java 風格（囉嗦版）

```
public class MainActivity extends Activity \{
@Override
protected void onCreate(Bundle b) \{
super.onCreate(b);
setContentView(R.layout.activity);
\}
\}
```

Kotlin 風格（清爽版）

```
class MainActivity : ComponentActivity() \{
override fun onCreate(b: Bundle?) \{
super.onCreate(b)
setContent \{ App() \}
\}
\}
```

同樣一件事，Kotlin 少了一堆花括號和樣板。第 50 章咱們會正式學它，現在你只需要記住：安卓開發的“普通話”叫 Kotlin。

三、一個 App 是怎麼誕生的？

很多人以為“開發 App”就是開啟電腦狂寫程式碼。實際上，一個 App 從想法到使用者手機上，大致要經過這樣一條流水線：

想法

設計（介面 + 功能）

寫程式碼

編譯打包（APK/AAB）

測試（模擬器 + 真機）

釋出（商店/安裝包）

圖 1：一個安卓 App 從想法到上架的全流程

流程裡有兩個概念你現在就要認識：

- APK：安卓應用的安裝包格式（.apk），連點兩下就能裝到手機上，就像 Windows 的.exe。平時自己測試、給朋友傳著玩，用它。

- AAB：Android App Bundle（.aab），上架 Google Play 用的” 進階打包格式”。商店會根據使用者的手機型號動態裁剪，讓每個人下載的包更小。自己裝不了，只能交給商店。

另外你還會不斷聽到一個詞：SDK（Software Development Kit，軟體開發工具包）。它就是安卓官方提供的一整套“開發裝備”——包括各版本的 API 庫、除錯工具、模擬器映象等。你寫程式碼時呼叫的 Activity、Button 這些類，都來自 SDK。

四、安卓技術棧全景：你的“武器清單”

正式學習前，先認識一下 2026 年主流安卓專案的標準配置——這張清單你現在看不懂沒關係，全書學完回來看，會豁然開朗：

層次	主流選擇（2026）	幹什麼用的
語言	Kotlin 2.x	寫邏輯的語言
介面	Jetpack Compose + Material 3	用程式碼畫介面（本書主線）
架構	MVVM + ViewModel	把介面和邏輯分開，程式碼不亂
資料儲存	Room（資料庫）+ DataStore（配置）	把資料存進手機
網路	Retrofit + OkHttp + Kotlin 協程	和伺服器對話
構建工具	Gradle + Kotlin DSL	把程式碼變成安裝包
開發環境	Android Studio	谷歌官方的” 全家桶”IDE

2026 年安卓主流技術棧速覽

別被這一串英文嚇到。它們就像廚房裡的鍋碗瓢盆——你現在只需要知道“做飯要進廚房”，具體每樣工具怎麼用，後面章節逐個拆給你看。而且好訊息是：Android Studio 建立新專案時會自動幫你把這一整套配置好，你只管炒菜。

五、安卓 vs iOS：開發者的第一堂“選邊課”

你手裡可能正同時有安卓和蘋果裝置。作為開發者，兩邊的體驗差異值得提前瞭解——不是“哪個更好”的站隊，而是“哪邊更適合你起步”的現實帳：

維度	安卓開發	iOS 開發
開發電腦	Windows / Mac / Linux 都行	必須 Mac（Xcode 獨佔）
開發工具	Android Studio（免費）	Xcode（免費）+ 上架年費 99 美元
上架稽核	寬鬆，幾小時 ~ 幾天	嚴格，1-7 天是常態
裝機試玩	APK 隨便發、隨便裝	分發受限，測試需簽名/TestFlight
收入環境	廣告/國內管道為主	App Store 內購付費習慣更好
學習成本	概念較多（四大元件等）	生態封閉但體系統一、檔案精良

安卓與 iOS 開發路徑對比（2026 年 8 月）

說句實在話：如果你的電腦不是 Mac，那這道題根本不用選——安卓是唯一選項。如果你有 Mac，本書第八、九兩篇可以對照著讀，程式設計思路是相通的，學完一邊再學另一邊會快很多。

來動手：看看你手機裡的”安卓”

開啟手機的 [設定] [關於手機]（或“關於本機”），找到“安卓版本”或“Android 版本”那一欄——連續點它幾下，還能觸發系統的彩蛋小遊戲。記下你的版本號，對照上面的版本表看看它屬於哪一代。你的手機，就是你的第一個“開發測試機”。

本章小結

- 安卓 2008 年誕生，2026 年是全球市場份額最大的移動系統，活躍裝置數十億
- 最新正式版 Android 17；主流 App 最低支援 Android 8-10，使用者盤子才夠大
- 新專案一律用 Kotlin：程式碼少 30%+、防空指標、官方欽定
- APK 自己裝，AAB 上商店；SDK 是官方開發裝備包
- 2026 標準技術棧：Kotlin + Compose + MVVM + Room + Retrofit + Gradle + Android Studio

第四十九章：搭建開發環境——Android Studio 與第一個專案

工欲善其事，必先利其器。這一章咱們把“廚房”搭好：裝好 *Android Studio*、跑通模擬器、建立人生第一個安卓專案，再把工程結構拆開看個明白。裝好環境之後，你會第一次看到自己寫的程式碼變成一個能點的按鈕——這個瞬間，很多人會記一輩子。

一、安裝 Android Studio

Android Studio 是谷歌官方的整合開發環境（IDE），免費，功能全家桶：寫程式碼、看介面預覽、除錯、打包全在裡面。安裝四步：

1. 開啟官網 developer.android.com/studio，下載最新穩定版（2026 年 8 月是 Narwhal 之後的版本，Windows/Mac/Linux 都有）；

2. 執行安裝包，一路” 下一步”——安裝嚮導會自動帶上 Android SDK 和模擬器元件；

3. 首次啟動時按提示下載 SDK 元件（預設勾選就好，可能需要幾分鐘和幾個 GB 的空間）；

4. 完成後你會看到歡迎頁——恭喜，” 廚房” 搭好了。

注意：硬體要求心裡有數

Android Studio 是個“吃記憶體”的傢伙：建議 16GB 記憶體起步（8GB 也能跑，但模擬器一開會比較卡）、磁碟預留 20GB 以上、CPU 支援虛擬化（BIOS 裡開啟 VT-x/AMD-V）。如果電腦確實老舊，可以用“真機除錯”代替模擬器（本章第四節會講），體驗反而更流暢。

二、建立你的第一個專案

在歡迎頁點 New Project，選 Empty Activity 模板（最乾淨的新手模板），然後填三樣東西：

配置項	填什麼	說明
Name	My First App	應用程式名稱字，顯示在手機桌面上
Package name	com.example.myfirstapp	全球唯一標識，習慣是” 域名反寫”，先按預設
Minimum SDK	API 26（Android 8.0）	最低支援版本，覆蓋 95%+ 裝置

建立專案要填的三件事

點 Finish 後，Android Studio 會用 Gradle 自動下載依賴、構建專案（第一次要等幾分鐘，之後每次會快很多）。構建完成後，你就有了一個能跑的 App——雖然它還只會顯示一行“Hello Android!”。

小貼士：包名的門道

包名（package name）就像 App 的“身份證號”，全球不能重複，上架後就不能改了。個人練習用 com.example.xxx 就行；正式產品要換成你自己的域名反寫（比如 com. 你的名字. 應用程式名稱）。別用 com.example 上架——商店會拒收。

三、跑起來：模擬器與真機

程式碼寫好了，得找個“手機”來跑。你有兩個選擇：

模擬器（Emulator）

模擬器是在電腦裡虛擬出一臺安卓手機。在 Android Studio 裡開啟 Device Manager Create Device，挑一臺裝置（推薦 Pixel 系列）選一個系統映象（帶 Google APIs 的）下載啟動。然後點工具欄的綠色三角 Run，幾秒鐘後，“手機”螢幕上就會出現你的 App。

真機除錯（更推薦）

真機永遠是最真實的測試環境——滑動手感、真網路、真感測器，模擬器都比不了。安卓真機除錯很簡單：

1. 手機上開啟 [設定] [關於手機]，連點” 版本號” 7 次，開啟” 開發者選項”；
2. 在 [開發者選項] 裡開啟 USB 除錯；
3. 用資料線連電腦，手機跳出視窗選” 允許 USB 除錯”；
4. 回到 Android Studio，工具欄的裝置下拉選單裡就會出現你的手機，點 Run。

避坑提醒：USB 除錯別常開

USB 除錯相當於給電腦開了“後門”。在公共場合充電、連線不信任的電腦時，務必關閉 USB 除錯，防止別人透過它讀取你的資料。平時開發用完就關，養成習慣。

四、工程結構拆解：每個檔案是幹嘛的

第一次開啟專案，左側的檔案樹可能讓你頭暈。別慌，日常開發你 90% 的時間只跟下面這幾個打交道：



圖 2：新手必懂的工程結構（口訣：只碰 app/ 目錄）

- MainActivity.kt：App 的” 大門”。使用者點開圖示，第一個被喚醒的就是它。現在的模板裡它負責顯示”Hello Android!”。
- AndroidManifest.xml：App 的” 戶口本”——宣告這個 App 叫什麼、圖示是哪個、有哪些頁面、要用哪些權限（相機、定位等）。
- res/ 目錄：資源倉庫，存圖片、圖示、字串、顏色。把文字放資原始檔而不是寫死在程式碼裡，是安卓開發的好習慣（換語言、換主題都方便）。
- build.gradle.kts：構建配方——宣告最低支援版本、依賴哪些庫（比如 Compose）。

五、小試牛刀：把“Hello”換成你的名字

開啟 MainActivity.kt，你會看到 Compose 模板生成的程式碼。找到 Text(“Hello Android!”) 這一行，把引號裡的字換成你的名字，比如 Text(“你好，張三！這是我的第一個 App”)，然後按 Ctrl+R（Mac 是 Cmd+R）重新執行。幾秒後，模擬器上就顯示了你的名字。

```
setContent {
    MyFirstAppTheme {
        Surface(modifier = Modifier.fillMaxSize()) {
            // ↓ 把這裡改成你自己的文字
            Text("你好，張三！這是我的第一個App")
        }
    }
}
```

恭喜，你已經完成了一次完整的“改程式碼編譯安裝執行”迴圈。這個迴圈，就是未來每一天的開發日常。趁熱打鐵，試著再改點別的：把字號調大、換個顏色——下一章我們就會學到這些“魔法”是怎麼來的。

六、常見報錯急救：新手前兩週的“急診室”

環境搭建和首次構建階段，你會大機率遇到下面三種報錯。別慌，都有解藥：

報錯現象	常見原因	解藥
Gradle sync failed / 依賴下載慢	網路訪問倉庫不暢	給 Gradle 配置國內映象倉庫（阿里雲/騰訊雲 maven 映象），網上搜”gradle 國內映象” 照著改
SDK 元件缺失	安裝時跳過了 SDK 下載	Settings SDK Manager 補裝缺失的平臺與工具
模擬器啟動失敗 / 黑屏	虛擬化未開啟、映象損壞	BIOS 開 VT-x/AMD-V；刪掉模擬器重建；或直接用真機除錯

新手環境報錯急救表

小貼士：搜報錯的方法論

報錯資訊第一行是最有價值的——把它原樣複製進搜尋引擎，加“android studio”關鍵字。別搜“我裝不上環境怎麼辦”這種籠統問題，搜“Gradle sync failed: Could not resolve com.android.tools.build”這種具體錯誤。精確的問題，才有精確的答案。

來動手：完成你的”開發儀式感”清單

(1) 裝好 Android Studio 並跑通模擬器；(2) 用真機跑一次自己的 App 並截圖留念；(3) 把 Hello 文字改成你的名字；(4) 把專案壓縮存個檔。做完這四步，你就可以在朋友圈宣佈：我也是安卓開發者了（截圖為證）。

本章小結

- Android Studio = 官方免費 IDE；建議 16GB 記憶體 + 20GB 磁碟 + 開虛擬化
- 新增專案三要素：名字、包名（全球唯一，com.example 別上架）、最低 SDK（API 26 起）
- 兩種執行方式：模擬器（電腦裡虛擬手機）與真機（開 USB 除錯，用完記得關）
- 工程口訣：只碰 app/ 目錄——MainActivity.kt 是大門，Manifest 是戶口本，res/ 是資源倉庫
- 開發日常 = 改程式碼 Run 看效果，迴圈往復

第五十章：Kotlin 快速上手——安卓開發的第一語言

這一章咱們正式學語言。別緊張——*Kotlin* 是出了名的“對新手友好”，而且你不需要背語法書：先掌握 *20%* 的語法，就能看懂和寫出 *80%* 的日常程式碼。剩下的邊做專案邊查。記住咱們的學習原則：看懂 > 記住，會用 > 精通。

一、變數與型別：給資料起名字

程式就是“資料 + 邏輯”。變數，就是給資料起個名字存起來。Kotlin 有兩種宣告方式：

```
val name = "張三"          // val：只讀變數（推薦預設用這個）
var age = 18               // var：可變變數（以後還能改）

age = 19                   // <i class="fas fa-check-circle mr-1"></i> var 可以改
// name = "李四"           // <i class="fas fa-times-circle mr-1"></i> val 不能改，編譯會報錯

val score: Int = 100       // 也可以顯式寫明型別
val pi = 3.14              // 不寫也能自動推斷
```

常用型別就幾種，一張表記住：

型別	存什麼	示例
Int	整數	42
Double	小數	3.14
String	文字	” 你好”
Boolean	真/假	true / false
List< T>	一組資料	listOf(1, 2, 3)

Kotlin 常用基礎型別

小貼士：val 優先原則

Kotlin 社群的鐵律：預設用 val，確有必要再用 var。能用 val 的地方用 var，IDE 會黃字警告你。這個習慣能幫你少掉一大類“變數被意外修改”的 bug——程式碼越少變化，越容易推理。

二、函式：把邏輯打包成“按鈕”

函式是一段有名字的邏輯，像遙控器上的按鈕——按一下（呼叫），就執行一串動作。Kotlin 裡用 fun 宣告：

```
fun greet(name: String): String {
    return "你好，$name！"      // $ 符號把變數插進字串
}

println(greet("張三"))         // 輸出：你好，張三！

// 單行函式可以簡寫成表示式
fun add(a: Int, b: Int) = a + b

println(add(3, 5))             // 輸出：8
```

幾個要點：引數寫在括號裡（名字在前、型別在後，用冒號隔開）；返回型別寫在括號後；字串模板 $ 變數名是 Kotlin 的招牌便利——不用再像 Java 那樣“字串 + 變數”地拼來拼去。

三、條件與迴圈：讓程式會“思考”

程式的智慧來自兩件事：會判斷（if/when）和會重複（for/while）。

```
// 判斷：if / else
val age = 17
if (age >= 18) {
    println("成年人")
} else {
    println("未成年")
}

// 多分支：when（比 switch 好用一百倍）
val score = 85
val level = when {
    score >= 90 -> "優秀"
    score >= 60 -> "及格"
    else -> "再接再厲"
}

// 重複：for 迴圈
```

```
val fruits = listOf("蘋果", "香蕉", "橙子")
for (fruit in fruits) {
    println("我愛吃$fruit")
}
```

when 是 Kotlin 裡最常用的多分支寫法，安卓開發裡你會天天用到它（比如根據狀態顯示不同介面）。

四、空安全：Kotlin 的“保命符”

Java 時代最著名的 bug 叫 NullPointerException（空指標異常）——程式訪問了一個“什麼都沒有”的東西，直接崩潰。據統計它是安卓崩潰排行榜的常年第一。Kotlin 從語言層面解決了它：預設不允許空值。

```
var name: String = "張三"   // 普通型別：絕不允許是 null
// name = null                 // <i class="fas fa-times-circle mr-1"></i> 編譯錯誤，直接把你攔住

var nickname: String? = null // 加個問號：允許為 null
nickname = "三兒"

// 使用可空型別時，必須安全地處理：
println(nickname?.length)    // 安全呼叫：null 就返回 null，不崩
val len = nickname?.length ?: 0   // Elvis 運運算元：null 時給預設值
println(len)
```

注意：? 和?: 別搞混

String? 是“允許為空”，a ?: b 是“a 為空時用 b”（長得像貓王髮型，所以叫 Elvis 運運算元）。剛開始寫程式碼時，看到編譯器報“null 相關錯誤”，八成是你忘了處理可空型別——這正是語言在保護你。

五、類與物件：造“零件”的模具

類是物件的“模具”，物件是類的“成品”。安卓裡的一切——頁面、按鈕、資料——都是物件。Kotlin 裡有個神器叫 data class（資料類），專門用來“裝資料”，一行頂 Java 幾十行：

```
// 普通類：模具
class Dog(val name: String) {
    fun bark() = println("$name 汪汪！")
}

val dog = Dog("旺財")       // 造物件
dog.bark()                  // 旺財 汪汪！
```

```
// data class：裝資料的盒子（自動帶 equals/copy 等）
data class User(val id: Int, val name: String, val email: String)

val me = User(1, "張三", "zhangsan@example.com")
val twin = me.copy(name = "張三豐")   // 複製並改一個欄位
```

以後你做記帳本，一筆帳就是一個 data class：data class Record(val amount: Double, val category: String, val date: String)。看，你已經能寫出真實專案的程式碼結構了。

六、lambda 與進階函式：Kotlin 的“語法糖”

安卓程式碼裡你會天天看到 { ... } 這種“沒名字的函式”，它叫 lambda（匿名函式）。Button 的 onClick = { count++ } 就是一個 lambda——把“點選後做什麼”打包成一坨，遞給按鈕。

```
// 普通寫法：定義一個有名函式再傳入
fun double(x: Int) = x * 2
val result = listOf(1, 2, 3).map(::double)

// lambda 寫法：就地定義，輕快直接
val result2 = listOf(1, 2, 3).map { it * 2 }   // it = 當前元素

// 篩選 + 變換連招（真實專案裡極常見）
val bills = listOf(12.5, 88.0, 5.0, 200.0)
val bigBills = bills.filter { it > 50 }.sorted()
// [88.0, 200.0]——"超過 50 的帳單，排好序"
```

幾個高頻“集合連招”記一下：filter（篩選）、map（變換）、sorted（排序）、sum（求和）。你未來 90% 的列表處理，都是這四招的組合。Kotlin 的哲學就是：能用一句話說清楚的邏輯，絕不寫五句。

七、寫給零基礎的語法速查卡

別試圖一天背完所有語法。把這張卡存下來，寫程式碼時對照著查，兩週後自然就熟了：

要做什麼	寫法
定義只讀變數	`val x = 1`
定義可變變數	`var x = 1`
定義函式	fun 名字 (引數): 返回型別 { ... }
判斷	if (條件) {...} else {...}
多分支	when { 條件 -> 結果}
遍歷	for (x in 列表) { ... }
字串拼接	” 你好，$name”
處理可空	`x?.length ?: 0`
定義資料類	`data class X(val a: Int, val b: String)`

Kotlin 語法速查卡（新手版）

來動手：不裝任何軟體，先玩 10 分鐘 Kotlin

瀏覽器開啟 play.kotlinlang.org（Kotlin 官方線上演練場），把本章的每一段程式碼都粘進去跑一遍，改改數字和文字，看看輸出怎麼變。這個“先玩後學”的 10 分鐘，會讓你第 51 章上手時輕鬆一倍。

本章小結

- val 優先，var 慎用；型別可自動推斷，冒號後顯式宣告也行
- 函式 = 打包的邏輯；字串模板 $ 是招牌便利
- if/when 判斷，for 迴圈；when 是安卓開發裡的” 多面手”
- 空安全是 Kotlin 的保命符：普通型別拒絕 null，可空型別必須? / ?: 處理
- data class 是裝資料的盒子，一個欄位一行，真實專案天天用
- 20% 語法幹 80% 的活：先會用，再精通

第五十一章：介面開發——Jetpack Compose 入門

介面是使用者看到的一切。在 *Compose* 出現之前，安卓介面用 *XML* 寫——像填空題一樣繁瑣；*2021* 年穀歌推出了 *Jetpack Compose*：直接用 *Kotlin* 程式碼“畫”介面，所見即所得，改一行程式碼預覽立刻重新整理。到 *2026* 年，*Compose* 已是新專案的絕對主流。這一章，咱們學會“畫畫”。

一、Compose 的思想：介面就是函式

Compose 的核心思想一句話：介面 = 函式。每個能顯示在螢幕上的東西（文字、按鈕、圖片）都是一個“可組合函式”（Composable），用 @Composable 標記。把它們像積木一樣巢狀起來，就是介面。

```
@Composable
fun Greeting(name: String) {
    Text(text = "你好，$name！")      // 一個"文字積木"
}

// 巢狀組合：大積木套小積木
@Composable
fun HomeScreen() {
    Column {                            // 豎著排列
        Greeting(name = "張三")
        Greeting(name = "李四")
        Button(onClick = { /* 點選做什麼 */ }) {
            Text("點我")
        }
    }
}
```

注意 @Composable 這個“帽子”——它告訴編譯器“這是一個介面積木”。積木可以無限巢狀，這就是 Compose 的全部秘密。

二、基礎元件：文字、按鈕、輸入框、圖片

Compose 的元件就是“積木塊”，最常見的四種：

元件	幹什麼	必備引數
Text	顯示文字	text（要顯示的字）
Button	按鈕	onClick（點選回撥）
TextField	輸入框	value、onValueChange（輸入變化回撥）
Image	顯示圖片	painter（圖片資源）

Compose 四大基礎元件

```
@Composable
fun BasicWidgets() {
    Column {
        Text(text = "這是一段文字")
        Button(onClick = { println("被點選了") }) {
            Text("按鈕")
        }
        var input by remember { mutableStateOf("") }
        TextField(
            value = input,
            onValueChange = { input = it },
            label = { Text("輸入點什麼") }
        )
    }
}
```

輸入框那兩行裡出現了 remember { mutableStateOf(“”) }——這就是 Compose 的“狀態”魔法，本章第五節細講。現在先混個臉熟。

三、佈局三件套：Column、Row、Box

元件有了，怎麼擺放？記住三個“盒子”：

- Column：豎著排（像排隊打飯）；
- Row：橫著排（像並排坐）；
- Box：疊著放（像撲克牌摞一起，適合做角標、遮罩）。

```
Column {
    Text("第一行")
    Row {
```

```
        Text("左邊")
        Text("右邊")
    }
    Box {
        Text("底層")
        Text("疊在上面的角標")
    }
}
```

想讓佈局更好看？給元件加 Modifier（修飾符）——它就像給積木“上妝”：

```
Text(
    text = "好看的文字",
    modifier = Modifier
        .padding(16.dp)           // 四周留白
        .fillMaxWidth()           // 撐滿寬度
        .background(Color.Blue)   // 背景色
        .clickable { /* 點選 */ }// 可點選
)
// 鏈式呼叫：一個接一個，順序有意義（先 padding 再背景，背景不含邊距）
```

注意：Modifier 的順序有講究

padding(16.dp).background(Blue) 和 background(Blue).padding(16.dp) 效果不同——前者是“先留白再塗色”（背景不含邊距），後者是“先塗色再留白”（背景包含邊距）。新手最容易在這上面畫錯“奇怪的白邊”。不確定時，調換順序試試，預覽立刻告訴你答案。

四、狀態：讓介面“活”起來

靜態介面沒意思。想讓按鈕點選後數字 +1、輸入框能打字，就需要狀態（State）。Compose 的規則：介面永遠忠實反映狀態，狀態變了，介面自動重畫。

```
@Composable
fun Counter() {
    // remember + mutableStateOf：記住這個會變化的值
    var count by remember { mutableStateOf(0) }

    Column {
        Text(text = "你點了 $count 次")
        Button(onClick = { count++ }) {   // 改狀態
            Text("點我 +1")                // 介面自動重新整理
        }
    }
}
```

三行程式碼，一個活的計數器。這就是 Compose 比老式寫法爽的地方：不用手動“重新

整理介面”，改資料就行。記住兩個關鍵函式：remember（讓值在介面重組時不被清空）和 mutableStateOf（告訴 Compose “這個值變了要重畫”）。

使用者點選

狀態改變（count++）

Compose 感知到變化

自動重畫介面（重組）

圖 3：Compose 的“狀態介面”單向資料流

五、主題與深色模式：一次配置，全域性生效

模板專案裡的 MyFirstAppTheme { ... } 就是主題。它基於 Material 3 設計規範，幫你把顏色、字型、形狀統一管理。支援深色模式只需三步：

1. 在主題裡定義好淺色/深色兩套配色（模板已自動生成 ColorScheme）；
2. 系統切到深色模式時，isSystemInDarkTheme() 自動判斷；
3. 用主題提供的顏色（如 MaterialTheme.colorScheme.primary）而不是寫死顏色值。

小貼士：Material 3 動態取色

Android 12 起的“Material You”能讓 App 從你的桌布裡提取顏色。Compose 模板預設支援（dynamicColor）。你的 App 在不同使用者手機上會“自動換裝”，這就是現代安卓的浪漫。

六、預覽、動畫與實時編輯：Compose 的“外掛”

Compose 三個開發體驗上的“外掛”，用上就回不去：

- @Preview 預覽：給 Composable 加 @Preview 註解，Android Studio 右側立刻渲染出介面——改程式碼所見即所得，不用每次裝到手機上；
- 實時編輯（Live Edit）：2026 年的 Android Studio 支援在模擬器/真機上” 熱更新” 部分改動，改文字顏色之類的小調整，裝置上秒變；
- 動畫 API：animateFloatAsState、AnimatedVisibility 幾行程式碼就能讓元素平滑過渡——” 進階感” 往往就差這幾十行。

```
@Composable
fun AnimatedBox() {
    var big by remember { mutableStateOf(false) }
    // 值變化時自動平滑過渡到目標值
```

```
    val size by animateDpAsState(if (big) 120.dp else 60.dp)

    Box(Modifier
        .size(size)
        .background(MaterialTheme.colorScheme.primary)
        .clickable { big = !big }
    )
}
```

小貼士：先功能後動畫

新手常犯的毛病是“介面還沒做對，就開始糾結動畫”。先讓功能跑通，最後再統一加動效——這也是“簡單至上”原則：動畫是錦上添花，功能是雪中送炭。

來動手：做一個”心情按鈕”

把模板專案改成一個心情切換器：一個按鈕，點選後在開心難過生氣之間迴圈切換，文字和背景色跟著變。提示：用 when 根據狀態選文案和顏色。做完發給朋友玩，收穫第一波“哇你會做 App 了”。

本章小結

- Compose 思想：介面 = @Composable 函式，積木式巢狀
- 四大元件：Text / Button / TextField / Image
- 佈局三件套：Column 豎排、Row 橫排、Box 疊放
- Modifier 鏈式”上妝”：padding、fillMaxWidth、background、clickable——順序有意義
- 狀態是靈魂：remember + mutableStateOf，改資料介面自動重畫
- 主題用 Material 3 + 動態取色，深色模式一次配置全域性生效

第五十二章：四大元件與 Activity 生命週期

如果說 *Compose* 是 *App* 的“臉”，那這一章講的就是 *App* 的“骨架”。安卓系統有四大基本元件，所有 *App* 都是由它們拼成的；而其中最重要的 *Activity*，有一張每個安卓開發者都必須刻在腦子裡的“生命週期圖”。本章學完，你就能理解“為什麼切個後臺回來，我的 *App* 狀態就沒了”這類千古之謎。

一、四大元件：安卓的“四梁八柱”

安卓官方定義了四種基本元件，分別負責一類工作：

元件	職責	打個比方
Activity	一個可見的介面	店鋪的門面——使用者能看見、能互動
Service	後臺幹活的”隱形人”	倉庫工人——放音樂、下載檔案，不需要介面
BroadcastReceiver	接收系統廣播	傳達室——”電池沒電了””網路斷了”的喇叭
ContentProvider	跨 App 共享資料	公共資料櫃——比如把聯絡人共享給別的 App

安卓四大元件速覽

新手階段你 95% 的時間都在跟 Activity 打交道；Service 和 BroadcastReceiver 做音樂播放、訊息推送時會用到；ContentProvider 用得最少（系統已把常用資料封好了）。另外注意：四大元件都要在 AndroidManifest.xml 裡“登記戶口”，不登記系統就找不到它們。

二、Activity 生命週期：安卓開發的第一張“藏寶圖”

手機上的 App 隨時可能被打斷：來電話了、使用者切去回微信、系統記憶體不夠把你殺了……Activity 的一生，就是在這張圖上迴圈旅行：

onCreate（出生：初始化）

onStart（可見了）

onResume（前臺互動中）

onPause（部分被遮住）

onStop（完全看不見）

onDestroy（銷燬）

onRestart / onStart / onResume：回到前臺時原路返回

圖 4：Activity 生命週期（前進方向：上排出生下排離場，回來時原路折返）

回撥	什麼時候觸發	典型用途
onCreate	Activity 建立時（一次）	載入介面、初始化資料
onStart	介面變為可見	開始一些輕量邏輯
onResume	拿到焦點、可互動	開始動畫、還原播放
onPause	被部分遮擋/切走瞬間	暫停動畫、儲存草稿
onStop	完全不可見	釋放重資源、暫停下載
onDestroy	被銷燬前	最後的清理

六個生命週期回撥的典型用途

避坑提醒：螢幕旋轉會“殺死”Activity

新手最經典的翻車現場：App 轉一下螢幕，輸入框裡的字全沒了。原因：旋轉螢幕預設會銷燬並重建 Activity（走一遍 onDestroy onCreate）。解法：用 rememberSaveable 儲存狀態（Compose 裡替代 remember），它會在重建時自動把值找回來。第 6、7 章會反覆用到。

三、Intent：App 裡外通行的“車票”

Intent（意圖）是元件之間通訊的“車票”。你想從一個頁面跳到另一個頁面，或者開啟瀏覽器、調起相機，都要“買票上車”：

```
// 方式一：顯式 Intent——明確指定去哪個頁面
val intent = Intent(this, DetailActivity::class.java)
intent.putExtra("name", "張三")      // 車上捎帶"行李"（資料）
startActivity(intent)

// 方式二：隱式 Intent——只說要幹什麼，系統找誰幹
val web = Intent(Intent.ACTION_VIEW, Uri.parse("https://developer.android.com"))
startActivity(web)   // 系統會開啟瀏覽器/或讓你選
```

兩種方式各有用途：顯式用於自己 App 內部的頁面跳轉；隱式用於呼叫系統或其他 App 的能力（開網頁、打電話、分享）。Compose 專案裡，在 Composable 中用 LocalContext.current 拿到 Context 就能 startActivity。

四、多頁面與導航

真實 App 都有多個頁面。Compose 官方推薦的導航方案是 Navigation Compose（依賴庫 androidx.navigation）：

```
val navController = rememberNavController()

NavHost(navController, startDestination = "home") {
    composable("home") { HomeScreen(navController) }
    composable("detail/{name}") { entry ->
        val name = entry.arguments?.getString("name")
        DetailScreen(name ?: "未知")
    }
}

// 在頁面裡跳轉：
navController.navigate("detail/張三")      // 帶引數
navController.popBackStack()               // 返回上一頁
```

小貼士：頁面多了怎麼辦？

專案長大後，把所有頁面路由集中寫在一個檔案裡（比如 AppNavGraph.kt），統一管理，別讓“跳轉到哪”的邏輯散落在各個頁面。這是從“能跑”走向“可維護”的第一步。

五、順帶認識：Fragment 是什麼（以及為什麼本書沒教）

查老教學時你一定會撞見 Fragment——它是 Activity 裡的“可複用介面片段”，在老式多頁面架構裡是絕對主角。但 Compose 時代，頁面直接用 Composable 函式 + Navigation 路由就夠了，官方也已弱化 Fragment 的地位。本書因此沒把它放進主線。

給你的建議：新專案不主動用 Fragment；但讀老程式碼、面試被問到時，知道它是“Activity 裡嵌的介面片段”即可。學習資源永遠優先選 2023 年以後的——安卓技術棧的“老黃曆”更新得比想象中快。

來動手：給計數器裝上”記憶”

把第 51 章的計數器改造一下：(1) 把 remember 換成 rememberSaveable，旋轉螢幕驗證數字還在；(2) 加一個“關於”按鈕，用 Navigation 跳到第二個頁面顯示版本號；(3) 在兩個頁面的 onCreate/onPause 裡列印日誌（Log.d），用 Logcat 觀察生命週期呼叫順序。做完這三步，生命週期就不再是紙上談兵。

本章小結

- 四大元件：Activity（門面）、Service（後臺）、BroadcastReceiver（喇叭）、ContentProvider（資料櫃）
- 生命週期六回撥：onCreate onStart onResume onPause onStop onDestroy

- 旋轉螢幕會銷燬重建 Activity：用 rememberSaveable 保住狀態
- Intent 是” 車票”：顯式跳自家頁面，隱式調系統能力
- 多頁面用 Navigation Compose：NavHost + composable 路由

第五十三章：資料儲存與網路請求

一個只能“看”的 *App* 是死的；能存資料、能連網路的 *App* 才是活的。這一章認識安卓資料層的“三件套”：*DataStore*（存小配置）、*Room*（存結構化資料）、*Retrofit*（和伺服器對話），外加它們的“總排程”——協程。學完這章，你就能寫會記住使用者習慣、會實時更新內容的真 *App* 了。

一、儲存方案怎麼選：先看這張表

方案	適合存什麼	特點	本書哪裡用
DataStore	零散配置（主題、開關）	鍵值對，簡單安全	第 54 章記帳本設定
Room	結構化資料（賬目、使用者）	真正的 SQLite 資料庫封裝，可查詢	第 54 章賬目記錄
Retrofit	（不存，聯網取）伺服器資料	HTTP 請求庫的事實標準	第 55 章天氣資料

資料三件套的分工

記一個口訣：“小配置進 DataStore，成批資料進 Room，外面的資料走 Retrofit”。舊教學裡常見的 SharedPreferences 已被 DataStore 取代，SQLite 手寫 SQL 也已被 Room 取代——直接學新的，別走老路。

二、DataStore：存點小配置

想記住“使用者上次選的幣種”“是否開啟深色模式”這類零散值，用 DataStore（Preferences 版）：

```
val Context.dataStore by preferencesDataStore(name = "settings")

// 寫：存一個值
suspend fun saveTheme(isDark: Boolean) {
    context.dataStore.edit { prefs ->
        prefs[THEME_KEY] = isDark
    }
```

```
}

// 讀：取出來（Flow 會自動通知變化）
val themeFlow: Flow<Boolean> = context.dataStore.data
    .map { it[THEME_KEY] ?: false }
```

注意返回的是 Flow——一個會“流動”的資料流。值一旦改變，所有觀察它的地方自動收到新值。這個概念（響應式）是安卓現代的基石，第 54 章會實戰。

三、Room：給資料安個“真正的家”

記帳本有成百上千筆賬，還要按分類、按月份查詢統計——這需要資料庫。Room 是 SQLite 的官方封裝，把“建表、查資料”變得像寫 Kotlin 一樣自然。三樣東西：

```
// 1. Entity：表結構（一張表 = 一個 data class）
@Entity(tableName = "records")
data class Record(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val amount: Double,
    val category: String,
    val note: String,
    val date: String
)

// 2. Dao：操作介面（增刪改查）
@Dao
interface RecordDao {
    @Query("SELECT * FROM records ORDER BY date DESC")
    fun getAll(): Flow<List<Record>>

    @Insert
    suspend fun insert(record: Record)

    @Delete
    suspend fun delete(record: Record)

    @Query("SELECT SUM(amount) FROM records WHERE category = :c")
    suspend fun sumByCategory(c: String): Double
}

// 3. Database：總裝
@Database(entities = [Record::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun recordDao(): RecordDao
}
```

這套“三件套”模板可以原樣搬進你的任何專案——換換欄位名就行。查詢返回 Flow，介面跟著資料自動重新整理，妙不可言。

四、Retrofit：和伺服器說話

天氣、新聞、匯率……這些資料不在手機裡，在別人的伺服器上。Retrofit 讓“向伺服器要資料”變得像調函式一樣簡單。三步走：

```
// 1. 定義介面：把 HTTP 請求描述成函式
interface WeatherApi {
    @GET("v1/weather")
    suspend fun getWeather(@Query("city") city: String): WeatherResponse
}

// 2. 建立例項
val api = Retrofit.Builder()
    .baseUrl("https://api.example.com/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()
    .create(WeatherApi::class.java)

// 3. 像調函式一樣呼叫（在協程裡）
val result = api.getWeather("北京")
```

避坑提醒：兩條網路鐵律

(1) 網路請求絕不能在主執行緒做——會卡死介面甚至崩潰（NetworkOnMainThreadException）。所有網路呼叫必須放在協程（viewModelScope.launch）裡。(2) 明文 HTTP 預設被禁止：安卓 9 起要求 HTTPS。測試時若要用 http:// 的本地地址，需要在 Manifest 裡顯式宣告 usesCleartextTraffic——但正式產品必須全走 HTTPS。

五、JSON 解析：給網路資料“翻譯”

伺服器返回的資料大多是 JSON——一串像“巢狀字典”的文字。Retrofit 需要一個“翻譯官”把它變成 Kotlin 物件。2026 年的兩大主流方案：

方案	特點	推薦場景
Gson / Moshi	老牌反射方案，配置零成本	快速起步、欄位不多的小專案
kotlinx.serialization	官方編譯期方案，更快更安全	新專案首選（2026 年主流）

JSON 解析兩大方案

```
// 給資料類加 @Serializable 註解
@Serializable
data class WeatherResponse(
```

```
    val weather: List<Weather>,
    val main: Main,
    val name: String
)

// Retrofit 掛上對應 Converter（以 Retrofit 2.11+ 官方支援為例）：
// .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
```

無論哪種方案，鐵律都是：先看真實返回的 JSON，再定義欄位。很多“解析失敗”的 bug，都是因為欄位名寫錯、巢狀層級猜錯——把原始 JSON 列印出來對照，十秒定位。

六、協程：安卓併發的“普通話”

八、協程：安卓併發的“普通話”

上面反覆出現的 suspend 和 launch 來自 Kotlin 協程（Coroutine）——安卓官方的併發方案。你可以粗暴地理解為：讓“要等一會兒”的操作（網路、資料庫）不阻塞介面。

```
// 在 ViewModel 裡啟動協程
viewModelScope.launch {
    val records = dao.getAll().first()  // suspend：會自動掛起等待
    // 更新介面狀態……
}

// 關鍵字速記：
// suspend   —— "我要等待，別攔著別人"
// launch    —— "開工！"
// viewModelScope —— 跟著頁面走的"工作間"，頁面銷燬自動收工
```

零基礎先記住用法就好，背後的“掛起與還原”原理可以以後再深究。日常開發你寫的 90% 都是這個模板。

來動手：做一個”每日一句”小應用

找一個免費的名言 API（比如 hitokoto 之類），用 Retrofit 拉取“每日一句”，顯示在螢幕中央，加個“換一句”按鈕（協程裡重新請求）。再把喜歡的句子用 Room 存進“收藏夾”。這個練習把 DataStore/Room/Retrofit/協程全用了一遍——做完它，第 7、8 章就是小菜一碟。

本章小結

- 口訣：小配置進 DataStore，成批資料進 Room，外面的資料走 Retrofit
- DataStore 返回 Flow：值變了，觀察者自動收到
- Room 三件套：@Entity（表）+ @Dao（操作）+ @Database（總裝），模板可直接複用

- Retrofit 三步：定義介面建立例項協程裡呼叫
- 兩條鐵律：網路不進主執行緒；正式產品必須 HTTPS
- 協程 = 不阻塞介面的等待；viewModelScope 是頁面專屬工作間

第五十四章：專案一——我的記帳本（本地版）

歡迎來到你的第一個“正經專案”。這個記帳本雖然不聯網，但五臟俱全：能記一筆賬、能看流水、能按月統計、能刪改。做完它，你就完整走了一遍“資料邏輯介面”的全流程。別急著抄程式碼——先想清楚再動手，這是咱們一貫的原則。

一、需求思考：先問五個問題

動手之前，用“需求五要素”把想法捋清楚：

要素	答案
Who（給誰用）	自己用，家人朋友也行
What（做什麼）	記收支、看流水、按月彙總、按分類統計
Why（為什麼）	練手 + 搞清楚錢花哪兒了
How（怎麼做）	Kotlin + Compose + Room + MVVM，純本地無後端
Constraint（約束）	一週內完成；最低支援 Android 8；不上架也先按上架標準寫

記帳本需求五要素

然後拆解成“每一步都能用”的小步驟：(1) 建 Room 資料層（能存一筆賬）；(2) 列表頁（能看流水）；(3) 錄入頁（能加賬）；(4) 刪除與編輯；(5) 月度彙總卡片。每完成一步，App 都是一個“能用的東西”。

二、資料層：讓“賬”有個家

先定義一筆賬長什麼樣——欄位要夠用但別貪多：

```
@Entity(tableName = "records")
data class Record(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val type: String,        // "income" 收入 / "expense" 支出
```

```
    val amount: Double,       // 金額（元）
    val category: String,     // 分類：餐飲、交通、工資……
    val note: String = "",    // 備註（可選）
    val date: String          // "2026-08-15"
)
```

Dao 里加兩條“統計用”的查詢——這是記帳本的靈魂：

```
@Dao
interface RecordDao {
    @Query("SELECT * FROM records ORDER BY date DESC, id DESC")
    fun getAll(): Flow<List<Record>>

    @Insert
    suspend fun insert(record: Record)

    @Delete
    suspend fun delete(record: Record)

    @Query("SELECT COALESCE(SUM(amount),0) FROM records WHERE type='income' AND
    date LIKE :month || '%'")
    suspend fun monthIncome(month: String): Double   // month 形如 "2026-08"

    @Query("SELECT COALESCE(SUM(amount),0) FROM records WHERE type='expense' AND
    date LIKE :month || '%'")
    suspend fun monthExpense(month: String): Double
}
```

小貼士：金額為什麼用 Double？

嚴謹的金融 App 會用 BigDecimal 或“以分為單位的 Int”來避免浮點誤差（0.1+0.2≠0.3 的老問題）。練手專案用 Double 足夠；如果哪天做真金融產品，記得換成“分”為單位——這是無數前輩踩出來的坑。

三、MVVM：介面與邏輯“分家”

直接讓介面運算元據庫？能跑，但介面程式碼會又臭又長。MVVM 架構讓各司其職：View（介面）只管畫，ViewModel（邏輯中樞）管資料，Model（Room）管儲存。

View（Compose 介面）

ViewModel（狀態與邏輯）

Model（Room 資料層）

圖 5：MVVM 三層結構——資料單向流動，各層互不越界

```
class RecordViewModel(private val dao: RecordDao) : ViewModel() {
```

```
    // 把 Flow 轉成介面能觀察的狀態
    val records = dao.getAll()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(), emptyList())

    // 使用者操作：呼叫 suspend 函式
    fun addRecord(type: String, amount: Double, category: String, note: String,
    date: String) {
        viewModelScope.launch {
            dao.insert(Record(type = type, amount = amount,
                category = category, note = note, date = date))
        }
    }

    fun deleteRecord(record: Record) {
        viewModelScope.launch { dao.delete(record) }
    }
}
```

介面（Compose）裡這樣用：val vm: RecordViewModel = viewModel()，然後 val list by vm.records.collectAsState()——資料一變，列表自動重新整理，你什麼都不用做。這就是 MVVM + Flow + Compose 三劍合璧的威力。

四、介面：列表頁 + 錄入頁

列表頁用 LazyColumn（懶載入列表）——資料再多也不卡：

```
@Composable
fun RecordListScreen(vm: RecordViewModel) {
    val records by vm.records.collectAsState()

    Column {
        // 頂部彙總卡片（收入-支出=結餘）
        SummaryCard(records)

        // 流水列表
        LazyColumn {
            items(records, key = { it.id }) { record ->
                RecordRow(record) { vm.deleteRecord(record) }
            }
        }

        // 懸浮"記一筆"按鈕  彈錄入對話方塊
        FloatingActionButton(onClick = { showAddDialog = true }) {
            Text("+")
        }
```

```
    }
}

@Composable
fun RecordRow(record: Record, onDelete: () -> Unit) {
    Row(Modifier.fillMaxWidth().padding(12.dp)) {
        Text(record.category, fontWeight = FontWeight.Bold)
        Spacer(Modifier.weight(1f))
        Text(if (record.type == "income") "+${record.amount}" else "-${record.
    amount}",
             color = if (record.type == "income") Color(0xFF27AE60) else Color(0
    xFFC0392B))
    }
}
```

錄入用 AlertDialog（對話方塊）最省事：幾個輸入框 + 收入/支出切換 + 分類下拉，點確定就調 vm.addRecord(...)。

注意：輸入校驗別偷懶

金額輸入框一定要校驗：空值、負數、“abc”這類非法輸入都要攔住（用 toDoubleOrNull() 判斷）。App 崩掉最常見的原因之一就是“把使用者的輸入當成了乖孩子”。記住：使用者輸入的一切，都要當敵人審查。

五、編輯與刪除：讓資料“能進能出”

只會加不會刪的記帳本，用三天就廢了。補兩個操作：

```
// Dao 裡補兩條：
@Query("UPDATE records SET amount=:amount, category=:category, note=:note WHERE id
    =:id")
suspend fun update(id: Long, amount: Double, category: String, note: String)

@Query("DELETE FROM records WHERE id=:id")
suspend fun deleteById(id: Long)

// 介面上：長按列表項彈"編輯/刪除"選單（Compose 用 DropdownMenu 或 AlertDialog）
// 刪除前最好彈確認框——"誤刪"是記帳類 App 使用者最痛恨的事
```

互動細節兩條經驗：刪除必須二次確認（防止手滑），編輯儘量預填原值（改一個字比重新輸入省事）。別小看這兩個細節——產品感就是這些細節堆出來的。

六、驗收：三層檢查 ＋ 兩個進階挑戰

八、驗收：三層檢查 ＋ 兩個進階挑戰

專案做完，按“功能邊界架構”三層驗收：能不能記、能不能刪、重開 App 資料還在不在（功能）；金額輸 0 或負數會不會崩、轉屏會不會丟資料（邊界）；介面和資料有沒有混在一起、欄位有沒有硬編碼（架構）。

基礎版驗收透過後，給自己加兩道“進階菜”：

1. 月度篩選：頂部加月份切換（比如 2026-08 v），列表只顯示當月流水；
2. 分類餅圖：用 Compose 的 Canvas 或引入輕量圖表庫，把”錢花哪兒了”畫成餅圖——做完這個，記帳本就有點”產品樣”了。

來動手：本週完成它

按“資料層 ViewModel 列表頁錄入頁彙總卡片”的順序動手，每天一步。卡住了就把報錯原樣丟給搜尋引擎或 AI（還記得第七篇《AI 程式設計實戰篇》嗎？描述問題時說清“期望行為、實際行為、復現步驟”）。完成後用它記一週賬——自己做的 App 用起來，成就感會推著你繼續往前走。

本章小結

- 需求五要素拆成”每步能用”的小步驟，再動手
- 資料層：Record 實體 ＋ Dao（含月份統計 SQL）＋ Database
- MVVM 分工：View 只管畫，ViewModel 管邏輯，Model 管儲存
- Flow ＋ stateIn ＋ collectAsState：資料一變介面自動重新整理
- LazyColumn 懶載入列表；AlertDialog 做錄入
- 三層驗收（功能/邊界/架構）＋ 進階挑戰（月度篩選、分類餅圖）

第五十五章：專案二——天氣查詢 App（網路版）

記帳本是“練內功”，天氣 *App* 是“見世面”——它要和網際網路打交道。這個專案你會學到：申請免費 *API*、解析 *JSON*、處理“載入中/成功/失敗”三種介面狀態，以及怎麼把 *API* 金鑰藏好。做完它，你就具備了做任何“聯網 *App*”的能力——新聞、匯率、翻譯，套路一模一樣。

一、申請免費天氣 API

個人練手，推薦 OpenWeatherMap 或國內的和風天氣（QWeather）——都有免費額度，註冊即用。以 OpenWeatherMap 為例：

1. 註冊帳號在 API keys 頁面生成一個 Key（apikey）；
2. 免費版可呼叫 Current Weather（當前天氣）介面：api.openweathermap.org/data/2.5/weather?q=beijing&appid= 你的 Key&units=metric；
3. 瀏覽器裡先試一次，看看返回的 JSON 長什麼樣——這是接任何 API 前必做的” 預習”。

避坑提醒：金鑰絕不能寫死在程式碼裡

API Key 相當於你的“錢包密碼”，寫進程式碼一提交到 GitHub，幾小時內就會被爬蟲掃走盜刷。正確做法：放進 local.properties（這個檔案已被 gitignore 忽略），再透過 Gradle 的 buildConfig 讀進程式碼。搜尋“Android 隱藏 API Key 的最佳實踐”照著改即可。

二、網路層：Retrofit + JSON 解析

先照 API 返回的 JSON 定義“資料模型”，欄位對不上的一律忽略：

```
// JSON: {"weather":[{"description":"晴"}],"main":{"temp":26.5,"humidity":40},"name
    ":"北京"}
data class WeatherResponse(
    val weather: List<Weather>,
    val main: Main,
    val name: String
```

```
)
data class Weather(val description: String)
data class Main(val temp: Double, val humidity: Int)

interface WeatherApi {
    @GET("data/2.5/weather")
    suspend fun getWeather(
        @Query("q") city: String,
        @Query("appid") key: String,
        @Query("units") metric: String = "metric",
        @Query("lang") lang: String = "zh_cn"
    ): WeatherResponse
}
```

Gson 會自動把 JSON 塞進這些 data class——欄位名一致，魔法自動發生。

三、三種介面狀態：載入中 / 成功 / 失敗

聯網 App 的靈魂在於“狀態管理”。網路慢、斷網、城市名打錯……每種情況介面都要給使用者交代。用一個 sealed class（密封類）優雅地表示：

```
sealed class UiState {
    object Loading : UiState()                    // 轉圈中
    data class Success(val data: WeatherResponse) : UiState()  // 成功
    data class Error(val message: String) : UiState()          // 失敗
}

// ViewModel 裡：
fun search(city: String) {
    viewModelScope.launch {
        _state.value = UiState.Loading
        _state.value = try {
            UiState.Success(api.getWeather(city))
        } catch (e: Exception) {
            UiState.Error("查不到這個城市，換個名字試試？")
        }
    }
}

// 介面裡：
when (val s = state) {
    is UiState.Loading -> CircularProgressIndicator()   // 轉圈
    is UiState.Success -> WeatherContent(s.data)        // 展示
    is UiState.Error -> Text(s.message)                 // 報錯
}
```

這個 Loading/Success/Error 三段式是聯網介面的“萬金油模板”，先背下來，以後所有聯網 App 照葫蘆畫瓢。

四、實戰介面程式碼：把天氣畫出來

狀態有了，介面長這樣（關鍵片段）：

```
@Composable
fun WeatherContent(data: WeatherResponse) {
    Column(
        Modifier.fillMaxSize().padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(data.name, fontSize = 28.sp, fontWeight = FontWeight.Bold)
        Text(
            "${data.main.temp}°C",
            fontSize = 72.sp,
            fontWeight = FontWeight.Light
        )
        Text(
            data.weather.firstOrNull()?.description ?: "未知",
            fontSize = 20.sp,
            color = Color.Gray
        )
        Spacer(Modifier.height(16.dp))
        Row {
            Text(" 溼度 ${data.main.humidity}%")
            Spacer(Modifier.width(24.dp))
            Text(" 體感 ${data.main.temp}°C")
        }
    }
}
```

幾個小細節值得學：firstOrNull() 安全取第一個元素（列表可能為空），?: “未知”給兜底文案——空安全原則在介面裡同樣適用。溫度用 72.sp 的大字號撐場面，“輕”字重（Light）讓數字顯得優雅，這是天氣類 App 的經典配方。

五、介面與互動細節

七、介面與互動細節

- 搜尋框：頂部 TextField，輸城市名點搜尋（鍵盤上設” 搜尋” 動作，回車即搜）；
- 天氣卡片：大字號溫度（26.5°）、天氣描述（晴）、溼度、城市名；
- 收藏城市：把常看的城市存進 DataStore，下次開啟自動查——順手複習第 6 章；

- 下拉重新整理：Compose 裡用 PullToRefreshBox（Material 3 提供），手感立刻”專業”起來。

小貼士：圖片讓介面活起來

給每種天氣配一個 emoji 或圖示（），用 when(description) 對映——三行程式碼，介面溫度感翻倍。也可以載入第三方的天氣圖示庫，但練手階段 emoji 足夠。

來動手：天氣 App 驗收清單

(1) 正常搜尋出天氣；(2) 斷開網路，App 優雅報錯不崩潰（開飛航模式測！）；(3) 輸一個不存在的城市，提示友好；(4) 收藏兩個城市，重啟 App 還在；(5) 轉屏不丟狀態。五條全過，這個專案才算畢業。

本章小結

- 先瀏覽器試 API、看懂 JSON，再寫程式碼——”預習”永遠不虧
- API Key 藏進 local.properties，絕不進程式碼庫
- Retrofit 介面 + data class：欄位對上，JSON 自動解析
- Loading/Success/Error 三段式狀態是聯網介面萬金油
- 斷網、空輸入、城市不存在——錯誤處理是聯網 App 的及格線

第五十六章：打磨與發佈——測試、除錯與上架 Google Play

App 做出來了，然後呢？這一章回答“從能跑”到“能交付”的三件事：怎麼高效地除錯找 bug、怎麼寫測試防迴歸、怎麼把 App 送上 Google Play 讓全世界下載。上架不只是儀式感——它是把一個作品變成產品的分水嶺。

一、除錯三板斧：Logcat、斷點、UI 檢查器

工具	幹什麼	怎麼用
Logcat	看系統/自己打的日誌	程式碼裡 Log.d(”TAG”,” 值 =$x”)，面板裡搜 TAG 過濾
斷點除錯	讓程式停在某行，看變數值	行號旁點一下出紅點，Debug 模式執行，逐步走
Layout Inspector	看執行中的介面結構	選單 Tools Layout Inspector，實時看元件樹

除錯三板斧

小貼士：報錯怎麼讀？

崩潰後 Logcat 裡會有紅色堆疊（Stacktrace），從下往上找第一個自己包名開頭的行（比如 com.example.yourapp），那行基本就是案發現場。先看“Exception 型別”（空指標？型別轉換？），再看“at ... 你的檔案: 行號”。這招能解決 80% 的崩潰。

二、測試：給程式碼上保險

手測靠不住——改了這個功能，另一個悄悄壞了。測試就是“後悔藥”兼“防彈衣”。安卓測試分三層，新手先吃透第一層：

層級	測什麼	放哪裡	速度
單元測試	純邏輯（計算、校驗）	app/src/test/	秒級
儀器測試	真實裝置上的介面行為	app/src/androidTest/	分鐘級
手動測試	真人感受、體驗	——	永遠需要

測試三層

```
// 把"金額字串轉數字"這種邏輯抽成純函式
fun parseAmount(input: String): Double? {
    val v = input.trim().toDoubleOrNull()
    return if (v != null && v > 0) v else null
}

// 測試它（JUnit4）
@Test
fun emptyInputReturnsNull() {
    assertEquals(null, parseAmount(""))
}
@Test
fun negativeReturnsNull() {
    assertEquals(null, parseAmount("-5"))
}
@Test
fun normalValueParses() {
    assertEquals(12.5, parseAmount("12.5"))
}
```

記帳本里的金額校驗、統計計算，都該有單元測試罩著。跑測試：右鍵 test 目錄 Run。綠色全過，放心改程式碼。

三、簽名打包：給 App 蓋上“印章”

安卓要求所有安裝包必須簽名——就像蓋公章，證明“這個包是我出的，沒被篡改”。除錯時 Android Studio 自動用 debug 簽名；發佈要用自己的 release 簽名。流程：

1. 選單 Build Generate Signed Bundle / APK；
2. 建立 keystore 檔案（金鑰庫）——設定密碼並妥善保管；
3. 選擇 release + AAB 格式，構建完成。

避坑提醒：keystore 丟了 = App 丟了

上架後更新 App 必須用同一個簽名，否則商店會拒絕更新。keystore 檔案和密碼一旦丟失，你只能換個新應用重新上架——老使用者全都接不上。務必把 keystore 和密碼備份到安全的地方（雲盤加密 + 記事本），這是安卓開發者最昂貴的教訓之一。

四、上架 Google Play 全流程

註冊開發者帳號

準備商店素材

上傳 AAB

填資料安全表

稽核（數小時 ~ 數天）

上架

圖 6：Google Play 上架六步走

- 註冊：一次性 25 美元註冊費，需能用的 Google 帳號；
- 素材：應用圖示（512×512）、截圖（至少 2 張）、功能描述、隱私政策連結（收集資料必填）；
- 資料安全表：如實申報收集了哪些資料——不聯網的記帳本可以” 零收集”；
- 稽核：新帳號首次上架常被人工稽核，數小時到數天不等，被打回就按意見改。

國內使用者注意：Google Play 需要特殊網路環境，且中國大陸地區分發通常走國內商店（華為、小米、應用寶等）。上架國內商店的流程類似，但各商店有自己的稽核口味——同一個 AAB/APK 可以多店分發。作為練手，先走通一家就贏了。

五、商店素材怎麼做：第一印象就是下載量

商店頁是使用者“逛店”時看到的櫥窗。三個素材決定第一印象：

- 圖示：512×512，別用截圖湊合。沒設計能力就用 AI 工具生成一個簡潔幾何圖形，在手機桌面上縮小後還能一眼認出才算合格；
- 截圖：至少 2 張，最好 4-6 張，展示最核心的功能場景；每張配一句人話文案（” 一秒記一筆” 比” 高效的記帳解決方案” 好十倍）；
- 描述：第一句說清” 這是什麼 + 給誰用”，別堆砌” 最專業、最強”——商店搜尋權重主要看關鍵詞和下載轉化，真誠比誇張有效。

小貼士：A/B 看看同行

上架前逛一圈商店裡同類 App 的前十名，截圖對比：他們圖示什麼風格？截圖講什麼賣點？標題怎麼寫？模仿不是抄襲——學套路、換自己的內容，這是所有產品冷啟動最快的一課。

五、效能最佳化與崩潰上報小清單

App 上架前，這兩件事值得各花半小時：

效能自查四條

1. 列表用 LazyColumn：長列表千萬別用 Column 硬排——懶載入是安卓效能的第一課；
2. 圖片別硬塞原圖：用 Coil 圖片庫自動縮放快取，一張 5MB 原圖足以卡爆低端機；
3. 避免在主執行緒乾重活：資料庫、網路、大檔案都在協程裡做；
4. 用 Profiler 量一量：Android Studio 自帶的效能分析器能看記憶體、CPU、耗電——先測量，再最佳化，別憑感覺。

崩潰上報

你測試時沒崩，不代表使用者手機不崩。上架前接入 Firebase Crashlytics（谷歌官方，免費）：幾行配置，使用者崩潰時自動上報堆疊，你在後臺就能看到“哪個機型、哪個版本、崩在哪一行”。沒有崩潰監控的上架，等於閉著眼睛開車。

來動手：把記帳本”交付”一次

(1) 給 parseAmount 等邏輯補 3 個單元測試；(2) 用 Logcat 親手抓一個自己製造的崩潰並修好；(3) 生成一個 release 簽名的 APK，發給朋友安裝（這才是“作品”的儀式感）；(4) 有餘力就註冊開發者帳號，把記帳本真正提交上架。前三條今晚就能做完。

本章小結

- 除錯三板斧：Logcat 看日誌、斷點看變數、Layout Inspector 看介面
- 讀崩潰：從堆疊下往上找自己包名的第一行
- 測試三層：單元（秒級，先學這個）/儀器/手動
- 發佈必須簽名：keystore 和密碼備份好，丟了 App 就”換人”了
- 上架六步：註冊素材上傳 AAB 資料安全表稽核上架

第五十七章：進階之路——權限、通知與現代架構

能做出 *App* 只是開始。這一章聊聊從“會做”到“做好”的幾道坎：怎麼優雅地申請權限、怎麼發通知不討人厭、怎麼把架構再往上抬一層，以及——在 *2026* 年，怎麼和 *AI* 搭檔寫安卓程式碼。每一條都是你在真實專案裡馬上會遇到的。

一、執行時權限：先敲門，再進門

相機、定位、通訊錄、通知……這些“敏感地帶”，安卓要求 App 執行時向使用者申請（Android 6 起）。規矩是：先宣告（Manifest），再申請（程式碼），使用者拒絕要優雅降級。Compose 裡用官方庫 Accompanist 的 rememberPermissionState 最省心：

```
// Manifest 裡先宣告：
// <uses-permission android:name="android.permission.CAMERA" />

@Composable
fun CameraButton() {
    val permission = rememberPermissionState(
        permission = android.Manifest.permission.CAMERA
    )
    Button(onClick = {
        when {
            permission.status.isGranted -> openCamera()
            permission.status.shouldShowRationale -> showExplainDialog()
            else -> permission.launchPermissionRequest()
        }
    }) { Text("拍照") }
}
```

注意：被永久拒絕怎麼辦？

使用者勾了“不再詢問”後，launchPermissionRequest 不會再跳出視窗。此時應引導使用者去系統設定裡手動開啟（用 Intent 跳到應用設定頁），並在介面裡說明“這個權限用來做什麼”。態度好一點，使用者開權限的意願會高很多——申請權限的文案，值得認真寫。

二、通知：在正確的時間說有用的話

通知是 App 觸達使用者的通道，也是最容易被使用者“拉黑”的功能。發通知三條紀律：只發有用的、讓使用者能關、別半夜騷擾。基礎用法：

```
// Android 13+ 必須先申請 POST_NOTIFICATIONS 許可權（見上一節）

// 1. 建立通知渠道（Android 8+ 必須）
val channel = NotificationChannel(
    "reminder", "提醒", NotificationManager.IMPORTANCE_DEFAULT
)
val manager = getSystemService(NotificationManager::class.java)
manager.createNotificationChannel(channel)

// 2. 發通知
val notification = NotificationCompat.Builder(this, "reminder")
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle("記帳提醒")
    .setContentText("今天還沒記帳哦")
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    .build()
if (ActivityCompat.checkSelfPermission(this,
        Manifest.permission.POST_NOTIFICATIONS) == PackageManager.
    PERMISSION_GRANTED
) {
    manager.notify(1, notification)
}
```

定時任務（比如“每天 20:00 提醒記帳”）用 WorkManager——它是官方推薦的“後臺排程管家”，能保證任務在合適時機執行，不用自己操心電池最佳化。鬧鐘類需求用它準沒錯。

三、架構進階：依賴注入與模組化

專案變大後，兩個“甜蜜的煩惱”會找上門：

- 物件建立亂：每個頁面都在自己 new 資料庫、new API——用 Hilt（依賴注入框架）統一管理，”誰要什麼，開口就有人遞”；
- 程式碼堆成山：單模組專案越改越慢——拆成多模組（app / core / feature-xxx），各功能獨立開發、獨立測試。

小貼士：別過度設計

Hilt 和模組化都是好工具，但單人專案先別急著上。記帳本單模組 + 手動建立依賴完全夠用。等程式碼量超過幾萬行、編譯開始變慢、多人協作時，再按需引入。工具是服務目標的，不是用來炫耀的——這也是咱們 AI 程式設計書裡的“簡單至上”原則。

四、2026 年的新玩法：AI 輔助安卓開發

2026 年，AI 已經深度嵌入安卓開發流程。如果你想提速，這套組合拳很成熟：

環節	AI 怎麼幫	你的職責
寫程式碼	Android Studio 內建 Gemini/第三方 AI 外掛補全與生成	描述清楚需求，審查產出
查資料	把官方文件和報錯資訊丟給 AI 解讀	關鍵結論回官方文件核實
寫測試	讓 AI 按”正常/邊界/異常”三類生成用例	確認用例符合業務語義
改 bug	貼堆疊 + 相關程式碼，AI 定位修復	看懂改動，跑測試驗證

AI 輔助安卓開發的四個場景

具體方法論——怎麼給 AI 下指令、怎麼驗收、怎麼避坑——完整的 11 章內容在本書第七篇《AI 程式設計實戰篇》（第 37-47 章）裡，強烈建議配合閱讀：安卓語法交給本篇，AI 協作的心法交給第七篇。

五、Compose 效能最佳化與除錯工具

Compose 寫起來爽，但“重組過多”也會卡。三個實用技巧：

- 穩定引數：給資料類加 @Immutable 或使用 derivedStateOf，避免無關狀態變化觸發大面積重組；
- 記得用 key：items(list, key = { it.id }) 讓列表增刪時只動該動的項——不加 key，刪除一項可能整表重畫；
- Layout Inspector 的 Compose 模式：實時看每個 Composable 的重組次數，誰頻繁重組一目瞭然，照著熱點最佳化。

另外把 Compose 編譯器報告（Metrics）開啟，能生成一份“哪些元件不穩定”的報告——進階路上，資料永遠比感覺可靠。

來動手：給記帳本加”三件套”

(1) 給記帳本加“每天 20:00 記帳提醒”（WorkManager + 通知管道 + Android 13 通知權限）；(2) 把資料庫和 API 的建立改成 Hilt 注入；(3) 試著用 AI 工具讓“月度篩選”功能提速實現，然後按三層驗收法檢查產出。三件套做完，你的專案就有“正規軍”的樣子了。

本章小結

- 敏感權限：Manifest 宣告 + 執行時申請 + 被拒後優雅引導
- 通知三紀律：只發有用的、能關、不騷擾；管道 + 權限缺一不可
- 定時任務用 WorkManager，別自己造輪子
- Hilt 管物件、模組化管程式碼——單人小專案先別急著上
- AI 輔助四場景：寫碼、查資料、寫測試、改 bug；方法論見第七篇（第 37-47 章）

第五十八章：速查手冊與資源

本章是一份可以獨立使用的速查手冊。當你記不清某個寫法或需要快速決策時，直接翻到這裡。建議把它當成“口袋卡片”：常用程式碼速查、常見問題、學習資源，一頁不落。

一、常用程式碼速查卡

要做什麼	關鍵程式碼
記住一個可變狀態	var x by remember { mutableStateOf(初值) }
轉屏不丟狀態	rememberSaveable { mutableStateOf(初值) }
豎排/橫排佈局	`Column { }/Row { }`
懶載入列表	LazyColumn { items(列表) { ... } }
頁面跳轉	navController.navigate(” 路由”)
協程裡乾等活兒	`viewModelScope.launch { ...}`
觀察資料流	`val list by flow.collectAsState()`
存小配置	context.dataStore.edit { it[KEY] = 值}
存結構化資料	Room 三件套：@Entity / @Dao / @Database
聯網取資料	Retrofit 介面 + suspend 函式
發通知	NotificationChannel + NotificationCompat.Builder
打日誌	`Log.d("TAG", "message")`

安卓開發速查卡（2026 年 8 月版）

二、常見問題 FAQ

Q1：我完全不會程式設計，學安卓開發要多久？

按本書路徑走：第 1-2 周裝環境、學 Kotlin 基礎；第 3-4 周學 Compose 做出介面；第 5-6 周完成記帳本專案；第 7-8 周做完天氣專案。兩個月，從零到能獨立做小 App。前提是每天投入 1-2 小時，且動手敲程式碼而不是隻看書。

Q2：Java 還要學嗎？

不用。新專案、新教學、新崗位基本都以 Kotlin 為準。只在“維護十年老專案”時才會遇到大量 Java。學完 Kotlin 後如果工作需要，再補 Java 會很快——兩者底層是同一個虛擬機器生態。

Q3：模擬器還是真機？

開發除錯用模擬器很方便（快照、多機型），但驗收一定要過真機——觸感、效能、真實網路、感測器，模擬器取代不了。條件允許的話，買一台中階真機當測試機，是最符合成本效益的投資。

Q4：安卓開發好找工作嗎？

2026 年的行情：純“會寫介面”的初級崗位競爭激烈，但“懂 Kotlin + 會架構 + 有上線作品”的開發者依然搶手。學習期的你，最好的簡歷就是商店裡那個自己做的 App。

Q5：遇到報錯卡住了怎麼辦？

黃金三步：(1) 把報錯第一行原樣搜 StackOverflow/搜尋引擎；(2) 查官方文件對應頁面；(3) 還不行，就把“期望行為 + 實際現象 + 報錯全文 + 相關程式碼”打包發給 AI 或社群提問。絕大多數問題，第一步就解決了。

三、推薦學習資源

型別	資源	推薦理由
官方	developer.android.com	安卓官方文件，Compose/Kotlin 教學都在裡面，第一權威
官方	Kotlin 官網 (kotlinlang.org)	語言官方文件 + 線上演練場
官方	Google Codelabs	手把手的分步實驗課，新手友好度滿分
社群	Stack Overflow (android 標籤)	報錯求救第一站
社群	掘金 / CSDN 安卓專區	中文實戰文章多，注意甄別新舊（先看日期）
影片	Google Developers 官方頻道	每年的開發者大會新技術講解
配套	第七篇《AI 程式設計實戰篇》（第 37-47 章）	用 AI 輔助安卓開發的完整心法

安卓開發學習資源清單（2026 年 8 月）

四、Gradle 常用命令速查

平時點按鈕就夠，但命令列有時更高效（尤其是配合 AI 工具或 CI 流水線時）：

命令	作用
`./gradlew assembleDebug`	打一個除錯版 APK（Windows 把./ 去掉）
`./gradlew assembleRelease`	打發布版（需配好簽名）
`./gradlew test`	跑全部單元測試
`./gradlew lint`	靜態檢查：找出潛在 bug 和不規範寫法
`./gradlew clean`	清空構建產物（”重啟大法”的編譯版）
`./gradlew :app: dependencies`	檢視依賴樹（排查版本衝突神器）

Gradle 常用命令速查

小貼士：構建出問題先 clean

“昨天還能跑今天就不行”的靈異事件，十有六七是構建快取錯亂。./gradlew clean 之後重新構建，能治癒一大半玄學問題。還不行再重啟 Android Studio——這招請配合 File Invalidate Caches 一起服用。

五、結語：從第一個 App 到第 N 個 App

七、結語：從第一個 App 到第 N 個 App

回頭看，你從“不知道 Activity 是什麼”，到裝好環境、學會 Kotlin、用 Compose 畫出介面、給 App 接上資料庫和網路，再到測試、打包、上架——這條路上你已經走完了從零到一最難的一段。

技術的世界變化很快：框架會升級，工具會換代，AI 會接手越來越多“敲鍵盤”的活。但有一樣東西不會過時：你把一個模糊的想法，拆解成清晰的步驟，再一步步實現它的能力。工具是放大器，思路是地基。放大器可以替換，地基不能動搖。

你不再只是”會玩手機的人”。你能把腦海裡的想法，變成三億安卓裝置上真實執行的 App。從今天開始，用鍵盤去創造——這就是安卓開發者的浪漫。

參考文獻

1. Android Developers. (2026). Android Developer Guides and Jetpack Compose Documentation. developer.android.com.
2. Kotlin Foundation. (2026). Kotlin Language Documentation. kotlinlang.org.
3. Android Open Source Project. (2026). Android Platform Version History and API Levels.
4. Google Play. (2026). Play Console Help: Publishing and App Signing.
5. AndroidX Release Notes. (2026). Room / DataStore / Navigation / WorkManager releases.
6. Square. (2026). Retrofit Documentation. github.com/square/retrofit.

九、iOS 開發實戰篇（第 59-69 章）

第五十九章：認識 iOS——蘋果生態與開發生態

恭喜你做出這個決定——從“用 iPhone 的人”變成“給 iPhone 寫應用的人”。在動手之前，咱們先把地圖攤開：iOS 從哪裡來？蘋果的“圍牆花園”對開發者意味著什麼？一個 App 從你的手指敲鍵盤到使用者手機上，中間都發生了什麼？

一、iOS 的身世：一部手機改變一個行業

2007 年 1 月，喬布斯在發佈會上掏出了第一代 iPhone。當時它連第三方應用都不支援——直到 2008 年，蘋果推出 App Store 和 iPhone SDK，允許全世界開發者給 iPhone 寫軟體。那一刻，“應用經濟”誕生了：到今天，App Store 上有數百萬個應用，為開發者累計分成超過數千億美元。

系統本身也在進化：最初叫 iPhone OS，2010 年更名 iOS；版本號一路走到 iOS 17（2023），然後在 2025 年，蘋果把命名改為跟隨年份——iOS 26 於 2025 年發佈，2026 年 8 月的當前版本是 iOS 27。命名會變，但一件事沒變過：iOS 的使用者付費習慣和軟體品質口碑，是移動生態裡公認最好的——同樣的 App，iOS 端的收入往往顯著高於安卓端。

節點	年份	對開發者重要的事
App Store 上線	2008	第三方開發生態開啟
Swift 發佈	2014	現代 iOS 開發語言登場
SwiftUI 發佈	2019	宣告式介面框架，本書主線
iOS 26（改名元年）	2025	命名跟隨年份；Apple Intelligence 全面整合
iOS 27	2026	當前最新版（本書基準）

iOS 開發大事記（截至 2026 年 8 月）

二、圍牆花園：蘋果生態的“規矩”與“紅利”

蘋果生態常被形容為“圍牆花園”（Walled Garden）——規則多、管得嚴：App 必須稽核才能上架、開發必須用 Mac 和 Xcode、分發主要走 App Store。聽起來憋屈？但對開發者，這套規矩換來了真金白銀的紅利：

- 裝置碎片化極小：iPhone 型號就那麼幾款，螢幕尺寸、系統版本高度統一——你適配 5 臺機型，就覆蓋了 95% 的使用者（安卓可能要適配幾十款）；
- 系統版本更新率極高：新系統發佈幾周內就有七八成使用者升級，新 API 可以大膽用，不用照顧” 五年前的老系統”；
- 使用者付費習慣好：App Store 使用者更願意為應用和內購花錢；
- 稽核雖然嚴格，但商店乾淨：山寨、流氓軟體少，好產品更容易被看見。

注意：先說清楚成本

iOS 開發有硬門檻：必須有一臺 Mac（Xcode 是 macOS 獨佔），上架需要 99 美元/年的開發者帳號（個人開發者免費帳號可以本地真機除錯，但不能上架）。想清楚這筆投入再啟程——不過好訊息是，學習階段用免費帳號完全夠用，第 60 章會講。

三、用什麼語言寫：Objective-C 還是 Swift？

iOS 開發歷史上也有兩位“主角”。Objective-C 是開國元老——從 2008 年用到 2014 年；Swift 是蘋果 2014 年發佈的新語言，此後一路進化到 Swift 6（2024 年發佈，引入嚴格併發檢查），到 2026 年，新專案幾乎全部用 Swift。

給零基礎的你一句實在話：直接學 Swift，Objective-C 連看都不用看——除非你入職後要維護十年前的遺產程式碼。Swift 的好處：語法清爽（沒有 C 語言時代的分號與標頭檔案包袱）、可選值（Optional）機制從語言層面消滅“空指標崩潰”、和 SwiftUI 無縫配合、還有 iPad 上的 Swift Playgrounds 可以邊玩邊學。

Objective-C 風格（上古遺蹟）

```
NSString *name = @"張三"; [name stringByAppendingString:@"你好"]; // 方括號 + 指標 + 分號三件套
```

Swift 風格（現代清爽）

```
var name = "張三" name += "你好" // 像自然語言一樣直白
```

四、一個 App 是怎麼誕生的？

和所有平臺一樣，iOS App 的誕生也有一條流水線。但蘋果版多了兩道“安檢”：

想法

設計（介面 + 功能）

寫程式碼（Swift + SwiftUI）

編譯簽名

測試（模擬器 + 真機）

蘋果稽核

上架 App Store

圖 1：一個 iOS App 從想法到上架的全流程（注意多出的“稽核”環節）

流程裡的關鍵概念：

- 簽名（Signing）：每個 iOS App 都必須”簽名”才能安裝——證明”這個包出自你，沒被篡改”。學習階段 Xcode 用你的 Apple ID 自動簽名；上架時用開發者證書。第 67 章細講。
- 模擬器（Simulator）：Mac 上虛擬出的 iPhone，開發除錯的主戰場。它跑的是 Mac 的 CPU，所以特別流暢，比安卓模擬器體驗好一大截。
- TestFlight：蘋果官方的”內測分發”工具——上架前把測試版發給最多 10000 名測試使用者，他們用 TestFlight 應用安裝試用。這是 iOS 生態獨有的優雅。

五、iOS 技術棧全景：你的“武器清單”

層次	主流選擇（2026）	幹什麼用的
語言	Swift 6	寫邏輯的語言
介面	SwiftUI + 蘋果設計規範（HIG）	用程式碼畫介面（本書主線）
架構	MVVM + @Observable	把介面和邏輯分開
資料儲存	SwiftData（資料庫）+ UserDefaults（配置）	把資料存進手機
網路	URLSession + async/await + Codable	和伺服器對話
構建工具	Xcode Build System / xcodebuild	把程式碼變成安裝包
開發環境	Xcode（macOS 獨佔）	蘋果官方 IDE

2026 年 iOS 主流技術棧速覽

注意到沒：這套清單比安卓的短一截——因為蘋果把很多事都“包辦”了。封閉生態的代價是自由度，回報是省心：不用糾結選哪個網路庫、哪個資料庫，官方給的往往就是最優解。

來動手：看看你手機的”蘋果戶口”

開啟 iPhone 的 [設定] [通用] [關於本機]，記下三樣：iOS 版本、機型名稱、可用容量。再去 App Store 的“今天”頁隨便逛逛，注意那些精品 App 的截圖和文案——從今天起，你逛商店的視角要從“使用者”切換到“開發者”了。

本章小結

- 2008 年 App Store 上線，” 應用經濟” 誕生；2026 年當前版本 iOS 27
- 圍牆花園的賬：規矩多，但裝置少、版本新、使用者肯花錢、商店乾淨
- 硬門檻：必須有 Mac；上架要 99 美元/年開發者帳號；學習階段免費帳號夠用
- 語言直接學 Swift；Objective-C 只屬於遺產程式碼
- 流水線比安卓多一環：蘋果稽核；TestFlight 是內測分發神器
- 2026 標準技術棧：Swift 6 + SwiftUI + MVVM + SwiftData + URLSession + Xcode

第六十章：搭建開發環境——Xcode 與第一個專案

工欲善其事，必先利其器。這一章咱們把“廚房”搭好：裝好 Xcode、跑通模擬器、建立人生第一個 iOS 專案，再把工程結構拆開看個明白。裝好環境之後，你會第一次看到自己寫的程式碼變成一個能點的按鈕——這個瞬間，很多人會記一輩子。

一、準備一臺 Mac

先說門檻：iOS 開發必須在 macOS 上進行。如果你沒有 Mac，選項有三：買一臺（哪怕是二手 Mac mini 或 MacBook Air 都夠用）、短期租雲 Mac、或者先讀第八篇《安卓開發實戰篇》（Windows 就能幹）。

配置項	最低要求	舒適線（推薦）
晶片	Apple Silicon（M1 起）	M2 及更新
記憶體	8GB	16GB
磁碟	預留 40GB+（Xcode 本體很胖）	256GB 起步
系統	最新 macOS	保持自動更新

Mac 開發機配置參考（2026 年 8 月）

小貼士：沒有 Mac 也能先學語言

iPad 上免費的 Swift Playgrounds 能學 Swift 語言基礎（不用 Mac）；甚至 iPhone 上也能裝。先把第 61 章的語言知識在 Playgrounds 裡玩熟，等拿到 Mac 後直接起飛。磨刀不誤砍柴工，說的就是這個。

二、安裝 Xcode

Xcode 是蘋果官方的整合開發環境，免費。三步裝好：

1. Mac 上開啟 App Store，搜尋”Xcode”，點安裝——注意它有好幾個 GB，找個網速好的時段；

2. 裝完首次啟動，同意協議、裝完元件（自動完成）；
3. 選單 Xcode Settings Locations，確認 Command Line Tools 已選中——命令列工具是很多指令碼和第三方工具的前提。

注意：版本和系統的”匹配遊戲”

新 Xcode 往往要求最新 macOS，舊 Mac 則會卡在老版本。上架 App Store 時，蘋果會要求用較新的 Xcode 構建（通常給一個寬限期）。所以系統更新別拖太久——iOS 開發者是“追新版”最勤快的一群人。

三、建立你的第一個專案

開啟 Xcode Create New Project 選 iOS App 模板，然後填：

配置項	填什麼	說明
Product Name	MyFirstApp	應用程式名稱字
Team	你的 Apple ID（免費）	簽名用，個人帳號即可
Organization Identifier	com.example	和 Product Name 拼成包標識
Interface	SwiftUI	本書主線框架
Language	Swift	不解釋

建立專案要填的五件事

點 Create 後，Xcode 會生成一個能直接跑的專案。左上角選一臺模擬器（比如 iPhone 17），點 > 執行——幾秒後，模擬器螢幕上出現一個寫著“Hello, world!”的視窗。你的第一個 iOS App 已經誕生了。

四、工程結構拆解：每個檔案是幹嘛的

左側是工程導航器，新手只需認識這幾個：

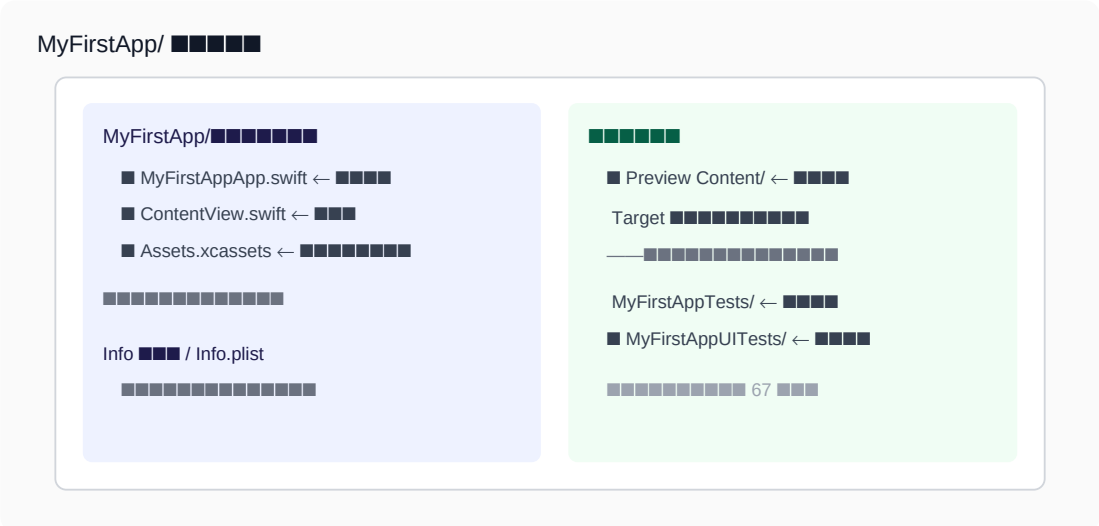


圖 2：新手必懂的 Xcode 工程結構

- MyFirstAppApp.swift：程式入口，SwiftUI 裡它只有幾行——”啟動，然後顯示主介面”；
- ContentView.swift：主介面檢視，你現在看到的一切都在這裡；
- Assets.xcassets：資源倉庫——圖示、圖片、顏色都放這；
- Target 設定：點左側藍色工程圖示進去，版本號、最低 iOS 版本、簽名都在裡面。App 顯示名、圖示也在這配置。

五、真機除錯：把 App 裝進你的 iPhone

五、Xcode 效率技巧：新手的五個“快速鍵外掛”

用熟這幾個，你會顯得“很專業”：

操作	快速鍵	作用
執行	CmdR	編譯並跑起來
開啟/切換畫布預覽	OptionCmd	所見即所得的開關
格式化縮排	CtrlI	自動整理選中程式碼的縮排
快速開啟檔案	CmdO	輸入檔名秒開（最常用！）
註釋/取消註釋	Cmd/	除錯時隨手開關程式碼

Xcode 高頻快速鍵（新手五件套）

另外兩個隱藏福利：多開模擬器（同時跑 iPhone 和 iPad 對比佈局）和程式碼片段庫（把常用的幾行程式碼拖進右側 Snippets 面板，下次一鍵插入）——效率工具先用起來，手感自然就上來了。

六、真機除錯：把 App 裝進你的 iPhone

模擬器雖好，真機才是終點。用免費 Apple ID 也能真機除錯：

1. 資料線連上 iPhone（或同 Wi-Fi 無線除錯）；
2. Xcode 頂部裝置列表選你的手機，首次會提示登入 Apple ID 並自動配置簽名；
3. iPhone 上 [設定] [通用] [VPN 與裝置管理] 信任你的開發者證書；
4. 點 >，App 裝進真機。

注意：免費帳號的”七天限制”

免費 Apple ID 簽署的 App 7 天後會失效（重新執行一次即可續期）。這是蘋果的防濫用機制，學習階段完全夠用；付費開發者帳號（99 美元/年）則沒有這個限制，第 67 章會細說。

七、小試牛刀 + 常見報錯急救

九、小試牛刀 + 常見報錯急救

開啟 ContentView.swift，把 Text(“Hello, world!”) 裡的字換成你的名字，按 CmdR 重新執行——幾秒後模擬器上就顯示你的名字。恭喜，你完成了第一次完整的“改程式碼編譯安裝執行”迴圈。

報錯現象	常見原因	解藥
Signing 報錯	Team 沒選、證書過期	Target Signing 裡勾上你的帳號；免費帳號重跑一次
模擬器黑屏/打不開	映象損壞	選單 Device Erase All Content；或刪掉重加模擬器
Xcode 下載慢/裝不上	App Store 網路波動	換時段；或到蘋果開發者網站下載.xip 離線包

新手環境報錯急救表

來動手：完成你的”開發儀式感”清單

(1) 裝好 Xcode 並跑通模擬器；(2) 用真機跑一次自己的 App 並截圖留念；(3) 把 Hello 文字改成你的名字；(4) 到 Target 設定裡把 App 顯示名改成“我的第一個 App”。做完這四步，你就是有作品的人了。

本章小結

- 硬門檻：Mac + Xcode；沒有 Mac 先用 iPad 上的 Swift Playgrounds 學語言
- Xcode 從 App Store 裝；新版本要求新系統，更新別拖
- 新增專案五要素：名字、Team、組織標識、SwiftUI、Swift
- 工程口訣：程式碼都在 MyFirstApp/ 目錄；Target 設定管版本與簽名
- 免費 Apple ID 真機除錯有 7 天簽名限制；付費帳號無此限制
- 開發日常 = 改程式碼 CmdR 看效果，迴圈往復

第六十一章：Swift 快速上手——蘋果開發的第一語言

這一章咱們正式學語言。別緊張——*Swift* 是出了名的“對新手友好”，而且你不需要背語法書：先掌握 *20%* 的語法，就能看懂和寫出 *80%* 的日常程式碼。剩下的邊做專案邊查。記住咱們的學習原則：看懂 > 記住，會用 > 精通。

一、變數與型別：給資料起名字

程式就是“資料 + 邏輯”。變數，就是給資料起個名字存起來。Swift 用 let 和 var 宣告：

```
let name = "張三"          // let：常量（推薦預設用這個）
var age = 18               // var：變數（以後還能改）

age = 19                   // <i class="fas fa-check-circle mr-1"></i> var 可以改
// name = "李四"           // <i class="fas fa-times-circle mr-1"></i> let 不能改，編譯報錯

let score: Int = 100       // 也可以顯式寫明型別
let pi = 3.14              // 不寫也能自動推斷

print("你好，\(name)")     // 字串插值：反斜槓 + 括號
```

常用型別一張表記住：

型別	存什麼	示例
Int	整數	42
Double	小數	3.14
String	文字	” 你好”
Bool	真/假	true / false
[Int]（陣列）	一組資料	[1, 2, 3]

Swift 常用基礎型別

小貼士：let 優先原則

Swift 社群的鐵律：預設用 let，確有必要再用 var。能用 let 的地方用 var，Xcode 會黃字警告你。這個習慣能幫你少掉一大類“變數被意外修改”的 bug——程式碼越少變化，越容易推理。

二、可選值（Optional）：Swift 的“防崩護身符”

這是 Swift 最重要的概念，沒有之一。一句話：一個值可能“不存在”時，Swift 強迫你處理它——從語言層面消滅了“空指標崩潰”這個行業噩夢。

```
var name: String = "張三"    // 普通型別：絕不允許是 nil
// name = nil                 // <i class="fas fa-times-circle mr-1"></i> 編譯錯誤，直接攔住

var nickname: String? = nil  // 加個問號：允許為 nil（可選值）
nickname = "三兒"

// 使用可選值，必須"解包"：
if let n = nickname {        // if let：安全解包
    print("你好，\(n)")
} else {
    print("還沒有暱稱")
}

let len = nickname?.count ?? 0   // ?? 給預設值（空合運算子）
print(len)
```

注意：！是”高危操作”

程式碼裡寫 nickname!（強制解包）等於對編譯器喊“我保證它不是 nil”——一旦猜錯，App 當場崩潰。新手期原則：儘量用 if let / ?? ，不用!。Xcode 補全程式碼時偶爾會順手加!，記得改掉它。

三、函式與閉包：把邏輯打包成“按鈕”

函式是一段有名字的邏輯，像遙控器上的按鈕。閉包（Closure）則是“沒名字的函式”，可以隨手遞給別人：

```
// 函式：名字在前，引數和返回型別在後
func greet(name: String) -> String {
    return "你好，\(name)！"
}
print(greet(name: "張三"))      // 呼叫時要帶引數名

// 閉包：就地定義，輕快直接
```

```
let numbers = [1, 2, 3, 4]
let doubled = numbers.map { $0 * 2 }   // $0 = 當前元素
// [2, 4, 6, 8]

let big = numbers.filter { $0 > 2 }    // 篩選
// [3, 4]
```

SwiftUI 裡閉包無處不在：按鈕的 Button(“點我”) { ... } 後面的大括號就是一個閉包。map、filter、sorted 這幾個“集合連招”請記牢——未來 90% 的列表處理都是它們的組合。

四、結構體與協議：Swift 的“樂高”體系

Swift 的官方偏好很明確：優先用結構體（struct），需要繼承時才用類（class）。結構體是“值型別”——複製時各過各的，天然少一類 bug：

```
// 結構體：裝資料的盒子
struct Record {
    var id = UUID()
    var amount: Double
    var category: String
    var date: Date

    // 結構體自帶"成員逐一構造器"：直接就能建立
}
let r = Record(amount: 25.5, category: "餐飲", date: Date())

// 協議：約定"必須具備哪些能力"
protocol Identifiable {
    var id: UUID { get }
}
struct Record: Identifiable { ... }   // 遵守協議 = 承諾實現其要求
```

SwiftUI 裡幾乎一切都是 struct（View 就是協議）。這句現在不懂沒關係，第 62 章會天天見面——在 Swift 的世界裡，先擁抱 struct。

五、寫給零基礎的語法速查卡

要做什麼	寫法
定義常量/變數	`let x = 1`/`var x = 1`
字串插值	”你好，\(name)”
定義函式	func 名字 (引數: 型別) -> 返回型別 { ... }
判斷	if 條件 { ... } else { ... }
遍歷	for x in 陣列 { ... }
安全解包	if let x = 可選值 { ... }
預設值	x ?? 預設值
定義結構體	`struct X { var a: Int; var b: String }`
閉包	`{ $0 * 2 }`

Swift 語法速查卡（新手版）

來動手：在 Playgrounds 裡玩 15 分鐘 Swift

有 Mac 就開啟 Xcode Playground；沒 Mac 就用 iPad 上的 Swift Playgrounds 或網頁版演練場。把本章每一段程式碼都跑一遍，改改數字和文字，故意製造幾個錯誤看看 Xcode 怎麼提示。這個“先玩後學”的 15 分鐘，會讓你第 62 章上手時輕鬆一倍。

本章小結

- let 優先，var 慎用；型別自動推斷，冒號後顯式宣告也行
- 可選值是 Swift 的靈魂：可能為空必須處理，if let / ?? 是護身符，! 是高危操作
- 函式呼叫要帶引數名；閉包 = 沒名字的函式，SwiftUI 裡無處不在
- 優先 struct，遵守協議（Protocol）；SwiftUI 幾乎一切都是 struct
- 20% 語法幹 80% 的活：先會用，再精通

第六十二章：介面開發——SwiftUI 入門

介面是使用者看到的一切。*SwiftUI* 是蘋果 *2019* 年推出的宣告式介面框架——你只需要“宣告”介面長什麼樣，系統自動負責渲染、適配深色模式、適配各種螢幕。到 *2026* 年，*SwiftUI* 已是新專案的絕對主流。這一章，咱們學會“畫畫”。

一、SwiftUI 的思想：宣告式，不是命令式

老式介面開發是“命令式”：先告訴系統“建立按鈕放在 (20,30) 設定顏色”，一步一步指揮。SwiftUI 是“宣告式”：你直接描述“介面上有一個紅色按鈕寫著’點我’”，怎麼畫、什麼時候重畫，系統自己操心。

```
import SwiftUI

struct ContentView: View {
    var body: some View {      // body：描述"介面長什麼樣"
        VStack {               // 豎著排列
            Text("你好，世界！")
                .font(.largeTitle)      // 修飾符：大標題
            Button("點我") {
                print("被點選了")
            }
            .buttonStyle(.borderedProminent)  // 藍色實心按鈕
        }
        .padding()
    }
}
```

關鍵點：檢視 = 遵守 View 協議的結構體，body 屬性描述介面，. 修飾符 () 鏈式“上妝”。這就是 SwiftUI 的全部基本盤。

二、基礎檢視：文字、按鈕、輸入框、圖片

檢視	幹什麼	示例
Text	顯示文字	Text(” 你好”)
Button	按鈕	Button(” 點我”) { 動作}
TextField	輸入框	TextField(” 提示”, text: $ 文字)
Image	顯示圖片	`Image(systemName: "star.fill")`

SwiftUI 四大基礎檢視

注意 Image(systemName:)——蘋果內建了 SF Symbols 圖示庫（幾千個免費圖示），“star.fill”這種名字就能直接顯示一個實心星星。這是 iOS 開發的隱藏福利：做介面幾乎不用找圖示素材。

三、佈局三件套：VStack、HStack、ZStack

- VStack：豎著排（像排隊打飯）；
- HStack：橫著排（像並排坐）；
- ZStack：疊著放（像撲克牌摞一起，適合做角標、遮罩）。

```
VStack(alignment: .leading, spacing: 12) {
    HStack {
        Text("帳單")
        Spacer()                    // 彈性空間：把右邊推到盡頭
        Text("-25.5 元").foregroundColor(.red)
    }
    Divider()                       // 分割線
    Text("備註：午飯")
}
.padding(16)                        // 四周留白
.background(Color(.systemGray6)) // 淺灰底
.cornerRadius(12)                   // 圓角

// 修飾符順序有意義：先 padding 再 background，背景不含邊距
```

注意：修飾符順序有講究

padding(16).background(灰) 和 background(灰).padding(16) 效果不同——前者“先留白再塗色”（背景不含邊距），後者“先塗色再留白”（背景包含邊距）。新手最常見的“奇怪白邊”都源於此。不確定時調換順序，預覽立刻告訴你答案。

四、狀態與繫結：讓介面“活”起來

靜態介面沒意思。SwiftUI 的規則：介面永遠忠實反映狀態，狀態變了，介面自動重畫。

```
struct CounterView: View {
    @State private var count = 0   // @State：這個值變了要重畫

    var body: some View {
        VStack {
            Text("你點了 \(count) 次")
            Button("點我 +1") {
                count += 1          // 改狀態，介面自動重新整理
            }
        }
    }
}
```

輸入框要用“雙向繫結”——$ 符號把“值和修改權”一起傳進去：

```
struct InputView: View {
    @State private var text = ""

    var body: some View {
        TextField("輸入點什麼", text: $text)  // $ 雙向繫結
            .textFieldStyle(.roundedBorder)
        Text("你輸入了：\(text)")   // 輸入時實時更新
    }
}
```

使用者操作

狀態改變（count += 1）

SwiftUI 感知到變化

自動重畫介面（body 重新計算）

圖 3：SwiftUI 的“狀態介面”單向資料流

五、預覽、主題與動畫：SwiftUI 的“外掛”

- #Preview 宏：檔案底部加 #Preview { ContentView() }，右側畫布實時渲染——改程式碼所見即所得；
- 深色模式免費：用系統顏色（Color(.systemGray6)、.primary）而不是寫死顏色，深色模式自動適配，一行不用多寫；
- 動畫一行搞定：withAnimation { count += 1 } 或給檢視加.animation(.default, value: count)——過渡絲滑。

```
struct AnimatedHeartView: View {
    @State private var liked = false

    var body: some View {
        Button {
            withAnimation(.spring(duration: 0.4)) {
                liked.toggle()
            }
        } label: {
            Image(systemName: liked ? "heart.fill" : "heart")
                .font(.system(size: 48))
                .foregroundColor(liked ? .red : .gray)
                .scaleEffect(liked ? 1.3 : 1.0)
        }
    }
}
```

小貼士：先功能後動畫

新手常犯的毛病是“介面還沒做對，就開始糾結動畫”。先讓功能跑通，最後再統一加動效——動畫是錦上添花，功能是雪中送炭。這也是咱們一貫的“簡單至上”原則。

來動手：做一個”心情按鈕”

六、常用修飾符速查：一頁在手，介面不愁

想做什麼	修飾符
留白	`.padding(16)`
圓角	`.cornerRadius(12)`
背景色	`.background(Color(.systemGray6))`
字型與字號	`.font(.title.bold())/.font(.system(size: 20))`
前景色（文字/圖示）	`.foregroundColor(.red)`
撐滿寬度	`.frame(maxWidth: .infinity)`
固定尺寸	`.frame(width: 100, height: 44)`
對齊方式	`VStack(alignment: .leading)`
透明度	`.opacity(0.6)`
陰影	`.shadow(radius: 5)`
點選手勢	`.onTapGesture { ...}`
鍵盤型別	`.keyboardType(.decimalPad)`

SwiftUI 常用修飾符速查

記不住沒關係——這頁就是用來查的。用得最多的其實就前四個：padding、圓角、背景、字型。

把模板專案改成一個心情切換器：一個按鈕，點選後在開心難過生氣之間迴圈切換，文字和背景色跟著變。提示：用陣列存三組（文字 + 顏色），@State 存下標，點選時 (index + 1) % 3。做完發給朋友玩，收穫第一波“哇你會做 App 了”。

本章小結

- SwiftUI 是宣告式：描述” 長什麼樣”，系統負責畫和重畫
- 檢視 = 遵守 View 協議的結構體；body 描述介面；修飾符鏈式” 上妝”
- 四大基礎檢視：Text / Button / TextField / Image；SF Symbols 圖示庫免費管飽
- 佈局三件套：VStack 豎排、HStack 橫排、ZStack 疊放；Spacer 是彈性空間
- @State 讓介面活起來；$ 雙向繫結餵給輸入框
- #Preview 所見即所得；系統顏色自動適配深色模式；withAnimation 一行動效

第六十三章：檢視、狀態與導航

單頁面的玩具已經會做了。這一章解決兩個“長大”問題：怎麼在多個頁面之間跳來跳去（導航），以及資料在頁面之間怎麼流動（狀態管理）。學完本章，你就能搭出“列表詳情編輯”這種經典的多頁面 *App* 骨架。

一、狀態管理的“資料流全景”

SwiftUI 的狀態工具不止 @State 一個。先看這張全景圖，搞懂“什麼時候用哪個”：

工具	管什麼	誰來用	典型場景
@State	頁面私有的臨時狀態	當前檢視	計數器、輸入框文字
@Binding	借用別人的狀態	子檢視改父檢視	自定義元件
@StateObject / @Observable	跨頁面共享的邏輯	檢視模型（ViewModel）	資料載入、業務邏輯
@EnvironmentObject / .environment	全域性共享的狀態	整個 App	使用者登入態、主題設定

SwiftUI 狀態工具全景（2026 年，@Observable 已是主流）

```
// iOS 17+ 的現代寫法：@Observable 宏（推薦新專案使用）
import Observation

@Observable
class CounterModel {                    // 檢視模型
    var count = 0
    func increment() { count += 1 }
}

struct CounterView: View {
    @State private var model = CounterModel()   // 持有模型

    var body: some View {
        VStack {
            Text("你點了 \(model.count) 次")
            Button("點我") { model.increment() }
```

```
        }
    }
}
```

給新手的粗暴建議：頁面私有的用 @State，跨頁面/有邏輯的用 @Observable 類。老教學裡的 @StateObject/@ObservableObject 是上一代寫法，能讀懂即可，新程式碼按上面來。

二、NavigationStack：頁面跳轉的“棧”

iOS 16 起，多頁面導航用 NavigationStack——像一摞盤子，推一個進一個，返回時逐個跳出：

```
struct HomeView: View {
    var body: some View {
        NavigationStack {
            List {
                NavigationLink("張三的帳本") {
                    DetailView(name: "張三的帳本")   // 目標頁面
                }
                NavigationLink("李四的帳本") {
                    DetailView(name: "李四的帳本")
                }
            }
            .navigationTitle("帳本列表")   // 頂部大標題
        }
    }
}

struct DetailView: View {
    let name: String
    var body: some View {
        Text("\(name) 的詳情頁")
            .navigationTitle(name)
            .navigationBarTitleDisplayMode(.inline)
    }
}
```

NavigationLink 就是“能點的跳轉連結”；navigationTitle 自動生成導航欄標題和返回按鈕。要程式設計式跳轉（比如點完“儲存”自動返回）時，用 @Environment(\.dismiss) 關閉當前頁面——這兩招配合，90% 的導航需求就齊了。

三、跳出視窗與底部標籤：sheet 與 TabView

除了“推頁面”，還有兩種經典互動：

```
// 1. 底部彈出半屏/全屏（錄入表單最愛）
```

```
struct HomeView: View {
    @State private var showAdd = false
    var body: some View {
        Button("記一筆") { showAdd = true }
            .sheet(isPresented: $showAdd) {
                AddRecordView()      // 彈出的錄入頁
            }
    }
}

// 2. 底部標籤欄（App 的"骨架"）
struct MainTabView: View {
    var body: some View {
        TabView {
            RecordListView()
                .tabItem { Label("帳本", systemImage: "book") }
            StatsView()
                .tabItem { Label("統計", systemImage: "chart.pie") }
            SettingsView()
                .tabItem { Label("設定", systemImage: "gear") }
        }
    }
}
```

小貼士：什麼時候用哪種？

經驗法則：主流程用 NavigationStack 推頁面（列表詳情），臨時操作用 sheet 跳出視窗（新增、編輯、確認），並列的幾大塊用 TabView（底部標籤）。三者組合，就是絕大多數 App 的骨架。

四、List：iOS 的“國民列表”

iOS 上最常用、最有“蘋果味”的元件就是 List——天然帶分割線、滑動刪除、點選高亮：

```
struct RecordListView: View {
    @State private var records = ["午飯 25.5", "地鐵 6.0", "工資 +8000"]

    var body: some View {
        List {
            ForEach(records, id: \.self) { r in
                Text(r)
            }
            .onDelete { indexSet in          // 左滑刪除
                records.remove(atOffsets: indexSet)
            }
        }
```

```
    }
}
```

列表 + NavigationLink + sheet + TabView——把這四樣組合起來，你已經能搭出“帳本列表點開詳情點 + 彈錄入”的完整 App 骨架了。下一章的記帳本專案，就是這麼搭的。

來動手：搭一個”三頁骨架”

五、程式生命週期：前臺與後臺

和安卓的 Activity 生命週期類似，iOS App 也有“出生、前臺、後臺、死亡”的狀態變化。SwiftUI 裡用 scenePhase 環境值觀察：

```
@main
struct MyApp: App {
    @Environment(\.scenePhase) private var phase

    var body: some Scene {
        WindowGroup {
            ContentView()
        }
        .onChange(of: phase) { _, newPhase in
            switch newPhase {
            case .active:    print("回到前臺，恢復重新整理")
            case .inactive:  print("過渡中（來電/切後臺瞬間）")
            case .background: print("進後臺，儲存草稿")
            @unknown default: break
            }
        }
    }
}
```

典型用途：回前臺時重新整理資料，進後臺時儲存草稿。iOS 對後臺執行管得很嚴——普通 App 進後臺後很快被掛起（凍結），別指望在後臺偷偷乾重活；真要定時任務，用第 68 章的本地通知，而不是“後臺常駐”。

做一個假資料版本的記帳本骨架：TabView 三個標籤（帳本/統計/設定）帳本頁用 List 顯示假資料、點行跳詳情（NavigationStack）右上角“+”按鈕彈錄入 sheet。不存資料、不聯網，純練骨架——這個骨架做好，第 65 章往裡填真資料就是水到渠成。

本章小結

- 狀態四工具：@State（頁面私有）、@Binding（借用）、@Observable（邏輯）、.environment（全域性）
- 新專案用 @Observable 宏寫檢視模型；老教學的 @ObservableObject 能讀即可
- NavigationStack 推頁面；@Environment(\.dismiss) 程式設計式返回

- 主流程推頁面、臨時操作彈 sheet、並列大塊用 TabView
- List 是” 國民列表”：分割線、滑動刪除開箱即用

第六十四章：資料儲存與網路請求

一個只能“看”的 *App* 是死的；能存資料、能連網路的 *App* 才是活的。這一章認識 *iOS* 資料層的“三件套”：*UserDefaults*（存小配置）、*SwiftData*（存結構化資料）、*URLSession*（和伺服器對話），外加解析 *JSON* 的 *Codable* 和現代化的 *async/await*。學完這章，你就能寫會記住使用者習慣、會實時更新內容的真 *App* 了。

一、儲存方案怎麼選：先看這張表

方案	適合存什麼	特點	本書哪裡用
UserDefaults	零散配置（開關、偏好）	鍵值對，簡單直接	第 65 章設定、第 66 章收藏城市
SwiftData	結構化資料（賬目、記錄）	官方資料庫封裝，@Model 一行宣告	第 65 章賬目記錄
URLSession	（不存，聯網取）伺服器資料	系統自帶網路框架，無第三方依賴	第 66 章天氣資料

資料三件套的分工

記一個口訣：“小配置進 UserDefaults，成批資料進 SwiftData，外面的資料走 URLSession”。舊教學裡的 Core Data 依然能用（SwiftData 就是它的現代馬甲），手寫 SQLite 則基本不用碰——直接學新的，別走老路。

二、UserDefaults：存點小配置

```
// 寫：存一個值
UserDefaults.standard.set(true, forKey: "isDarkMode")
UserDefaults.standard.set("北京", forKey: "favoriteCity")

// 讀：取出來（沒有就返回預設值）
let isDark = UserDefaults.standard.bool(forKey: "isDarkMode")
let city = UserDefaults.standard.string(forKey: "favoriteCity") ?? "北京"

// 刪：移除一個鍵
UserDefaults.standard.removeObject(forKey: "isDarkMode")
```

注意它只適合零散的小配置——幾十上百條記錄請交給 SwiftData，別把 UserDefaults 當資料庫用（會拖慢啟動、也不便查詢）。

三、SwiftData：給資料安個“真正的家”

SwiftData 是 iOS 17 起的官方資料庫方案，把“建表、增刪改查”簡化到了極致——一個 @Model 註解，資料類就變成資料庫表：

```
import SwiftData

// 1. @Model：這個類自動變成"表"
@Model
class Record {
    var amount: Double
    var category: String
    var note: String
    var date: Date

    init(amount: Double, category: String, note: String, date: Date) {
        self.amount = amount
        self.category = category
        self.note = note
        self.date = date
    }
}

// 2. App 入口掛上容器
@main
struct MyApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
        }
        .modelContainer(for: Record.self)   // 資料庫就緒
    }
}

// 3. 檢視裡用 @Query 讀、@Environment 寫
struct RecordListView: View {
    @Query(sort: \Record.date, order: .reverse) var records: [Record]
    @Environment(\.modelContext) private var context

    var body: some View {
        List(records) { r in
            Text("\(r.category) \(r.amount) 元")
        }
```

```
    }

    func add(amount: Double, category: String) {
        let r = Record(amount: amount, category: category, note: "", date: Date())
        context.insert(r)     // 存
        try? context.save()   // 落盤
    }
}
```

這套“@Model + modelContainer + @Query”模板可以原樣搬進你的任何專案——換換欄位名就行。資料一變，@Query 自動重新整理介面，妙不可言。

四、Codable：給 JSON“翻譯”

伺服器的資料大多是 JSON 文字。Swift 的 Codable 協議讓“JSON 結構體”的轉換自動發生——欄位名對得上，魔法就生效：

```
// JSON: {"name":"北京","main":{"temp":26.5},"weather":[{"description":"晴"}]}
struct WeatherResponse: Codable {
    let name: String
    let main: Main
    let weather: [Weather]
}
struct Main: Codable { let temp: Double }
struct Weather: Codable { let description: String }

// JSON 字串  結構體
let decoder = JSONDecoder()
let data = jsonString.data(using: .utf8)!
let result = try decoder.decode(WeatherResponse.self, from: data)
print(result.weather[0].description)   // "晴"
```

鐵律：先看真實返回的 JSON，再定義欄位。很多“解析失敗”的 bug 都是欄位名寫錯、型別猜錯（伺服器發“26.5”字串，你宣告成 Double）——把原始 JSON 列印出來對照，十秒定位。

五、URLSession + async/await：和伺服器說話

iOS 聯網用系統自帶的 URLSession，配合 Swift 的 async/await（非同步函式），“等網路”就像“等快遞”一樣自然：

```
struct WeatherService {
    // 非同步函式：async 標記"我要等一會兒"，throws 標記"可能失敗"
    func fetchWeather(city: String) async throws -> WeatherResponse {
        let url = URL(string:
            "https://api.example.com/weather?city=\(city)")!
```

```
        let (data, _) = try await URLSession.shared.data(from: url)
        return try JSONDecoder().decode(WeatherResponse.self, from: data)
    }
}

// 呼叫處（在 .task 裡）：
.task {
    do {
        let weather = try await service.fetchWeather(city: "北京")
        // 更新介面……
    } catch {
        print("出錯了：\(error)")
    }
}
```

避坑提醒：兩條網路鐵律

(1) 聯網必須 async/await，絕不阻塞主執行緒——卡住介面是崩潰級體驗。Swift 的結構化併發讓這變得很自然，但別忘了 try await 和 do/catch。(2) ATS 預設禁止明文 HTTP：iOS 要求 HTTPS。測試本地 http:// 服務時需要在 Info.plist 裡臨時開例外（NSAppTransportSecurity），但正式產品必須全走 HTTPS——稽核也會盯這個。

來動手：做一個”每日一句”小應用

六、網路錯誤也要“分級”：別把一切報錯都丟給使用者

斷網、超時、404、伺服器 500……不同的錯，使用者該看到的話不一樣：

```
func friendlyMessage(_ error: Error) -> String {
    if let urlError = error as? URLError {
        switch urlError.code {
        case .notConnectedToInternet: return "網路斷了，檢查一下連線？"
        case .timedOut:               return "請求超時，稍後再試？"
        case .cannotFindHost:         return "伺服器找不到了"
        default:                      return "網路出問題了：\(urlError.code)"
        }
    }
    if let d = error as? DecodingError {
        return "資料格式不對（可能是介面變了）"
    }
    return "出錯了：\(error.localizedDescription)"
}
```

給使用者看“人話”，給日誌看“原話”（print 完整 error）。別把 error.localizedDescription 的英文天書直接甩給使用者——產品感就藏在這些細節裡。

找一個免費的名言 API（比如 hitokoto 之類），用 URLSession + async/await 拉取“每日一句”，顯示在螢幕中央，加個“換一句”按鈕。再把喜歡的句子用 SwiftData 存進“收藏

夾”。這個練習把 UserDefaults/SwiftData/Codable/URLSession 全用了一遍——做完它，第 7、8 章就是小菜一碟。

本章小結

- 口訣：小配置進 UserDefaults，成批資料進 SwiftData，外面的資料走 URLSession
- SwiftData 三板斧：@Model 宣告表、modelContainer 掛容器、@Query 讀 + context.insert 寫
- Codable 自動”翻譯”JSON：欄位對得上，魔法就生效
- URLSession + async/await：聯網像等快遞；do/catch 處理失敗
- 兩條鐵律：不阻塞主執行緒；正式產品必須 HTTPS（ATS）

第六十五章：專案一——我的記帳本（本地版）

歡迎來到你的第一個“正經專案”。這個記帳本雖然不聯網，但五臟俱全：能記一筆賬、能看流水、能按月統計、能刪改。做完它，你就完整走了一遍“資料邏輯介面”的全流程。別急著抄程式碼——先想清楚再動手，這是咱們一貫的原則。

一、需求思考：先問五個問題

要素	答案
Who（給誰用）	自己用，家人朋友也行
What（做什麼）	記收支、看流水、按月彙總、按分類統計
Why（為什麼）	練手 + 搞清楚錢花哪兒了
How（怎麼做）	Swift + SwiftUI + SwiftData + MVVM，純本地無後端
Constraint（約束）	一週內完成；最低支援 iOS 17（SwiftData 門檻）

記帳本需求五要素

然後拆解成“每一步都能用”的小步驟：(1) 建 SwiftData 資料層（能存一筆賬）；(2) 列表頁（能看流水）；(3) 錄入跳出視窗（能加賬）；(4) 刪除與編輯；(5) 月度彙總卡片。每完成一步，App 都是一個“能用的東西”。

二、資料層：讓“賬”有個家

先定義一筆賬長什麼樣——欄位要夠用但別貪多：

```
import SwiftData

@Model
class Record {
    var type: String      // "income" 收入 / "expense" 支出
    var amount: Double    // 金額（元）
```

```
    var category: String  // 分類：餐飲、交通、工資……
    var note: String      // 備註（可選）
    var date: Date        // 記帳日期

    init(type: String, amount: Double, category: String, note: String = "", date:
    Date = Date()) {
        self.type = type
        self.amount = amount
        self.category = category
        self.note = note
        self.date = date
    }
}
```

小貼士：金額為什麼用 Double？

嚴謹的金融 App 會用 Decimal（精確十進位制）來避免浮點誤差（0.1+0.2≠0.3 的老問題）。練手專案用 Double 足夠；如果哪天做真金融產品，記得換成 Decimal 或“以分為單位的 Int”——這是無數前輩踩出來的坑。

三、MVVM：介面與邏輯“分家”

直接讓介面運算元據庫？能跑，但介面程式碼會又臭又長。MVVM 架構讓各司其職：View（介面）只管畫，ViewModel（邏輯中樞）管資料，Model（SwiftData）管儲存。

View（SwiftUI 介面）

ViewModel（@Observable 邏輯中樞）

Model（SwiftData 資料層）

圖 4：MVVM 三層結構——資料單向流動，各層互不越界

```
import Observation

@Observable
class RecordViewModel {
    var records: [Record] = []
    private let context: ModelContext

    init(context: ModelContext) { self.context = context }

    func load() {
        // SwiftData 的 #Predicate 篩選 + 排序
        let descriptor = FetchDescriptor<Record>(
            sortBy: [SortDescriptor(\.date, order: .reverse)]
        )
        records = (try? context.fetch(descriptor)) ?? []
```

```
    }

    func add(type: String, amount: Double, category: String, note: String) {
        let r = Record(type: type, amount: amount, category: category, note: note)
        context.insert(r)
        try? context.save()
        load()                          // 重新載入，介面重新整理
    }

    func delete(_ record: Record) {
        context.delete(record)
        try? context.save()
        load()
    }
}
```

介面裡 @State private var vm = RecordViewModel(...)，然後直接 vm.records 渲染列表——資料一變，介面自動重新整理。

四、介面：列表頁 + 錄入跳出視窗

```
struct RecordListView: View {
    @State private var vm: RecordViewModel
    @State private var showAdd = false

    var body: some View {
        NavigationStack {
            List {
                Section("本月結餘") {           // 彙總卡片
                    SummaryRow(records: vm.records)
                }
                Section("流水") {
                    ForEach(vm.records) { r in
                        RecordRow(record: r)
                    }
                    .onDelete { idx in
                        idx.forEach { vm.delete(vm.records[$0]) }
                    }
                }
            }
            .navigationTitle("我的記帳本")
            .toolbar {
                Button { showAdd = true } label: {
                    Image(systemName: "plus")
                }
            }
```

```
            .sheet(isPresented: $showAdd) {
                AddRecordView(vm: vm)          // 錄入彈窗
            }
        }
    }
}
```

錄入跳出視窗要點：金額輸入框鍵盤用.keyboardType(.decimalPad)（數字鍵盤），收入/支出用分段控制元件（Picker），分類用選單選擇，點“儲存”調 vm.add(...) 並 dismiss() 關窗。

注意：輸入校驗別偷懶

金額輸入框一定要校驗：空值、負數、“abc”這類非法輸入都要攔住（用 Double(text) 判斷，失敗返回 nil）。App 崩掉最常見的原因之一就是“把使用者的輸入當成了乖孩子”。記住：使用者輸入的一切，都要當敵人審查。

五、編輯、刪除與彙總

- 刪除：List 的.onDelete 左滑刪除 + 確認跳出視窗（誤刪是記帳類 App 使用者最痛恨的事）；
- 編輯：點行進編輯跳出視窗，預填原值（改一個字比重新輸入省事）；
- 彙總：用 filter + reduce 算” 本月收入 / 支出 / 結餘”，頂部卡片展示：

```
func monthSummary(_ records: [Record]) -> (income: Double, expense: Double) {
    let cal = Calendar.current
    let monthRecords = records.filter {
        cal.isDate($0.date, equalTo: Date(), toGranularity: .month)
    }
    let income = monthRecords
        .filter { $0.type == "income" }
        .reduce(0) { $0 + $1.amount }
    let expense = monthRecords
        .filter { $0.type == "expense" }
        .reduce(0) { $0 + $1.amount }
    return (income, expense)
}
```

六、驗收：三層檢查 + 兩個進階挑戰

專案做完，按“功能邊界架構”三層驗收：能不能記、能不能刪、重啟 App 資料還在不在（功能）；金額輸 0 或負數會不會崩（邊界）；介面和資料有沒有混在一起、欄位有沒有硬編碼（架構）。

基礎版驗收透過後，給自己加兩道“進階菜”：

1. 分類統計：加一個” 統計” 標籤頁，用 Swift Charts 把分類支出畫成餅圖/柱狀圖——蘋果官方圖表庫，幾行程式碼出圖；

2. iCloud 同步：SwiftData 開 iCloud 同步只需配幾項設定——帳本在 iPhone/iPad 之間自動同步，瞬間” 進階” 起來。

來動手：本週完成它

按“資料層 ViewModel 列表頁錄入跳出視窗彙總卡片”的順序動手，每天一步。卡住了就把報錯原樣丟給搜尋引擎或 AI（還記得第七篇《AI 程式設計實戰篇》嗎？描述問題時說清“期望行為、實際行為、復現步驟”）。完成後用它記一週賬——自己做的 App 用起來，成就感會推著你繼續往前走。

本章小結

- 需求五要素拆成” 每步能用” 的小步驟，再動手
- 資料層：@Model 模型 + modelContainer 容器 + FetchDescriptor 查詢
- MVVM 分工：View 只管畫，@Observable 的 ViewModel 管邏輯，SwiftData 管儲存
- 介面：List + Section + toolbar + sheet 錄入跳出視窗
- 三層驗收（功能/邊界/架構）+ 進階挑戰（Swift Charts、iCloud 同步）

第六十六章：專案二——天氣查詢 App（網路版）

記帳本是“練內功”，天氣 *App* 是“見世面”——它要和網際網路打交道。這個專案你會學到：申請免費 *API*、解析 *JSON*、處理“載入中/成功/失敗”三種介面狀態，以及怎麼把 *API* 金鑰藏好。做完它，你就具備了做任何“聯網 *App*”的能力——新聞、匯率、翻譯，套路一模一樣。

一、申請免費天氣 API

個人練手，推薦 OpenWeatherMap 或國內的和風天氣（QWeather）——都有免費額度，註冊即用。以 OpenWeatherMap 為例：

1. 註冊帳號在 API keys 頁面生成一個 Key（apikey）；
2. 免費版可呼叫 Current Weather 介面：api.openweathermap.org/data/2.5/weather?q=beijing&appid= 你的 Key&units=metric；
3. 瀏覽器裡先試一次，看看返回的 JSON 長什麼樣——這是接任何 API 前必做的” 預習”。

避坑提醒：金鑰絕不能寫死在程式碼裡

API Key 相當於你的“錢包密碼”，寫進程式碼一提交到 GitHub，幾小時內就會被爬蟲掃走盜刷。iOS 的正確做法：放進 Config.xcconfig（配置檔案，已被 gitignore 忽略），透過 Info.plist 讀入程式碼。搜尋“iOS xcconfig 管理金鑰”照著改即可。

二、網路層：URLSession + Codable

先照 API 返回的 JSON 定義模型，再寫服務：

```
// JSON: {"weather":[{"description":"晴"}],"main":{"temp":26.5,"humidity":40},"name
    ":"北京"}
struct WeatherResponse: Codable {
    let weather: [Weather]
    let main: Main
    let name: String
```

```
}
struct Weather: Codable { let description: String }
struct Main: Codable { let temp: Double; let humidity: Int }

struct WeatherService {
    let key: String   // 從 xcconfig 讀入

    func fetchWeather(city: String) async throws -> WeatherResponse {
        var comps = URLComponents(string: "https://api.openweathermap.org/data/2.5/
    weather")!
        comps.queryItems = [
            URLQueryItem(name: "q", value: city),
            URLQueryItem(name: "appid", value: key),
            URLQueryItem(name: "units", value: "metric"),
            URLQueryItem(name: "lang", value: "zh_cn")
        ]
        let (data, _) = try await URLSession.shared.data(from: comps.url!)
        return try JSONDecoder().decode(WeatherResponse.self, from: data)
    }
}
```

三、三種介面狀態：載入中 / 成功 / 失敗

聯網 App 的靈魂在於“狀態管理”。網路慢、斷網、城市名打錯……每種情況介面都要給使用者交代。用列舉 + switch 優雅地表示：

```
enum LoadState {
    case loading                  // 轉圈中
    case success(WeatherResponse) // 成功
    case error(String)            // 失敗
}

@Observable
class WeatherViewModel {
    var state: LoadState = .loading
    private let service = WeatherService(key: Config.apiKey)

    func search(city: String) async {
        state = .loading
        do {
            let data = try await service.fetchWeather(city: city)
            state = .success(data)
        } catch {
            state = .error("查不到這個城市，換個名字試試？")
        }
    }
```

```
}

// 介面裡：
switch vm.state {
case .loading: ProgressView("載入中…")
case .success(let d): WeatherContent(data: d)
case .error(let msg): Text(msg).foregroundColor(.secondary)
}
```

這個 loading/success/error 三段式是聯網介面的“萬金油模板”，先背下來，以後所有聯網 App 照葫蘆畫瓢。

四、實戰介面程式碼：把天氣畫出來

```
struct WeatherContent: View {
    let data: WeatherResponse

    var body: some View {
        VStack(spacing: 12) {
            Text(data.name)
                .font(.title.bold())
            Text("\(Int(data.main.temp))°C")
                .font(.system(size: 76, weight: .thin))
            Text(data.weather.first?.description ?? "未知")
                .font(.title3)
                .foregroundColor(.secondary)
            HStack(spacing: 24) {
                Label("溼度 \(data.main.humidity)%", systemImage: "humidity")
                Label("體感 \(Int(data.main.temp))°C", systemImage: "thermometer")
            }
        }
        .padding()
    }
}
```

幾個小細節值得學：first? 安全取第一個元素（列表可能為空），?? “未知”給兜底文案；76 號細字重的溫度數字是天氣類 App 的經典配方；Label 是“圖示 + 文字”的官方元件，比手拼 HStack 更規範。

五、互動細節與收藏城市

- 搜尋框：頂部.searchable 修飾符，系統自帶搜尋互動，提交時調 vm.search；
- 收藏城市：把常看的城市存進 UserDefaults，下次開啟自動查；

- 下拉重新整理：.refreshable { await vm.search(...) } 一行搞定；
- 天氣圖示：用 Image(systemName:) 按描述對映（sun.max / cloud.rain / snowflake），介面立刻活起來。

小貼士：中文城市的”梗”

OpenWeatherMap 對中文城市名支援一般，搜“北京”可能無結果而“beijing”正常。產品化做法是內建一份“城市名英文/座標”對映表，或接國內天氣服務。練手階段先支援拼音/英文城市名即可，別和它較勁。

來動手：天氣 App 驗收清單

六、網路請求怎麼除錯：列印、抓包與模擬慢網

聯網 App 出問題，十有八九在“請求沒發對”或“響應沒解析對”。三個除錯手段：

1. 列印原始資料：把 String(data: data, encoding: .utf8) 列印出來看原始 JSON——欄位名、型別一目瞭然，比猜快十倍；
2. 抓包工具：Mac 上免費的 Proxyman / 開源 Charles 能截獲 App 的全部請求和響應，像 X 光一樣看清網路層；
3. 模擬弱網：模擬器選單 Features Network Link Conditioner（需從開發者網站下載工具），能模擬 3G、高延遲——測出”轉圈半天”的體驗問題。

小貼士：請求沒過 ATS？

如果請求一發出就報“App Transport Security”錯誤，八成是用了 http:// 明文地址。正式介面必須 HTTPS；本地除錯臨時開例外（Info.plist 加 NSAllowsLocalNetworking），上架前記得清乾淨。

(1) 正常搜尋出天氣；(2) 斷開網路，App 優雅報錯不崩潰（模擬器裡關掉 Mac 的網路測！）；(3) 輸一個不存在的城市，提示友好；(4) 收藏兩個城市，重啟 App 還在；(5) 深色模式下介面不刺眼。五條全過，這個專案才算畢業。

本章小結

- 先瀏覽器試 API、看懂 JSON，再寫程式碼——”預習”永遠不虧
- API Key 藏進 xcconfig，絕不進程式碼庫
- Codable 模型 + URLSession async/await：欄位對上，JSON 自動解析
- loading/success/error 三段式狀態是聯網介面萬金油
- 斷網、空輸入、城市不存在——錯誤處理是聯網 App 的及格線

第六十七章：打磨與發佈——測試、除錯與上架 App Store

App 做出來了，然後呢？這一章回答“從能跑”到“能交付”的三件事：怎麼高效地除錯找 bug、怎麼寫測試防迴歸、怎麼把 App 送上 App Store 讓全世界下載。上架不只是儀式感——它是把一個作品變成產品的分水嶺。

一、除錯三板斧：print、斷點、預覽

工具	幹什麼	怎麼用
print + 控制檯	看執行日誌	程式碼裡 print(” 值 =\(x)”)，Xcode 底部控制檯看輸出
斷點除錯	讓程式停在某行，看變數值	行號旁點一下出藍標，執行到那裡暫停，逐步走
SwiftUI 預覽 + 偵錯程式	實時看介面與檢視層級	#Preview 畫布 + 執行時的 View Hierarchy 檢查器

除錯三板斧

小貼士：崩潰了怎麼讀？

Swift 崩潰時控制檯會列印“Fatal error”加原因，Xcode 會停在出錯那一行並高亮。真機上的歷史崩潰可以在 Xcode Window Organizer Crashes 裡看（需付費帳號）。關鍵詞優先搜“錯誤原因第一句 + swiftui”——精確的問題，才有精確的答案。

二、測試：給程式碼上保險

手測靠不住——改了這個功能，另一個悄悄壞了。測試就是“後悔藥”兼“防彈衣”。iOS 用 XCTest 框架，分兩種：

層級	測什麼	Target	速度
單元測試	純邏輯（計算、校驗）	MyFirstAppTests	秒級
UI 測試	真實介面行為	MyFirstAppUITests	分鐘級
手動測試	真人感受、體驗	——	永遠需要

iOS 測試層級（模板專案已預置兩個 Test Target）

```
import XCTest

// 把"金額字串轉數字"這種邏輯抽成純函式
func parseAmount(_ input: String) -> Double? {
    guard let v = Double(input.trimmingCharacters(in: .whitespaces)),
          v > 0 else { return nil }
    return v
}

final class AmountTests: XCTestCase {
    func testEmptyInputReturnsNil() {
        XCTAssertNil(parseAmount(""))
    }
    func testNegativeReturnsNil() {
        XCTAssertNil(parseAmount("-5"))
    }
    func testNormalValueParses() {
        XCTAssertEqual(parseAmount("12.5"), 12.5)
    }
}
```

跑測試：CmdU。全綠之後再改程式碼，心裡有底。記帳本里的金額校驗、彙總計算，都該有單元測試罩著。

三、簽名與打包：給 App 蓋上“印章”

每個 iOS App 都必須簽名才能安裝——證明“這個包出自你，沒被篡改”。層級關係：

- 學習階段：免費 Apple ID + Xcode 自動簽名（Personal Team），真機可跑，7 天續期一次；
- 分發階段：付費開發者帳號（99 美元/年）簽發證書與描述檔案，才能上架和長期安裝。

打包：Xcode 選單 Product Archive（歸檔），構建完成後在 Organizer 裡 Distribute App 選 App Store Connect 上傳。歸檔必須用 Generic iOS Device 作為目標（不是模擬器），這是新手常忘的一步。

避坑提醒：證書丟了 = 更新斷檔

簽名證書和私鑰一旦丟失，你無法再給舊 App 發更新——使用者只能裝“全新”應用。妥善備份證書（鑰匙串匯出.p12）並記好密碼。好在蘋果提供了“自動簽名”和雲證書管理（Xcode 裡勾選 Automatically manage signing），能省掉大部分手工操作的坑。

四、上架 App Store 全流程

註冊開發者帳號

準備商店素材

上傳構建包

填隱私與稽核資訊

蘋果稽核（1-7 天）

上架

圖 5：App Store 上架六步走

- 註冊：developer.apple.com，個人帳號 99 美元/年；
- 素材：圖示（1024×1024，別帶圓角——蘋果自己切）、截圖（iPhone 各尺寸）、描述、隱私政策連結；
- 隱私問卷：如實申報收集了哪些資料——不聯網的記帳本可以” 不收集任何資料”；
- 稽核：首次提交常有 1-7 天等待，被拒就按” 稽核備註” 逐條改——蘋果的拒絕理由通常寫得非常具體，照做即可；
- TestFlight：上架前先把測試版發給朋友內測（最多 10000 人），收集一輪反饋再正式提交，專業範兒十足。

五、商店素材怎麼做：第一印象就是下載量

商店頁是使用者“逛店”時看到的櫥窗。三個素材決定第一印象：

- 圖示：1024×1024，別用截圖湊合。沒設計能力就用 AI 工具生成一個簡潔圖形，在桌面上縮小後還能一眼認出才算合格；
- 截圖：至少 2 套（iPhone 主流尺寸），展示最核心的功能場景；每張配一句人話文案（” 一秒記一筆” 比” 高效的記帳解決方案” 好十倍）；
- 描述：第一句說清” 這是什麼 + 給誰用”，別堆砌” 最專業、最強”——真誠比誇張有效。

來動手：把記帳本” 交付” 一次

六、崩潰上報與效能最佳化小清單

App 上架前，這兩件事值得各花半小時：

崩潰監控

你測試時沒崩，不代表使用者手機不崩。蘋果在 Xcode Organizer 裡提供免費的崩潰報告（付費帳號可看真機崩潰日誌）；更省心的方案是接入第三方 SDK（如 Firebase Crashlytics）。

沒有崩潰監控的上架，等於閉著眼睛開車。

效能自查四條

1. 列表用 LazyVStack / List：長列表別用普通 VStack 硬排——懶載入是效能第一課；
2. 圖片別硬塞原圖：用 AsyncImage（遠端圖）和系統快取，5MB 原圖足以卡爆老機型；
3. 別在主執行緒乾重活：網路、大檔案都走 async/await；
4. 用 Instruments 量一量：Xcode 自帶的效能分析器（Time Profiler、記憶體洩漏檢查）——先測量，再最佳化，別憑感覺。

(1) 給 parseAmount 等邏輯補 3 個單元測試（CmdU 跑綠）；(2) 用斷點親手抓一個自己製造的崩潰並修好；(3) Product Archive 打一個包，走 TestFlight 邀請朋友內測；(4) 有餘力就註冊開發者帳號，把記帳本真正提交上架。前三步今晚就能做完。

本章小結

- 除錯三板斧：print 日誌、斷點、SwiftUI 預覽與檢視檢查器
- XCTest：單元測試（CmdU）+ UI 測試 + 手動測試，模板已預置
- 簽名：免費帳號自動簽名（7 天續期）；付費帳號才能分發
- 打包：Generic iOS Device + Archive + Distribute；證書務必備份
- 上架六步：註冊素材上傳隱私問卷稽核上架；TestFlight 先內測

第六十八章：進階之路——權限、通知與現代架構

能做出 *App* 只是開始。這一章聊聊從“會做”到“做好”的幾道坎：怎麼優雅地申請權限、怎麼發通知不討人厭、怎麼把架構再往上抬一層，以及——在 *2026* 年，怎麼和 *AI* 搭檔寫 *iOS* 程式碼。每一條都是你在真實專案裡馬上會遇到的。

一、權限：先敲門，再進門

相機、定位、通訊錄、通知……這些“敏感地帶”，iOS 要求 App 在用到時向使用者跳出視窗申請。規矩是“宣告 + 申請 + 優雅降級”三步：

```
// 1. Info.plist 裡宣告用途文案（不宣告直接崩潰！）
// Privacy - Location When In Use Usage Description: "用於查詢你所在城市的天氣"

// 2. 用的時候申請
import CoreLocation

@Observable
class LocationModel {
    private let manager = CLLocationManager()

    func request() {
        switch manager.authorizationStatus {
        case .notDetermined: manager.requestWhenInUseAuthorization()
        case .denied, .restricted: /* 引導去設定裡開 */
        default: /* 已授權，開幹 */
        }
    }
}

// 3. 被拒後優雅降級：給提示 + 跳設定
// UIApplication.shared.open(URL(string: UIApplication.openSettingsURLString)!)
```

注意：iOS 權限的”一次性”

iOS 的權限跳出視窗只彈一次——使用者拒絕後再也不會彈。所以：申請的時機要選在“使用者正要用的那一刻”（而不是一啟動就彈一堆），文案要說明用途。被拒後的介面引導（“去設定裡開啟定位”）是標配，別讓使用者困在死衚衕裡。

二、通知：在正確的時間說有用的話

通知是 App 觸達使用者的通道，也是最容易被使用者“拉黑”的功能。iOS 用 UserNotifications 框架，發通知三條紀律：只發有用的、讓使用者能關、別半夜騷擾。基礎用法：

```
import UserNotifications

// 1. 申請通知許可權（彈窗一次）
UNUserNotificationCenter.current()
    .requestAuthorization(options: [.alert, .sound, .badge]) { granted, _ in
        print(granted ? "已授權" : "被拒絕")
    }

// 2. 安排一條"每天 20:00 記帳提醒"
func scheduleDailyReminder() {
    let content = UNMutableNotificationContent()
    content.title = "記帳提醒"
    content.body = "今天還沒記帳哦"
    content.sound = .default

    var comps = DateComponents()
    comps.hour = 20
    let trigger = UNCalendarNotificationTrigger(dateMatching: comps, repeats: true)

    let request = UNNotificationRequest(
        identifier: "daily-reminder",
        content: content,
        trigger: trigger
    )
    UNUserNotificationCenter.current().add(request)
}
```

注意：iOS 的通知權限同樣“只彈一次”，且使用者隨時可以在系統設定裡關掉——所以通知內容的價值感很重要，濫發一次可能就永遠失去使用者。

三、架構進階：MVVM 深入與可測試性

第 65 章用過 MVVM，這裡補三個進階要點：

- 協議抽象：讓 ViewModel 依賴協議（protocol）而不是具體類——測試時可以注入”假資

料來源”，這是可測試性的基石；

- 單一職責：一個 ViewModel 只管一件事（記帳的不管天氣）；功能多了就拆；
- 別過度設計：單人小專案，”@Observable 類 + 協議抽象” 就夠了。VIPER、Clean Architecture 這類重架構，等團隊和複雜度找上門再說——工具是服務目標的，不是用來炫耀的。

小貼士：SwiftUI 的” 生態紅利”

iOS 生態還有兩個“只此一家”的進階玩法：小工具（WidgetKit）讓使用者不開啟 App 就能看資訊（記帳本的“本月支出”放桌面），快捷指令（App Intents）讓使用者用 Siri 或自動化呼叫你的功能。兩者都有官方框架，入門成本不高，但對使用者體驗的加成巨大——進階路上值得優先解鎖。

四、2026 年的新玩法：AI 輔助 iOS 開發

2026 年，AI 已經深度嵌入蘋果開發流程。如果你想提速，這套組合拳很成熟：

環節	AI 怎麼幫	你的職責
寫程式碼	Xcode 內建程式碼補全 + 第三方 AI 外掛生成	描述清楚需求，審查產出
查資料	把官方文件和報錯資訊丟給 AI 解讀	關鍵結論回官方文件核實
寫測試	讓 AI 按” 正常/邊界/異常” 三類生成 XCTest 用例	確認用例符合業務語義
改 bug	貼崩潰日誌 + 相關程式碼，AI 定位修復	看懂改動，跑測試驗證

AI 輔助 iOS 開發的四個場景

具體方法論——怎麼給 AI 下指令、怎麼驗收、怎麼避坑——完整的 11 章內容在本書第七篇《AI 程式設計實戰篇》（第 37-47 章）裡，強烈建議配合閱讀：Swift 語法交給本篇，AI 協作的心法交給第七篇。

來動手：給記帳本加” 三件套”

五、Swift 併發要點：@MainActor 與 Task

Swift 6 的併發體系有兩條新手必懂的規則：

- 介面只能在主執行緒更新：SwiftUI 檢視預設是 @MainActor，網路回撥回到主執行緒再改狀態（await MainActor.run { ... } 或把 ViewModel 標成 @MainActor）；
- 非同步任務用 Task 管理：.task { } 修飾符隨檢視出現自動開始、消失自動取消——這是 SwiftUI 裡啟動非同步工作的標準姿勢，別在 body 裡亂寫 Task 裸奔。

```
struct WeatherScreen: View {
    @State private var vm = WeatherViewModel()

    var body: some View {
        Group {
            switch vm.state {
            case .loading: ProgressView()
            case .success(let d): WeatherContent(data: d)
            case .error(let m): Text(m)
            }
        }
        .task {                          // 出現即開始，消失即取消
            await vm.search(city: "北京")
        }
    }
}
```

這兩條規則背後是 Swift 6 的“資料競爭安全”設計——編譯器會在你寫錯時直接報錯，而不是等 App 崩了才發現。編譯器的報錯不是找茬，是替你擋子彈。

(1) 給記帳本加“每天 20:00 記帳提醒”（通知權限 + 本地通知）；(2) 把 ViewModel 的資料來源改成協議抽象，並寫一個注入假資料的單元測試；(3) 加一個“本月支出”桌面小工具。三件套做完，你的專案就有“正規軍”的樣子了。

本章小結

- 權限三步：Info.plist 宣告（漏了必崩）、用時申請、被拒引導去設定
- iOS 權限跳出視窗只彈一次：時機選在” 正要用的那一刻”，文案寫清用途
- 通知三紀律：只發有用的、能關、不騷擾；UserNotifications 框架
- MVVM 進階：協議抽象（可測試）、單一職責、不過度設計
- 生態紅利：WidgetKit 小工具 + App Intents 快捷指令
- AI 輔助四場景：寫碼、查資料、寫測試、改 bug；方法論見第七篇（第 37-47 章）

第六十九章：速查手冊與資源

本章是一份可以獨立使用的速查手冊。當你記不清某個寫法或需要快速決策時，直接翻到這裡。建議把它當成“口袋卡片”：常用程式碼速查、常見問題、學習資源，一頁不落。

一、常用程式碼速查卡

要做什麼	關鍵程式碼
頁面私有狀態	@State private var x = 初值
跨頁面邏輯	`@Observable class VM { var x = 0 }`
豎排/橫排佈局	`VStack { }/HStack { }`
彈性空間	`Spacer()`
列表	List { ForEach(陣列) { ... } }
頁面跳轉	`NavigationStack + NavigationLink`
底部跳出視窗	.sheet(isPresented: $show) { 內容}
關當前頁	`@Environment(\.dismiss) var dismiss`
聯網取資料	`try await URLSession.shared.data(from: url)`
JSON 解析	`struct X: Codable+JSONDecoder().decode`
存小配置	UserDefaults.standard.set(值, forKey: ”鍵”)
存結構化資料	`@Model+modelContainer+@Query`
深色模式適配	用系統顏色（.primary / Color(.systemGray6)），不寫死
跑測試	CmdU

iOS 開發速查卡（2026 年 8 月版）

二、常見問題 FAQ

Q1：我沒有 Mac，能學 iOS 開發嗎？

能學語言，不能做完整開發。iPad/iPhone 上的 Swift Playgrounds 可以學完第 61 章的全部內容；但 Xcode 是 macOS 獨佔，最終必須有一臺 Mac。預算有限可以先租雲 Mac 或買二

手 Mac mini 過渡，學完語言再決定投入。

Q2：學 iOS 開發要花多少錢？

學習階段：0 元（Xcode 免費 + 免費 Apple ID 真機除錯）。發佈階段：開發者帳號 99 美元/年（約 700 元）。加上一台 Mac 的硬體投入，這就是全部成本。相比回報——App Store 上哪怕一個小工具也有付費使用者——這筆帳算下來非常值得。

Q3：SwiftUI 和 UIKit 先學哪個？

SwiftUI。2026 年的新專案、新教學、新崗位都以 SwiftUI 為主，本書全程用它。UIKit 是上一代框架，仍在大量老專案中使用——等你 SwiftUI 熟練後，按需補 UIKit（官方有互操作教學），上手會很快。順序別反了。

Q4：iOS 開發好找工作嗎？

2026 年的行情：純“會畫介面”的初級崗位競爭激烈，但“懂 Swift + SwiftUI + 有上架作品”的開發者依然搶手。iOS 崗位的薪資中位數在各技術棧裡常年靠前。學習期的你，最好的簡歷就是商店裡那個自己做的 App。

Q5：App 被稽核拒絕了怎麼辦？

別慌，很正常——首次提交被拒的機率不低。蘋果的拒絕理由通常寫得非常具體（哪個頁面、哪條準則），逐條修改後在 App Store Connect 裡回覆說明，重新提交即可。常見的三大拒絕原因：權限用途說不清、介面有 bug、後設資料（截圖/描述）與實物不符。提前自查這三項，能省一輪等待。

三、Xcode 高頻快速鍵速查

操作	快速鍵
執行 / 測試	CmdR / CmdU
開啟畫布預覽	OptionCmd
快速開啟檔案	CmdO
全域性搜尋	CmdF
格式化縮排	CtrlI
註釋 / 取消註釋	Cmd/
清理構建快取	CmdK
跳轉到定義	Cmd 點按
重新命名（全域性安全改名）	右鍵 Refactor Rename

Xcode 高頻快速鍵速查

小貼士：編譯玄學先清快取

“昨天還能跑今天就不行”的靈異事件，十有六七是構建快取錯亂。CmdK 清理後重新構建，能治癒一大半玄學問題。還不行就重啟 Xcode（連 Mac 一起），再不行就“刪掉 DerivedData 資料夾”——這是 iOS 開發者口口相傳的終極三連。

四、推薦學習資源

六、推薦學習資源

型別	資源	推薦理由
官方	developer.apple.com	蘋果官方文件，SwiftUI/Swift 教學都在裡面，第一權威
官方	Apple 的 SwiftUI Tutorials	官方手把手課程，新手友好度滿分
官方	Swift 官網 (swift.org)	語言官方文件與線上演練場
社群	Hacking with Swift	Paul Hudson 的免費教學庫，品質公認第一梯隊
社群	Stack Overflow (swiftui 標籤)	報錯求救第一站
影片	Apple Developer 官方頻道（WWDC 影片）	每年新技術的權威講解
設計	Apple 人機介面指南（HIG）	做出” 蘋果味” 介面的官方聖經
配套	第七篇《AI 程式設計實戰篇》（第 37-47 章）	用 AI 輔助 iOS 開發的完整心法

iOS 開發學習資源清單（2026 年 8 月）

五、結語：從第一個 App 到第 N 個 App

回頭看，你從“不知道 SwiftUI 是什麼”，到裝好 Xcode、學會 Swift、用 SwiftUI 畫出介面、給 App 接上資料庫和網路，再到測試、打包、上架——這條路上你已經走完了從零到一最難的一段。

技術的世界變化很快：框架會升級，工具會換代，AI 會接手越來越多“敲鍵盤”的活。但有一樣東西不會過時：你把一個模糊的想法，拆解成清晰的步驟，再一步步實現它的能力。工具是放大器，思路是地基。放大器可以替換，地基不能動搖。

你不再只是” 會玩手機的人”。你能把腦海裡的想法，變成數億臺 iPhone 上真實執行的 App。從今天開始，用鍵盤去創造——這就是 iOS 開發者的浪漫。

參考文獻

1. Apple Inc. (2026). SwiftUI Framework Documentation. developer.apple.com.

2. Apple Inc. (2026). Swift Programming Language Guide. swift.org.

3. Apple Inc. (2026). SwiftData Framework Documentation. developer.apple.com.

4. Apple Inc. (2026). App Store Review Guidelines. developer.apple.com/app-store/review.

5. Apple Inc. (2026). Human Interface Guidelines (HIG). developer.apple.com/design.

6. Apple Inc. (2026). Xcode 27 Release Notes and App Distribution Guide.

十、附錄

附錄 A：新手機開箱“保姆級”流程

開箱驗機“三部曲”：

- **看外觀**：檢查機身、螢幕有無劃痕、磕碰。
- **對資訊**：核對盒子上的 IMEI/序列號和發票上的是否一致。
- **試功能**：撥個電話試試聽筒話筒，拍張照片檢查攝像頭。

啟用與初始設定“重點提醒”：

- **網路**：先連上穩定的 Wi-Fi。
- **賬號**：登入你的蘋果 ID、華為賬號、小米賬號等。
- **基礎設定**：把” 查詢我的手機” 開啟，設定一個靠譜的鎖屏密碼。
- **eSIM 設定**：如果你的手機支援 eSIM（如 iPhone、部分安卓旗艦），可以在設定中新增 eSIM 套餐，無需插入實體 SIM 卡。出國時 eSIM 尤其方便，可以直接購買當地運營商的 eSIM 套餐。
- **資料遷移**：從舊手機遷移資料時，確保兩臺手機電量充足。iPhone 用” 快速開始”，安卓用官方換機 App。

裝 App“安全第一”：

- **首選官方商店**：蘋果用 App Store，安卓用手機自帶的應用商店。
- **許可權別亂給**：安裝時彈出的許可權請求，想清楚” 這個 App 真的需要嗎？”

附錄 B：動手練習小參考

- **基礎篇練習：**清理手機儲存，目標就是找到並刪除那些”忘了是啥”的舊影片、重複截圖和很久不用的 App。
- **軟體篇練習：**設定微信的儲存清理，路徑通常是【我】-【設定】-【通用】-【儲存空間】。
- **進階篇練習：**檢視電池健康度，iPhone 在【設定】-【電池】-【電池健康】裡。
- **通用技巧：**遇到問題，先重啟！這是解決一半以上手機小毛病的萬能鑰匙。

附錄 C：極簡術語“黑話”對照表

你可能聽到的詞	其實就是指……	一句話解釋
SoC / 晶片	手機的大腦和心臟	決定手機快不快、遊戲卡不卡的核心部件。
RAM (運存)	手機的” 短期工作桌”	桌越大，能同時開的 App 就越多，切換越流暢。
ROM (儲存)	手機的” 大倉庫”	存照片、影片、App 的地方。128G、256G 說的就是它。
OLED / LCD	螢幕的兩種材質	OLED 色彩更豔、更省電；LCD 更護眼、成本低。
NFC	近距離無線通訊	讓你手機能變身公交卡、門禁卡、銀行卡的” 貼貼” 功能。
OTA 更新	無線系統升級	在【設定】裡點的那個” 系統更新”，就是它。
Root	安卓超級管理員許可權	獲得後可以修改系統檔案，但也有風險。
BL 鎖	Bootloader 鎖	廠商加的安全鎖，解鎖後才能刷機。
NPU	神經網路處理器	專門處理 AI 任務的晶片，比 CPU 快幾十倍、功耗更低。
端側 AI	手機本地執行的 AI	AI 在手機上直接執行，不需要聯網，速度快、隱私好。
Qi2	新一代無線充電標準	帶磁吸定位的無線充電，類似 MagSafe，充電更方便。
eSIM	嵌入式 SIM 卡	沒有實體卡的 SIM，直接寫入手機，換運營商不用換卡。
LTPO	自適應重新整理率技術	螢幕重新整理率根據內容自動調節（1-120Hz），流暢又省電。

附錄 D：靠譜資源去哪裡找？

官方支援是首選：

- **蘋果使用者**：apple.com/cn 和 iCloud.com。
- **安卓使用者**：手機品牌的官網都有” 服務與支援” 板塊。
- **通用工具**：蘋果用 iTunes，華為用華為手機助手，小米用小米換機。

實用網站與博主：

- **查技巧**：在 B 站搜尋”【你的手機型號】技巧”。
- **看測評**：多看幾個不同博主的測評影片。
- **學拍照**：關注手機攝影教程博主。

安全自查一句話清單：

1. 鎖屏密碼設了嗎？
2. ” 查詢我的手機” 開了嗎？
3. 重要賬號的雙重認證開了嗎？
4. 定期給照片、微信聊天記錄做備份了嗎？
5. 記住防騙口訣：不輕信、不洩露、不轉賬、不貪心、不急躁。

附錄 E：感謝與說明

感謝你堅持讀完這本“手機課”！寫作過程中，我們參考了大量廠商的官方說明書、使用者手冊以及公開的技術科普文章，以確保資訊的準確性。

技術的發展日新月異，書中內容基於撰稿時的主流技術和產品。當你讀到它時，或許又有新的功能和產品出現。希望這本書傳授的思路和方法，能幫助你從容面對未來更多的數字新玩意。

6 版更新要點

- 擴充” 數字資產價值與管理理念”，新增” 圖片與檔案系統管理” 章節（時間 + 事件命名、分類整理、72 小時自動清理、EXIF 脫敏）。
- 完善” 純淨/防護模式” 與” 電子寄生蟲” 防護體系。
- 新增 AI 時代常見詐騙套路（換臉擬聲/冒充權威/虛假投資）與破局防護思路。
- 重寫” 開盒” 與社工攻擊防護。
- 擴充無障礙章節（旁白/TalkBack 完整操作、放大與色彩濾鏡、AI 影像描述、實時字幕、RTT/TTY 通話雙通道）。
- 用” 角色-背景-任務-限制” 思路重寫提示詞框架，附 3 個完整場景案例。
- 新增三篇技術進階附錄：第一章附：安卓手機記憶體拓展解析（本質定義、zRAM/swap 原理、速度瓶頸、適用場景邊界、各廠商命名與開啟方式）。
- 第二十章附：Termux——Android 裝置上的行動式 Linux 終端環境（核心功能、外掛生態、安裝配置、移動開發/伺服器運維/安全測試等典型場景）。
- 第二十三章附：GitHub 入門指南——從程式碼託管到協作平臺（Git vs GitHub 區分、核心概念、註冊上手、Pull Request 協作流、安全提醒、國內訪問方案）。

7 版更新要點

一、新增 4 章內容

- 第 29 章：數字健康——螢幕使用時間分析、App 限額、勿擾模式、20-20-20 護眼法則、頸椎自救、睡眠管理、資訊焦慮” 選擇性失明”、一週數字健康挑戰。
- 第 30 章：螢幕投放與車載——AirPlay / Miracast / DLNA 三陣營對照、有線螢幕投放方案、Samsung DeX 桌面模式、CarPlay / Android Auto / HiCar / CarLife 四大車載方案、會議室螢幕投放實戰。
- 第 31 章：短影音創作——選題三問、拍攝 5 步設定、運鏡五招（推拉搖移跟）、剪映六步工作流、封面製作要點、” 前 3 秒定生死” 內容心法。
- 第 32 章：安全急救手冊——丟失黃金 30 分鐘 SOP、被騙 72 小時處理流程、中毒安全模式排查、帳號被盜奪回六步、勒索資訊應對、日常預防五件事。

二、新增兩章內容

- 第 27 章：適老化與親情——簡易模式品牌對照表、防詐騙六大騙局與三道防線、遠端協助（暢連/向日葵）、微信關懷模式與語音輸入、SOS / 跌倒檢測 / 用藥提醒。
- 第 28 章：選購指南——三步選機法、” 只看五項” 參數列、五檔預算推薦、七大場景選機表、五大購機坑、購買時機與管道分析。

三、新增朗讀與互動功能

- 浮動朗讀控制欄：支援播放/暫停、上一段/下一段、0.5x-2.5x 倍速調節、停止。
- 右側快捷耳機按鈕：一鍵啟動朗讀，自動從當前閱讀位置開始。
- 鍵盤快速鍵：R 切換朗讀，ESC 停止。
- 朗讀體驗最佳化：自動連續翻章，列印時朗讀控制元件自動隱藏。

四、體系更新

目錄、學習路徑（四步五步）、章節地圖、版本號（1.61.7）、附錄編號（五六）、後設資料全部同步重新整理。

五、內容時效性更新

- 處理器標準：驍龍 8 Elite Gen5 / 天璣 9500 劃入頂級檔（2026 年基準）。
- 旗艦機型參考：iPhone 17 Pro、三星 S26 Ultra、小米 17 Ultra、華為 Pura 80 Ultra、vivo X300 Pro。
- AI 章節：更新到 DeepSeek R1、ChatGPT-5、端側 NPU 80 TOPS 算力標準。
- 摺疊屏：價格區間下探到 4000-5000 元檔。
- 選購指南：全部預算分檔和場景推薦重新整理至 2026 年市場。
- 網路與生態：Wi-Fi 7 / 5G-A / 鴻蒙生態 / LE Audio 全部更新到最新規範。

六、新增功能模組：學習儀表盤（位於 Hero 區域下方）

- 閱讀進度環形圖：實時顯示已讀章節百分比，配 SVG 圓環動畫。
- 手機健康自檢：8 項安全檢查清單（密碼、雙重認證、查詢手機、雲備份、微信備份、支付安全、反詐 App、免密清理）。
- 狀態持久化：勾選狀態寫入 localStorage，健康分動態計算並以漸變進度條呈現。
- 分級提示語：根據得分顯示” 滿分/不錯/及格/安全隱患” 等不同反饋。

8 版更新要點

新增第七篇“AI 程式設計實戰篇”（第 37-47 章，共 11 章）：正規化革命與 Agentic Engineering、AI 程式設計五大思維原則、上下文工程與規範驅動開發（SDD）、需求工程、2026 年 AI 程式設計工具全景、大型語言模型全景與 API 實戰、對話工程、從 0 到 1 完整專案全流程、品質保障、進階之路（MCP/多 Agent 協作）與速查手冊。同步更新目錄、學習路徑（五步六步）、章節地圖與速查索引。內容源自《AI 程式設計思維：從零基礎到全棧實戰的方法論指南》2026 年 8 月版。

新增第八篇“安卓開發實戰篇”（第 48-58 章，共 11 章）與第九篇“iOS 開發實戰篇”（第 59-69 章，共 11 章）：兩大平臺均覆蓋環境搭建（Android Studio / Xcode）、語言速成（Kotlin / Swift）、介面框架（Jetpack Compose / SwiftUI）、資料與網路、兩個完整實戰專案（記帳本 + 天氣查詢）、測試與上架（Google Play / App Store）及速查手冊。同步更新目錄、學習路徑（六步八步）、章節地圖與速查索引。內容源自《從零開始的安卓開發》與《從零開始 iOS 開發》2026 年 8 月版。

內容增補：實用軟體與騷擾攔截

- 第十三章新增第六節「實用系統工具」：整理清濁等清理工具、系統自帶儲存清理、國家反詐中心、品牌手機管家、電信業者防騷擾與 AdGuard，並給出「系統優先 / 官方補充 / 第三方剋制」的選型原則。
- 手機自帶騷擾攔截品牌對照表：覆蓋 iOS、小米/Redmi、華為/榮耀、OPPO/realme/一加、vivo/iQOO、三星的入口路徑與推薦開啟項。
- 第八章騷擾攔截能力擴充：補充來電識別、黑白名單、簡訊關鍵詞過濾、iOS 靜音未知來電、勿擾聯動等實操說明，並與第十三章互鏈。
- 三層防護建議：系統攔截 + 電信業者防騷擾 + 國家反詐中心，強調白名單防誤殺。

內容增補：天氣預警 / 自然災害預警 / 地震預警

- 第十三章新增第七節：天氣預警（官方氣象管道 + 系統天氣 App + 顏色等級）、自然災害預警（氣象/應急多管道）、地震預警原理與誤區（預警 $\neq$ 預報）。

- 地震預警品牌開啟表：小米、華為/榮耀、OPPO 系、vivo、iPhone、三星等常見入口與權限注意事項。
- 第三十二章新增「場景零」：地震警報後的避險順序、暴雨/颱風應對、與衛星 SOS / 無網通訊的銜接；急救手冊與第十三章預警設定互鏈。
- 天氣小節聯動：旅行出行中的天氣 App 條目補充預警通知與「開啟通知才收得到」提醒。

內容增補：跨裝置協同與效率工具

- 第十三章第八節：手機遠控電腦（向日葵、ToDesk）及反詐紅線；AI 辦公（Trae / WorkBuddy / Dumate）與手機遙控電腦工作流；米哈遊人工桌面動態桌布。
- 區域網快傳與剪貼簿：LocalSend、飛鴿傳書、隔空投送；KDE Connect、通用剪貼簿、品牌手機電腦互傳。
- 動態口令 2FA：Aegis、Microsoft Authenticator，備用碼與換機注意。
- 滴答清單與紙質掃描：日程/任務用法展開；掃描類工具與拍照技巧補全。

祝你玩轉手機，享受科技帶來的便利與樂趣，也成為一個清醒、自信的數字公民。

——你的數字生活嚮導

附錄 F：摺疊屏選購與使用專題

摺疊屏已經從“未來科技”變成“現在能買”的成熟品類。但它和直板機在形態、使用習慣、保養方式上差異很大，值不值得買、怎麼選、怎麼用，這一篇幫你理清楚。

一、摺疊屏現在發展到什麼程度了？

先說結論：**摺疊屏已經不是“嚐鮮玩具”，但也不是“人人該買”的主流形態**。它介於小眾極客產品和大眾消費品之間，適合特定人群。

目前主流分兩類：

- **橫向摺疊（書本式）**：合上是普通手機大小，展開是接近方形的小平板。代表：華為 Mate X 系列、三星 Galaxy Z Fold 系列、小米 MIX Fold 系列、vivo X Fold 系列。主打”一部手機 = 手機 + 平板”。
- **縱向摺疊（翻蓋式）**：合上像粉餅盒，展開是正常尺寸手機。代表：華為 Pocket 系列、三星 Galaxy Z Flip 系列、OPPO Find N Flip、小米 MIX Flip。主打”小巧便攜 + 外屏速覽”。

2024-2026 年的關鍵進步：鉸鏈壽命普遍標稱 50 萬次摺疊（每天開合 100 次能用 13 年）、UTG 超薄玻璃取代早期塑膠膜、摺痕肉眼可見但手感已很輕微、價格從萬元檔下探到 4000-5000 元檔。

一句話判斷

如果你現在用直板機完全夠用、沒有“螢幕不夠大”的痛點，就別買摺疊屏。它是“錦上添花”的裝置，不是“雪中送炭”的裝置。

二、什麼人適合買摺疊屏？

根據真實使用者反饋，這幾類人買摺疊屏“真香”機率高：

1. **重度閱讀/文件處理者**：經常在手機上看 PDF、批註文件、讀長文。展開後接近 7.5 英寸的內屏，看 A4 PDF 不用縮放，體驗質變。

2. **經常出差/開會的商務使用者**：不想帶平板又嫌手機屏小。橫向摺疊屏展開看 PPT、簽合同、雙屏對照郵件，比直板機效率高一截。

3. **喜歡”懸停態”創作的人**：摺疊到 90 度放在桌上，上半屏取景下半屏按快門（自拍/延時攝影）、上半屏影片下半屏記筆記、上半屏直播下半屏看評論——這是直板機做不到的形態。

4. **追求便攜又想要大屏的女性使用者**：縱向摺疊屏合上能塞進小包，展開是正常手機，外屏還能當化妝鏡、看通知。

這幾類人別買

- 手機只用來刷短影片、聊微信、打電話的——摺疊屏對你毫無增益，反而更重更厚。
- 手小或手腕有腱鞘炎的——橫向摺疊屏展開後普遍 230g+，單手握持累。
- 預算緊張、一部手機要用 4 年以上的——摺疊屏二手保值率低於直板機，維修成本高。

三、選購要點：看這 6 個指標

別被“摺疊”兩個字晃花了眼，選購時重點看：

- **鉸鏈型別**：水滴鉸鏈（摺痕淺、壽命長，目前主流）優於 U 型鉸鏈（摺痕深、早期方案）。問銷售或查評測確認是哪種。
- **內屏材質**：必須是 UTG 超薄玻璃，不要早期那種塑膠膜。UTG 手感接近玻璃，塑膠膜指甲劃過會有凹陷感且易留痕。
- **摺痕表現**：去線下店親手展開看。2026 年的合格水平是：亮屏時正面幾乎看不出，斜光下能看出一條淺溝，手指劃過有輕微凹陷但不割手。
- **外屏尺寸**：橫向摺疊屏的外屏越大越好（最好 6.4 英寸以上），合上能當正常手機用；縱向摺疊屏的外屏越大越實用（4 英寸以上能回訊息、看導航）。
- **重量與厚度**：橫向摺疊展開後 230-260g 是常態，低於 230g 算優秀；合上厚度 12-14mm，低於 12mm 算薄。縱向摺疊合上後能輕鬆放進襯衫口袋才算合格。
- **售後與保修**：重點看內屏碎裂的維修價格。有些品牌首年免費換一次內屏，有些自費要 2000-4000 元。這是摺疊屏最大的隱性成本。

線下體驗清單

1. 親手開合 10 次，感受鉸鏈阻尼是否順滑、有沒有異響。
2. 展開後斜對著燈光看摺痕，再正面亮屏看。
3. 單手握持展開狀態 30 秒，感受重量是否能接受。
4. 試用懸停態：折 90 度放桌上，試試拍照和看影片。

5. 問清楚內屏單獨維修價格和保修政策。

四、摺疊屏獨有的使用技巧

買回來之後，這幾個玩法是直板機體驗不到的：

分屏與多工

橫向摺疊屏展開後，大部分系統支援上下/左右分屏。典型場景：上半屏看網課影片，下半屏記筆記；左邊微信右邊文件；上面地圖導航下面音樂。比直板機分屏實用得多，因為螢幕夠大，每半邊都不擠。

懸停態（90 度摺疊）

這是摺疊屏的“殺手鐧”用法：

- **自拍/合影**：放桌上折 90 度，上半屏取景，下半屏按快門，不用找人幫忙拍。
- **延時攝影/夜景**：同樣放桌上穩定拍攝，省三腳架。
- **視訊通話**：解放雙手，對方看你角度也更自然。
- **看影片**：放桌上當小電視，飛機/高鐵小桌板上特別實用。
- **外屏預覽 + 內屏拍攝**：用後置攝像頭拍自己時，外屏能實時顯示畫面，構圖更準。

外屏速覽

縱向摺疊屏合上後，外屏可以：看時間/通知/天氣、回微信（語音轉文字或快捷回覆）、刷公交卡、看導航、當化妝鏡（呼叫前置攝像頭）。不用每次都展開，省電又省事。

電腦模式

部分橫向摺疊屏（華為、三星）展開後可切換“電腦模式”，外接顯示器鍵鼠後像臺簡易 PC，臨時處理文件、回郵件夠用。不是主力生產力工具，但應急很香。

五、保養要點：讓摺疊屏活得久一點

摺疊屏最嬌貴的就是內屏和鉸鏈，這幾條習慣能顯著延長壽命：

- **別在低溫下強行開合**：0℃ 以下 UTG 玻璃變脆，開合前先揣懷裡暖幾分鐘。這是官方說明書都會寫但沒人看的重點。
- **避免指甲/硬物接觸內屏**：內屏雖有保護層但比外屏軟，用指腹操作，別用指甲戳。

- **合上時確認屏內無異物**：一粒沙子夾在中間合上，可能直接壓壞內屏。合上前掃一眼。
- **不要貼普通鋼化膜**：內屏只能用官方專用膜或 UTG 保護膜，普通鋼化膜會影響摺疊和觸控。建議去官方售後貼膜。
- **鉸鏈別進灰塵液體**：雖然多數標稱 IPX8 防水，但防塵等級普遍較低。沙灘、工地等環境要小心。
- **開合力度均勻**：別單手暴力甩開，雙手從中間勻速展開最穩。

內屏損壞的高發場景

- 合上時夾住耳機線/充電線——直接壓裂內屏。
- 放包裡和鑰匙混裝——外屏沒事，內屏被鑰匙硌出裂痕。
- 冬天從冷處進暖房立刻開合——溫差導致玻璃應力變化。
- 用了一年後覺得” 摺痕變深了” 去撕原廠膜——撕膜會連內屏一起損壞。

六、摺疊屏的“坑”和心理預期

買之前心裡要有這幾本賬：

- **摺痕會一直在**：2026 年的技術做不到” 完全無痕”，只能做到” 亮屏正面幾乎看不出”。如果你是完美主義者，每天盯著摺痕難受，別買。
- **更重更厚**：橫向摺疊屏展開後普遍 230g+，單手握持比直板機累；合上後厚度 12-14mm，褲兜裡更鼓。
- **續航普遍一般**：因為要塞鉸鏈機構，電池空間被壓縮，同價位續航通常不如直板機。
- **保值率低**：一年後二手價通常是新機的 50-60%，比直板機跌得快。
- **維修貴**：內屏總成維修價 2000-4000 元，鉸鏈維修 500-1500 元。買之前查清楚保修政策。
- **App 適配參差**：主流 App 已適配大屏，但小眾 App 可能只是” 拉伸放大”，體驗一般。

摺疊屏不是“更好的手機”，而是“另一種形態的手機”。買它是因為你需要它獨特的形態，不是因為它“更先進”。

——關於摺疊屏的選購哲學

七、主流品牌怎麼選（2026 年）

不寫具體型號（會過時），只寫品牌調性，幫你快速定位：

- **華為 Mate X 系列**：橫向摺疊標杆，鴻蒙生態協同強，外屏比例接近正常手機，合上能當主力機用。價格高但保值率相對最好。
- **三星 Galaxy Z Fold/Flip**：摺疊屏鼻祖，鉸鏈調校成熟，Z Flip 是縱向摺疊的顏值擔當。國內售後網點較少是短板。
- **小米 MIX Fold/Flip**：價效比路線，配置堆料足，價格相對親民。適合想嚐鮮又不想花太多的使用者。
- **vivo X Fold 系列**：橫向摺疊裡外屏比例做得最接近正常手機的，合上當直板機用體驗好。
- **OPPO Find N 系列**：橫向摺疊裡尺寸偏緊湊的，單手可握持，適合覺得大摺疊太重的使用者。
- **華為 Pocket 系列**：縱向摺疊裡外屏最大最實用的，化妝鏡、防曬檢測等女性向功能做得細緻。

選購建議：預算充足追求生態選華為 Mate X；想要最成熟鉸鏈選三星 Z Fold；價效比首選小米/vivo；縱向摺疊看華為 Pocket 和三星 Z Flip。

最後提醒

摺疊屏技術每年都在迭代，本附錄寫於 2026 年。買之前務必查最新評測和使用者反饋，重點關注：鉸鏈壽命實測、摺痕表現、內屏維修政策這三項。線下親手摸永遠比看評測重要。

結語：保持好奇，保持清醒

恭喜你讀到了這裡。

這本書從手機硬體拆到軟體生態，從日常設定挖到開發者模式，從安全防護聊到數字遺產。如果它幫你解決了一個實際問題——哪怕只是學會了關掉一個不該開的許可權、養成了一次備份的習慣——那這些字就沒白寫。

但比任何具體技巧都重要的，是兩件事：

- **保持好奇**：手機每年都在變，AI 在變，摺疊屏在變，衛星通訊在變。今天學的操作步驟明天可能就過時了，但”想知道為什麼”的好奇心永遠不會過時。多問一句”這個許可權是幹什麼的””這個資料存在了哪裡””這個功能會不會洩露隱私”，比記住一百個操作步驟都有用。
- **保持清醒**：手機是工具，不是主人。它設計出來就是為了吸引你的注意力、佔據你的時間、引導你的消費。用它是你的選擇，被它用是你的失控。定期抬頭看看螢幕時間，定期想想”我是在用手機做事，還是在被手機消磨”，這不是矯情，這是數字時代的生存本能。

技術在變，但人的需求沒變——我們需要安全感、需要連線、需要被尊重。手機只是一個載體。真正的“手機課”不在書裡，在你每天解鎖螢幕的那一瞬間。

最後一個練習

合上這本書，拿起你的手機，做一件事：**用 30 秒想想，你的手機裡有哪三樣東西是“丟了會真正心疼”的？**然後，確保它們被備份了。這本書就值了。

附錄 G：全書速查索引

按拼音排序，方便你快速定位到對應章節。

A-B

- BL 鎖第二十四章
- ADB 第二十章
- App 安裝與管理第四章
- AI 助手第二十二章
- AI 手機第二十六章
- AI 換臉詐騙第三十二章
- AI 程式設計第三十七～四十七章
- AI 程式設計五大原則第三十八章
- API Key 第四十二章
- Agent 智慧體第三十七章
- CLAUDE.md 第三十九章
- 安卓開發第四十八～五十八章
- Android Studio 第四十九章
- Activity 生命週期第五十二章
- App Store 上架第六十七章

C-D

- 車機互聯第三十四章
- 電池與續航第十五章

- 密碼管理第十四章
- Passkey 第十四章
- 開發者模式第二十章
- 兒童與手機第三十五章
- Claude Code / Cursor 第四十一章
- DeepSeek 第四十二章
- Compose 第五十一章
- DataStore 第五十三章

F-H

- Foldable 摺疊屏附錄 F
- 故障排查第十九章
- Google Play 上架第五十六章
- GaN 充電器第十五章
- iCloud 第十六章
- 智慧家居第三十六章
- 計算攝影第十一章附

K-R

- 拍照技巧第十一章
- 權限管理第四章
- SafetyNet 認證第二十章
- SIM swap 攻擊第三十二章
- Root 權限第二十章 / 第二十四章
- Termux 第二十章附
- MCP 協議第四十六章
- PRD 需求文件第四十章
- Prompt 工程第四十三章
- Kotlin 第五十章

- MVVM 架構第五十七章・第六十八章
- Room 資料庫第五十三章
- Retrofit 第五十三章

S-Z

- 數字遺產第二十五章
- 資料備份第十八章
- 騷擾攔截第八章・第十三章
- 天氣預警第十三章
- 自然災害預警第十三章・第三十二章
- 地震預警第十三章・第三十二章
- 實用軟體與清理工具第十三章
- 清濁第十三章
- UWB 超寬頻第十七章
- 網路安全第二十一章
- 選購指南第二十八章
- 雲服務第十六章
- SDD 規範驅動開發第三十九章
- Token 第四十二章
- Vibe Coding 第三十七章
- Swift 第六十一章
- SwiftUI 第六十二章
- SwiftData 第六十四章
- URLSession 第六十四章
- Xcode 第六十章
- iOS 開發第五十九～六十九章

其他

- 檔案管理第二章
- 適老化第二十七章
- 工具類 App 推薦第十三章
- 防騷擾 / 國家反詐中心第十三章・第二十七章
- 遠控電腦（向日葵/ToDesk）第十三章
- AI 辦公（Trae 等）第十三章・第四十一章
- LocalSend / 區域網快傳第十三章
- KDE Connect / 剪貼簿同步第十三章・第十七章
- 2FA 動態口令（Aegis 等）第十三章・第十四章
- 滴答清單第十三章
- 紙質檔案掃描第十三章
- 人工桌面（動態桌布）第十三章
- 地震 / 災害手機應急第三十二章
- 螢幕投放與車載第三十章
- 短影音創作第三十一章
- 安全急救第三十二章
- Git/GitHub 第二十三章附
- Git 版本控制第四十五章
- 上下文工程第三十九章
- 需求工程第四十章
- 記帳本 App 第五十四章・第六十五章
- 天氣 App 第五十五章・第六十六章
- WidgetKit 小工具第六十八章

附錄 H：版本路線圖

這本書還在持續更新。以下是未來版本的規劃方向，也歡迎你在反饋中告訴我們你最想看的內容。

v1.8 已落地：新增第七、八、九篇「AI 程式設計實戰篇」「安卓開發實戰篇」「iOS 開發實戰篇」（第 37-69 章，共 33 章）

從 AI 程式設計的正規化革命、五大思維原則、上下文與需求工程、2026 工具與模型全景、API 實戰、對話工程、三個完整專案、品質保障與進階之路——帶你從“會用 AI”升級到“指揮 AI 寫程式碼”；到安卓與 iOS 兩大平臺的完整開發鏈路：環境搭建（Android Studio / Xcode）、語言速成（Kotlin / Swift）、介面框架（Jetpack Compose / SwiftUI）、資料與網路、實戰專案（記帳本 + 天氣查詢）、測試與上架（Google Play / App Store）及速查手冊——帶你從數位使用者一路成長為 App 開發者。

下一版（v1.9）計劃方向

- 手機與可穿戴裝置：Apple Watch、華為手錶、智慧戒指（Oura/Galaxy Ring）的深度協同指南
- 旅行與手機：eSIM 實操、國際用量方案、離線地圖、海外支付、翻譯 App 對比
- 手機與健康管理：運動健康、睡眠監測、心率血氧、女性健康、心理健康 App
- AI 短影音創作：剪映 AI、可靈 AI、數字人、AI 配音的實操教學
- 豎屏短劇入門：劇本結構、拍攝技巧、平臺政策

關於時效性的說明

手機技術迭代極快，本書內容基於 2026 年中的技術和市場狀況編寫。AI 能力、晶片型號、系統版本、價格區間都可能在你閱讀時已發生變化。涉及具體操作步驟時，請以你裝置的實際系統版本為準；涉及購買決策時，請查閱最新評測和使用者反饋。